

FROM NATURAL LANGUAGE REQUIREMENTS TO GRAPHICAL USER INTERFACES: AUTOMATED PROTOTYPING AND VERIFICATION WITH PRETRAINED LANGUAGE MODELS

Dissertation
zur Erlangung des Doktorgrades
der Naturwissenschaften

vorgelegt von
KRISTIAN KOLTHOFF
aus Nagold

genehmigt von der Fakultät für
Naturwissenschaften, Mathematik und Informatik
der Technischen Universität Clausthal

Tag der mündlichen Prüfung
29.04.2026

Dekan	Prof. Dr. René Wilhelm, TU Clausthal
Vorsitzender der Promotionskommission	Prof. Dr. Rüdiger Ehlers, TU Clausthal
Betreuer	Prof. Dr. Christian Bartelt, TU Clausthal
Gutachter	Prof. Dr. Jörg P. Müller, TU Clausthal
Gutachter	Prof. Dr. Simone Paolo Ponzetto, Universität Mannheim

ACKNOWLEDGEMENTS

Throughout the course of my doctoral journey, I have received invaluable support, guidance and encouragement from many individuals, for which I am deeply grateful.

First and foremost, I wish to express my sincere gratitude to my two supervisors, Prof. Dr. Christian Bartelt and Prof. Dr. Simone Paolo Ponzetto. From the beginning of my doctoral journey, Christian contributed substantially to shaping my research direction through countless hours of engaging discussions, provided continuous guidance and support, and offered steady encouragement, particularly during challenging periods. Beyond his significant influence on my academic development, Christian's mentorship also played an important role in my personal growth, for which I am sincerely thankful. Simone likewise supported my research journey in many meaningful ways through insightful discussions, consistently constructive feedback and encouragement, which profoundly improved the quality of my research and this thesis.

I am also deeply grateful to Prof. Dr. Alexander Mädche from the Karlsruhe Institute of Technology for the inspiring collaboration and insightful feedback over the final two years of my doctoral journey. In particular, I wish to express my sincere gratitude to Dr. Felix Kretzer for the close collaboration during this period, which had a substantial impact on this thesis. I greatly appreciated both the professional and personal exchange, and I am especially thankful for his remarkable commitment and perseverance during demanding phases marked by tight deadlines, late-night work and countless discussions.

I am also grateful to my friends and former colleagues at the Institute for Enterprise Systems, University of Mannheim, and my friends and colleagues at the Clausthal University of Technology, Institute for Software and Systems Engineering, for their contributions to this thesis through insightful discussions and diverse perspectives.

I would also like to acknowledge the many researchers I encountered over the years at conferences for the insightful discussions, inspiring perspectives and valuable exchanges. These interactions helped broaden my perspective and refine many of my research ideas.

I am also deeply grateful to my parents, Cornelia Kolthoff and Dr. med. Dr. phil. Helmut Kolthoff, whose constant support and encouragement throughout my life provided the foundation for my academic journey and made this work possible.

Lastly, my deepest gratitude goes to my family – my wonderful wife Yaru, and my son, Lio – for your unconditional love, understanding and endless support, which carried me through every stage of this journey. Without your love, constant encouragement and many sacrifices, pursuing a PhD and completing this work would not have been possible.

ABSTRACT

Requirements elicitation is a crucial activity before and during the development of interactive software systems, ensuring that the resulting product meets the needs of the stakeholders. Typically, the process of requirements elicitation relies heavily on natural language (NL) communication between requirements analysts and stakeholders, being prone to misunderstandings due to the inherent ambiguity of NL. While formal software specifications serve as ambiguity mitigation, they necessitate technical knowledge for proper interpretation. Therefore, Graphical User Interface (GUI) prototyping has emerged as an effective requirements elicitation and validation technique, providing tangible, visual artifacts representing the implemented requirements and acting as a common language between analysts and stakeholders. In particular, employing GUI prototyping improves the elicitation and validation of requirements by enabling fruitful discussions and integrating stakeholders more closely into the development process. Although GUI prototyping offers these advantages in comparison to other elicitation and validation approaches, it simultaneously necessitates increased manual effort, which is time-consuming and costly, especially for high-fidelity GUI prototypes that enable stakeholders to provide feedback of higher quality. While requirements validation ensures that the software system aligns with the actual needs of the stakeholders, verification of requirements is equally important, ensuring that the implementation of requirements in the software system and prototypes aligns with the specification. However, similar to validation, verification is still often a manual and effort-demanding task, while existing solutions offer mostly merely rule-based, static techniques and approaches.

In this work, we propose several approaches to address the mentioned challenges, namely (**C1**) reducing the effort required to transform NL requirements into GUI prototypes to facilitate requirements elicitation and validation, and (**C2**) reducing the effort required for conducting requirements verification in GUI applications and prototypes. Specifically, we tackle the first challenge (**C1**) by initially presenting novel NL-based GUI retrieval and reranking approaches, enabling to rapidly map NL requirements to tangible GUI prototypes, while creating new benchmarks and achieving state-of-the-art performance. Furthermore, we propose several methods for efficiently adapting LLMs to GUI generation and devise novel techniques to efficiently enable LLM-based generation for proprietary GUI representations, allowing to quickly translate NL requirements into customizable GUI prototypes. We evaluated these techniques on a novel benchmark encompassing an extensive collection of human annotations created via crowdsourcing,

showing their high effectiveness for GUI generation. To tackle the second challenge (**C2**), we initially present novel LLM-based methods enabling the verification of semantically complex NL requirements on static GUI prototypes. To assess their performance, we created a high-quality benchmark, which links NL requirements to GUI prototypes, indicating high effectiveness for requirements verification on GUI prototypes. Moreover, we propose a multi-modal (M)LLM-based agent approach that enables verification of complex functional and non-functional NL requirements of fully-fledged, highly dynamic GUI prototypes and applications by automatically creating and evaluating meaningful interaction trajectories, showing high effectiveness on a novel benchmark of human-annotated NL requirements and GUI applications. To summarize, by providing effective methods for rapidly mapping NL requirements to GUI prototypes, we facilitate automated GUI prototyping and substantially increase prototyping productivity, positively impacting elicitation and validation activities. Moreover, with our novel and effective MLLM-based agent approach, we substantially improve and facilitate the automated verification of complex NL requirements in GUIs.

ZUSAMMENFASSUNG

Die Erhebung von Anforderungen ist eine entscheidende Tätigkeit vor und während der Entwicklung interaktiver Softwaresysteme und stellt sicher, dass das resultierende Produkt den Bedürfnissen der Stakeholder entspricht. Typischerweise stützt sich der Prozess der Anforderungserhebung stark auf die Kommunikation in natürlicher Sprache (NL) zwischen Anforderungsanalysten und Stakeholdern und ist aufgrund der inhärenten Mehrdeutigkeit der natürlichen Sprache anfällig für Missverständnisse. Während formale Softwarespezifikationen zur Reduzierung von Mehrdeutigkeiten dienen, erfordern sie technisches Fachwissen für eine korrekte Interpretation. Daher hat sich das Prototyping grafischer Benutzeroberflächen (Graphical User Interface (GUI)) als effektive Technik zur Erhebung und Validierung von Anforderungen etabliert, da es greifbare, visuelle Artefakte bereitstellt, die die implementierten Anforderungen repräsentieren und als gemeinsame Sprache zwischen Analysten und Stakeholdern fungieren. Insbesondere verbessert der Einsatz von GUI-Prototyping die Erhebung und Validierung von Anforderungen, indem er fruchtbare Diskussionen ermöglicht und Stakeholder enger in den Entwicklungsprozess einbindet. Obwohl GUI-Prototyping diese Vorteile gegenüber anderen Erhebungs- und Validierungsansätzen bietet, erfordert es gleichzeitig einen erhöhten manuellen Aufwand, der zeitaufwendig und kostspielig ist – insbesondere bei hochausgereiften GUI-Prototypen, die es Stakeholdern ermöglichen, qualitativ hochwertigeres Feedback zu geben. Während die Anforderungvalidierung sicherstellt, dass das Softwaresystem den tatsächlichen Bedürfnissen der Stakeholder entspricht, ist die Verifikation von Anforderungen ebenso wichtig, da sie gewährleistet, dass die Umsetzung der Anforderungen im Softwaresystem und in Prototypen mit der Spezifikation übereinstimmt. Allerdings ist die Verifikation – ähnlich wie die Validierung – nach wie vor häufig eine manuelle und aufwändige Aufgabe, während bestehende Lösungen meist lediglich regelbasierte, statische Techniken und Ansätze bieten.

In dieser Arbeit schlagen wir mehrere Ansätze vor, um die genannten Herausforderungen zu adressieren, nämlich (**C1**) den Aufwand zur Transformation von NL-Anforderungen in GUI-Prototypen zu reduzieren, um die Erhebung und Validierung von Anforderungen zu erleichtern, und (**C2**) den Aufwand für die Durchführung der Anforderungsverifikation in GUI-Anwendungen und -Prototypen zu verringern. Konkret adressieren wir die erste Herausforderung (**C1**), indem wir zunächst neuartige NL-basierte Ansätze zur GUI-Suche und -Neubewertung (Reranking) vorstellen, die eine schnelle Abbildung von NL-Anforderungen auf greifbare GUI-Prototypen ermögli-

chen, während wir neue Benchmarks erstellen und eine Leistungsfähigkeit auf dem Stand der Technik erreichen. Darüber hinaus schlagen wir mehrere Methoden zur effizienten Anpassung großer Sprachmodelle (LLMs) an die GUI-Generierung vor und entwickeln neuartige Techniken, um die LLM-basierte Generierung von proprietären GUI-Repräsentationen effizient zu ermöglichen, sodass NL-Anforderungen schnell in anpassbare GUI-Prototypen übersetzt werden können. Wir evaluierten diese Techniken anhand eines neuartigen Benchmarks, der eine umfangreiche Sammlung von durch Crowdsourcing erstellten menschlichen Annotationen umfasst, und zeigen deren hohe Effektivität für die GUI-Generierung. Zur Bewältigung der zweiten Herausforderung (C2) präsentieren wir zunächst neuartige LLM-basierte Methoden, die die Verifikation semantisch komplexer NL-Anforderungen auf statischen GUI-Prototypen ermöglichen. Zur Bewertung ihrer Leistungsfähigkeit erstellten wir einen hochwertigen Benchmark, der NL-Anforderungen mit GUI-Prototypen verknüpft und eine hohe Effektivität für die Anforderungsverifikation auf GUI-Prototypen zeigt. Darüber hinaus schlagen wir einen multimodalen (M)LLM-basierten Agentenansatz vor, der die Verifikation komplexer funktionaler und nicht-funktionaler NL-Anforderungen vollständig ausgereifter, hochdynamischer GUI-Prototypen und -Anwendungen ermöglicht, indem automatisch sinnvolle Interaktionspfade erzeugt und evaluiert werden. Dieser Ansatz zeigt eine hohe Effektivität auf einem neuartigen Benchmark mit von Menschen annotierten NL-Anforderungen und GUI-Anwendungen. Zusammenfassend erleichtern wir durch die Bereitstellung effektiver Methoden zur schnellen Abbildung von NL-Anforderungen auf GUI-Prototypen das automatisierte GUI-Prototyping und steigern die Produktivität des Prototypings erheblich, was sich positiv auf Erhebungs- und Validierungsaktivitäten auswirkt. Darüber hinaus verbessern und erleichtern wir mit unserem neuartigen und effektiven MLLM-basierten Agentenansatz die automatisierte Verifikation komplexer NL-Anforderungen in GUIs erheblich.

Contents

List of Publications	xv
List of Figures	xvii
List of Tables	xxi
List of Acronyms	xxiii
1 Introduction	2
1.1 Motivation	2
1.2 Problem Statement	6
1.3 Solution Overview	9
1.4 Contributions	10
1.5 Outline	14
2 Theoretical Background	16
2.1 Requirements and Software Engineering (RE/SE)	16
2.1.1 Requirements Elicitation (REl)	21
2.1.2 Requirements Validation (RVal)	22
2.1.3 Software Validation (SVal) and Verification (SVer)	23
2.2 GUI Prototyping	25
2.2.1 History of GUI Prototyping Tools	26
2.2.2 GUI Prototyping Processes and Approaches	28
2.2.3 GUI Prototype Characteristics	29
2.3 Computational Methods and Techniques	31
2.3.1 Machine Learning (ML)	31
2.3.2 Information Retrieval (IR)	33
2.3.3 Pretrained Language Models (PLMs)	35
2.3.4 Prompt and Context Engineering (PE/CE)	40
I PROTOTYPING: GUI RETRIEVAL VIA NATURAL LANGUAGE	44
3 Natural-Language-Based GUI Retrieval	46
3.1 Motivation	46
3.2 Approach: RaWi	49

3.2.1	GUI Repository and Text Extraction	50
3.2.2	GUI Retrieval and Text Preprocessing	52
3.2.3	Deriving Editable GUI Prototypes	56
3.2.4	Prototype Implementation	57
3.3	Experimental Evaluation	59
3.3.1	RQ ₁ : GUI Retrieval Performance	59
3.3.2	RQ ₂ : Productivity of Rapid Prototyping	66
3.3.3	RQ ₃ : Perceived Usefulness	68
3.4	Results & Discussion	68
3.4.1	RQ ₁ : GUI Retrieval Performance	68
3.4.2	RQ ₂ : Productivity of Rapid Prototyping	72
3.4.3	RQ ₃ : Perceived Usefulness	74
3.5	Threats to Validity	75
3.6	Limitations	76
3.7	Related Work	77
3.8	Conclusion	79
4	GUI-ReRank: Enhancing GUI Retrieval with Multi-Modal LLM-based Reranking	80
4.1	Motivation	80
4.2	Approach: GUI-ReRank	82
4.2.1	Automated GUI Repository Annotation	82
4.2.2	Embedding-based Constrained GUI Retrieval	83
4.2.3	Multi-Modal (M)LLM-based GUI Reranking	84
4.2.4	Prototype Implementation	85
4.3	Experimental Evaluation	86
4.3.1	RQ ₁ : Effectiveness of MLLM-based GUI Reranking	86
4.3.2	RQ ₂ : MLLM-based Reranking Effectiveness-Cost Trade-offs	87
4.4	Results & Discussion	88
4.4.1	RQ ₁ : Effectiveness of MLLM-based GUI Reranking	88
4.4.2	RQ ₂ : MLLM-based Reranking Effectiveness-Cost Trade-offs	90
4.5	Threats to Validity	92
4.6	Limitations	92
4.7	Related Work	93
4.8	Conclusion	93
5	Self-Elicitation of Requirements with Automated GUI Prototyping	94
5.1	Motivation	94
5.2	Approach: SERGUI	96
5.2.1	Interaction Model	97
5.2.2	NLR-Based GUI Retrieval	98
5.2.3	NLR-based GUI Feature Retrieval	100
5.2.4	LLM-based GUI Feature Recommendation	101

CONTENTS

5.2.5	Feature-based GUI Reranking	103
5.2.6	GUI Prototype Output	103
5.2.7	Prototype Implementation	104
5.3	Experimental Evaluation	104
5.3.1	User Study	105
5.3.2	RQ ₁ : Feature Recommendation Relevance	106
5.3.3	RQ ₂ : GUI Feature Retrieval Relevance	106
5.3.4	RQ ₃ : GUI Reranking Performance	107
5.3.5	RQ ₄ : Perceived Usefulness	107
5.4	Results & Discussion	107
5.4.1	RQ ₁ : Feature Recommendation Relevance	107
5.4.2	RQ ₂ : GUI Feature Retrieval Relevance	109
5.4.3	RQ ₃ : GUI Reranking Performance	110
5.4.4	RQ ₄ : Perceived Usefulness	111
5.5	Threats to Validity	112
5.6	Limitations	113
5.7	Related Work	114
5.8	Conclusion	115
 II PROTOTYPING: GUI GENERATION VIA NATURAL LANGUAGE		116
 6 Tailoring LLMs to Prototyping: Zero-Shot GUI Generation with Retrieval-Augmentation and Self-Critique Prompting		118
6.1	Motivation	119
6.2	Approach	121
6.2.1	Baselines: ZS-Instruction and ZS-CoT	121
6.2.2	PD for GUI Generation	121
6.2.3	RAGG	122
6.2.4	SC Looping for GUI Generation	124
6.2.5	LLM-based GUI Content Generation	124
6.3	Experimental Evaluation	125
6.3.1	GUI Requirements Dataset	126
6.3.2	RQ ₁ : Effectiveness of Prompting for GUI Generation	127
6.3.3	RQ ₂ : Effectiveness of Example Number on RAGG	129
6.3.4	RQ ₃ : Effectiveness of Number of Loops in SC	130
6.3.5	RQ ₄ : Effectiveness of GUI Content Generation	130
6.4	Results & Discussion	130
6.4.1	RQ ₁ : Effectiveness of Prompting for GUI Generation	130
6.4.2	RQ ₂ : Effectiveness of Example Number on RAGG	133
6.4.3	RQ ₃ : Effectiveness of Number of Loops in SC	133
6.4.4	RQ ₄ : Effectiveness of GUI Content Generation	134
6.5	Threats to Validity	134
6.6	Limitations	135

6.7	Related Work	136
6.8	Conclusion	136
7	Closing the Loop Between User Stories and GUI Prototypes: An LLM-based Assistant for Cross-Functional Integration in Software Development	138
7.1	Motivation	139
7.2	Approach	140
7.2.1	LLM-Based Approaches	140
7.2.2	GUI Prototyping Plugin (Control)	144
7.2.3	GUI Prototyping Plugin (Treatment)	146
7.3	Experimental Evaluation	149
7.3.1	RQ ₁ : GUI Component Generation	149
7.3.2	RQ ₂ , RQ ₃ : GUI Prototyping Assistant Evaluation	151
7.3.3	RQ ₄ : User Stories Generation	153
7.4	Results	154
7.4.1	RQ ₁ : GUI Component Generation	154
7.4.2	RQ ₂ , RQ ₃ : GUI Prototyping Assistant Evaluation	155
7.4.3	RQ ₄ : User Stories Generation	159
7.5	Discussion	160
7.5.1	RQ ₁ : Effectiveness of Generating Prototype Components	160
7.5.2	RQ ₂ , RQ ₃ : LLM-Based Assistant Impact on GUI Quality	160
7.5.3	RQ ₄ : Effectiveness of Generating US from Components	162
7.5.4	Generating GUI Prototypes from USs or Vice-Versa?	162
7.6	Threats to Validity	163
7.7	Limitations	164
7.8	Related Work	165
7.9	Conclusion	166
8	GUIDE: LLM-Driven GUI Generation Decomposition for Automated Prototyping	168
8.1	Motivation	169
8.2	Approach: GUIDE	170
8.2.1	Figma Plugin User Interface	170
8.2.2	GUI Generation Decomposition	171
8.2.3	Prototype Implementation	172
8.3	Experimental Evaluation	172
8.3.1	RQ ₁ : Effectiveness of GUI Prototype Generation	172
8.4	Results & Discussion	174
8.4.1	RQ ₁ : Effectiveness of GUI Prototype Generation	174
8.5	Threats to Validity	175
8.6	Limitations	176
8.7	Related Work	176

8.8	Conclusion	176
III	VERIFICATION: (M)LLM-BASED APPROACHES	178
9	Interlinking GUIs and User Stories: A Semi-Automatic LLM-Based Approach	180
9.1	Motivation	180
9.2	Approach	182
9.2.1	GUI Prototype Abstraction	183
9.2.2	User Story Implementation Detection	183
9.2.3	User Story GUI Component Matching	184
9.3	Experimental Evaluation	184
9.3.1	Data Collection	185
9.3.2	RQ ₁ : User Story Implementation Detection	186
9.3.3	RQ ₂ : User Story GUI Component Matching	188
9.4	Results & Discussion	189
9.4.1	RQ ₁ : User Story Implementation Detection	189
9.4.2	RQ ₂ : User Story GUI Component Matching	190
9.5	Threats to Validity	192
9.6	Limitations	193
9.7	Related Work	193
9.8	Conclusion	194
10	GUISpector: An MLLM Agent Framework for Automated Verification of Natural Language Requirements in GUI Applications	196
10.1	Motivation	197
10.2	Approach: GUISpector	198
10.2.1	Human-in-the-Verification-Loop	199
10.2.2	MLLM GUI Agent Verification Loop	200
10.2.3	Agentic Implementation-Verification Loop	201
10.2.4	Prototype Implementation	202
10.3	Experimental Evaluation	202
10.3.1	RQ ₁ : Effectiveness of MLLM CUA for GUI Verification	202
10.3.2	RQ ₂ : Cost for MLLM-based CUA GUI Verification	204
10.4	Results & Discussion	204
10.4.1	RQ ₁ : Effectiveness of MLLM CUA for GUI Verification	204
10.4.2	RQ ₂ : Cost for MLLM-based CUA GUI Verification	208
10.5	Threats to Validity	210
10.6	Limitations	210
10.7	Related Work	210
10.8	Conclusion	211

IV	DISCUSSION AND CONCLUSION	212
II	Discussion	214
II.1	GUI Retrieval and GUI Generation Trade-offs	214
II.2	LLM-based Approaches Augment Human Capabilities	215
II.3	Cost-Effectiveness of LLM-based Approaches	215
II.4	Evaluation Challenges for GUI Prototyping Approaches	215
II.5	Hallucinations in LLM-based Approaches	216
12	Conclusion	218
	Bibliography	223
A	Published Resources	248
B	Appendix for Chapter 3	250
B.1	Experimental Details	250
C	Appendix for Chapter 4	255
C.1	Experimental Details	255
D	Appendix for Chapter 6	257
D.1	Experimental Details	257
E	Appendix for Chapter 9	260
E.1	Experimental Details	260

LIST OF PUBLICATIONS

This thesis is based on work that has been previously published in proceedings of premier international software engineering conferences including the *IEEE/ACM International Conference on Automated Software Engineering* and the corresponding *Automated Software Engineering Journal*, *IEEE/ACM International Conference on Software Engineering*, *ACM CHI Conference on Human Factors in Computing Systems* and the *IEEE International Requirements Engineering Conference*. These publications are annotated with their respective conference rankings (A*, a leading conference in the research area and A, a highly respected conference in the research area) based on the CORE Conference Ranking, which is a popular and widely adopted ranking system for conferences in computer science. The publications encompass text, tables and figures and are listed in chronological order regarding their publication time. Subsequent chapters will reference these publications accordingly. Additionally, the list includes work that has been submitted to top-tier conferences but is currently undergoing peer review, which we denote accordingly.

Kolthoff, Kristian. Automatic Generation of Graphical User Interface Prototypes from Unrestricted Natural Language Requirements. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE, A*)*, San Diego, CA, USA, January 2020, pages 1234–1237. IEEE.

Kolthoff, Kristian, Bartelt, Christian, and Ponzetto, Simone Paolo. GUI2WiRe: Rapid Wireframing with a Mined and Large-Scale GUI Repository using Natural Language Requirements. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE, A*)*, Melbourne, Australia (Virtual Event), January 2021, pages 1297–1301. ACM.

Kolthoff, Kristian, Bartelt, Christian, and Ponzetto, Simone Paolo. Automated Retrieval of Graphical User Interface Prototypes from Natural Language Requirements. In *Proceedings of the 26th International Conference on Applications of Natural Language to Information Systems (NLDB)*, Saarbrücken, Germany (Virtual Event), June 2021, pages 376–384, Cham: Springer International Publishing.

Kolthoff, Kristian, Bartelt, Christian, and Ponzetto, Simone Paolo. Data-Driven Prototyping via Natural-Language-based GUI Retrieval. *Automated Software Engineering*, March 2023, 30(1), 13, pages 1–34, Springer.

Kolthoff, Kristian*, Kretzer, Felix*, Bartelt, Christian, Maedche, Alexander, and Ponzetto, Simone Paolo. Interlinking User Stories and GUI Prototyping: A Semi-Automatic LLM-based Approach. In *Proceedings of the 32nd IEEE International Requirements Engineering Conference (RE, A)*, Reykjavik, Iceland, June 2024, pages 380–388. IEEE.

Kolthoff, Kristian, Bartelt, Christian, Ponzetto, Simone Paolo, and Schneider, Kurt. Self-Elicitation of Requirements with Automated GUI Prototyping. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE, A*)*, Sacramento, CA, USA, October 2024, pages 2354–2357. ACM.

Kretzer, Felix*, **Kolthoff, Kristian***, Bartelt, Christian, Ponzetto, Simone Paolo, and Maedche, Alexander. Closing the Loop Between User Stories and GUI Prototypes: An LLM-based Assistant for Cross-Functional Integration in Software Development. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI, A*)*, Yokohama, Japan, April 2025, pages 1–19. ACM.

Kolthoff, Kristian*, Kretzer, Felix*, Bartelt, Christian, Maedche, Alexander, and Ponzetto, Simone Paolo. GUIDE: LLM-Driven GUI Generation Decomposition for Automated Prototyping. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion) (ICSE, A*)*, Ottawa, ON, Canada, April 2025, pages 1–4. IEEE.

Kolthoff, Kristian, Kretzer, Felix, Bartelt, Christian, Maedche, Alexander, and Ponzetto, Simone Paolo. GUI-ReRank: Enhancing GUI Retrieval with Multi-Modal LLM-based Reranking. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE, A*)*, Seoul, Republic of Korea, January 2026, pages 1–4. ACM. (in press)

Kolthoff, Kristian*, Kretzer, Felix*, Bartelt, Christian, Maedche, Alexander, and Ponzetto, Simone Paolo. GUIspecter: An MLLM Agent Framework for Automated Verification of Natural Language Requirements in GUI Prototypes. In *Proceedings of the 48th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, April 2026, 1–4, Rio de Janeiro, Brazil. ACM. (in press)

Furthermore, we list papers that are currently available as ArXiv preprints and are undergoing review at the time of thesis submission.

Kolthoff, Kristian*, Kretzer, Felix*, Fiebig, Lennart, Maedche, Alexander, Ponzetto, Simone Paolo, and Bartelt, Christian. Zero-Shot Prompting Approaches for LLM-based Graphical User Interface Generation. arXiv preprint arXiv:2412.11328, 2024. The paper is currently under review for the *2026 ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*.

* Authors contributed equally.

List of Figures

1.1	Schematic overview of requirements elicitation, validation and GUI prototyping	6
1.2	Schematic overview of GUI verification	7
1.3	Overview and relatedness of the challenges addressed in the thesis . . .	8
1.4	Overview of solution approaches	10
2.1	Requirements template for structured documentation of Natural Language Requirements (NLR) (adapted from Pohl (2016))	17
2.2	Linear and incremental RE process model (adapted from Kotonya and Sommerville (1998))	18
2.3	Iterative RE process model (adapted from Loucopoulos and Karakostas (1995))	19
2.4	Waterfall model for SD (adapted from Sommerville (2011))	20
2.5	REl techniques categorization (adapted from Pacheco et al. (2018)) . .	21
2.6	Overview of RVal techniques and their characteristics	23
2.7	V-Model for SD (adapted from Sommerville (2011))	24
2.8	History overview of the development of GUI prototyping editors and tools (adapted from Silva et al. (2017))	26
2.9	Iterative GUI prototyping process model (adapted from Van der Veer and Van Vliet (2001))	28
2.10	User-Centered Design process model (adapted from Zimmermann and Grötzbach (2007))	29
2.11	GUI prototyping process model (adapted from Weichbroth and Sikorski (2015))	30
2.12	Three-level GUI prototype fidelity overview (adapted from Feng et al. (2023))	31
2.13	<i>Skip-gram</i> and <i>CBOW</i> architectures for learning semantic word representations (adapted from Mikolov et al. (2013a) and Lauscher (2021)) .	34
2.14	BERT approach with pretraining and finetuning of a <i>Transformer</i> encoder (adapted from Devlin et al. (2019))	37
2.15	SBERT approach with training and inference architecture based on the BERT model (adapted from Reimers and Gurevych (2019))	38
2.16	MLLM architecture of the <i>MiniGPT-4</i> model (adapted from Zhu et al. (2023))	40

2.17	<i>Prompt Engineering (PE)</i> techniques overview	42
3.1	Iterative GUI prototyping process with <i>RaWi</i>	49
3.2	Architecture overview of <i>RaWi</i>	50
3.3	Extracting text segments from GUI prototypes	52
3.4	PRF-KLD scoring example for AQE	54
3.5	BERT-LTR model architecture	55
3.6	Example for deriving partly editable GUI prototypes	57
3.7	Web-based GUI prototyping editor of <i>RaWi</i>	58
3.8	Overview of the gold standard creation pipeline for NLR-based GUI retrieval	60
3.9	Four example GUI-NLR pairs taken from the gold standard	61
3.10	Example of an NLR from the gold standard	62
3.11	Relevance level distributions for the gold standard	63
3.12	Boxplots for NLR-based GUI retrieval effectiveness	70
3.13	Ranking plots for Oracle, BM25 and BERT-LTR	71
3.14	Example top-5 rankings with BERT-LTR	72
3.15	Evaluation results of the <i>RaWi</i> user study	73
3.16	Perceived usability histograms	75
4.1	Overview of the <i>GUI-ReRank</i> approach	82
4.2	Overview of MLLM-based GUI annotation pipeline	83
4.3	Overview of embedding-based constrained multi-dimensional GUI re- trieval approach	84
4.4	MLLM-based GUI reranking approach	85
4.5	Boxplots for NLR-based GUI reranking effectiveness	88
4.6	Ranking plots for Oracle, BERT-LTR and MLLM-based reranking	89
4.7	Effectiveness-cost (API costs) trade-off for MLLM-based reranking	91
4.8	Effectiveness-cost (time) trade-off for MLLM-based GUI reranking	92
5.1	Overview of the interactive GUI prototyping procedure with <i>SERGUI</i>	96
5.2	Overview of the <i>SERGUI</i> architecture	97
5.3	NLR-based GUI retrieval approach	99
5.4	NLR-based GUI feature retrieval approach	100
5.5	LLM-based GUI feature recommendation approach	101
5.6	Example GUI summary outputs from <i>SERGUI</i>	103
5.7	Web-based <i>SERGUI</i> tool prototype	104
5.8	Boxplots for LLM-based GUI feature recommendation	109
5.9	Example GUI feature recommendations	110
5.10	Perceived usability histograms	112
6.1	Overview of ZS prompting approaches	120
6.2	Overview of ZS prompting approaches architecture	122
6.3	Overview of the LLM-based content generation pipeline	125

LIST OF FIGURES

6.4	Overview of the evaluation procedure	126
6.5	Boxplots for LLM-based GUI generation effectiveness	131
6.6	Mean input and output token usage for LLM-based GUI generation .	132
6.7	GUI examples from LLM-based GUI generation	133
7.1	Overview of the LLM framework integrated into <i>Figma</i>	141
7.2	Proposed LLM-based <i>Figma</i> GUI prototyping plugin	145
7.3	<i>Control</i> and <i>treatment</i> GUI prototyping assistant configuration . . .	146
7.4	New and changed US intervention	148
7.5	Overview of the evaluation procedure	150
7.6	Violin plots for LLM-based GUI component generation effectiveness	154
7.7	Example GUI prototypes created during the evaluation with the <i>control</i> and <i>treatment</i> GUI prototyping assistant configuration	155
7.8	Boxplots for US completion with <i>control</i> and <i>treatment</i> GUI prototyp- ing assistant configuration	156
7.9	NASA-TLX (raw) results for <i>control</i> and <i>treatment</i> GUI prototyping assistant configuration	159
7.10	Violin plots for LLM-based US generation	160
7.11	Discussion on example GUI prototypes created during the evaluation	161
8.1	Overview of the <i>GUIDE</i> architecture	170
8.2	Boxplots for <i>control</i> and <i>treatment</i> for GUI prototype quality	175
9.1	Overview of LLM-based approach for US implementation detection and GUI component matching	182
9.2	#Word/US and #Comps/US distributions across the dataset	186
9.3	Examples from the dataset with USs/GUI component annotations . .	187
9.4	Confusion matrices for LLM-based approaches for US implementation detection	189
9.5	GUI examples with respective USs for error analysis	190
9.6	Boxplots for LLM-based GUI component matching effectiveness . . .	192
9.7	Confusion matrices for LLM-based approaches for GUI component matching	192
10.1	Overview of <i>GUISpector</i> for MLLM-based CUA GUI verification . .	197
10.2	Overview of the <i>GUISpector</i> architecture	199
10.3	Tool prototype of <i>GUISpector</i> showing a requirements collection . . .	200
10.4	#Words/(NLR/AC) and NLR label distributions per app	203
10.5	Bar charts for MLLM-based CUA GUI verification effectiveness on requirements level	205
10.6	Bar charts for MLLM-based CUA GUI verification effectiveness on AC level	205
10.7	Confusion matrices for MLLM-based CUA GUI verification on the requirements level	206

10.8	Stacked bar charts with predicted GUI action distributions across apps	207
10.9	Example trajectory extracted from the gold standard run	208
10.10	Boxplots for cost-related performance metrics	209

List of Tables

2.1	Overview of popular LLMs/MLLMs	39
3.1	Summary statistics for the NLR-based GUI retrieval dataset	63
3.2	Evaluation results of NLR-based GUI retrieval approaches (1)	68
3.3	Evaluation results of NLR-based GUI retrieval approaches (2)	69
4.1	Evaluation results of MLLM-based GUI reranking approaches (1)	86
4.2	Evaluation results of MLLM-based GUI reranking approaches (2)	87
4.3	Cost-related performance metrics for MLLM-based GUI reranking	90
5.1	Evaluation results of LLM-based GUI feature recommendation	108
5.2	Evaluation results of GUI feature (<i>aspect</i> -GUI) retrieval	III
6.1	Likert-scale items and self-reported assessments for evaluating LLM-generated GUI prototypes	128
7.1	Feature comparison of <i>control</i> and <i>treatment</i> configuration of the GUI prototyping assistant	147
7.2	Mean ratings of Likert-scale items from crowdworkers for generated GUI prototypes	157
7.3	Perceived usability results between <i>control</i> and <i>treatment</i>	158
8.1	Evaluation results of LLM-driven GUI generation (1)	173
8.2	Evaluation results of LLM-driven GUI generation (2)	174
9.1	Summary statistics for GUI-US dataset	186
9.2	Evaluation results of LLM-based approaches for US implementation detection in GUIs	188
9.3	Evaluation results of LLM-based approaches for GUI component matching from USs	191
10.1	Summary statistics for NLR-based GUI verification gold standard	203
10.2	Evaluation results of MLLM-based CUA GUI verification	204
10.3	Cost-related performance metrics for MLLM-based CUA GUI verification from NLR	209

A.1	Overview of published resources and artifacts	248
A.2	Overview of published approach demonstration videos	249
B.1	<i>Holm</i> -corrected p -values for comparing NLR-based GUI retrieval approaches (1)	250
B.2	<i>Holm</i> -corrected p -values for comparing NLR-based GUI retrieval approaches (2)	251
B.3	<i>Holm</i> -corrected p -values for comparing NLR-based GUI retrieval approaches (3)	251
B.4	<i>Holm</i> -corrected p -values for comparing <i>RaWi</i> against <i>Mockplus</i> on <i>#GUI-comps</i> metric	252
B.5	<i>Holm</i> -corrected p -values for comparing <i>RaWi</i> against <i>Mockplus</i> on <i>#GUI-comps-div</i> metric	252
B.6	<i>Holm</i> -corrected p -values for comparing <i>RaWi</i> against <i>Mockplus</i> on <i>#GUI-comps-neg-div</i> metric	252
B.7	Mixed-effects model for <i>#GUI-comps</i>	253
B.8	Mixed-effects model for <i>#GUI-comps-div</i>	253
B.9	Mixed-effects model for <i>#GUI-comps-neg-div</i>	254
C.1	<i>Holm</i> -corrected p -values for comparing MLLM-based GUI reranking approaches (1)	255
C.2	<i>Holm</i> -corrected p -values for comparing MLLM-based GUI reranking approaches (2)	256
C.3	<i>Holm</i> -corrected p -values for comparing MLLM-based GUI reranking approaches (3)	256
D.1	<i>Friedman test</i> results for GUI generation (1)	257
D.2	<i>Friedman test</i> results for GUI generation (2)	257
D.3	<i>Holm</i> -corrected post-hoc comparisons for GUI generation (1)	258
D.4	<i>Holm</i> -corrected post-hoc comparisons for GUI generation (2)	258
D.5	<i>Holm</i> -corrected <i>Wilcoxon signed-rank tests</i> for RAGG with varying k	258
D.6	<i>Holm</i> -corrected <i>Wilcoxon signed-rank tests</i> for SC with varying k	259
D.7	Results for <i>Wilcoxon signed-rank tests</i> on content generation	259
E.1	<i>Holm</i> -corrected p -values from the <i>McNemar tests</i> for comparing LLM prompting methods for US implementation detection in GUIs	260
E.2	<i>Holm</i> -corrected p -values from pairwise one-sided <i>Wilcoxon signed-rank tests</i> on paired per-US <i>Fi</i> scores for GUI component matching	261

LIST OF ACRONYMS

AC	Acceptance Criteria. 12, 13, 18, 198–201, 203–208, 210, 211, 221
AMT	Amazon Mechanical Turk. 11, 59–61, 75, 76
AP	Average Precision. 59, 64, 69, 70, 72, 87, 88, 90–92, 104, 106–109, 250, 255
API	Application Programming Interface. 90, 91, 93, 172, 201, 203, 210, 215
AQE	Automatic Query Expansion. 12, 48, 50, 53, 54, 59, 64, 65, 68–70
BERT	Bidirectional Encoder Representations from Transformers. 12, 36–39, 46, 48, 50, 53–56, 59, 64, 66, 68, 70–72, 77–81, 86–90, 94, 99, 119, 218, 219, 221
BM₂₅	Best Matching 25. 34, 47, 53, 54, 58, 61, 64–66, 69–72, 77, 79, 80, 218
BoW	Bag-of-Words. 33
CBOW	Continuous Bag-of-Words. 34, 35
CE	Context Engineering. 16, 31, 41
CI	Confidence Interval. 173, 174, 203
CNN	Convolutional Neural Network. 99
CoT	Chain-of-Thought. 13, 41, 42, 121, 122, 128, 130, 134, 181, 182, 184, 188, 189, 191, 192, 217, 258, 259
CSI	Creativity Support Index. 149, 153, 158, 159
CSS	Cascading Style Sheets. 9, 11, 12, 31, 57, 104, 119–125, 127, 131, 139, 169, 202, 203
CUA	Computer Use Agent. 10, 13, 196–202, 204–211, 221
DCG	Discounted Cumulative Gain. 65
DSL	Domain-Specific Language. 13, 14, 119, 136
FFNN	Feed-Forward Neural Network. 34, 36, 37, 55, 56
FN	False Negative. 187, 189, 190
FP	False Positive. 51, 123, 135, 187, 189, 190, 205
FR	Functional Requirements. 3, 7, 10, 17, 30, 76, 83, 194, 198
FS	Few-Shot. 13, 41, 101, 102, 120, 160, 181, 184, 186, 188, 189, 191, 192
GPT	Generative Pretrained Transformer. 39, 87–92, 101, 106, 113, 119, 124, 129, 130, 136, 144, 166, 172, 183, 187, 202, 210

GUI	Graphical User Interface. 4–14, 16, 25–31, 46–115, 118–166, 168–176, 180–194, 196–211, 214–221, 248–254, 257, 258
HIT	Human Intelligence Task. 60, 61
HTML	Hypertext Markup Language. 8, 9, 11, 12, 31, 57, 104, 119–125, 127, 131, 139, 169, 202, 203, 220
IAA	Inter-Annotator Agreement. 64, 186, 193, 203, 210
ICL	In-Context Learning. 16, 40, 102, 120, 183
IDF	Inverse Document Frequency. 33, 34
IR	Information Retrieval. 7–9, 11, 16, 31, 33, 48, 50, 53, 59, 61, 64, 65, 77, 78, 86–88, 104, 105, 215, 218
JS	JavaScript. 9, 11, 12, 57, 58, 104, 202, 203
JSON	JavaScript Object Notation. 31, 142, 143, 171, 172, 201
KLD	Kullback-Leibler Divergence. 12, 33, 53, 54, 59, 64, 65, 69
LLM	Large Language Model. 6–14, 31, 38–42, 54, 82, 83, 93, 95–97, 101, 102, 104, 106–110, 113, 114, 118–127, 131–144, 146–150, 152–154, 159–164, 166, 168–172, 176, 180–186, 188–194, 196–203, 211, 214–221
LTR	Learning-To-Rank. 12, 46, 48, 50, 53–56, 59, 64, 66, 68, 70–72, 77, 79–81, 86–90, 94, 99, 119, 218, 219
MAP	Mean Average Precision. 65
MCP	Model Context Protocol. 14, 197–199, 201, 211, 221
ML	Machine Learning. 11, 16, 31, 32, 93, 136
MLE	Maximum Likelihood Estimates. 53, 54
MLLM	Multi-Modal Large Language Model. 7, 9–14, 39, 40, 80–94, 101, 119, 120, 123, 124, 129, 130, 134, 135, 143, 144, 172, 183, 187, 196–202, 204–211, 215, 217, 219, 221
MLM	Masked Language Modeling. 36, 37
MRR	Mean Reciprocal Rank. 59, 65, 69, 70, 87, 88, 90, 104–109, 111, 250, 255
NASA-TLX	National Aeronautics and Space Administration – Task Load Index. 149, 153, 158, 159, 161
nBoW	Neural Bag-of-Words. 35, 53, 61, 64, 65, 69, 70
NDCG	Normalized Discounted Cumulative Gain. 65, 68–70, 72, 86–90, 251, 256
NFR	Non-Functional Requirements. 3, 8, 10, 12, 17, 81, 83, 91, 93, 171, 198
NL	Natural Language. 3, 7, 13, 17, 18, 41, 48, 49, 52, 59, 78, 83, 100, 113, 114, 119, 181, 197, 199, 211
NLG	Natural Language Generation. 40
NLP	Natural Language Processing. 6, 14, 36–40, 54, 119, 122, 144, 181, 218
NLR	Natural Language Requirements. 3–13, 17, 18, 46–50, 52–56, 58–64, 66, 68–71, 73, 74, 77–87, 89, 91–100, 104, 114, 115, 118–130, 138–143, 149, 166,

LIST OF ACRONYMS

	169–174, 176, 180–182, 193, 194, 196–200, 202–204, 207, 208, 210, 211, 214–221, 250, 251
NLU	Natural Language Understanding. 81, 93, 113, 119, 144, 181, 193, 211, 214, 215, 218, 220, 221
PD	Prompt Decomposition. 13, 41, 42, 120–122, 128–134, 171, 257, 259
PE	Prompt Engineering. 16, 31, 40–42
PLM	Pretrained Language Model. 6, 9, 11, 16, 31, 32, 35, 36, 38, 218, 221
PRF	Pseudo-Relevance Feedback. 12, 53, 54, 59, 64, 65, 69, 70
RAG	Retrieval-Augmented Generation. 13, 42, 93, 120, 139, 140, 143, 144, 149, 150, 154, 155, 160, 166, 168–171, 176, 217, 220
RAGG	Retrieval-Augmented GUI Generation. 13, 120–123, 125, 128–135, 137, 215, 217, 220, 257–259
RaWi	Rapid Wireframing. 12, 47–49, 51, 57–59, 66, 72–79, 94, 95, 99, 114, 119, 219, 248, 252–254
RD	Requirements Documentation. 3, 17, 18
RE	Requirements Engineering. 2, 3, 11, 14, 16–23, 25, 28, 29
REI	Requirements Elicitation. 2–6, 9, 16–23, 25, 47–49, 66, 67, 81, 83, 91, 94–98, 104, 114, 121, 175, 180, 197, 216, 218, 220
REST	Representational State Transfer. 58, 172
RL	Reinforcement Learning. 32, 39
RSL	Requirements Specification Language. 193, 194
RVal	Requirements Validation. 3–6, 9, 17–20, 22, 23, 25, 47, 49, 81, 83, 91, 94–96, 180, 197, 218, 220
RVer	Requirements Verification. 18, 25
S2W	Screen2Words. 100, 107, 123
SBERT	Sentence Bidirectional Encoder Representations from Transformers. 38, 56, 59, 64, 66, 70, 72, 78, 99, 100, 106, 107, 110, 111, 123
SC	Self-Critique. 13, 41, 42, 120–122, 124, 125, 128–135, 137, 220, 257–259
SD	Software Development. 2, 17, 20, 139, 163
SDim	Search Dimension. 82–85
SDLC	Software Development Life Cycle. 3, 5, 20, 24, 25
SE	Software Engineering. 11, 14, 16, 17, 20–24, 28
SER	Self-Elicitation of Requirements. 13, 14, 95–98, 101, 103–107, 109–115, 119, 216, 219, 248, 249
SFT	Supervised Finetuning. 39
SGD	Stochastic Gradient Descent. 39
SLM	Small Language Model. 215, 221
SUD	System Under Development. 7, 17, 20, 22, 25
SUS	System Usability Scale. 59, 68, 74, 75, 96, 105, 107, 111, 112, 115, 149, 153, 158, 159, 219
SVal	Software Validation. 23, 25

SVer	Software Verification. 5, 6, 23, 25, 220
TF	Term Frequency. 33, 34
TF-IDF	Term Frequency-Inverse Document Frequency. 33–35, 47, 53, 61, 64, 65, 69, 70, 77, 218
TN	True Negative. 187, 192
TP	True Positive. 187
UCD	User-Centered Design. 28, 29
UI/UX	User Interface and User Experience. 120, 121, 127, 128, 139, 140, 145, 150–153, 162–165, 172, 173, 220
UML	Unified Modeling Language. 18, 19
URL	Uniform Resource Locator. 125, 199, 200
US	User Story. 12, 13, 18, 50, 96, 139–143, 145–151, 153–166, 181–194, 203, 217, 248, 249
UX	User Experience. 160, 161, 163
XML	Extensible Markup Language. 99, 101
ZS	Zero-Shot. 10, 11, 13, 41, 82, 83, 85, 118–124, 128–131, 133–135, 137, 138, 141, 143, 154, 164, 169, 171, 172, 176, 181, 183, 184, 188–192, 199, 200, 202, 215, 216, 220, 248, 257–259

To Yaru & Lio.

CHAPTER I

INTRODUCTION

I.1 Motivation

Requirements Elicitation (REl) represents a crucial activity within Requirements Engineering (RE) for Software Development (SD) projects, which is characterized by gathering, interpreting and understanding requirements from respective requirements sources, encompassing stakeholders, documents and legacy systems in operation (Zowghi and Coulin, 2005; Pohl, 2010, 2016). This activity involves two important parties: the requirements engineer or requirements analyst¹, who closely speaks the language of the stakeholders, is responsible for becoming acquainted with the corresponding application domain and creates the requirements artifacts, such as requirements documents, among others (Pohl, 2016). Second, the stakeholders, who support the requirements analyst to become familiar with their application domain, provide the requirements to the analyst based on the requirements activities conducted together and help inspect collected requirements as well as rapidly communicate requirements changes (Pohl, 2016).

To facilitate an effective REl process, several elicitation² techniques and approaches have been developed and are applied in real-world projects, which can be specifically selected and conducted depending on multiple conditions (Goguen and Linde, 1993; Zowghi and Coulin, 2005; Davis et al., 2006; Pohl, 2010, 2016; Dar et al., 2018; Pacheco et al., 2018). For example, these techniques include *surveying*, particularly in the form of unstructured, semi-structured and structured *interviews* (Bano et al., 2019) (e.g., in the form of *laddering* (Corbridge et al., 1994)) as well as *questionnaires* (Foddy and Foddy, 1993) and *introspection* (Zowghi and Coulin, 2005), among others. Moreover, *prototyping* represents another prominent elicitation technique, which is the construction of a preliminary, incomplete and simplified or abstracted representation of the envisioned software system, application or product, enabling stakeholders to experience tangible,

¹The terms *requirements engineer*, *requirements analyst* and the abbreviated form *analyst* are used interchangeably in the remainder of this work, referring to the individual(s) responsible for conducting the RE activities within a SD project.

²The terms *requirements elicitation* and the abbreviated version *elicitation* are used interchangeably in the remainder of this work, referring to the activities of gathering, interpreting and understanding requirements.

often visual artifacts of the system to be developed, which facilitates effectively gathering detailed feedback and relevant information (Hickey and Dean, 1998; Mannio and Nikula, 2001; Zowghi and Coulin, 2005; Pacheco et al., 2018; Abad et al., 2018). Prototyping has also been shown to enable more effective capturing of Functional Requirements (FR) and Non-Functional Requirements (NFR) compared to standalone elicitation meetings, with prototyping being absent (Abad et al., 2018). Notably, most of these commonly employed elicitation techniques and approaches are heavily based on Natural Language (NL) communication between stakeholders and analysts. Furthermore, Requirements Documentation (RD), the activity of storing gathered information and requirements for the system obtained within the RE activities, is most commonly conducted by employing Natural Language Requirements (NLR) (Pohl, 2016).

While NL provides the advantage of representing a common language between the stakeholders and analysts as well as enabling the expression of arbitrary requirements due to the unrestricted nature of NL, it inherently possesses ambiguity and provides the potential to create misinterpretations between the parties (Kamsties and Peach, 2000; Kamsties, 2005; Kiyavitskaya et al., 2008; Shah and Jinwala, 2015). Apparently, this is most prevalent when conducting NL-focused elicitation techniques, such as interviewing (Ferrari et al., 2016). Research has shown that various problems can arise from unclear, ambiguous and vague requirements, such as a failure to meet the expectations of customers, quality issues of the created software system as well as wasted effort (Bjarnason et al., 2011). Moreover, poor-quality and ambiguous requirements negatively affect all downstream activities in the Software Development Life Cycle (SDLC), including software architecture and design, the system implementation represented by source code and test artifacts. Defects detected only at later development stages are many times more expensive to resolve than defects identified early during the REI phase (Boehm and Papaccio, 1988; Firesmith, 2007). This can lead to heavy rework, increased project costs as well as project delays and schedule overruns (Boehm and Papaccio, 1988; Bjarnason et al., 2011).

To mitigate issues arising from NLR specified and documented during REI, Requirements Validation (RVal) plays a critical role, encompassing activities in RE intended to facilitate the detection of erroneous requirements, including ambiguity, vagueness, incompleteness as well as contradictions (Pohl, 2010, 2016). In order to support RVal, various techniques have been proposed before and are employed in practice (Bilal et al., 2016). For example, review techniques include *commenting*, where requirements are validated by expert co-workers (Pohl, 2016), *inspections*, which represent a systematic process of error identification based on development artifacts (Gilb and Graham, 1993; Laitenberger and DeBaud, 2000), and *walk-throughs*, which are a lightweight variant of an inspection (Pohl, 2016). Moreover, *prototyping* is one of the most popular approaches for RVal and a versatile technique, enabling not only effective elicitation by facilitating the discovery of novel requirements (Abad et al., 2018), but also an effective validation³ procedure via tangible, interactive representations of the software system (Pohl, 2016).

³The terms *Requirements Validation (RVal)* and the abbreviated version *validation* are used interchangeably in the remainder of this work, referring to the conducted activities and processes of ensuring that the elicited and documented requirements correctly reflect the actual stakeholder needs of the software system.

Particularly, employing Graphical User Interface (GUI) prototypes for RVal has been shown to be one of the most valuable validation techniques (Moore and Shipman, 2000; Kamalrudin and Grundy, 2011), effectively bridging the gap between potentially ambiguous NLR and the interpretation of the requirements by the analyst. Moreover, conducting RVal by employing GUI prototypes facilitates the incorporation of stakeholders closely into the system development activities, helps in sparking fruitful discussions and clarifying as well as refining existing requirements from previous elicitation iterations (Windsor and Storrs, 1992; Rudd et al., 1996; Ravid and Berry, 2000; Beaudouin-Lafon and Mackay, 2002; Mukasa and Kaindl, 2008).

GUI prototypes can be broadly characterized by different dimensions, including the reusability or development life cycle intent such as *throw-away* or *evolutionary* prototypes (Zowghi and Coulin, 2005; Sommerville, 2011), as well as the prototype fidelity (Rudd et al., 1996; Coyette et al., 2007), which represents how closely a prototype resembles the envisioned final system, among others. While throw-away prototypes are solely employed for the specific purpose of REI and RVal and subsequently discarded, evolutionary prototypes are reused in the process and continuously improved in later stages (Zowghi and Coulin, 2005; Sommerville, 2011). Furthermore, the fidelity level of prototypes plays an important role, ranging from low-fidelity GUI prototypes, such as pen-and-paper prototypes (Snyder, 2003; Davis et al., 2007), which are highly abstracted representations with restricted functionality and typically limited interaction possibilities, to high-fidelity GUI prototypes, which offer more functionality as well as interactivity, providing a resemblance closer to the final system (Rudd et al., 1996). Especially high-fidelity GUI prototypes offer a more effective prototyping mechanism compared to their low-fidelity variants, since high-fidelity prototypes provide the foundation for higher-quality discussions between stakeholders and analysts, and more detailed feedback can be gathered while conducting validation sessions with stakeholders (Landay and Myers, 1994; Rudd et al., 1996; Coyette et al., 2007). However, the benefits of high-fidelity GUI prototypes are accompanied by the shortcomings of requiring more time and effort to create, therefore entailing higher production costs, as well as requiring more technical experience to create them (Rudd et al., 1996). Moreover, since the creation of these prototypes necessitates time, multiple meetings between stakeholders and analysts are required, often spanning several weeks or months with long breaks in between, leading to outdated requirements and project delays (Schneider, 2007; Debnath et al., 2021b).

While several approaches have been developed over the years to facilitate the creation of GUI prototypes, providing support directly based on automated processing of NLR has been largely neglected in research before. For example, traditionally, proprietary and commercially available GUI prototyping editors are widely adopted in practical prototyping environments, providing visual representations and editing functionality to improve the GUI prototyping efficiency and simplicity, e.g., *Balsamiq* (Faranello, 2012), *Mockplus* (Mockplus), *Adobe XD* (Adobe XD), *Figma* (Figma) and *Sketch* (Sketch). These prototyping tools enable users to combine fundamental GUI components and building blocks as well as provide a small amount of predefined and reusable GUI templates. However, the employment of these tools still remains largely a manual, time-consuming

process, additionally requiring prototyping expertise. Moreover, multiple GUI retrieval approaches have been proposed before, which, however, necessitate GUI sketches or screenshots as input to retrieve similar GUIs, such as *GUIFetch* (Behrang et al., 2018), *Swire* (Huang et al., 2019), *VINS* (Bunian et al., 2021) and *Screen2Vec* (Li et al., 2021). While these approaches help in rapidly obtaining similar design ideas for inspiration, they are incapable of directly processing NLR. Likewise, the GUI prototyping assistant approach *GUIComp* (Lee et al., 2020) supports prototype developers via image similarity retrieval and usability metrics computations during the creation, but cannot support the creation of GUI prototypes from NLR. Therefore, providing effective approaches for automating and facilitating the rapid mapping of NLR to GUI prototypes represents the first major challenge (**C1**) addressed in this work.

Besides RVal, Software Verification (SVer) represents another vital activity conducted in the SDLC, closely related to the validation activities (Wallace and Fujii, 1989; Rakitin, 2001; Post et al., 2009). However, while validation focuses on ensuring alignment of the explicitly specified requirements with the stakeholders’ actual needs and expectations, verification⁴ is concerned with ensuring that the final and intermediate development artifacts, such as software components, and the resulting software system, adhere to their specification, i.e. correctly implement the explicitly specified requirements (Wallace and Fujii, 1989). Specifically, since SVer can be associated with different aspects of a software system, we focus on the verification of GUI applications, namely, the fully implemented version of the earlier developed GUI prototype used for REI and RVal.

Testing represents the most common verification technique, while GUI testing is particularly time-consuming and effort-demanding (Memon, 2001, 2007). To achieve verification for GUI applications and thereby improve the GUI quality and the correctness of the implementation, a plethora of testing approaches has been proposed in research (Xie and Memon, 2007; Amalfitano et al., 2012; Mao et al., 2016; Gu et al., 2019). For example, these approaches include manual testing via human-based interaction with the GUI application (Marick, 1998), traditionally the most popular approach, writing and employing test scripts, which rely on interacting with specifically named GUI components in a predefined manner (Xie and Memon, 2007), automated random exploration of the GUI application (Amalfitano et al., 2012; Mao et al., 2016), also referred to as *fuzzing*, such as in *GUIFuzz++* (Otto et al., 2025), replay techniques (Hicinbothom and Zachary, 1993; Memon, 2003) as well as model-based techniques (Gu et al., 2019). However, these GUI testing approaches necessitate the creation of test artifacts, which in turn require technical expertise. Moreover, they are not capable of directly processing NLR to automatically verify the GUI application and require experts to create respective test representations by translating them from NLR. Hence, providing support to automate the verification of GUI applications based on NLR represents the second major challenge (**C2**).

Despite their practical relevance, both introduced challenges are inherently difficult to address. On the one hand, NLR are inherently ambiguous, incomplete and context-

⁴The terms *Software Verification (SVer)* and the abbreviated version *verification* are used interchangeably in the remainder of this work, referring to activities and processes ensuring that the implemented software system aligns with the requirements and system specification.

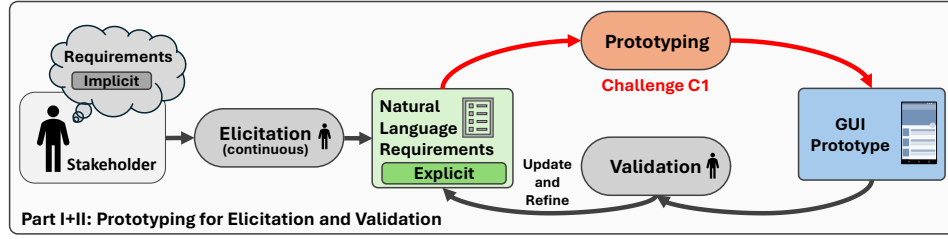


Figure 1.1: Schematic overview of the parties, artifacts, activities and their interplay for *elicitation* of requirements (REl), *validation* of requirements (RVal) and *GUI prototyping*.

dependent. Moreover, NLR are often heterogeneous (a mix of functional and non-functional aspects) and unstructured, lacking a fixed syntax and semantics, which renders parsing and transformation into representations optimized for automatic machine-based computations difficult. On the other hand, GUIs are complex, multi-dimensional objects, encompassing structural information (such as layout and component hierarchy), functional aspects (expressed through GUI components and their interactions), visual appearance and often complex, dynamic behavior (in the case of GUI applications). Therefore, the complexity and multi-dimensionality of GUIs render an optimized representation for automatic computations similarly difficult. This shows that an alignment between NLR and GUIs is inherently difficult, due to their individual complexity and the lack of a shared representation space. However, recent advances in Natural Language Processing (NLP) – through powerful foundational language processing models such as Pretrained Language Models (PLMs) and Large Language Models (LLMs) – provide large potential to tackle these challenges effectively. Therefore, this work explores the potential of such models for NLR-based automated GUI prototyping and verification.

1.2 Problem Statement

Despite the existence of numerous approaches aimed at simplifying and supporting the GUI prototyping and testing activities, a substantial research gap remains for facilitating and automating these processes directly from elicited NLR. While prior research has primarily focused on different techniques for creating GUI prototypes to support REl and RVal as well as developing GUI testing approaches for SVer, these methods are incapable of supporting NLR directly. Consequently, this work is particularly concerned with the following two guiding challenges:

- (C1) *Mapping NLR to GUI Prototypes*: the creation of high-fidelity GUI prototypes from NLR is a labor-intensive, costly endeavor, requiring expert knowledge and experience. How can we automate the GUI prototype creation based on NLR?
- (C2) *Verification of GUI Applications from NLR*: the verification of GUI applications against NLR remains predominantly a time-consuming, manual process, with existing solutions requiring expert knowledge and experience to be employed effectively. How can we automate the verification of NLR in GUI applications?

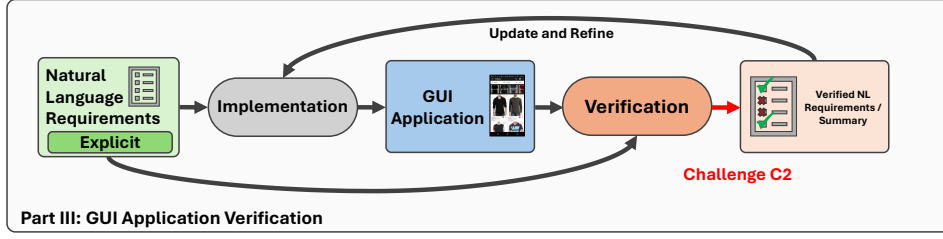


Figure 1.2: Schematic overview of the artifacts, activities and their interplay for verification of a GUI application based on NLR (verification drives iterative implementation updates).

A schematic and simplified overview of the *elicitation*, *validation* and *GUI prototyping* activities, together with their respective inputs and outputs, is presented in Figure 1.1. From the initial elicitation (e.g., in the form of interviews), explicit NLR are gathered and documented. Subsequently, GUI prototypes are developed and, together with the stakeholders, discussed in validation sessions, leading to updates and refinements (i.e. addition, modification and removal) of the explicit NLR. Furthermore, verification is performed by employing the NLR and an implemented GUI application to assess conformance with the specified requirements, as shown in Figure 1.2. In the presence of deviations between them, the GUI implementation is accordingly updated and refined.

To tackle the two major challenges introduced, we develop several approaches. The approach for automated mapping from NLR to GUI prototypes (**C1**) is divided into two parts: (I) first, we approach the mapping as a *retrieval* problem, i.e. given the NLR⁵ as input, we exploit a large-scale GUI repository to automatically compute a ranking over GUI prototypes according to their level of correspondence with the NLR (*Part I* of the thesis). Second, (II) we leverage an LLM to frame the mapping as a *translation* problem, with NLR as source and GUI code as target language (*Part II* of the thesis). Moreover, (III) we approach the automated verification of GUI applications from NLR by leveraging an LLM as well as proposing a Multi-Modal Large Language Model (MLLM)-based agent (*Part III* of the thesis). Each of these approaches is accompanied by its own sub-challenges, which we describe next. An overview of the addressed challenges and their relatedness to the posed main challenges is depicted in Figure 1.3. First, we state the sub-challenges regarding NLR-based GUI retrieval approaches (*Part I*):

C1-1 *NLR-based GUI Retrieval Gap*: traditional Information Retrieval (IR) techniques are mainly focused on retrieving text artifacts and cannot be readily employed due to the semantic gap between unstructured NLR and complex GUI prototypes. How can we adapt and optimize text-based retrieval methods and techniques to enable more effective NLR-based GUI retrieval?

C1-2 *Semantic Representation Gap in GUI Retrieval and Reranking*: while optimized text-based IR methods improve the ranking of GUI prototypes based on NLR

⁵When we refer to the terms NLR or requirements from now on, we specifically refer to functional requirements (FR) in natural language (NL) of the System Under Development (SUD) of an interactive, user-facing software system for the remainder of this work, if not explicitly stated otherwise.

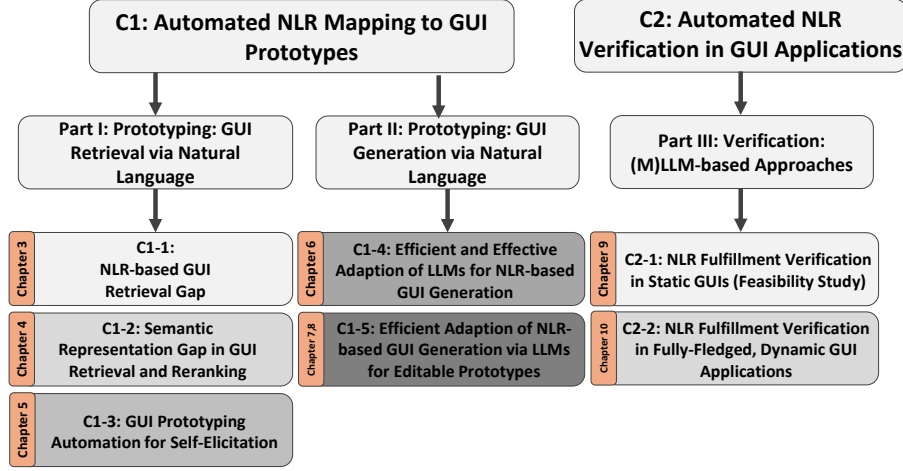


Figure 1.3: Overview and relatedness of the seven individual sub-challenges tackled in this work with regard to the two major challenges posed, where challenge **C1** is divided into NLR-based GUI retrieval and generation, and **C2** into NLR-based GUI app. verification.

over standard IR methods, the multi-dimensionality of GUI prototypes is mostly neglected. How can the semantic representation gap between NLR (including NFR) and multi-dimensional GUI prototypes be reduced to enable more effective GUI ranking?

C1-3 *GUI Prototyping Automation for Self-Elicitation*: while retrieval-based approaches enable rapid mapping from NLR to GUI prototypes, these techniques assume sufficient prototyping process knowledge to be employed effectively and provide no guidance for the prototyping and elicitation process. How can we provide automatic assistance for self-elicitation of requirements with GUI prototyping?

Second, we list the challenges regarding NLR-based GUI prototype generation (*Part II*):

C1-4 *Efficient and Effective Adaptation of LLMs for NLR-based GUI Generation*: while pretrained LLMs offer general capabilities to generate GUI prototypes from NLR, optimizing them for more effective GUI generation without inefficient and resource-intensive fine-tuning of LLMs remains open. How can we efficiently optimize LLMs for more effective GUI prototype generation?

C1-5 *Efficient Adaptation of NLR-based GUI Generation via LLMs for Editable Prototypes*: while pretrained LLMs have profound capabilities of generating GUIs in widely available code formats, for example, Hypertext Markup Language (HTML), these representations are not well suited for customization. How can LLMs be efficiently adapted to generate editable GUI prototype representations from NLR?

Third, we list the challenges regarding NLR-based verification of GUI artifacts (*Part III*):

- C1-1** *NLR Fulfillment Verification in Static GUIs*: existing verification techniques are predominantly rule-based and require technical expertise, limiting their ability to handle semantically complex NLR. How can NLR-based verification be automated for simplified, static GUI representations as a foundation for more complex GUI verification tasks?
- C1-2** *NLR Fulfillment Verification in Fully-Fledged, Dynamic GUI Applications*: similarly to the previous challenge, existing approaches are not capable of supporting highly dynamic, fully-fledged GUI application verification based on complex NLR. How can NLR-based verification be automated for highly dynamic, fully-fledged GUI applications?

In summary, we address seven individual challenges: optimizing NLR-based GUI retrieval approaches from text representations, embracing the multi-dimensionality of GUI prototypes and efficiently adapting LLMs for GUI generation regarding the first challenge **C1**. Moreover, we address the NLR-based verification of simplified GUI representations and highly dynamic, fully-fledged GUI applications regarding the major challenge **C2**.

1.3 Solution Overview

To address the challenges delineated in the previous section, we divided them into three different parts, as depicted in Figure 1.3. For each of these parts, we briefly outline the general solution notion subsequently. In addition, we provide a visual overview of these ideas in Figure 1.4, covering NLR-based GUI retrieval (*Part I*), NLR-based GUI generation (*Part II*) and LLM as well as MLLM-agent-based verification of GUI applications (*Part III*). Particularly, for both challenges **C1** and **C2**, we focus on GUIs from either *mobile* (specifically the *Android* platform) or *web* (i.e. implemented based on *HTML*, *Cascading Style Sheets (CSS)* and *JavaScript (JS)*) applications, since these software platforms are most prevalent and ubiquitous.

First, (*I*) we approach NLR-to-GUI prototype mapping as a *retrieval* problem. To this end, a large-scale GUI repository encompassing over 72,000 individual GUIs is exploited, enabling matching to a vast number of different NLR. Given the NLR, we compute a ranking score for each GUI encompassed in the repository and provide the best matches to users. Therefore, while restricted to the existing GUI repository, we enable the fast mapping from NLR to GUI prototypes for a broad spectrum of real-world GUIs. Specifically, we develop a text extraction approach from these GUIs, investigate the performance of different text-based IR methods, adapt existing methods and finetune a PLM for improved GUI ranking performance. Moreover, we investigate MLLM-based GUI reranking, reducing the semantic representation gap between NLR and multi-dimensional GUI prototypes. Finally, we provide a dialogue-based approach (with retrieval) for simplifying and guiding the GUI prototyping process for REI and RVal.

Second, (*II*) we tackle the NLR-to-GUI prototype mapping as a *translation* problem by employing a pretrained LLM, which encompasses vast amounts of general, programming and prototyping knowledge. In particular, we investigate several approaches to

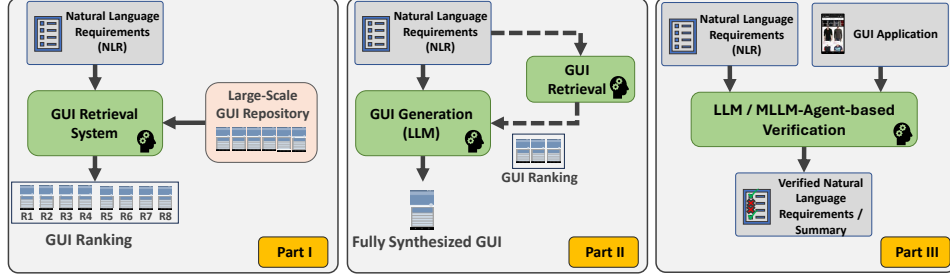


Figure 1.4: General solution overview for the two challenges divided into three parts in the thesis, including NLR-based GUI retrieval (*Part I*), NLR-based GUI generation (*Part II*) and LLM as well as MLLM-agent-based verification of GUI applications (*Part III*).

efficiently optimize a pretrained LLM to more effectively generate GUI prototypes. To this end, we follow a Zero-Shot (ZS) prompting approach and leverage a large-scale GUI repository combined with a GUI retrieval approach (developed in *Part I*) to efficiently extend the prototyping knowledge of the LLM. Additionally, we investigate several other prompting approaches to more effectively access and employ the GUI prototyping knowledge already stored in the parameters of the LLM. Furthermore, in a similar manner, we efficiently adapt an LLM via retrieval of proprietary GUI component libraries, enabling the LLM-based generation of fully editable GUI prototypes from NLR, integrated into a popular GUI prototyping editor with fully-fledged editing functionality.

Third, (*III*) we approach the NLR-based automatic verification of GUI applications in two stages. To begin with, we investigate an easier variant of the problem as a feasibility study by examining simplified, static GUI representations. Specifically, we focus on fixed GUI hierarchy data, encompassing the GUI components and their relationships, while excluding dynamic GUI behavior. To this end, we investigate different prompting techniques, optimize an LLM to compute binary predictions of NLR fulfillment and match NLR to corresponding GUI components given a static GUI representation. In the second step, we examine NLR-based verification of fully-fledged, highly dynamic web-based GUI applications by adapting an MLLM-based Computer Use Agent (CUA). This agent is capable of interacting with complex GUIs, predicting the most likely next interaction steps, executing them on the GUI application and obtaining the updated GUI states to repeat the same process again. Hence, complex NLR (both FR and NFR) can be verified automatically in dynamic, web-based GUI applications with high effectiveness.

1.4 Contributions

After motivating our research, presenting both challenges with their sub-challenges regarding NLR-based GUI prototyping and NLR-based GUI application verification, and briefly delineating our solution approaches, including GUI retrieval, GUI generation and MLLM-based verification, we build on top of these insights and outline our contribution

to the field of RE and Software Engineering (SE). We evaluated our newly proposed GUI prototyping and verification approaches on several novel datasets and conducted multiple insightful analyses, showing the effectiveness of automating GUI prototyping and verification using NLR. Subsequently, we present our contribution divided into created and published datasets (for fostering future research) and novel approaches proposed.

Datasets. During our research, we created several datasets spanning both challenges of NLR-based GUI prototyping (**C1**) and verification (**C2**) for enabling effective training as well as comprehensive evaluation of approaches focusing on PLMs, LLMs and MLLMs.

1. *NLR-based GUI Retrieval and Reranking Gold Standard:* to evaluate the GUI retrieval and reranking methods (see Chapters 3 and 4), we created a novel crowd-sourced benchmark and training dataset, consisting of NLR combined with GUI prototypes from the *Rico* GUI dataset (Deka et al., 2017) – obtained from IR-based pooling – and corresponding three-level relevance annotations. This dataset consists of 450 NLR, each combined with 20 GUIs, and we gathered over 40,500 relevance annotations using the crowd-sourcing platform Amazon Mechanical Turk (AMT). This represents the first dataset for the problem and can be employed to both evaluate GUI ranking models on the benchmark and train novel models on the training dataset. Particularly, we employed these datasets to optimize and improve text-based GUI retrieval (**C1-1**) as well as to reduce the semantic representation gap between NLR and multi-dimensional GUI prototypes (**C1-2**).
2. *Rico Annotation and Embeddings Dataset:* to support the multi-dimensionality of GUI prototypes during NLR-based retrieval (**C1-2**), we annotated the *Rico* GUI dataset (Deka et al., 2017) by employing an MLLM to obtain textual descriptions for each prototype regarding the aspects of *functionality*, *design*, *domain*, *GUI components* and *displayed text*. Moreover, we provide 3,072-dimensional *OpenAI* (OpenAI, 2023) embeddings for each annotated dimension and GUI prototype. Both datasets enable fostering future research by, for example, fueling other related downstream Machine Learning (ML) tasks such as GUI classification, advanced GUI retrieval techniques as well as semantic understanding of GUI prototypes.
3. *GUI Description and GUI Prototype Dataset:* to assess LLM-based GUI generation approaches (**C1-4**), we constructed a novel, high-quality GUI description dataset with more complex, information-dense and longer descriptions compared to our previously introduced benchmark and training dataset for NLR-based GUI retrieval, carefully gathered in a controlled lab-based setting. Moreover, we created a GUI prototype dataset (web-based prototypes consisting of *HTML*, *CSS* and *JS*) corresponding to the gathered NLR, which has been generated by different ZS prompting methods investigated in Chapter 6. This dataset provides potential for various future ML approaches (requiring high-quality GUI requirements) as well as analyses, including error analyses of LLMs in GUI generation, among others.

4. *Interlinked User Story (US) and GUI Components Dataset*: to conduct our feasibility study for verification of NLR in simplified, static GUI representations, we created a novel dataset, encompassing NLR in the popular form of User Stories (USs), widely adopted in industry and practice to document requirements, again gathered in a controlled, lab-based environment, ensuring high quality. Each US was collected with respect to a specific GUI from the *Rico* GUI dataset (Deka et al., 2017), where a GUI consists of multiple US annotations. In addition, this dataset provides a *one-to-many* mapping from US to individual GUI components in the *Rico* GUIs, enabling requirements verification research (**C2-1**). Furthermore, this dataset fosters requirements traceability research efforts as well as other approaches, where detailed mappings between USs and GUI components are of interest.
5. *NLR-based Verification of GUI Applications Gold Standard*: aiming at assessing NLR-based verification of highly dynamic, complex GUI applications (**C2-2**), we finally constructed a dataset consisting of USs, each combined with multiple, detailed Acceptance Criteria (AC). Overall, this dataset encompasses five GUI applications (web-based applications consisting of *HTML*, *CSS* and *JS*) from different domains (*Park-and-Pay*, *Budget Tracker*, *Recipe Generator*, *Fitness Quests*, *Cleaning Booking*), where each application is mapped to a collection of USs with their AC. In addition, the USs are annotated with a three-class label, indicating whether a US and its AC are *met*, *unmet* or *partially met*. This is the first dataset available in research offering this combination of requirements and application mapping, potentially fueling further research for agent-based GUI verification.

Approaches. In order to address the introduced challenges, we explore and propose multiple novel data-driven approaches and techniques, largely based on pretrained LLMs.

1. First, we develop a new text extraction approach based on GUI hierarchy data, fueling text-based GUI retrieval from NLR. In this context, we adapt a Pseudo-Relevance Feedback (PRF) method based on Kullback-Leibler Divergence (KLD) scoring for optimized Automatic Query Expansion (AQE) regarding GUIs. Moreover, we are the first to optimize a Bidirectional Encoder Representations from Transformers (BERT)-Learning-To-Rank (LTR) model to enhance the effectiveness of text-based GUI ranking (**C1-1**). Finally, the proposed GUI retrieval techniques are integrated into a novel, fully data-driven GUI prototyping editor (*RaWi*).
2. To reduce the semantic representation gap between NLR (including NFR) and multi-dimensional GUI prototypes (**C1-2**), we propose a novel GUI retrieval and reranking approach, supporting multi-dimensional, negative constraints (exclusion criteria). Particularly, NLR are deconstructed by a novel LLM-based query decomposition technique. Furthermore, we propose an MLLM-based GUI reranking approach, enabling efficient (text-based) or more effective (image-based) GUI reranking, achieving state-of-the-art performance for NLR-based GUI ranking. These techniques are integrated into a novel GUI retrieval tool (*GUI-ReRank*), which additionally enables the simple adaptation to arbitrary GUI (image) datasets.

3. To support users in the GUI prototyping and elicitation process (**C1-3**), we propose a novel approach based on the notion of Self-Elicitation of Requirements (SER) (Rietz and Maedche, 2019), enabling effective GUI prototyping via a dialogue-based NL interface. The approach integrates previously developed NLR-based GUI retrieval, a novel GUI feature retrieval and an LLM-based GUI feature recommendation approach, proactively stimulating stakeholder-driven elicitation of requirements. Furthermore, we propose a novel retrieval method which incorporates a GUI feature scoring mechanism, ensuring that GUI prototypes with more relevant features are scored higher in the ranking. The entire approach is implemented and integrated into a novel GUI prototyping tool (*SERGUI*).
4. For tackling the efficient adaptation of LLMs to effectively generate GUI prototypes (**C1-4**), we propose multiple ZS prompting techniques. In particular, we adapt Prompt Decomposition (PD) (Khot et al., 2022), which dissects the generation procedure into predefined subtasks with intermediate reasoning outputs. Moreover, we introduce Retrieval-Augmented GUI Generation (RAGG), a novel technique which combines the advantages of retrieval techniques and language generation approaches, by injecting potentially relevant GUI prototypes into the LLM context, thereby extending the prototyping knowledge of the LLM, which is particularly valuable for Out-of-Domain (OoD) problems. Furthermore, we adapt Self-Critique (SC) (Saunders et al., 2022), which iteratively generates new versions of the prototype and creates critiques, continuously improving the prototype.
5. We are the first to efficiently adapt LLMs to generate fully editable GUI prototypes (**C1-5**), integrating them into fully-fledged GUI prototyping editors. To this end, we adapt an LLM to generate a proprietary Domain-Specific Language (DSL) by employing a two-stage Retrieval-Augmented Generation (RAG) approach on custom GUI component libraries. Moreover, by creating mappings between NLR (in particular USs) and their implementations, expensive regenerations of entire prototypes can be avoided. The proposed approach is integrated into a novel *Figma* (Figma) plugin, enabling practitioners to easily apply these new techniques.
6. To enable verification of NLR (specifically USs) in GUI applications (**C2-1**), we initially propose an LLM-based approach, where USs are automatically matched against GUI components of the static GUI, thereby verifying the requirements fulfillment. We investigate different prompting paradigms, including ZS (Kojima et al., 2022) and Few-Shot (FS) prompting (Brown et al., 2020) as well as the Chain-of-Thought (CoT) technique (Kojima et al., 2022; Wei et al., 2022).
7. Last, we propose a novel MLLM-agent-based approach to enable NLR-based verification of highly dynamic, fully-fledged GUI applications (**C2-2**), by leveraging a CUA (OpenAI, 2025b), which is adapted to the GUI verification problem. To this end, the agent iteratively receives GUI states in the form of images and predicts next actions to best verify whether an AC of a requirement is fulfilled. These

actions are executed in the GUI environment and updated states are provided to the agent in turn. In addition, we provide a Model Context Protocol (MCP) server implementation for the verification agent, thereby being the first to enable completely autonomous implementation-verification loops for GUI applications with LLM-based programming agents. Finally, these techniques are integrated into an intuitive tool (*GUISpector*), facilitating rapid adoption by practitioners.

1.5 Outline

First, we provide a brief overview of the theoretical background (Chapter 2), outlining the foundations from the domains of RE, SE, GUI prototyping and NLP. Subsequently, Part I introduces text-based GUI retrieval (Chapter 3), MLLM-based GUI reranking (Chapter 4) and the SER approach (Chapter 5). Part II is composed of efficiently adapting LLMs to GUI generation (Chapter 6) and generation of editable GUIs with a proprietary DSL (Chapters 7 and 8). Part III introduces LLM and MLLM-agent-based approaches for GUI verification (Chapters 9 and 10). Each of these chapters provides a motivation, approach details, evaluation methodology, results, limitations and related work. Finally, Part IV provides an overall discussion and then concludes the thesis (Chapters 11 and 12).

CHAPTER 2

THEORETICAL BACKGROUND

In this chapter, we present the foundational topics and concepts underlying this thesis: (1) foundations of Requirements Engineering (RE) and Software Engineering (SE) (Section 2.1), (2) the history, process and important concepts of GUI prototyping (Section 2.2) and (3) relevant computational methods and techniques (Section 2.3), spanning Machine Learning (ML), Information Retrieval (IR), Pretrained Language Models (PLMs), In-Context Learning (ICL), Prompt Engineering (PE) and then Context Engineering (CE).

2.1 Requirements and Software Engineering (RE/SE)

Requirements Engineering (RE) is an active research area and simultaneously an important industry practice ensuring a systematic approach for eliciting, specifying and documenting, validating and verifying as well as managing requirements for the development of a software system (Pohl, 1997, 2010, 2016), and “[...] the part of development in which people attempt to discover what is desired” (Gause and Weinberg, 1989). The significance of RE is highlighted by earlier research indicating that approximately 60 % of software system failures originate in the early stages of projects, due to issues associated with the requirements (Boehm, 1984a), primarily as a result of unclear, ambiguous, incomplete or erroneous requirements, among others (Walia and Carver, 2009; de Bruijn and Dekkers, 2010). Specifically, a failure is a deviation of the software system behavior with respect to the expected requirements (Walia and Carver, 2009), or “[t]he inability of a system or component to perform its required functions within specified performance requirements” (IEEE, 1990) (IEEE 610.12-1990). The cost escalation of fixing errors associated with requirements in later development stages is substantial: the cost increase of detecting them during the design phase is estimated to be 3–8-fold higher, during the development phase 7–16 times higher, during the test and integration phase 21–78 times higher and during the operational phase 29–1500 times higher, in comparison to identifying them in the early REI phase (Stecklein et al., 2004).

Particularly, a requirement is defined as: “(1) A condition or capability needed by a user to solve a problem or achieve an objective. (2) A condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard,

2. THEORETICAL BACKGROUND

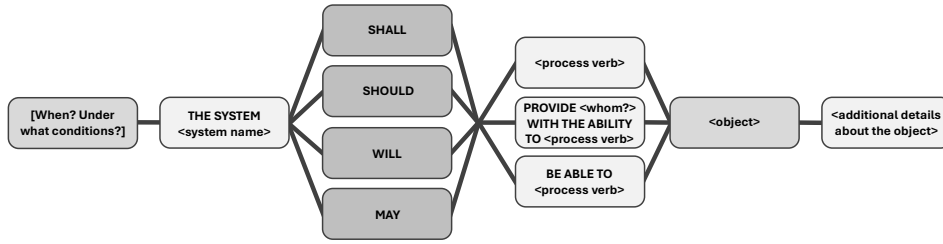


Figure 2.1: Requirements template for structured documentation of Natural Language Requirements (NLR) according to Pohl (2016), including temporal or logical constraints, the system name, a modal verb to distinguish between the different kinds of requirements, the process verb structure, the object of the requirement and additional details for the object, which can vary depending on the respective object (adapted from Pohl (2016)).

specification, or other formally imposed documents. (3) A documented representation of a condition or capability as in (1) or (2)” (iee, 1990) (IEEE 610.12-1990), as part of the standard glossary of SE terminology. Moreover, requirements are differentiated by their type, namely *FR*, which represent the actual functionality of the System Under Development (SUD), with regard to a result of a system behavior provided by a function of the software system (Pohl, 2010, 2016; Glinz, 2011). In addition, *NFR*, also called *quality requirements*, refer to quality-related aspects of the SUD, such as performance, scalability and availability, among others (Pohl, 2010, 2016; Glinz, 2011). Further, *constraints* are requirements which restrict the solution space for FR and NFR (Pohl, 2010, 2016; Glinz, 2011). The overall *functionality* of a system is defined by the collection of the respective FR (Glinz, 2011). The entirety of requirements for a system is directly or indirectly influenced by the *stakeholders*, such as the users and operators of the system, customers and testers, among others (Pohl, 2010, 2016). Identifying relevant stakeholders is a crucial aspect of RE, since this directly affects the quality of the requirements (Glinz and Wieringa, 2007).

RE encompasses a set of core activities, including REl, RD and specification, requirements negotiation, RVal, requirements verification and management (Kotonya and Sommerville, 1998; Pohl, 2010, 2016). First, the main goal of REl is to extract and gather all requirements from the stakeholders and related sources (e.g., legacy systems, existing documents), as well as refine requirements (Pohl, 2010, 2016). To this end, stakeholders and *system boundaries* need to be identified (Glinz and Wieringa, 2007), separating the SUD from the *system context*, which is the part of the environment that is relevant to the system (Nuseibeh and Easterbrook, 2000). Second, elicited requirements are documented by employing different techniques, most commonly using NL, but also via concept models (Pohl, 1996, 2016). This requirements specification represents the input to subsequent SD phases, which should be traceable, consistent and comprise the perspectives of all stakeholders (Pohl, 1996). In order to reduce limitations inherent to NL while documenting requirements, such as incompletely specified conditions or process verbs, *requirements templates* are often employed, which act as syntactical blueprints for the requirements,

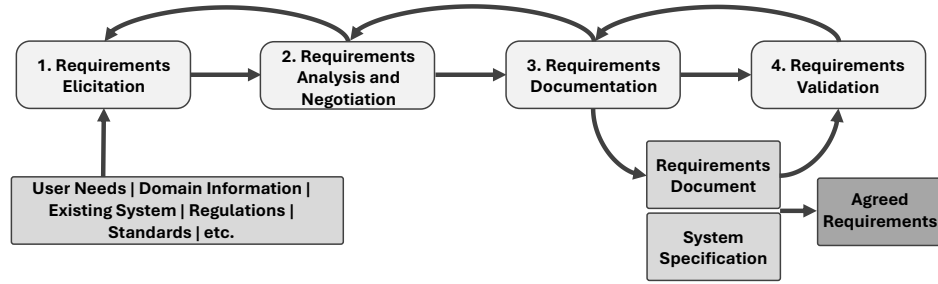


Figure 2.2: RE process model with different requirements artifacts, activities and their relationships: (1) REl, (2) requirements analysis and negotiation, (3) RD and (4) RVal, depicted as a linear, incremental process (adapted from Kotonya and Sommerville (1998)).

as depicted in Figure 2.1 (Pohl, 2016). Another widely applied and closely related technique of documenting NLR is the employment of USs, which are comprehensible, small functional descriptions that facilitate participatory design to tightly integrate users of the system into the elicitation and are primarily combined with agile methods (Cohn, 2004; Lucassen et al., 2016). Most often, the specification of USs is conducted by employing the *Connextra* template, namely, “As a <role>, I want to <action>, [so that <benefit>]” (Lucassen et al., 2016; Dalpiaz and Brinkkemper, 2018). When USs are employed for specification, they are typically combined with Acceptance Criteria (AC), which represent conditions for satisfaction, and are employed often in the form of the *Given-When-Then* template, composed of three main building blocks, namely, “Given that <aCondition> when <anAction> then <aDesiredConsequence>” (Ferreira et al., 2022). In contrast to the NL-based requirements specification, conceptual models are employed, for example, by using the Unified Modeling Language (UML) (Specification, 2007; Pohl, 2016), providing technically more precise representations. Third, negotiation of requirements should ensure agreement among stakeholders regarding the gathered requirements (Pohl, 1996). Fourth, RVal is concerned with ensuring that the correct requirements have been elicited and specified, i.e. ensuring that a software system is developed which is aligned with the actual needs and intentions of the stakeholders, while Requirements Verification (RVer) refers to checking the requirements regarding several aspects such as *consistency*, ensuring the absence of internal contradictions in the requirements, and *correctness*, referring to the absence of apparent mistakes in the requirements, among others (Pohl, 1996, 2010, 2016). Finally, requirements management is perceived as a continuous activity conducted orthogonally to other RE activities, ensuring the structuring of elicited and specified requirements, preparing requirements for later use and maintaining consistency as well as ensuring the implementation of the requirements (Pohl, 2016).

Prior research described different RE process models, encompassing the various RE activities and depicting their relationships. For example, Kotonya and Sommerville (1998) describe a linear, incremental process starting from elicitation, analysis and negotiation and progressing to documentation and validation, while activities allow interactions with preceding activities, as illustrated in Figure 2.2. Elicitation is fed by user needs, domain

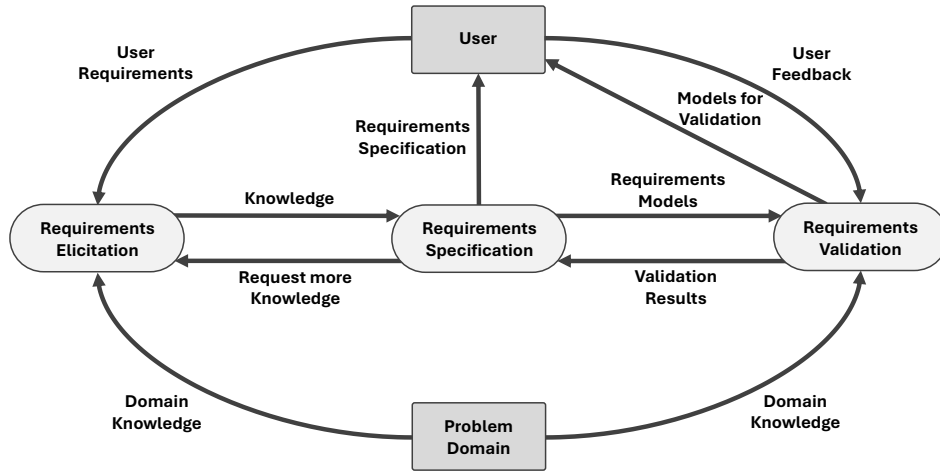


Figure 2.3: RE process model illustrating the user and domain, information flow and RE activities of REl, requirements specification and RVal, depicted as a non-linear, highly connected, iterative process model (adapted from Loucopoulos and Karakostas (1995)).

information, existing systems, regulations and standards, among others (Kotonya and Sommerville, 1998). Moreover, requirements documentation leads to a manifested requirements specification document, which is inspected during validation. In addition, the system specification is based on the created requirements specification. After successful requirements validation, the process model depicts an agreed requirements specification as the RE process model output (Kotonya and Sommerville, 1998).

In contrast, Loucopoulos and Karakostas (1995) describe the RE relationships between activities as a non-linear, iterative process, showing the RE activities of REl, requirements specification and RVal, as depicted in Figure 2.3. Knowledge from the elicitation phase fuels the specification and, if necessary, additional elicitation is conducted to improve the specification (Loucopoulos and Karakostas, 1995). Requirements models are derived (e.g., conceptual models or prototypes), which fuel the validation in combination with the specification, by incorporating the domain knowledge and the user to obtain feedback, which is fed back to the specification and, if necessary, updates to the specification are conducted. Moreover, user requirements directly fuel the elicitation, additionally supported by incorporating domain knowledge (Loucopoulos and Karakostas, 1995). As can be observed from the possible interactions and relationships between the RE activities, their process model emphasizes a highly non-linear, iterative procedure.

While we presented two different RE process models, other similar processes are discussed in the research literature. For example, Escalona and Koch (2003) describe the RE process using an UML activity diagram, indicating iterative interrelations between elicitation, specification and validation activities. Furthermore, Sawyer et al. (1997) present the RE process as a spiral, emphasizing the iterative nature of RE, and integrate elicitation, analysis and validation, as well as requirements negotiation into the process. Both the

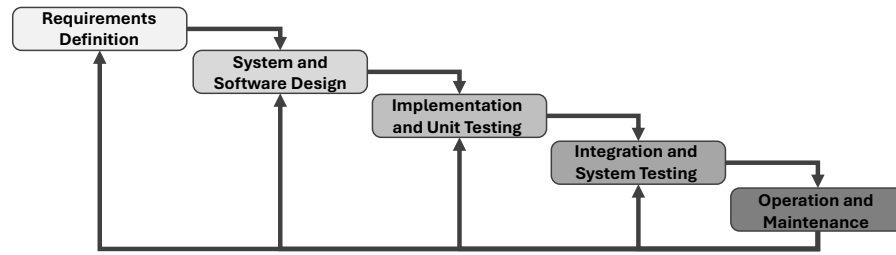


Figure 2.4: Waterfall Model for Software Development (SD) including requirements definition, system and software design, implementation and unit testing, integration and system testing as well as operation and maintenance (adapted from Sommerville (2011)).

cost and the quality of the requirements information increase with successive iterations (Sawyer et al., 1997). On the contrary, Macaulay (2012) describe a simple, linear RE process including conceptualization, problem analysis, feasibility, analysis and modeling as well as requirements documentation. Other research also indicates that RE activities are highly intertwined, yet argues that the RE process should rather be considered as a collection of different process chunks, instead of a monolithic process (Houdek and Pohl, 2000). Finally, while different RE process models are described in research with varying characteristics, most share commonalities regarding the encompassed RE activities and stress the heavy relatedness of activities as well as the iterative nature of RE.

Before diving deeper into the RE activities most important to this thesis, namely REI and RVal, we first provide a brief overview of the discipline of SE and its connection to RE. SE represents a systematic approach to the creation of software systems, encompassing a particular set of activities and techniques (Sommerville, 2011), or SE is described as “[...] [t]he application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software [...]”, as defined by the standard SE glossary (iee, 1990) (IEEE 610.12-1990). Specifically, the SE process is characterized by activities of *software specification*, ensuring that requirements are gathered and defined together with the stakeholders, *SD*, which is concerned with designing and programming the software system, *software validation*, ensuring that the SUD is aligned with the actual stakeholder needs and *software evolution*, ensuring a continuous adaptation and maintenance of the software to reflect changing requirements (Sommerville, 2011). Over the years, different SE processes have been described in research, commonly referred to as SDLC, describing the usual phases from early planning to late maintenance of software systems, where connections between these activities depend on the concrete development approach (iee, 1990) (IEEE 610.12-1990). For example, the *waterfall model* describes the SE process as a sequential, incremental one, where each phase depends on the completion of the preceding phase, as shown in Figure 2.4 (Sommerville, 2011). This particular variant of the waterfall model described by Sommerville (2011) allows for feedback loops to previous stages and consists of five phases, including requirements definition, system and software design, implementation and unit testing, integration and system testing as well as operation and

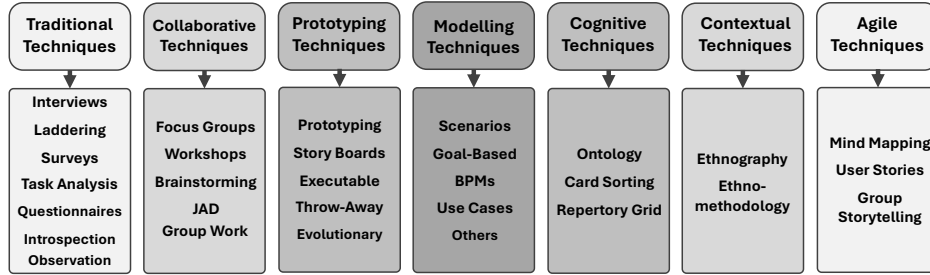


Figure 2.5: Categorization of REI techniques covering traditional, collaborative, prototyping, modeling, cognitive, contextual and agile (adapted from Pacheco et al. (2018)).

maintenance, while other variants such as the original one from Royce (1987) comprise seven phases and are strictly linear. As apparent from the described SE process, requirements definition represents the first phase and illustrates the interrelation between RE and SE. The previously described discipline of RE is an important part of SE, ensuring an understanding of the needs of the stakeholder and providing appropriate requirements specifications, while SE is concerned with building the software, encompassing the actual design and architecture, implementation, testing and integration activities.

On the contrary, agile SE processes emerged, emphasizing a more flexible, iterative approach (Fowler et al., 2001; Abad et al., 2018). Within agile SE processes, the individual iterations are considered self-contained, spanning all previously described SE activities from requirements definition and analysis to implementation and testing, resulting in a release, which represents an evolving subset of the final product (Boehm, 2007). Each iteration is usually conducted in a short time, enabling the quick incorporation of stakeholder feedback based on the previous and current product evolution and therefore rapid adaptation to changing requirements (Boehm, 2007). While different agile approaches exist, for example, *extreme programming* (Beck, 2000) and *adaptive software development* (Highsmith, 2013), the most popular approach is *Scrum* (Schwaber and Beedle, 2001).

2.1.1 Requirements Elicitation (REI)

REI represents an important activity comprised in the RE process, focusing on understanding the actual requirements of different stakeholders, including uncovering, extracting and refining requirements and providing respective requirements specifications as a means of communication to the developers (Zowghi and Coulin, 2005; Pohl, 2010, 2016). For conducting effective REI, initially, the application domain needs to be explored, investigated and understood, providing a foundation for subsequent REI activities (Zowghi and Coulin, 2005). Next, identifying and analyzing all relevant requirements sources is instrumental, which has a large effect on the quality of the elicited requirements (Glinz and Wieringa, 2007), and includes various stakeholders, existing documents, such as standards or domain- and organization-specific ones, as well as systems in operation, such as legacy systems (Alexander and Stevens, 2002; Zowghi and Coulin, 2005; Pohl, 2010,

2016). Afterwards, the appropriate RE approaches, techniques and tools are selected, which may vary depending on the problem context, domain, constraints and project characteristics (Zowghi and Coulin, 2005; Pohl, 2016), since no universal methodology or technique for effective elicitation exists (Hickey and Davis, 2003; Pohl, 2010, 2016). Most often, a combination of different elicitation techniques is employed, depending, for example, on the phase of the development, the types of requirements to be elicited, the experience of the analyst with a specific technique as well as budget and time constraints (Zowghi and Coulin, 2005; Pohl, 2010, 2016). The elicitation activities are typically conducted in an iterative manner, where each iteration provides an increasing level of detail and quality of the requirements specification (Zowghi and Coulin, 2005), embracing the natural evolution and co-creation of requirements during elicitation (Ferrari et al., 2022). As discussed earlier, the RE and SE activities are highly intertwined and iterative.

To effectively elicit requirements, a plethora of different approaches has been proposed in research before and a comprehensive categorization of these techniques is shown in Figure 2.5 (Pacheco et al., 2018). This includes traditional techniques, for example, *interviews* (Agarwal and Tanniru, 1990) conducted in an unstructured, structured or semi-structured approach, with varying levels of guidance based on predefined questions or templates (Zowghi and Coulin, 2005). A particular interview technique is represented by *laddering*, asking a series of probing questions and arranging the gathered knowledge in a hierarchical manner (Hinkle, 1965; Corbridge et al., 1994). Moreover, *task analysis* represents a top-down technique decomposing tasks into subtasks, to uncover all events and actions of processes the users and systems conduct (Carlshamre and Karlsson, 1996). In addition, *Joint Application Development (JAD)* represents a structured approach incorporating all stakeholders to jointly discuss problems and solutions, facilitating rapid decision making and resolution of issues (Wood and Silver, 1995). While *prototyping* is another effective elicitation technique leveraging abstracted versions of the SUD (Somerville, 2011), on which we focus in this work, we provide an introduction in Section 2.2. For a summary of other techniques, such as *scenarios*, *repertory grids* and *ethnographies*, we refer the interested reader to Zowghi and Coulin (2005) and Pacheco et al. (2018).

2.1.2 Requirements Validation (RVal)

RVal is the RE activity concerned with ensuring that the elicited and specified requirements correctly reflect the actual needs of the involved stakeholders (Boehm, 1984b; Pohl, 1996, 2010, 2016). In contrast, requirements verification encompasses procedures and techniques to ensure that the requirements specification is correct by detecting internal errors, such as contradictions or inconsistencies (Sakthivel, 1991). To increase validation quality, RVal incorporates multiple principles: ensuring that the adequate stakeholders are involved, decoupling the detection and correction of deviations, conducting validation from multiple perspectives, employing different requirements document types with varying advantages and weaknesses (e.g., text-based and graphical models), constructing development artifacts (e.g., test cases) and repeating validation, embracing the continuous evolution of the knowledge, understanding and the needs of stakeholders (Pohl, 2016).

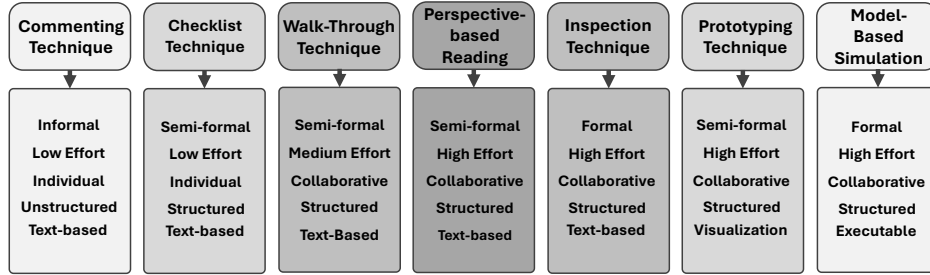


Figure 2.6: Overview of RVal techniques, including commenting, checklist, walk-through, perspective-based reading, inspection, prototyping and model-based simulation, each with characteristics of formality, effort, collaboration, structuredness and artifact type.

To ensure effective RVal, numerous techniques have been proposed in research before (Pohl, 1996). An overview of well-known and widely studied validation techniques, each with their main characteristics, is shown in Figure 2.6. For example, *commenting* is a technique where the author of a requirement gathers reviews from others (e.g., co-workers) and represents an informal, low-effort and text-based validation method (Pohl, 2010, 2016). Moreover, *inspections* represent a structured approach for collaboratively investigating a predetermined set of requirements for their correctness (Laitenberger and DeBaud, 2000; Pohl, 2016), while *walk-throughs* represent a lightweight version of an inspection, enforcing a less systematic and strictly organized error investigation approach, yet enabling collaborative improvement of the understanding of the requirements and detection of requirements errors (Pohl, 2016). In addition, *prototyping* enables stakeholders to experience the software system as a tangible artifact, enabling more detailed feedback (Jones, 2007; Pohl, 2010, 2016). Apparently, the prototyping technique that we previously discussed in the context of REI can similarly be applied to the problem of RVal. This indicates the versatility of incorporating simplified, abstracted versions of the software system in the RE process, to facilitate both REI and RVal. Next, we discuss the related concepts of both Software Validation (SVal) and Software Verification (SVer).

2.1.3 Software Validation (SVal) and Verification (SVer)

SVal and SVer (often also abbreviated by the acronym *V&V* (iee, 1990) (IEEE 610.12-1990)) represent two related activities in the SE process, ensuring that the developed software system performs according to specification as well as aligns with the actual stakeholder needs (Adrion et al., 1982; Boehm, 1984b; Sommerville, 2011). In particular, validation is defined as “[...] [t]he process of evaluating a system or component during or at the end of the development process to determine whether it satisfies specified requirements [...]”, and verification is defined as “[...] [t]he process of evaluating a system or component to determine whether the products of a given development phase satisfy the conditions imposed at the start of that phase [...]”, in the standard SE glossary (iee, 1990) (IEEE 610.12-1990). Moreover, according to William (1996) (PMBOK), validation

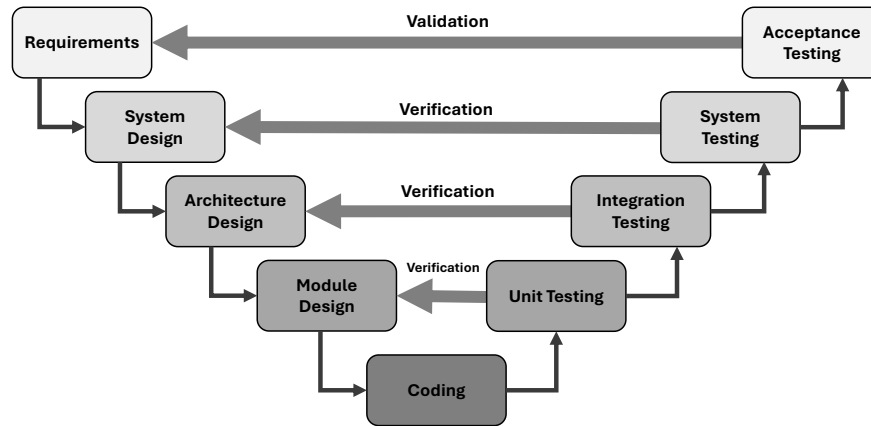


Figure 2.7: V-model with multiple abstraction levels from requirements definition, system specification, system architecture to component design on the right-hand side, and respective testing activities, namely component, integration, system and acceptance testing, on the left-hand side, with verification and validation (adapted from Sommerville (2011)).

is defined as “[t]he assurance that a product, service, or system meets the needs of the customer and other identified stakeholders [...]” and “[...] [i]t often involves acceptance and suitability with external customers [...]”, whereas verification is defined as “[t]he evaluation of whether or not a product, service, or system complies with a regulation, requirement, specification, or imposed condition [...]” and as “[...] [i]t is often an internal process [...]” (William, 1996). More informally, Boehm (1984b) defines validation as “[Are we] building the right product?” and verification as “[Are we] building the product right?”. From these various definitions, it can be observed that validation refers more to an external process of ensuring alignment with the actual needs of the stakeholders, while verification is rather an internal process, ensuring that the software products or intermediate development artifacts adhere to their specification. In addition to the previously discussed SE process models, or the SDLC models, another popular SE process representation is the *V-model*, clearly representing the validation and verification processes in their connection to other SDLC activities (Forsberg and Mooz, 1991; Johansson and Bucanac, 1999; Sommerville, 2011), which is illustrated in Figure 2.7. The SDLC activities in the V-model start on the left-hand side with the requirements specification, followed by the derived system design, the architecture design and the module design and finish at the implementation level, representing the lowest, most detailed representation, and each step represents a further manifestation of the specification from the preceding level. The vertical axis from top to bottom represents an increasing level of detail, while the horizontal axis represents the increasing time (Sommerville, 2011). Each activity on the left-hand side is directly linked to verification and validation activities on the right-hand side. They start from low-level unit testing of the modules implemented

according to the module specification and proceed to higher-level integration testing based on the architecture specification, to system testing given the system specification and finally to acceptance testing, which involves actual users. While the acceptance testing represents validation, involving the users to confirm that the system implements their actual needs, the remaining lower-level testing activities are considered verification, checking an implemented version of a specification against the actual specification at different granularity levels (Sommerville, 2011).

Apparently, similar to the previously discussed SDLC models, the V-model closely integrates the RE activities into the process, where lower-level specifications of the system are all directly derived from the high-level requirements specification. However, the notion of validation and verification in the RE discipline and SDLC models is slightly different. While RVal and SVal share the same idea of validating if the specified requirements align with actual stakeholder needs, they differ in the artifact that is being validated, namely, requirements or early prototypes for RVal and the actual software product in later stages of development for SVal. Within RVer, the requirements are inspected for internal consistency and correctness, while SVer focuses on testing the conformity of an implemented system or system component with its respective specification. Next, we discuss GUI prototyping, covering history, process models and prototype characteristics.

2.2 GUI Prototyping

GUI prototyping represents one of the most effective techniques for the elicitation and validation of requirements, facilitating the communication and discussion of specified and understood requirements via tangible, visual artifacts of the SUD (Baumer et al., 1996; Pohl, 1996; Zowghi and Coulin, 2005; Pohl, 2010; Teixeira et al., 2014; Pohl, 2016). GUI prototyping refers to a collection of techniques and processes for creating a simplified, abstracted, small-scale version of an actually complex software system, thereby reducing the costs and risks compared to developing the entire software system (Szekely, 1994; Baumer et al., 1996). Prototyping techniques range from creating highly abstract, simple pen-and-paper sketch-based prototypes to drawing more detailed mockups, defined as “[...] model[s] or replica[s] of a machine or structure, used for instructional or experimental purposes” (Dictionary, 1989) and creating sophisticated high-fidelity prototypes that possess a look-and-feel close to the final product (Szekely, 1994; Baumer et al., 1996). For REI, GUI prototyping is particularly relevant for interactive and user-facing software systems, often used in conjunction with other elicitation techniques (Zowghi and Coulin, 2005) and helps to elicit more requirements (Abad et al., 2018). Moreover, RVal is enhanced due to the GUI prototype providing a less ambiguous representation as a visual manifestation of the specified requirements (Jones, 2007; Pohl, 2010, 2016). Additionally, this facilitates the integration of stakeholders early in the development, reducing potential misunderstandings (Pohl, 2010, 2016). Next, we provide an overview of the history of GUI prototyping tools and editors, discuss common GUI prototyping process models from the research literature, and introduce GUI prototype characteristics such as different fidelity levels and types of prototypes as well as typical GUI representations.

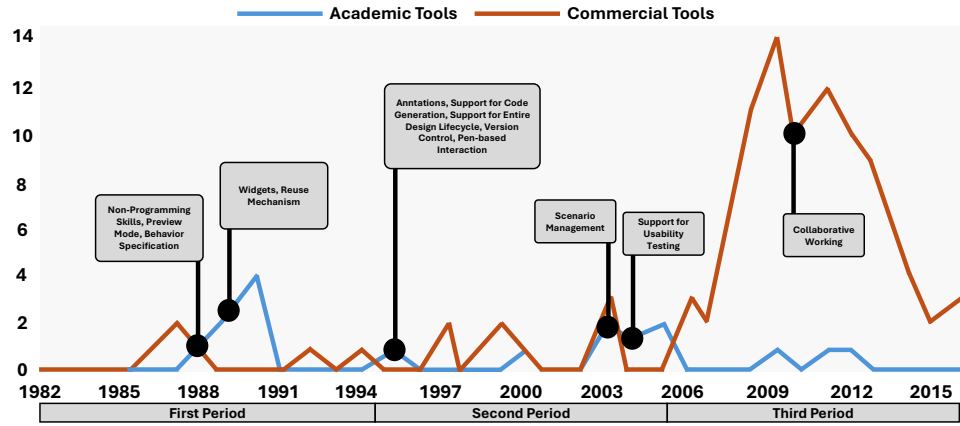


Figure 2.8: History overview (1982 to 2015) of the development of GUI prototyping editors and tools differentiated by academic and commercial tools, annotated with multiple milestones in the development of prototyping tools (adapted from Silva et al. (2017)).

2.2.1 History of GUI Prototyping Tools

The interest in providing support for GUI prototyping through tools and editors both in research and for commercial use dates back around forty years, with early approaches for rapid GUI prototyping such as *Mirage* (McDonald et al., 1988). Silva et al. (2017) provide a detailed historic overview of the development of GUI prototyping approaches, which is depicted in Figure 2.8, showing the yearly occurrence of novel tools both in research and for commercial use as well as milestones in the development. The rise of GUI prototyping approaches was largely based on the interest in User Interface Management Systems (UIMS) (Kasik, 1982), which tried to decouple the GUI representation from the business logic of the software system (Silva et al., 2017).

In the beginning, the focus of proposed approaches lay heavily on supporting non-programming skills for the creation of GUI prototypes, enabling a shift of the focus to the content and functionality of the GUI away from implementation details, thereby simplifying the overall prototyping procedure, such as in *DENIM* (Lin et al., 2000; Silva et al., 2017). To this end, one central notion was to enable users to directly manipulate the visual representation of the GUI (Silva et al., 2017), informally often referred to as *WYSIWYG* (*what you see is what you get*). Another line of early GUI prototyping tools focused on supporting pen-based interaction, enabling the recording of hand-drawn sketches of the GUI (Silva et al., 2017), while other approaches such as *SILK* (Landay and Myers, 1995) and *SketchiXML* (Coyette et al., 2006) also support the translation of sketches into less ambiguous widget-based prototypes with automated recognition techniques. GUI widgets represent predefined, essential GUI components, basically the smallest building blocks of GUI prototypes. Early tools, such as *Lapidary* (Zanden and Myers, 1991) and most modern prototyping tools, provide large widget libraries to choose from, accelerating and simplifying the prototyping process (Silva et al., 2017). Widget

libraries can incorporate more abstract widgets (e.g., basic shapes) for building low-fidelity prototypes, such as in *Balsamiq* (Faranello, 2012), or more detailed widgets (e.g., buttons, text input fields, drop-down selections, list items) possessing a visual representation close to the final system for high-fidelity prototypes, such as in *Mockplus* (Mockplus).

Another major aspect introduced early into GUI prototyping tools was the ability to specify dynamic behavior, typically characterized by a collection of GUI prototype states reachable via defined transitions, in addition to the otherwise static nature of GUI prototypes (Silva et al., 2017). For example, prototyping approaches based on sketches such as *Marvel* (Silva et al., 2017) enable simplistic dynamic behavior by employing *hotspots*, which represent rectangular shapes drawn on top of the sketch that can be linked to various actions and transition behaviors. Similarly, tools like *SILK* (Landay and Myers, 1995) and *DENIM* (Lin et al., 2000; Silva et al., 2017) support merely fundamental wireframe (a basic, simplified layout and structure of the GUI prototype) interactions, represented by direct links between prototype states. More advanced prototyping tools such as *Figma* (Figma) that rely on building GUI prototypes with widget libraries typically offer widget-specific event handlers, often also allowing the definition of variables and conditions to support more complex dynamic behavior specifications, but simultaneously require more effort to create.

An additional important development of GUI prototyping editor functionality includes the preview mode, rapidly enabling users to explore an interactive representation of the prototype based on the predefined dynamic behavior, implemented already in early tools such as *Lapidary* (Zanden and Myers, 1991), *Mirage* (McDonald et al., 1988), *SILK* (Landay and Myers, 1995) and most modern editors such as *Figma* (Figma). Another key functionality introduced in early prototyping editors was a reuse mechanism, for example, facilitated by the employment of widget libraries, predefined GUI templates and prespecified behaviors (Silva et al., 2017), fostering the acceleration of GUI prototyping. Much later introduced in prototyping tools was the ability to create GUI prototype annotations to capture additional important information such as user feedback, problem reports or design preferences, during the prototype creation and usability testing phases (Silva et al., 2017). At the same time, prototyping tools started to encompass code generation capabilities to directly transform the GUI prototype specification into executable program code, thereby reducing the development gap between prototyping and implementation stages (Silva et al., 2017). In later stages, prototyping editors incorporated scenario management, the ability to integrate written scenarios into built GUI prototypes, the support for usability testing, in which users are asked to conduct several tasks with the prototype and data is recorded for future analysis, and eventually collaborative working modes, which allow multiple developers to simultaneously work on an integrated and distributed GUI prototype (Silva et al., 2017).

To summarize, the first period of GUI prototyping editors was characterized by the discussed UIMS tools, decoupling the GUI code from the rest of the software system with a focus on non-programming skills, behavior specification and widget libraries (Silva et al., 2017). The second phase was mainly characterized by the introduction of additional useful features, including annotations, version control and code generation, while in the

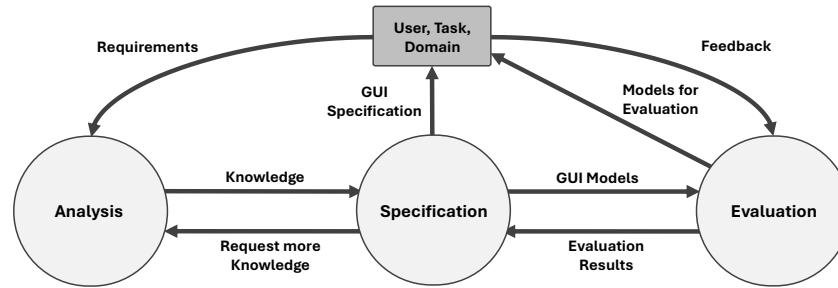


Figure 2.9: Iterative GUI prototyping process model including activities of requirements analysis, GUI specification and GUI model evaluation based on obtaining detailed feedback from the user, task and domain (adapted from Van der Veer and Van Vliet (2001)).

third stage there was a large increase in available commercial GUI prototyping tools (Silva et al., 2017). The three different phases are depicted in Figure 2.8. More recently, many commercial editors supporting GUI prototyping exist, such as *Sketch* (Sketch), *Adobe XD* (Adobe XD), *Mockplus* (Mockplus), *Figma* (Figma) and *Balsamiq* (Faranello, 2012).

2.2.2 GUI Prototyping Processes and Approaches

Multiple GUI prototyping process models were discussed in research before, describing the key activities and artifacts involved in the development of interactive systems. For example, Van der Veer and Van Vliet (2001) present an iterative GUI prototyping process model that includes the requirements analysis from the user, task and domain, the GUI specification and evaluation of GUI models based on their feedback, as depicted in Figure 2.9. As can be observed, this prototyping process model closely resembles the RE process model from Loucopoulos and Karakostas (1995), discussed as part of Section 2.1 in Figure 2.3, while key aspects are adapted to GUI models. Requirements are gathered and analyzed in a task-centered manner, based on existing inadequacies as well as articulation by the users. Accordingly, a GUI specification is derived, GUI models are created and evaluated based on feedback from the user, task and domain (Van der Veer and Van Vliet, 2001).

A similar process model is defined by User-Centered Design (UCD) for interactive systems (DIN EN ISO 9241-220) (din, 2020), which encompasses the planning of the UCD process, understanding and specifying the context of use followed by the specification of the user requirements, the creation of design solutions and finally the evaluation of the design solutions against the requirements (Zimmermann and Grötzbach, 2007; din, 2020), as depicted in Figure 2.10. Apparently, this process model is similar to the RE process model from Section 2.1 in Figure 2.2, while the focus here lies on design creation and evaluation. It represents an iterative and complementary process model to the presented RE and SE process models, particularly with a human-centered perspective (Zimmermann and Grötzbach, 2007). In addition, a similar but simplified variant of this model was presented by Greenberg (1996), representing a cyclic, iterative process with activities of designing, prototyping and user testing with evaluation. Weichbroth and

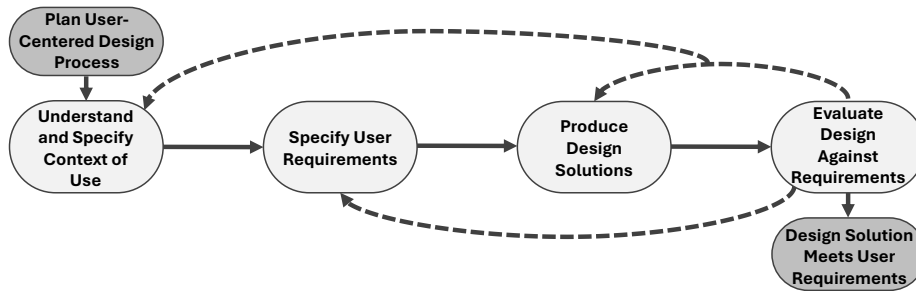


Figure 2.10: Iterative UCD process model including the planning of the design process, understanding and specification of context of use, specification of user requirements, creation of design solutions and finally the evaluation of design solutions against the previously specified requirements (adapted from Zimmermann and Grötzbach (2007)).

Sikorski (2015) present a GUI prototyping process model which displays the creation of prototypes based on GUI standards, such as navigation and structure patterns, the interface schema with primary components and data flow diagrams, as illustrated in Figure 2.11. Subsequently, the GUI prototypes are evaluated and verified whether they are Ready-to-Manufacture (RTM). Otherwise, the prototypes are validated with users and refined, and the GUI specifications are redefined (Weichbroth and Sikorski, 2015). To summarize, most GUI prototyping process models presented in the research literature agree on an inherently iterative procedure, with many overlapping activities. Moreover, many similarities exist between the GUI prototyping and RE process models.

A simple yet effective GUI prototyping technique is paper-based prototyping, often in the form of hand-drawn sketches (Snyder, 2003). In particular, this technique is cost-effective and prototypes can be created rapidly. However, the resulting prototypes are low-fidelity, i.e. highly abstracted with reduced information and a lack of interactivity. Further, low-fidelity prototypes might also be represented as wireframes, which encompass merely rough layouts and reduced information (Rudd et al., 1996). Moreover, high-fidelity GUI prototypes are typically constructed with the help of previously discussed prototyping editors, benefiting from advanced levels of detail and realism, which is advantageous for obtaining higher-quality feedback during user tests (Rudd et al., 1996). By employing the *Wizard-of-Oz* technique, interactions can be mimicked by a human, creating the impression of a fully implemented system (Bernsen et al., 1994) or the interactions can be fully implemented through predefined behaviors in the corresponding GUI prototype.

2.2.3 GUI Prototype Characteristics

GUI prototypes are simplified representations of the final systems' GUI and can be characterized by various properties, which we will discuss in the following. Hartson and Smith (1991) identify three orthogonal dimensions for GUI prototypes. First, they categorize prototypes based on their relation to the final product: *revolutionary*, which describes prototypes employed solely for their specific purpose (e.g., elicitation) and dis-

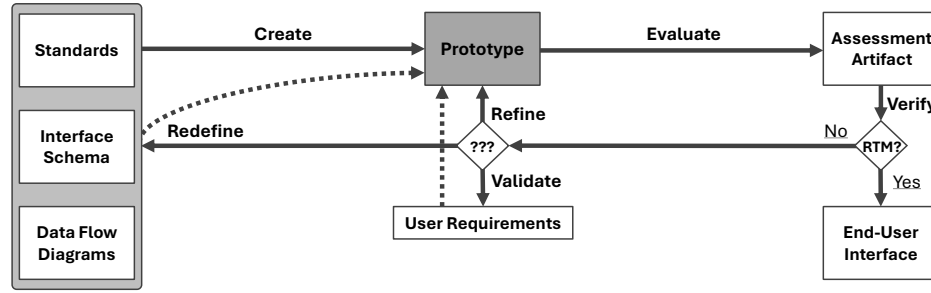


Figure 2.11: Iterative GUI prototyping process model including the creation of the prototype based on standards, interface schema and data flow diagrams, evaluating the GUI prototype artifact, validating requirements and accordingly refining the GUI prototype as well as the respective GUI specifications (adapted from Weichbroth and Sikorski (2015)).

carded afterwards (also referred to as *throw-away* (Kordon et al., 2002)), and *evolutionary*, which represents prototypes that are iteratively modified and refined, evolving into an implementation of the final software product (Hartson and Smith, 1991). Second, they differentiate between building the interface only and creating the entire software system, including the actual computational components (Hartson and Smith, 1991). The third property according to Hartson and Smith (1991) is related to execution: *intermittent*, if the prototype can demonstrate the software behavior merely at cyclic times during development, when the system has been completely created, and *continuous*, if the prototype is not dependent on development-specific times to be exercised. According to them, prototypes can possess any combination of properties along the discussed dimensions.

Moreover, the most defining characteristic of GUI prototypes is the level of fidelity, describing the similarity of the prototype in comparison to the final GUI (Rudd et al., 1996; McCurdy et al., 2006; Feng et al., 2023). Commonly, low-fidelity and high-fidelity levels are described (Rudd et al., 1996), while others encompass a third medium-fidelity level (Feng et al., 2023), as illustrated in Figure 2.12. Low-fidelity prototypes provide a simple, abstract starting point for the GUI, neglecting details of scaling and pixel-accuracy as well as detailed functional information (Rudd et al., 1996; Feng et al., 2023). Moreover, medium-fidelity prototypes provide more specifics compared to their low-fidelity counterparts, including pixel-accuracy for layouts, semantic icons and rough text descriptions, enabling increased possible interactivity (Feng et al., 2023). Finally, high-fidelity GUI prototypes are already close to the final system in terms of their look-and-feel (Rudd et al., 1996; Feng et al., 2023). However, we emphasize that GUI prototype fidelity should be regarded rather as a spectrum instead of fixed categories. Moreover, McCurdy et al. (2006) argue that prototypes possess five independent dimensions, while each of these dimensions can possess different fidelity levels. Specifically, the first dimension is represented by the level of visual refinement, where hand-drawn sketches reside on the low-fidelity side, while pixel-accurate mockups are defined as high-fidelity. Second, the breadth of functionality describes whether a highly functional prototype includes approximations for a large portion of the most important FR. Third, the depth of the



Figure 2.12: Three-level GUI prototype fidelity overview including low-, medium- and high-fidelity GUI prototype examples (adapted from Feng et al. (2023)), while the high-fidelity GUI shows a screenshot extracted from the *Rico* GUI dataset (Deka et al., 2017).

functionality represents the implementation detail for particular features. Fourth, the richness of interactivity, where prototypes with transitions between GUI states and enabled user input represent high-fidelity variants. Finally, the fifth dimension is the richness of the data model, representing the closeness of the employed data (such as names, etc.) in the prototype compared to the final system (McCurdy et al., 2006).

GUI prototypes can be represented in a large variety of different internal formats, ranging from commonly applied web standards (e.g., *HTML* and *CSS*, *JavaScript Object Notation (JSON)*, *Scalable Vector Graphics (SVG)*) to proprietary hierarchical or binary structures. These internal representation models define the different aspects of GUI prototypes, such as the GUI components or widgets, their layout, design and interaction behavior. Often, GUI prototypes are represented as tree-like hierarchical structures internally, which reflects the inherently nested, visual GUI component structure, where frames represent the roots, while they contain grouping containers and individual components.

2.3 Computational Methods and Techniques

In this section, we introduce key computational methods fundamental to our work. In particular, we introduce the basics of ML (Subsection 2.3.1), followed by classical document scoring and ranking methods as well as text representation methods for IR (Subsection 2.3.2). Next, we introduce main concepts of PLMs and LLMs (Subsection 2.3.3), including the Transformer architecture and related models. Finally, we discuss multiple methods for efficiently adapting LLMs, such as PE and CE (Subsection 2.3.4).

2.3.1 Machine Learning (ML)

ML represents a subfield of artificial intelligence, which, in comparison to explicitly programmed rules or instructions, aims at learning and inferring computational models

from experience (data) in order to make predictions or decisions for unseen data points and create representations of the underlying data (Mitchell, 1997). More formally, given a task $\mathcal{T}$, an ML model improves the performance measure $\mathcal{P}$ with exposure to additional experience $\mathcal{E}$ (data). Typically, ML techniques are differentiated into *unsupervised*, *supervised*, *semi-supervised*, *self-supervised* and *Reinforcement Learning (RL)* paradigms, each referring to different forms of supervisory information employed for deriving the model (Bishop and Nasrabadi, 2006; Rani et al., 2023).

In particular, for (1) unsupervised learning, supervisory information is absent and the performance measure $\mathcal{P}$ must rely on intrinsic properties of the data and the task itself, such as the intra-cluster similarity within a document clustering task. Next, for (2) supervised learning, a supervisory signal is available for optimizing the model, often based on human annotations. Specifically, given a dataset $D = \{(x_i, y_i)\}_{i=1}^N$, with $x \in X$ being the input data and $y \in Y$ being the respective labels, the task $\mathcal{T}$ is to learn a parametrized function $f_\theta : X \rightarrow Y$ mapping input examples to corresponding output labels. Binary classification represents a special instance of this problem, where, for example, $Y = \{0, 1\}$ represents a binary label set, and a potential ML model typically predicts a scalar score $s_\theta(x)$, converted into a probability with the sigmoid function:

$$P_\theta(y = 1 | x) = \sigma(s_\theta(x)) = \frac{1}{1 + e^{-s_\theta(x)}}. \quad (2.1)$$

Afterwards, a threshold is applied to create the binary class prediction. To optimize the model for the prediction task, different loss functions or learning objectives can be applied, depending on the concrete task $\mathcal{T}$. For example, to optimize the binary classification model, one objective is to minimize the *binary cross-entropy* (Ho and Wookey, 2019). By employing this loss, the prediction model is encouraged to maximize the prediction probabilities for the correct class across the entire training dataset. Particularly, the binary cross-entropy loss function (Ho and Wookey, 2019) is defined as

$$L_{\text{BCE}}(\theta) = -\frac{1}{N} \sum_{i=1}^N [y_i \log P_\theta(x_i) + (1 - y_i) \log(1 - P_\theta(x_i))] \quad (2.2)$$

While for binary prediction tasks the shown binary cross-entropy loss is commonly employed for optimizing ML models, for multi-class prediction tasks, i.e. with the label set $Y = \{1, 2, \dots, K\}$, $K > 2$, encompassing K -many labels, the *multinomial cross-entropy* loss function (Golik et al., 2013) is typically employed and minimized as:

$$L_{\text{CE}}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log P_\theta(y_i | x_i). \quad (2.3)$$

Furthermore, (3) self-supervised learning is a form of representation learning, exploiting inherent correlations of the data as a supervisory signal in comparison to human-created annotations (Liu et al., 2021). This paradigm plays a significant role in PLMs, which are

typically pretrained on self-supervised tasks, such as contextual or next word predictions. In addition, and similar to cross-entropy, another important measure of divergence between two probability distributions is the *KLD* (Kullback, 1951), defined as

$$D_{\text{KL}}(P \mid Q) = \sum_{x \in \mathcal{X}} P(x) \log \frac{P(x)}{Q(x)}. \quad (2.4)$$

where P represents the true probability distribution and Q refers to the approximating probability distribution, with both distributions defined over the respective support $\mathcal{X}$.

2.3.2 Information Retrieval (IR)

IR is concerned with retrieving relevant documents or resources given an information need (query) (Singhal et al., 2001; Schütze et al., 2008). More formally, given a corpus of documents $D = \{d_1, \dots, d_N\}$ and a set of possible queries Q , a modern IR system provides a scoring function $s : Q \times D \rightarrow \mathbb{R}$, which assigns a relevance score $s(q, d)$ to each document d associated with query q , ultimately creating a ranking from most to least relevant according to $s(q, d)$ over the entire collection D as $\pi(q) = \text{argsort}_{d \in D} s(q, d)$. We refer to the set of relevant documents with respect to a query q as $R(q) \subseteq D$. However, classical IR methods are based on the *Boolean Model* (Lancaster and Gallup, 1973), where documents are represented as collections of individual index terms $t_i \in V$, and queries are Boolean expressions in Conjunctive Normal Form (CNF) over V such as $q = (t_1 \vee \neg t_2) \wedge (t_3 \vee t_4)$, allowing solely exact matches (Lancaster and Gallup, 1973).

Bag-of-Words (BoW). To compute similarities between queries and documents, the textual information needs to be represented in a form which enables effective processing by computational methods. A simple representation of text documents is provided by the BoW model, which essentially is a text feature extraction technique. In this model, text is represented as an unordered collection of words, where each word of the vocabulary V corresponds to a dimension in a vector representing document d as $\mathbf{x}_d = (c_{d,1}, c_{d,2}, \dots, c_{d,|V|})$, where $c_{d,i}$ represents the frequency of word t_i within document d . However, this representation neglects word order and semantics entirely.

Term Frequency-Inverse Document Frequency (TF-IDF). To improve the simple BoW model and adjust the importance of terms with respect to their occurrence in the corpus, the Term Frequency (TF) is adjusted by the Inverse Document Frequency (IDF):

$$w_{t,d} = \frac{c_{d,t}}{\sum_{t'} c_{d,t'}} \log \left(\frac{N}{1 + n_t} \right) \quad (2.5)$$

with $w_{t,d}$ being the weight for word t in document d , $c_{d,t}$ the frequency of word t in document d , $N = |D|$ being the number of documents in the corpus and n_t the number of documents encompassing word t . Similarity between queries and documents can then be computed based on cosine similarity, i.e. $\text{sim}(d_1, d_2) = \frac{x_{d_1} \cdot x_{d_2}}{\|x_{d_1}\| \|x_{d_2}\|}$, with x_{d_1} and x_{d_2} representing the TF-IDF vectors of documents d_1 and d_2 , respectively. TF-IDF

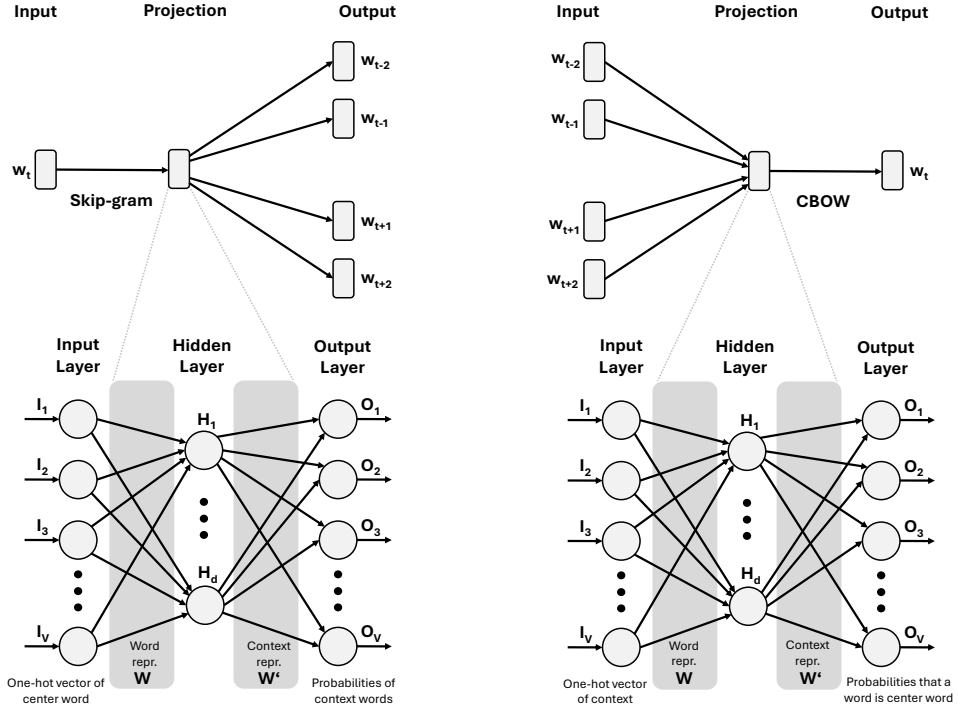


Figure 2.13: Two approaches for learning semantic word representations: (1) *Skip-gram* architecture (on the left), predicting the context words given the one-hot encoded vector of the center word and (2) *Continuous Bag-of-Words (CBOW)* (on the right), predicting the center word given one-hot encoded context words, both with a Feed-Forward Neural Network (FFNN) architecture (adapted from Mikolov et al. (2013a) and Lauscher (2021)).

scoring ensures that frequently occurring words (e.g., stop words, providing little extra information for distinction) accordingly receive lower weights (Salton and Buckley, 1988).

Okapi Best Matching 25 (BM25). A more advanced scoring function of the TF-IDF family is represented by Okapi BM25, which also incorporates both TF and IDF components with adjusted weightings (Robertson et al., 1995), and is defined as

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \frac{c_{d,t}(k_1 + 1)}{c_{d,t} + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)} \quad (2.6)$$

with $t \in q$ referring to the words t contained in the query q , $\text{IDF}(t)$ being a BM25-specific variant of the IDF for term t , while it should be noted that multiple IDF variants exist, $c_{d,t}$ the frequency of word t in document d , $|d|$ the number of terms t in document d , also referred to as the length of the document, avgdl the average document length across the corpus, while $k_1 \in [1.2, 2.0]$ and $b = 0.75$ are adjustable weighting terms.

word2vec. *word2vec* is one of the early approaches to enhance word representations over the simple sparse models, enabling them to capture semantic relationships (Mikolov et al., 2013b,a). Fundamental to the approach is the *distributional hypothesis*, which notes that words appearing in similar contexts tend to have similar meanings (Harris, 1954). Particularly, in such a word vector space, each word w_i is represented as a static, real-valued, d -dimensional vector $\mathbf{e}(w_i) \in \mathbb{R}^d$. To compute these embeddings, two common approaches exist. First, *Skip-gram* maximizes the average log probability

$$\frac{1}{T} \sum_{t=1}^T \sum_{j=-c}^c \log P(w_{t+j} \mid w_t) \quad (2.7)$$

for predicting the surrounding context words $w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c}$ given the center word w_t , where c denotes the context size, while T represents the number of words in the sequence, which is depicted in Figure 2.13 (Mikolov et al., 2013a; Lauscher, 2021). The probability $P(w_{t+j} \mid w_t)$ is computed with the *softmax* function as

$$P(w_{t+j} \mid w_t) = \frac{\exp(x'_{t+j} \top x_t)}{\sum_{i=1}^{|V|} \exp(x'_i \top x_t)} \quad (2.8)$$

with x_i and x'_i denoting the actual d -dimensional word and context vector representations, while V denotes the vocabulary. In contrast, the CBOW approach predicts the center word w_t given the sequence of surrounding context words $w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c}$ by optimizing the following average log probability (Mikolov et al., 2013b)

$$\frac{1}{T} \sum_{t=1}^T \log P(w_t \mid w_{t-c}, \dots, w_{t+c}) \quad (2.9)$$

Neural Bag-of-Words (nBoW). Since the discussed *word2vec* embeddings are static and computed for single words only, a simple approach to compute embeddings for a sequence of words, such as in document d , is to employ the *nBoW* technique (Sheikh et al., 2016), by computing a weighted average over the individual word embeddings as

$$x_d = \sum_{i=1}^L \alpha_i \mathbf{e}(w_i) \quad (2.10)$$

with L representing the number of words in the sequence $w_1, \dots, w_L$, $\mathbf{e}(w_i) \in \mathbb{R}^d$ being the d -dimensional embedding of word w_i and α_i its respective weight. As meaningful weights, the normalized TF-IDF scores can be applied, i.e. $\alpha_i = w_{i,d}$, emphasizing embeddings from more relevant words, while discounting less informative embeddings.

2.3.3 Pretrained Language Models (PLMs)

In contrast to the previously introduced static word representations, PLMs aim to learn generalized and contextualized text representations by leveraging large corpora for *pre-*

training, which can be adapted to various downstream NLP problems and tasks via subsequent *finetuning* approaches (Wang et al., 2023). In particular, the objective employed to optimize PLMs requires them to predict *tokens* based on their context. Modern PLMs rely on deep neural network architectures, especially the *Transformer* architecture (Vaswani et al., 2017), which we present subsequently, followed by different PLM variants.

Transformer Architecture. The most popular approach underlying PLMs is based on the *Transformer* architecture (Vaswani et al., 2017), particularly on the decoder component, which comprises multiple identical layers. Each of these layers encompasses a *self-attention* and a FFNN component. Relying on *self-attention*, the *Transformer* overcomes prior efficiency limitations of recurrent neural architectures (Sutskever et al., 2011). Specifically, the *self-attention* usually employs the *scaled dot-product (multiplicative) attention* (Vaswani et al., 2017). The *self-attention* mechanism operates on three matrices, namely, queries $\mathbf{Q}$, keys $\mathbf{K}$ and values $\mathbf{V}$, which are obtained from the input representations. Subsequently, the *self-attention* is computed using a *softmax* function as

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (2.11)$$

with $\frac{1}{\sqrt{d_k}}$ being a scaling factor. Hence, every token in the sequence is enabled to attend to every other token for long-range dependency coverage. To enable capturing diverse relationships across the different representation spaces, *Multi-Head Self-Attention (MHSA)* is introduced, which employs multiple *self-attention heads* in parallel. The computed outputs of each *self-attention head* are concatenated and projected with matrix $\mathbf{W}^O$ as

$$\text{MHSA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \mathbf{W}^O \quad (2.12)$$

In particular, this mechanism enables the model to simultaneously attend to diverse information from multiple representation spaces from various positions, creating richer contextual representation modeling (Vaswani et al., 2017). Each head_i is computed as

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V) \quad (2.13)$$

and projection matrices as $\mathbf{W}_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, and $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$, with the dimension d_k for queries and keys, and dimension d_v for values. While modern PLMs are based on the *Transformer* architecture as the state-of-the-art approach, they typically rely on decoder-only or encoder-only architectures.

Bidirectional Encoder Representations from Transformers (BERT). An early adaptation of the *Transformer* architecture as an encoder-only bidirectional model for creating language representations is BERT, which is trained on large-scale text corpora by incorporating the left and right context across all layers (Devlin et al., 2019). In particular, the model is pretrained with two training objectives: (1) a Masked Language Modeling (MLM) token-level task, which asks the model to predict randomly masked tokens in the

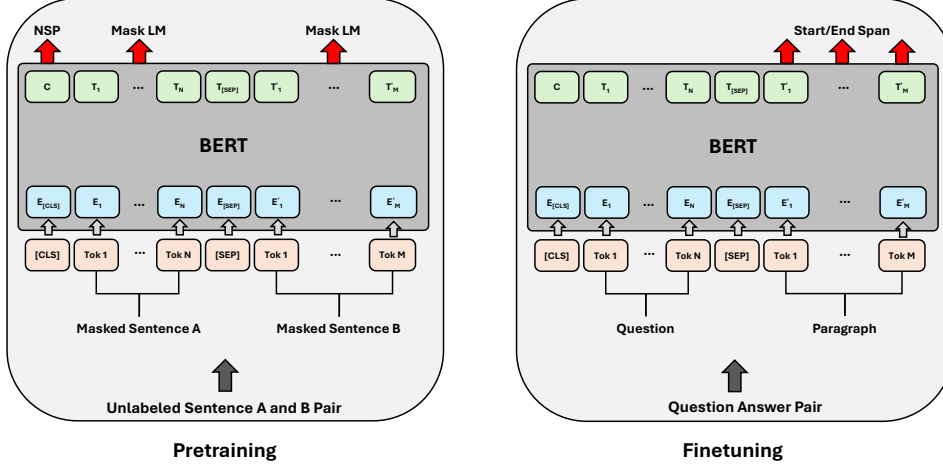


Figure 2.14: *BERT* approach based on a *Transformer* encoder with (1) pretraining phase, which includes two training objectives: Masked Language Modeling (MLM) and NSP, and (2) the finetuning stage with the same parameters from pretraining for NLP tasks (e.g., question answering or query-document scoring) (adapted from Devlin et al. (2019)).

sequence and (2) a Next Sentence Prediction (NSP) sentence-level task, which requires the model to predict whether two sentences are adjacent in the text, as depicted in Figure 2.14. The input to the model is divided into token, segment and position embeddings (Devlin et al., 2019). Specifically, BERT employs the two loss functions together in the pretraining stage as $\mathcal{L}_{\text{BERT}} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{NSP}}$, with $\mathcal{L}_{\text{MLM}}$ and $\mathcal{L}_{\text{NSP}}$ defined as follows:

$$\mathcal{L}_{\text{MLM}} = - \sum_{t \in \mathcal{M}} \log P_{\theta}(x_t | \tilde{X}). \quad (2.14)$$

$$\mathcal{L}_{\text{NSP}} = - [y \log P_{\theta}(1) + (1 - y) \log P_{\theta}(0)]. \quad (2.15)$$

with $\mathcal{M}$ being the set of randomly masked tokens, x_t the masked token and $\tilde{X}$ the masked input token sequence. BERT utilizes two special input tokens: (1) [CLS], representing the starting token, whose hidden state $C \in \mathbb{R}^H$ represents the aggregated sequence embedding, which can be used for sentence-level classification tasks (e.g., NSP), and (2) [SEP], which represents the separation between the first (e.g., sentence A) and the second (e.g., sentence B) input. BERT is available in different sizes, but a popular variant is **BERT_{BASE}** with L=12 layers, H=768 hidden dimension size, A=12 self-attention heads and 110M parameters in total (Devlin et al., 2019). To finetune BERT for different tasks, the input sequence requires preparation encompassing the special tokens. For example, considering a ranking task, query-document pairs (q, d_i) can be inputted to BERT, while a prediction head (e.g., a simple FFNN) processes the encoded BERT state to compute ranking score predictions. The entire BERT model can be finetuned end-to-end, which updates all parameters, or solely the prediction head can be finetuned for the problem.

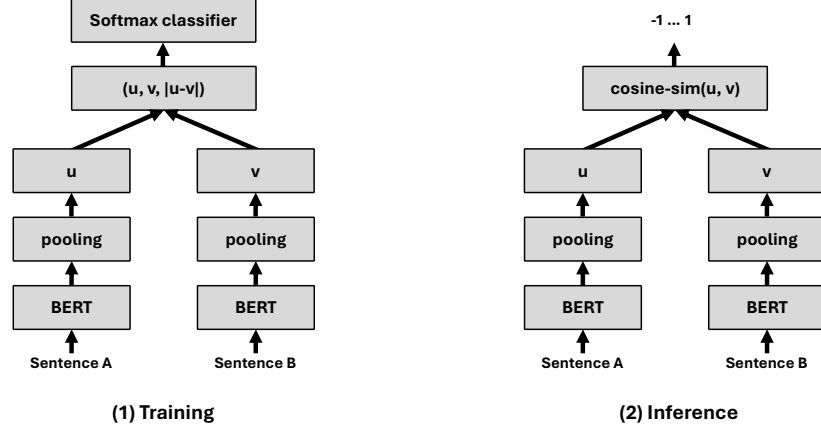


Figure 2.15: *Sentence Bidirectional Encoder Representations from Transformers (SBERT)* approach with (1) training architecture (on the left), showing a siamese network structure with shared parameters based on the BERT model and (2) inference architecture with *cosine similarity* output (on the right) (adapted from Reimers and Gurevych (2019)).

SBERT. While BERT provided state-of-the-art performance across various NLP tasks (before LLMs), the computational overhead for tasks such as computing Semantic Textual Similarity (STS) is immense (Reimers and Gurevych, 2019). To mitigate this, Reimers and Gurevych (2019) proposed SBERT, which enables the computation of semantic sentence embeddings for downstream tasks, such as similarity scoring and text retrieval problems. The model is based on a Siamese Neural Network (SNN) (Chicco, 2021), which replicates its structure and conducts parameter sharing for both input sentences, as shown in Figure 2.15. Each sentence is fed through the BERT model and the outputs are pooled based on different strategies (e.g., employing the $[CLS]$ token, *mean* over output vectors), concatenated with element-wise difference $|u - v|$, which are then multiplied by trainable weights W_t and fed through $\text{softmax}(W_t(u, v, |u - v|))$ to optimize cross-entropy loss. Finally, the text representations obtained from SBERT can be meaningfully applied to compare text via *cosine similarity*, i.e. $\text{sim}(u, v) = \frac{u^\top v}{\|u\| \|v\|}$, useful for ranking.

LLMs. Based on the large success of the *Transformer* architecture, its adaptation for state-of-the-art PLMs and the insight that scaling these architectures up leads to improved capabilities (Kaplan et al., 2020), the term LLMs has been coined, referring to massive variants of PLMs, typically decoder-only, which encompass billions of parameters and are trained on vast amounts of text data (Zhao et al., 2023). In particular, LLMs are initially optimized via self-supervised pretraining on large token sequences by optimizing a common (autoregressive) language model objective (Radford et al., 2018) as follows

$$\sum_i \log P_\theta(u_i | u_{i-k}, \dots, u_{i-1}) \quad (2.16)$$

2. THEORETICAL BACKGROUND

Model Name	Organization	Parameters	Context	MM	Reference
GPT-3	OpenAI	175B	2,048	×	(Brown et al., 2020)
GPT-4o	OpenAI	UD	128k	✓	(OpenAI, 2025e)
GPT-4.1	OpenAI	UD	1M	✓	(OpenAI, 2025d)
GPT-5	OpenAI	UD	400k	✓	(OpenAI, 2025f)
LLaMA 2	Meta	7,13,70B	4k	×	(Meta, 2025a)
LLaMA 3.2	Meta	3,11,90B	128k	✓	(Meta, 2025b)
Gemini 2.5 Pro	Google	UD	1M	✓	(Google, 2025a)
Gemini 3 Pro	Google	UD	1M	✓	(Google, 2025b)
Claude Sonnet 4.5	Anthropic	UD	200k, 1M	✓	(Anthropic, 2025b)
Claude Opus 4.5	Anthropic	UD	200k	✓	(Anthropic, 2025b)
Qwen 2	Alibaba	0.5, 1.5, 7.2B	32k	×	(Yang et al., 2024)
Qwen 3	Alibaba	4, 8, 14, 32B	128k	×	(Yang et al., 2025)
DeepSeek	DeepSeek	7, 64B	4k	×	(Bi et al., 2024)
DeepSeek-VL	DeepSeek	1.3, 7B	4k	✓	(Lu et al., 2024)

Table 2.1: LLM overview (OpenAI, Meta, Google, Anthropic, Alibaba, DeepSeek) with organization, parameters, context, multi-modal (MM) support (UD = Undisclosed).

with u_i referring to a token, k being the context size and θ the parameters of a neural network trained with Stochastic Gradient Descent (SGD)-based optimization (Amari, 1993). Similar to the discussed BERT model, LLMs typically undergo Supervised Finetuning (SFT) for specific NLP problems after generic pretraining using an objective such as

$$\sum_{(x,y)} \log P(y \mid x_1, \dots, x_m) \quad (2.17)$$

with $x_1, \dots, x_m$ referring to an input token sequence, while y denotes a corresponding label for the task (Radford et al., 2018). While increasing the parameter size of the models constantly improves their performance under scaling laws (Kaplan et al., 2020), the ability to follow human instructions is restricted (Ouyang et al., 2022). Therefore, Ouyang et al. (2022) proposed an adaptation pipeline consisting of three phases: (1) conduct SFT of the LLM given high-quality task completions with respect to a prompt, (2) train a reward model based on sampling prompts and multiple model completions, which are subsequently ranked by annotators and (3) optimize a policy with regard to the reward model by employing RL. The resulting model shows increased ability to follow provided instructions over previous models. Table 2.1 provides an overview of well-known proprietary and open-source LLMs, showing their parameter size (while often undisclosed for many proprietary LLMs), the context size and their multi-modal support. Most prominent is *OpenAI*, providing one of the earliest LLMs, with the GPT model series, achieving state-of-the-art results across diverse NLP tasks. Next, we discuss MLLMs.

MLLMs. While the increasing capabilities of LLMs for text-based tasks are promising and achieve state-of-the-art results, incorporating other modalities such as images represents an important problem. Specifically, MLLMs extend LLMs with multi-modal

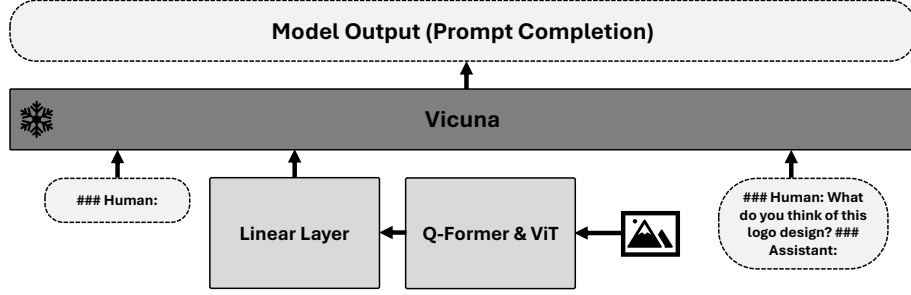


Figure 2.16: Example *MLLM* architecture of *MiniGPT-4* employing a vision encoder based on a pretrained *Vision Transformer (ViT)* model and *Q-Former* network, a linear projection layer, which is integrated into *Vicuna* LLM (adapted from Zhu et al. (2023)).

processing capabilities. Zhu et al. (2023) integrated a visual encoder with frozen weights into an advanced LLM via a single projection layer, as depicted in Figure 2.16. Particularly, the results indicate that, when visual features are properly aligned with an LLM, the resulting model possesses multi-modal abilities, such as image captioning or generating website code given a respective screenshot (Zhu et al., 2023). Instead of creating and training a MLLM from scratch end-to-end, Wu et al. (2023) propose *Visual-ChatGPT* by integrating various ViT models into LLMs via a high-level prompt manager, which provides the LLM with access to the image information via conversion by utilizing ViT.

2.3.4 Prompt and Context Engineering (PE/CE)

With increasing parameter size, LLMs have been shown to possess emergent abilities such as ICL, which refers to their capability of task adaptation during inference time through examples in their context, requiring no parameter updates or training (Brown et al., 2020; Liu et al., 2023). More formally, given the context $C = \{I, (x_1, y_1), \dots, (x_k, y_k)\}$ of k examples as input to the LLM, with I denoting optional instructions in natural language, while (x_i, y_i) refers to an example pair of input text x_i with the respective label y_i , the LLM predicts the label for the next unseen example x_{k+1} (Dong et al., 2024) as

$$P_{\theta}(y_{k+1} \mid x_{k+1}, C) = \text{LLM}_{\theta}(C, x_{k+1}). \quad (2.18)$$

by computing the respective $\hat{y}$ that achieves the maximum probability among all possible solutions based on its parameters θ , i.e. $\hat{y} = \arg \max_{y_j \in Y} P_{\theta}(y_j \mid x)$. Following this paradigm, Liu et al. (2023) coined the overall approach as “*pre-train, prompt, and predict*” or “*prompt-based learning*”. This enables efficient adaptation of LLMs to a plethora of different NLP tasks, including classification and task-specific Natural Language Generation (NLG) (Dong et al., 2022). Furthermore, research has shown that LLMs can be effectively adapted to various tasks in the complete absence of examples, solely by providing meaningful instructions in natural language (Wei et al., 2021; Kojima et al., 2022). To optimize the effectiveness of LLMs in such settings, the discipline of *Prompt Engineering*

(*PE*) has emerged (Marvin et al., 2023; Schulhoff et al., 2024), referring to the process of creating, refining and optimizing instructions given to the LLM in order to solve a specific task effectively, thereby leveraging the full potential of the LLM. More recently, the term *Context Engineering (CE)* (Haseeb, 2025) has been coined, referring to the systematic construction of the input context of an LLM, by providing all task-specific information and external knowledge relevant for effectively solving a task, including necessary tools. Subsequently, we provide an overview of effective PE techniques proposed in research.

Zero-Shot (ZS). Research has shown that providing solely meaningful instructions in NL can be sufficient for LLMs to effectively solve certain tasks (Wei et al., 2021; Kojima et al., 2022). More formally, this approach is referred to as ZS and the input context $C = \{I\}$ to the LLM consists merely of the instruction I . Typically, *prompt templates* are employed to replace example-specific information, while retaining generic textual instructions, i.e. context $C = \{I(x_1, \dots, x_k)\}$, with x_i referring to task-specific inputs.

Few-Shot (FS). In contrast to ZS (only instructions and task-specific information), FS prompting relies on providing multiple input-output pair examples in the context, i.e. $C = \{I, (x_1, y_1), \dots, (x_k, y_k)\}$ with $k \geq 2$ (Brown et al., 2020). This approach can be conducted both with and without task-specific instructions I given in the context C .

One Shot (OS). A particular instance of FS prompting is OS prompting, which provides merely a single example in the input context of the LLM, i.e. $C = \{I, (x, y)\}$ (Brown et al., 2020). However, ZS and FS are more widely used in research and practice.

Chain-of-Thought (CoT). An early approach to improve the reasoning capabilities of LLMs by PE techniques alone is represented by CoT (Wei et al., 2022), which asks the LLM to introduce a sequence of intermediate reasoning steps instead of immediately providing the output answer, as depicted in Figure 2.17 (1). More formally, the input context of the LLM is constructed as $C = \{I, (x_1, y_1, r_1), \dots, (x_k, y_k, r_k)\}$, with x_i being the input problem of the i -th example, y_i denoting the correct output for the i -th example and $r_i = \{r_i^1, \dots, r_i^l\}$ providing l -many intermediate reasoning steps for the i -th example. Multiple variations of CoT have been proposed, such as the FS variant (FS-CoT), an OS variant with $k = 1$ (OS-CoT) and a ZS variant (ZS-CoT), which directly instructs the model to think step-by-step without examples (Kojima et al., 2022).

Prompt Decomposition (PD). To enable LLMs to handle more complex problems, Khot et al. (2022) proposed a novel PE technique referred to as PD. In comparison to CoT, PD divides the overall problem into smaller subtasks, with each subtask being delegated to a collection of task-specific prompting strategies, as shown in Figure 2.17 (2).

Self-Critique (SC). Another important PE technique to improve the task-solving effectiveness of LLMs is *SC* (Saunders et al., 2022), depicted in Figure 2.17 (3). Instead of generating an answer y directly by the LLM from the input context, i.e. $y = LLM_\theta(I, x)$,

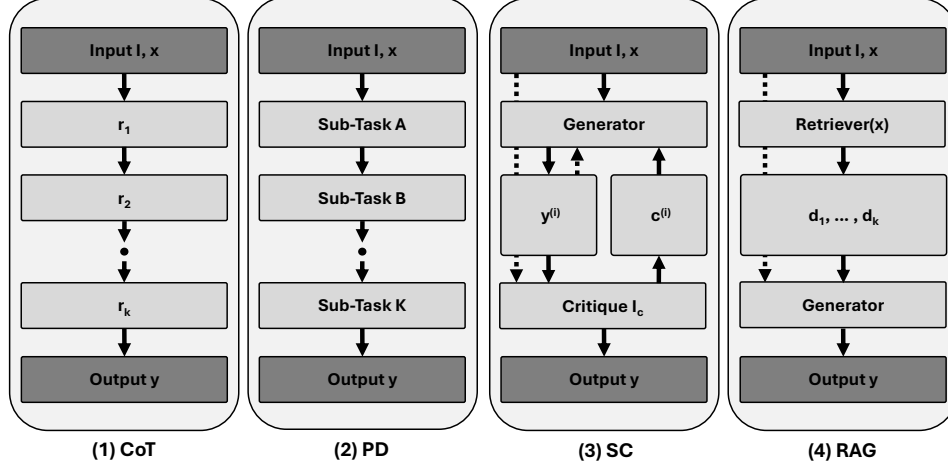


Figure 2.17: *PE* techniques overview encompassing four well-known and popular approaches proposed in research before such as (1) *OS-CoT*, (2) *PD*, (3) *SC* and (4) *RAG*.

SC first (1) generates an initial solution response $y^{(0)} = LLM_{\theta}(I, x)$, which (2) is followed by a *self-critique* as $c^{(0)} = LLM_{\theta}(I, x, y^{(0)}, I_c)$ with critique instructions I_c to (3) produce an enhanced response $y^{(1)} = LLM_{\theta}(I, x, y^{(0)}, c^{(0)})$. In particular, as indicated by the indices, *SC* can also be conducted in an iterative manner, consisting of multiple rounds of output and critique generation, depending on the problem context.

RAG. Since the previously discussed *PE* techniques mostly rely on static, predefined input context, LLMs might miss contextually relevant external information for a given problem and example. Furthermore, without access to external knowledge, LLMs are forced to solely rely on information stored in their parameters during pretraining. To mitigate this problem, *RAG* (Lewis et al., 2020; Li et al., 2022) has been proposed in research, exploiting external knowledge bases to inject context-specific information into the input context, i.e. $C = \{I, x, d_1, \dots, d_k\}$, with $D = \{d_1, \dots, d_k\}$ referring to the top- k external documents or chunks retrieved by a respective retrieval model $R(x) = D$, depicted in Figure 2.17 (4). This *PE* technique proves particularly effective for problems that tackle domains and knowledge outside of the LLMs’ pretraining data.

Other *PE* Techniques. While we discussed several of the most important *PE* techniques, particularly relevant in the context of this work, many other *PE* approaches have been proposed in research, of which we briefly summarize a selection. For example, *self-consistency* (Wang et al., 2022a) samples multiple, diverse reasoning paths and aggregates the results to provide an improved output response, while *Tree-of-Thoughts (ToT)* (Long, 2023; Yao et al., 2024) generalizes the notion of *CoT* to tree-like structures, allowing LLMs to self-evaluate decision paths and conduct backtracking when necessary. Moreover, *Graph-of-Thoughts (GoT)* represents a generalization of *ToT*, where intermediate thoughts or reasoning steps are interpreted as graph vertices, while edges between these nodes represent arbitrary dependencies, enabling the modeling of complex problems.

Part I

PROTOTYPING: GUI RETRIEVAL VIA NATURAL LANGUAGE

CHAPTER 3

NATURAL-LANGUAGE-BASED GUI RETRIEVAL

Given the context of the first main challenge (C1), namely, mapping NLR to GUI prototypes, in this chapter, we provide details about the first solution approach, the creation of a novel benchmark for evaluation and the results obtained. In particular, we investigate text-based retrieval methods with NLR as input and rankings over GUI prototypes as the output target. The following sections are based on previous work and were already partially published before (Kolthoff, 2019; Kolthoff et al., 2020, 2021, 2023)¹. Our interactive prototype, source code, datasets (including the novel gold standard) and demonstration video are all publicly available to foster future research for NLR-based GUI retrieval².

Personal Contribution. I ideated and created the concept for the work, constructed the entire approach and architecture, implemented the BERT-LTR model and data-driven tool prototype. Moreover, I designed and conducted the evaluation, including the novel gold standard, conducted data analysis and was the sole author of the manuscript.

3.1 Motivation

As we already discussed in the introduction of this work, GUI prototyping represents an important technique to visualize the analysts’ understanding of the NLR from stakehold-

¹This section is adapted from: (1) **Kolthoff, Kristian**, Bartelt, Christian, and Ponzetto, Simone Paolo. Data-Driven Prototyping via Natural-Language-based GUI Retrieval. *Automated Software Engineering*, March 2023, 30(1), 13, pages 1–34, Springer. Sections are directly reused with only minor adaptations (i.e. thesis Section 3.1 corresponds to paper Section 1) (2) **Kolthoff, Kristian**, Bartelt, Christian, and Ponzetto, Simone Paolo. GUI2WiRe: Rapid Wireframing with a Mined and Large-Scale GUI Repository using Natural Language Requirements. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE, A*)*, Melbourne, Australia (Virtual Event), January 2021, pages 1297–1301. ACM. (3) **Kolthoff, Kristian**, Bartelt, Christian, and Ponzetto, Simone Paolo. Automated Retrieval of Graphical User Interface Prototypes from Natural Language Requirements. In *Proceedings of the 26th International Conference on Applications of Natural Language to Information Systems (NLDB)*, Saarbrücken, Germany (Virtual Event), June 2021, pages 376–384, Cham: Springer International Publishing.

²Materials for this chapter available at <https://github.com/kristiankolthoff/RaWi> and <https://github.com/kristiankolthoff/GUI2WiReTool>, video at <https://youtu.be/2nN-Xr2Hk7I>

ers, enable their validation by stakeholders as a tangible artifact, provide the foundation for incorporating stakeholders early into the application development and lead to fruitful discussions, clarifications and refinements of requirements (Windsor and Storrs, 1992; Rudd et al., 1996; Pohl, 1996; Ravid and Berry, 2000; Beaudouin-Lafon and Mackay, 2002; Mukasa and Kaindl, 2008; Pohl, 2010, 2016). Especially high-fidelity GUI prototypes have been proven to be effective, since these prototypes provide the foundation for discussions of higher quality between stakeholders and analysts, and more detailed feedback can be obtained during user tests compared to low-fidelity approaches (Landay and Myers, 1994; Rudd et al., 1996; Coyette et al., 2007). However, the benefits of high-fidelity GUI prototypes are accompanied by the disadvantages of increased time and experience required for their development (Rudd et al., 1996).

To facilitate and simplify GUI prototyping, a plethora of approaches has been proposed before. Many popular GUI prototyping tools are available and employed in practical prototyping environments such as *Sketch* (Sketch), *Adobe XD* (Adobe XD), *Mockplus* (Mockplus), *Figma* (Figma) and *Balsamiq* (Faranello, 2012). These approaches typically allow users to combine basic GUI components and a small number of GUI templates. However, they are time-consuming and demand a wide prototyping experience for effective application. More recent GUI retrieval approaches such as *GUIFetch* (Behrang et al., 2018), *Swire* (Huang et al., 2019), *Screen2Vec* (Li et al., 2021) and *VINS* (Bunian et al., 2021) use hand-drawn sketches or GUI screenshots as input for retrieving GUIs to support prototyping. However, they initially require a basic GUI representation and the retrieved GUI images cannot be directly reused or adapted. Overall, these approaches focus on supporting the prototyping of the final GUI design by showing GUI design alternatives, but cannot be applied for interactive GUI prototyping for REI and RVal.

In addition, *Guigle* (Bernal-Cárdenas et al., 2019) represents the first basic search engine for GUIs of mobile apps supporting a query language and stylistic searches. However, it proposes only a simple retrieval approach and the obtained GUI screenshots are not directly reusable. Specifically, *Lucene* (Lucene) was employed, focusing on simple, traditional TF-IDF and BM25 methods. However, these methods are optimized for the same modality and assume similar text representations between *query* and *documents* to work effectively. When considering the problem at hand, the input representations in the form of NLR represent high-level, abstract concepts and descriptions, while GUIs are multi-dimensional objects, typically represented as layout hierarchies encompassing GUI components with a diverse set of properties. Therefore, instead of applying these methods without any adaptation to NLR-based GUI retrieval emphasizing the representation mismatch, it is crucial to tailor them to this problem. Hence, this work is based on the leading research question of challenge **C1-I**: *How can we adapt and optimize text-based retrieval methods and techniques to enable more effective NLR-based GUI retrieval?*

NLR-based GUI retrieval for GUI prototyping. In this work, we introduce *RaWi*, a data-driven rapid GUI prototyping approach (to create high-fidelity GUI prototypes) that (i) exploits a large-scale semi-automatically created GUI repository of mobile applications by employing NLR-based ad-hoc GUI retrieval and automatically derives editable

GUI screens for reuse and (ii) thus effectively leverages the GUI prototyping knowledge embodied in the repository to facilitate prototyping and improve prototyping productivity. Due to enabling rapid mapping of NL fragments from users to GUI prototypes, our approach allows for significantly faster GUI prototyping compared to a traditional approach and thus can be applied more effectively by analysts to elicit requirements with stakeholders via interactive GUI prototyping. Moreover, we envision supporting novice analysts with little or no prior experience in GUI prototyping with our approach by providing easy access to plenty of reusable GUIs and domain knowledge embodied in the GUI repository. Due to the automatic derivation of partly editable GUI screens from a large-scale GUI screenshot repository, our approach is able to provide a vast number of reusable GUI screens, which eliminates the manual effort typically required for providing GUI templates in traditional prototyping approaches. *RaWi* integrates both the improved GUI retrieval mechanism and editable GUI screen derivation techniques in a novel, data-driven and web-based graphical prototyping editor.

Contributions. With our work, we make the following four research contributions:

- *BERT-LTR model for GUI ranking:* we present a BERT-LTR approach that is trained on large-scale GUI relevance data harvested using crowdsourcing techniques, demonstrating substantial improvements over traditional retrieval and ranking methods.
- *Novel gold standard for NLR-based GUI retrieval:* we create a novel and comprehensive gold standard for NLR-based GUI retrieval with crowdsourcing techniques to enable a systematic evaluation of NLR-based GUI ranking models and make it publicly available to foster further research.
- *In-depth IR method comparison:* we conduct the first in-depth analysis and evaluation of various traditional IR, AQE and trained BERT-based LTR models for NLR-based GUI ranking using the novel gold standard.
- *Evaluation methodology:* we propose a comprehensive evaluation methodology including multiple metrics for measuring GUI prototyping productivity and conduct an extensive user study to assess the usefulness of the GUI prototyping approach in a practical rapid prototyping environment.

Focus on the REI process. While GUI prototypes are employed with different characteristics for different scenarios, the focus of early-stage elicitation lies in uncovering the functionality of the system. Therefore, prototypes optimally should possess high-fidelity with regard to the functionality dimension, while the final GUI design plays a less significant role at that stage (McCurdy et al., 2006). In this work, we particularly focus on supporting GUI prototyping with respect to the functionality of the system. As stated before, previous research showed that by employing high-fidelity GUIs for elicitation, the feedback quality and detail are improved (Rudd et al., 1996; Coyette et al.,

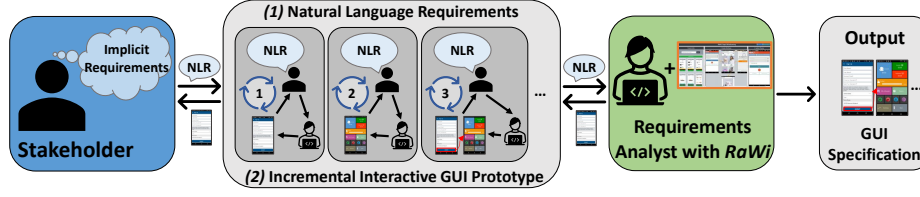


Figure 3.1: REI and RVal via interactive GUI prototyping assisted by our approach *RaWi* to rapidly map NLR into GUI prototypes and iteratively create a GUI specification.

2007; Landay and Myers, 1994). In an interactive elicitation session with stakeholders, the available time for GUI prototyping is typically very limited and requires fast mapping of NL fragments from stakeholders to GUI prototypes with the respective functionality, as shown in Figure 3.1. Here, our *RaWi* approach provides automatic assistance especially for inexperienced analysts. Accordingly, in our conducted experiments we restricted the available time for GUI prototyping and quantified productivity in terms of selecting the necessary GUI components (see Section 3.3.2). Similarly, our process of creating a gold standard for GUI retrieval using crowdsourcing techniques is meant to capture the scenario of stakeholders with no particular experience in GUI prototyping providing NL queries, since participants on crowdsourcing platforms have a variety of backgrounds.

Structure of the chapter. The remainder of this chapter is structured as follows. In Section 3.2, we provide a summary of our GUI retrieval and GUI prototyping approach *RaWi*. Section 3.3 presents the details on the experimental setting used for evaluation, whose results are presented in Section 3.4. Threats to validity and limitations of this work are presented in Section 3.5 and 3.6, respectively. In Section 3.7, we provide an overview of related work. Section 3.8 covers the concluding remarks and presents future work.

3.2 Approach: RaWi

The main goal of our approach is to (i) enable fast GUI retrieval from a large-scale GUI repository via NLR-based search queries to leverage the embodied GUI prototyping knowledge and (ii) to allow users to rapidly build prototypes using automatically derived editable GUI screens. Fig. 3.2 shows an overview of the architecture of *RaWi* separated into four components. First, (A) we employ the large-scale semi-automatically created GUI repository *Rico* (Deka et al., 2017) for mobile applications as the basis for GUI retrieval. This dataset encompasses a large number of *Android* applications that are crawled from the *Google Play* store. To improve the quality of the GUI repository, we initially cleanse the GUIs based on multiple criteria. Afterwards, we create textual representations of the GUIs by extracting different text segments from the GUI hierarchy data. Second, (B) we apply an identical pipeline of text preprocessing techniques to both the textual GUI representations and the NLR-based search queries. In particular, in the context of this approach, NLR refer to short, abstract descriptions representing the

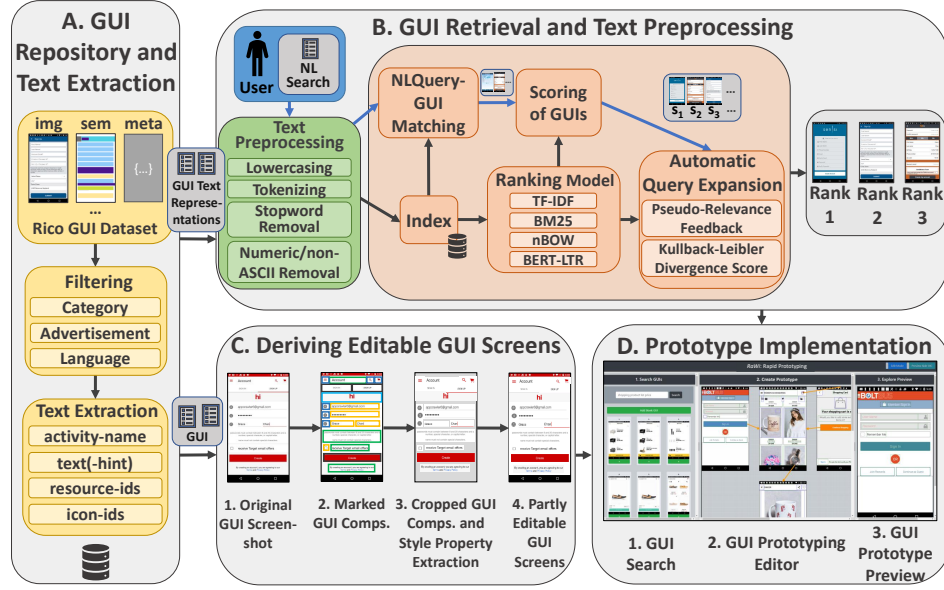


Figure 3.2: Architecture overview of *RaWi* with four components: (A) the large-scale GUI repository *Rico* and GUI text extraction, (B) GUI retrieval and text preprocessing pipeline, (C) automatic GUI screen derivation and (D) the rapid GUI prototyping editor.

main functionality of a GUI (which could also be extracted from USs). The retrieval architecture employed in *RaWi* follows a multi-stage retrieval and reranking pipeline³. To this end, we compute an index over the GUI text representations and match the NLR-based queries against it to obtain initial matches. Subsequently, we compute a ranking over the GUI matches using popular IR and state-of-the-art BERT-based LTR models. In addition, we experimented with multiple AQE techniques. Third, (C) we automatically create partly editable GUI screens by exploiting the GUI screenshots and GUI hierarchy data and extract basic style properties of some GUI components. Finally, (D) we provide a web-based implementation of our approach that integrates all of the previously discussed components. Subsequently, we describe each component in detail.

3.2.1 GUI Repository and Text Extraction

To retrieve relevant GUIs for NLR-based search queries, we first require a suitable GUI repository. Recent data-driven design research constructed and released several GUI datasets appropriate for our approach by automatically crawling a vast number of *Android* applications from *Google Play* such as *ReDraw* (Moran et al., 2018), *ERICA* (Deka et al., 2016), *Rico* (Deka et al., 2017) and *Enrico* (Leiva et al., 2020). *ReDraw* collects GUIs by automatically exploring 5,416 *Android* applications comprising a total of 14,382 unique GUI screenshots and hierarchy data using GUI testing automation techniques to simulate

³While retrieval refers to selecting an initial candidate set of documents across the entire document collection with a fast, high-recall model, reranking refers to computing an improved ranking on a document subset (usually the top- k documents) with a more expressive but computationally more expensive model.

user input. Screenshots of the GUIs are captured either with third-party frameworks or platform-dependent utilities. In contrast, *ERICA* solely relies on manual human-based GUI exploration and is primarily developed for capturing user interaction traces. *ERICA* comprises around 18,600 unique GUIs harvested from 2,400 applications. *Rico*, an extension of the *ERICA* dataset, mines GUI screenshots, GUI hierarchy data, application metadata and interaction traces with both human-based and automatic exploration techniques and constitutes the largest design dataset of the discussed ones with 72,219 GUIs collected from 9,772 unique *Android* applications (from 27 different application categories). *Rico* employs a custom *Android* service to access detailed information about all GUI components such as text, absolute position on the screen, visibility indicators, resource identifiers and the activity name, among others. For our *RaWi* approach, we employ *Rico* for multiple important reasons: (i) the large number of GUIs encompassed, (ii) the coverage of many diverse application domains and (iii) the comprehensive textual information provided, particularly valuable for text-based GUI retrieval approaches.

GUI Filtering. Due to the partly automated extraction of GUIs in *Rico*, the GUI repository naturally contains erroneous GUIs inapplicable to our approach. We recognized multiple GUI types to be excluded through manual inspection of the dataset. First, (1) GUIs of the *entertainment* category (e.g. *gaming*) are discarded since *RaWi* should specifically support rapid prototyping of business and utility applications. Second, (2) GUIs displaying *advertisement overlays* and *full-screen web views* are identified heuristically by particular patterns of component labels and discarded from the repository (based on the presence of a specific number of components and component types, e.g., only *web view*, *web view* and *icons*, etc.). By employing this filtering heuristic, a common hierarchy-GUI-screenshot mismatch error type, in which the GUI screenshot appears to be a normal GUI with many low-level GUI components, but the *Rico* view hierarchy data represents the entire GUI only as a single full-screen web view, is identified and removed. This mismatch occurs often since, e.g., developers directly embed their web apps into *Android* apps instead of developing a native application. Third, (3) we discard non-English GUIs identified through a language detection framework that computes language probabilities by accumulating character-level *n*-gram spelling feature probabilities (Shuyo, 2010). Other languages could be supported by adapting the retrieval models (e.g., by employing multilingual embeddings or simply translating the corpus).

After applying filtering, the GUI repository encompasses 57,764 unique GUIs. Furthermore, research identified several other error types encompassed in the *Rico* GUI dataset (Leiva et al., 2020). Many of these errors are related to component-level mismatches (e.g., vertical offset, missing background image, wrong component type), which do not affect our GUI retrieval methods. However, the *Rico* GUI dataset also rarely contains completely mismatched hierarchy-GUI-screenshot pairs, where the GUI screenshot shows completely different functionality compared to the available view hierarchy. These cases can potentially create False Positives (FPs) during GUI retrieval. However, they cannot be identified easily. Available research provides no evaluation of the extent of their occurrences in *Rico* and based on our manual inspection, the occurrence of these cases



Figure 3.3: Multiple GUI text segments (*activity name* (*act-name*), *icon* (*icon-id*) and *resource identifiers* (*res-id*)) that are extracted and preprocessed for NL-based GUI retrieval.

appears to be infrequent. Moreover, the potential negative impact of erroneous GUIs on the retrieval performance is mitigated by the vast number of available GUIs in the dataset.

GUI Text Extraction. To support GUI retrieval from NLR-based search queries over the GUI repository, we construct a textual representation for each GUI by extracting several text segments through *XPath* (Olteanu et al., 2002) expressions from the GUI hierarchy. First, we extract *displayed text* and *text hints* that are marked as being visible on the captured GUI screenshot. Based on initial experiments, we decided to neglect the extraction of text from components marked as non-visible on the captured GUI screenshot since these text sections often tend to mislead the retrieval models. Similarly, we exploit the *full activity name* of the GUI and the *resource identifier* of each GUI component since developers often provide semantically descriptive naming especially valuable for GUI retrieval. However, these strings (for example `"com.sample.sens.register.CreateNewAccountActivity"`) require additional preprocessing through a tokenization pipeline to make them fully searchable. To this end, we apply *punctuation*, *camel* and *snake case* tokenization and finally use a probabilistic tokenizer based on English Wikipedia unigram frequencies to extract tokens from the remaining parts. To clean the created tokens from non-descriptive and general tokens (for example `"com"`, `"main"` and `"activity"`), we employ a custom domain-specific stopwords list for filtering, containing the most frequent words in the corpus. Furthermore, we exploit the semantic labels of icons provided by *Rico* due to their similarly meaningful semantic descriptions. Fig. 3.3 shows an example GUI retrieved with *RaWi* and multiple text segments that are extracted for retrieval, including the *activity name*, the *icon labels* and the *resource identifiers*. By extracting these dimensions of text, our approach supports both the search for entire GUIs and the retrieval of GUIs that only contain parts relevant to the NLR search query.

3.2.2 GUI Retrieval and Text Preprocessing

For text preprocessing of both the GUI text representations and the NLR input query, we apply a standard preprocessing pipeline. First, we lowercase the text and apply tok-

enization. Tokens are then excluded by multiple filters. We discard *stopwords* and words comprising *numeric* or *non-ASCII* (*American Standard Code for Information Interchange (ASCII)*) characters. Subsequently, we describe the used baseline GUI retrieval techniques, the adapted AQE techniques and the semantic BERT-based LTR models.

Baseline Retrieval Models. For enabling retrieval of relevant GUIs with NLR-based search queries from the introduced *Rico* GUI repository, we adopt well-known baseline retrieval models that showed their effectiveness in various domains before and are established in general-purpose search engines (Manning et al., 2008). Therefore, we decided to adopt the TF-IDF (Salton and Buckley, 1988), BM25 (Robertson et al., 1995) and a TF-IDF-weighted nBoW method (Sheikh et al., 2016; Galke et al., 2017) using pretrained dense word embeddings for similarity scoring to establish a first baseline for GUI ranking.

Automatic Query Expansion (AQE). To further enhance the retrieval performance and tackle the vocabulary mismatch problem (or synonymy), we additionally examined AQE techniques (Manning et al., 2008; Azad and Deepak, 2019). Synonyms such as *"choose"* and *"select"* cannot be matched by traditional IR methods such as BM25 (although the later-described BERT-LTR approach mitigates this problem). AQE methods attempt to extend and refine the initial query automatically with relevant terms in order to improve the ranking performance. In the absence of user feedback, AQE methods employ *PRF*, where the initially retrieved top- k documents (from a base model such as BM25) are assumed to be relevant and thus used as input to compute the query expansion terms. For our GUI retrieval approach, we therefore focused on the incorporation and evaluation of PRF methods. Based on the notion of PRF, many expansion term scoring techniques have been devised in AQE research before (Azad and Deepak, 2019). These scoring techniques generally follow a similar procedure. By comparing the term distributions of term t in the relevant documents D_R and the entire corpus D_C , the importance of a term in the relevant documents D_R can be estimated. The initial NLR query can then be expanded with the top- n terms according to their ranking score. For our GUI retrieval system, we compute the KLD score (Kullback, 1951; Carpineto et al., 2001) for each term $t \in D_R$ as

$$Score_{KLD}(t) = p(t \mid D_R) \cdot \log \frac{p(t \mid D_R)}{p(t \mid D_C)}$$

with $p(t \mid D_R)$ and $p(t \mid D_C)$ being the probability of term t occurring in the relevant documents D_R and in the entire document collection D_C , respectively. To compute these probabilities, we use the *Maximum Likelihood Estimates (MLE)*, i.e. $p(t \mid D_x) = \frac{f_{t,x}}{\sum_{d \in D_x} |d|}$, where d is a document of D_x with $|d|$ tokens and $f_{t,x}$ represents the frequency of term t across all documents in D_x . Applying and evaluating the KLD score in our AQE experiments is based on the notion that this term scoring showed its effectiveness compared to other expansion term scoring methods before (Carpineto et al., 2001). In addition to using this method for expansion term selection solely, we evaluated a second variant that includes the KLD score as a weight for the expanded terms in the retrieval model in order to control their effect on the GUI document ranking. Since

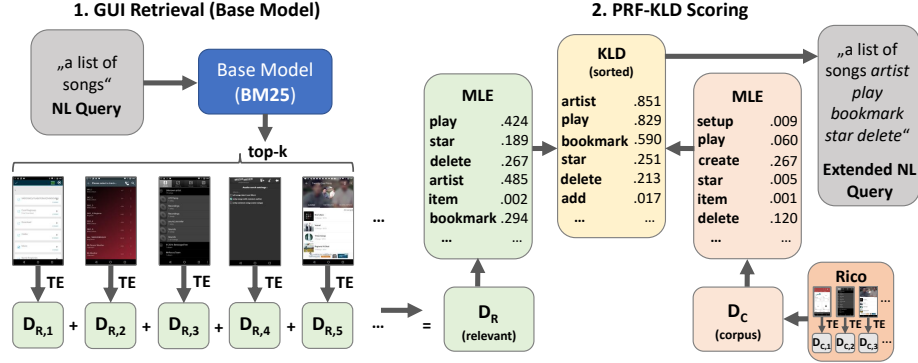


Figure 3.4: Example of using the PRF-KLD scoring: (1) Initial top- k GUI retrieval with a base ranking model (e.g., BM25) and (2) PRF-KLD score calculation based on the MLEs.

the GUI prototypes are represented through multiple text segments, we adapt the PRF-KLD score to compute expansion candidates for each GUI text segment individually and aggregate the top- n terms of the different text segments to obtain the query expansion terms. If duplicate top- n terms from different text segments appear, we only incorporate them once in the expanded query. Similarly to the previously discussed method, we additionally evaluated a second variant of the *text segment-wise* PRF-KLD by weighting the expansion terms with their respective scores to control their influence on the ranking. Figure 3.4 exemplifies our AQE by (1) retrieving the initial top- k GUIs and extracting the text (TE) to obtain D_R , (2) computing the MLEs for D_R and D_C to obtain the scores.

BERT-based LTR Models. The models presented previously can be applied to the GUI text documents, but as motivated earlier, GUI ranking differs from other text-based ranking tasks in multiple aspects. First, GUIs are not standard well-structured text documents, but typically provide solely *brief text fragments* (e.g., button or short label text), *named components* (often with proprietary abbreviations), *many weakly relevant data items* (e.g., product descriptions or names) and *no natural ordering* of the text. Accordingly, the semantic gap between the high-level, abstracted NLR queries and the available GUI text representations is large. Therefore, more sophisticated semantic models are required to further improve the ranking performance. To this end, we finetune a state-of-the-art BERT-based (Devlin et al., 2019) LTR model (Han et al., 2020). BERT is a generally effective pretrained and large-scale language model that outperforms many traditional approaches across different NLP tasks⁴ (before the advent of LLMs).

In particular, LTR approaches represent supervised models that learn a scoring function given a training set $T = \{(\mathbf{x}, \mathbf{y}) \in \mathcal{X}^n \times \mathbb{R}_+^n\}$, with $\mathcal{X}$ representing the space of document-query pairs, $\mathbf{x}$ denoting an n -dimensional vector of individual items x_i and $\mathbf{y}$ denoting an n -dimensional vector of non-negative real-valued relevance labels y_i

⁴Writing the original manuscript and publication occurred before the advent of more powerful LLMs.

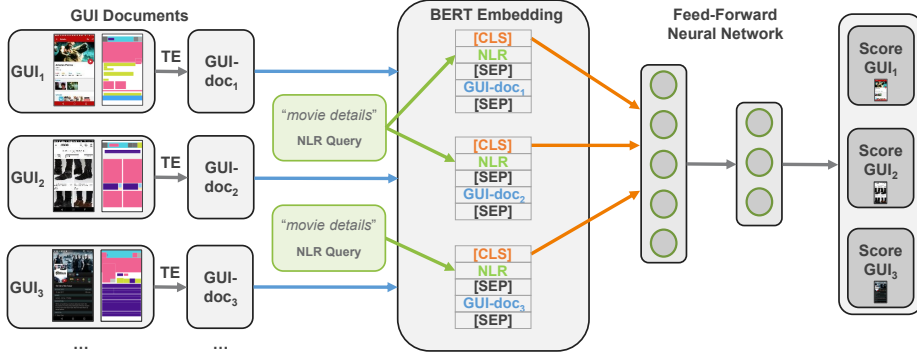


Figure 3.5: BERT-LTR model based on Han et al. (2020) and adapted to the problem of NLR-based GUI ranking using a pretrained BERT model with FFNN output processing.

(Bruch et al., 2019). Here, an item x_i refers to a representation of a query-document pair (q, d) , which is matched with a respective relevance label y_i . The scoring function of LTR models $f_\theta : \mathcal{X}^n \rightarrow \mathbb{R}^n$ is learned by optimizing the parameters θ through employing the training data and should create scores that optimize the ranking of an ordered document list with respect to a query. To create query-document pair representations, we utilize BERT. Since the BERT model requires a specific input structure for the text, we prepare the BERT model by concatenating the [CLS] token (the first BERT-specific token), followed by the NLR query, a separator [SEP] token (the second BERT-specific token) and the potentially truncated GUI document text, followed by a final [SEP] token as the input to the BERT-LTR model. Next, this BERT model input structure is shown:

$$[\text{CLS}] \quad \text{NLR}_{t_1} \quad \text{NLR}_{t_2} \quad \dots \quad [\text{SEP}] \quad \text{GUI}_{t_1} \quad \text{GUI}_{t_2} \quad \dots \quad [\text{SEP}]$$

where NLR_{t_i} refers to token t_i of the NLR input and GUI_{t_j} represents token t_j of the GUI document input, provided by applying the BERT tokenizer to both inputs. Subsequently, the hidden representation h_{CLS} of the special BERT token [CLS] is utilized as input to a ranking model based on a simple FFNN to obtain a ranking score $s(q, d)$ for the pair of query q (i.e. the NLR) and document d (i.e. the GUI text representation), i.e. $s(q, d) = \text{FFNN}(h_{\text{CLS}})$. To finetune BERT and train the ranking model, we experimented with three different approaches by varying the loss function. First, we utilized a *pointwise* approach, which treats each query-document pair independently and forces the model to predict a single document score without comparison or contextualization to other documents in the list. To this end, we utilized the (i) *sigmoid cross-entropy loss* defined as

$$\mathcal{L}_{\text{point}(\theta)} = -\frac{1}{|T|} \sum_T \sum_i \left[y_i \log \sigma(s_i) + (1 - y_i) \log (1 - \sigma(s_i)) \right] \quad (3.1)$$

with $|T|$ referring to the number of examples in the training set and $\sigma(s_i)$ denoting the *sigmoid* function applied to the score s_i . For this particular loss function, the relevance

labels should be binary, i.e. $y_i \in \{0, 1\}$. Since this loss neglects the context of other documents, we experimented with a second variant, namely, a *pairwise* LTR approach. To this end, we applied a different loss function, which compares two documents and therefore incorporates a more relative ranking perspective in contrast to *pointwise* LTR. In particular, we employ a (2) *pairwise logistic loss* to learn the scoring function defined as

$$\mathcal{L}_{pair}(\theta) = \frac{1}{|T|} \sum_T \sum_i \sum_j \mathbb{I}[y_i > y_j] \log(1 + \exp(-(s_i - s_j))) \quad (3.2)$$

with $|T|$ again referring to the number of examples in the training set, $\mathbb{I}(\cdot)$ denoting the *indicator* function, which compares whether label y_i is larger than label y_j , s_i and s_j referring to the scores of query-document pair i and j (having the same query), respectively. For query-document pairs with larger relevance labels compared to other pairs, the model should create scores that are higher in comparison. While comparing query-document pairs forces the model to better optimize the actual ranking between two documents with respect to a query, *listwise* LTR approaches incorporate the entire list of documents for comparison. To this end, we employ a *softmax cross-entropy loss* (Bruch et al., 2019)

$$\mathcal{L}_{list}(\theta) = -\frac{1}{|T|} \sum_T \sum_i y_i \log \left(\frac{\exp(s_i)}{\sum_j \exp(s_j)} \right) \quad (3.3)$$

with $|T|$ again referring to the number of examples in the training set, while s_i and s_j denote the scores of query-document pair i and j (having the same query), respectively. For this loss function, labels y_i have to be projected to a probability distribution, i.e. $\frac{y_i}{\sum_j y_j}$ (Bruch et al., 2019). The resulting values can be interpreted as probabilities that a particular document x_i appears at the top position of the ordered list. This loss function was proposed as part of *ListNet* (Cao et al., 2007). We employ the BERT-LTR model proposed by (Han et al., 2020) and illustrate its application for NLR-based GUI ranking in Figure 3.5. The respective GUIs are first transformed into GUI text documents, concatenated with the NLR query and fed into the BERT model to obtain the BERT representations. To compute the ranking score for a GUI, the embedding is fed through a FFNN. On this basis, the discussed *pointwise*, *pairwise* and *listwise* losses can be computed to optimize the FFNN and finetune the pretrained BERT model. Moreover, as an additional baseline for semantic models, we included a pretrained SBERT model (Reimers and Gurevych, 2019) in our evaluation, in order to better understand the performance gained by the proposed BERT-LTR models, which are trained specifically for the NLR-based GUI ranking task. SBERT was also used as a *text-only* baseline in *Screen2Vec* (Li et al., 2021).

3.2.3 Deriving Editable GUI Prototypes

Since *Rico* (Deka et al., 2017) provides solely GUI screenshots which are not directly reusable for rapid GUI prototyping and editing, we propose a simple yet effective algorithm to transform them into partly editable GUI screens. The automatic derivation of

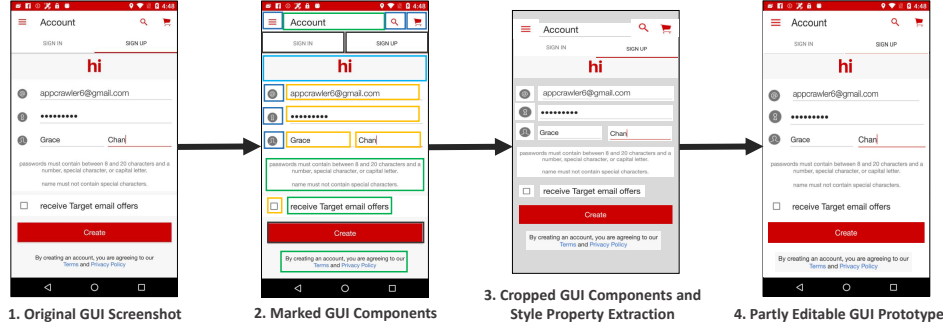


Figure 3.6: Approach for automatically deriving partly editable GUI prototypes from *Rico* (Deka et al., 2017) GUI screenshots and their corresponding GUI view hierarchies.

editable GUIs from the GUI repository is integrated into the graphical editor of *RaWi*, which will be described in more detail in the subsequent section. Figure 3.6 illustrates the entire procedure for creating partly editable GUI prototypes. In particular, we can exploit the GUI hierarchy data and semantic labels for GUI components encompassed in *Rico* to extract them accordingly from the accompanying GUI screenshots. Therefore, (1) we initially obtain the original GUI screenshot provided by *Rico* and (2) crop each contained GUI component from the screenshot based on its absolute position. We applied this procedure to both individual GUI components and additionally identified layout components. Third, (3) for three GUI component types including *labels*, *buttons* and *text-input*, we extract multiple style properties in our current prototype to provide more detailed editing capabilities. For the remaining components, we currently simply reuse their image crop in the editable GUI screens. For each layout group, we compute the background color as the top-ranked RGB color in its histogram. For *labels*, we identify the font color by first obtaining the background color as before and then computing a normalized color distance to the background color for all colors in the histogram. The first top-ranked color in the histogram that exceeds a predefined distance threshold is selected as the font color, being a simple yet effective heuristic. In addition, we estimate the font size by comparing the bounding boxes from the hierarchy data and an instantiated bounding box containing the text. Since *Rico* often provides only rough bounding boxes for the *labels*, we compute refined bounding boxes using Tesseract-OCR (Smith, 2007). For *buttons* and *text-inputs*, we similarly extract the background and font colors. We enrich *Rico* with the identified style properties to enable proper GUI component instantiation in the editor later. Finally, (4) the cropped GUI components are placed on their exact GUI screen positions. Subsequently, we describe our prototypical tool implementation.

3.2.4 Prototype Implementation

The presented GUI retrieval and GUI prototype derivation components are integrated in a tool prototype for a completely data-driven, web-based rapid prototyping editor using HTML, CSS and JS. Our current prototype provides (1) a *GUI search view*, (2) a *graphical prototyping editor* and (3) an interactive and dynamically updated *preview* of the created

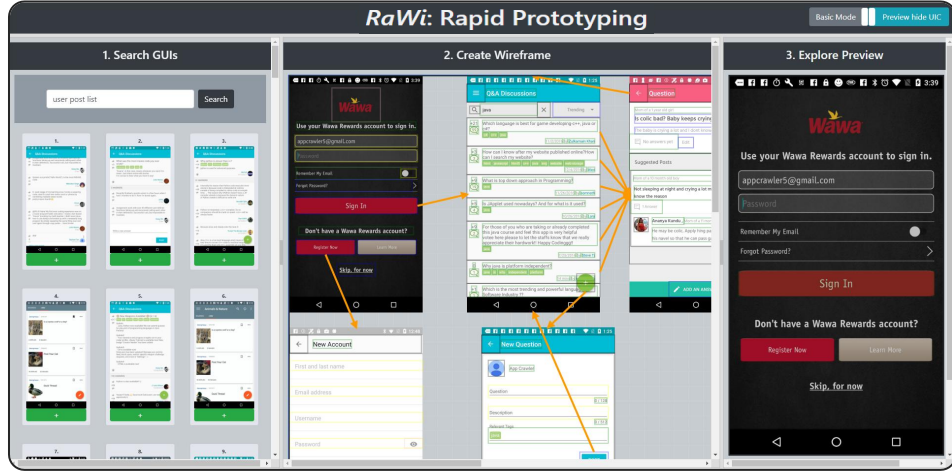


Figure 3.7: Web-based GUI prototyping editor of our approach *RaWi* with three components: (1) the *GUI search view*, (2) the *graphical editor* and (3) the *prototype preview*.

application, as illustrated in Figure 3.7. Subsequently, we provide more details about the features of our current prototypical tool implementation and each of the components.

NLR-based GUI Search. NLR-based search queries can be submitted by users through a simple search interface with autocompletion capabilities. The *Python*-based GUI retrieval component is accessible through a *Representational State Transfer (REST)* interface, which is implemented as a *Django* application. For efficiency, the default is BM25.

GUI Prototyping Editor. Users are enabled to add retrieved GUI prototypes to the graphical prototyping editor. Here, the previously created enriched GUI hierarchy data is employed to create the editable GUI prototypes. Initially, we place all the image crops of layout and individual GUI components at their absolute positions on the GUI screen to preserve the complete original appearance of the GUI. By clicking on *labels*, *buttons* or *text-input* components, an editable version of the GUI component is instantiated and basic properties such as *text*, *color* and *size* can be modified. Furthermore, users can create custom GUI components directly or reuse different components from other retrieved GUI screens. In addition, we currently provide basic editing functionality: GUI components can be *removed*, *copied* and *bound* to other screens. To create an interactive prototype, *RaWi* provides the capabilities to create simple click-event transitions on GUI components. The editor is based on the *2D-canvas* JS library *KonvaJS* (Konva.js).

GUI Prototype Preview. We provide a dynamically updated preview of the created prototype showing all modifications similar to traditional prototyping approaches. The preview enables users to directly explore an interactive version of the created application prototype, which is based on the previously created GUI transitions as the foundation.

3.3 Experimental Evaluation

In this section, we present the design and methodology of our experimental procedure to evaluate *RaWi*. In particular, the main goals of our experimental evaluation are (i) to measure the performance of the employed GUI retrieval techniques and conduct a comprehensive comparison between them on a newly created gold standard, (ii) to assess the productivity improvements of *RaWi* compared to a traditional rapid prototyping approach in a practical GUI prototyping environment and (iii) to measure the perceived usefulness and gain user insights for our approach. To this end, we investigate the following three research questions in the subsequent experimental evaluation:

- **RQ₁:** *Which method performs best for GUI retrieval on the basis of NLR search queries?* To answer this question, we compare the presented baseline, adapted AQE techniques (particularly PRF-KLD scoring methods), SBERT and the three BERT-LTR variants. To evaluate the effectiveness, we utilize standard IR metrics such as *Average Precision (AP)* and *Mean Reciprocal Rank (MRR)*, among others.
- **RQ₂:** *Does RaWi increase the GUI prototyping productivity compared to a traditional prototyping tool?* To answer this question, we compare our proposed data-driven prototyping tool against *MockPlus* (Mockplus), which represents a popular, traditional prototyping approach. To measure productivity gains, we conduct a *within-subjects* user study and compare the created GUI prototypes.
- **RQ₃:** *Do users perceive RaWi as useful for rapid high-fidelity GUI prototyping?* To answer this question, we asked participants several questions on a five-point Likert scale questionnaire regarding the perceived usefulness of the data-driven prototyping tool. Additionally, we computed the *System Usability Scale (SUS)*.

3.3.1 RQ₁: GUI Retrieval Performance

Gold Standard. Due to the absence of an evaluation dataset for GUI retrieval from NLR queries, we decided to build a new and comprehensive gold standard through crowdsourcing techniques (Carvalho et al., 2011) and publicly release it to foster future research. The construction of the retrieval gold standard is divided into three main phases: (i) the collection of NLR queries, (ii) the collection of relevance annotations for potentially relevant GUIs with regard to NLR queries and (iii) the derivation of the final gold standard via data cleansing. To accomplish the large-scale data collection, we used the well-established crowdsourcing platform Amazon Mechanical Turk (AMT) (Paolacci et al., 2010). The GUI retrieval gold standard is available at our accompanying repository, including detailed descriptive statistics of the dataset (such as the *distribution of contained app categories*, the *average number of tokens per query*), which will also be discussed briefly at the end of this section. Subsequently, we explain the three phases of the gold standard construction in detail as shown in Figure 3.8.

First, (i) the collection of NLR queries poses a challenge due to the difficulties of assuring high data quality for free-form NL-based crowdsourcing tasks (Rashtchian et al., 2010). Therefore, we decided to employ AMT for data collection but restricted the tasks

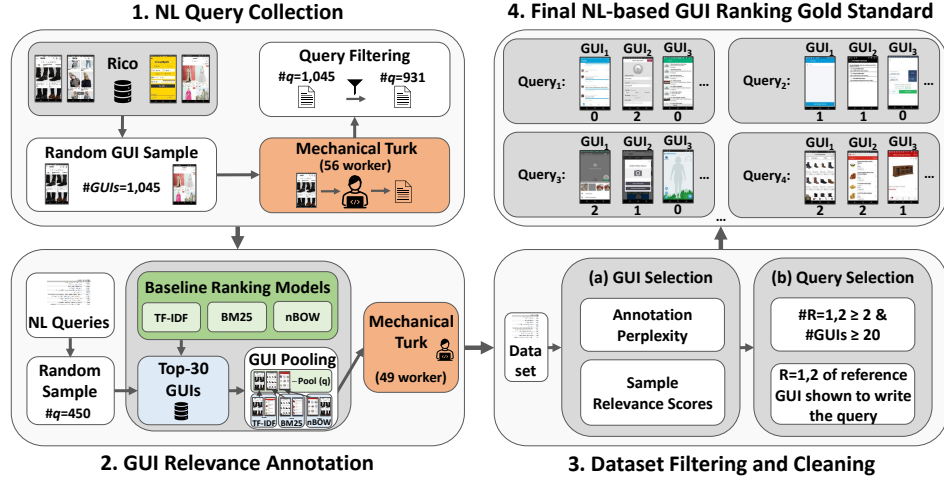


Figure 3.8: Overview of the pipeline to create the gold standard for NLR-based GUI retrieval using crowdsourcing with the three main steps: (1) NLR query collection through AMT, (2) GUI relevance annotation through AMT and (3) dataset filtering and cleaning.

to workers that we instructed (hired) specifically for the completion of this task. These annotators represent a good proxy for the population in our scenario, since workers on AMT typically have diverse demographics (Ross et al., 2010; Difallah et al., 2018) (potential threats to validity on worker selection are discussed later). As the foundation for writing queries and to avoid bias, we randomly sampled GUI screenshots from the filtered *Rico* dataset and asked the workers to write queries for them in a *Human Intelligence Task (HIT)*. Overall, for each of 1,045 unique GUIs from the sample, we gathered one unique NLR query written by one of our overall 56 unique workers (resulting in 1,045 GUI-NLR-query pairs). For additional quality assurance, we manually reviewed the queries and filtered six individual queries for GUIs that were erroneous or unusable screenshots from *Rico* (e.g. *white screen*, *foreign language* not detected by the language detection framework, among others) and all queries from six workers that made systematic errors due to misunderstandings of the task. By filtering these queries, solely apparently erroneous NLR queries were discarded. Thus, no bias is introduced, but the quality of the dataset is enhanced. After filtering, we retain 931 GUI-NLR-query pairs with queries written by 50 unique workers and apply basic text preprocessing methods (*dequoting*, *stripping* and *removal* of special characters). By employing AMT for query collection, we are enabled to gather a large number of different NLR queries from a variety of annotators, thereby increasing the heterogeneity of the NLR queries the retrieval and ranking models have to handle. Four GUI-NLR-query pairs taken from the gold standard are shown in Figure 3.9. The examples illustrate the diversity of the gathered NLR queries ranging from short and general queries (e.g., Figure 3.9 (a)) to long and detailed search queries (e.g., Figure 3.9 (c)) across a broad selection of different application domains.

Second, (ii) to accomplish the collection of relevance annotations, we initially require a collection of potentially relevant GUI documents to be annotated for each query. Due

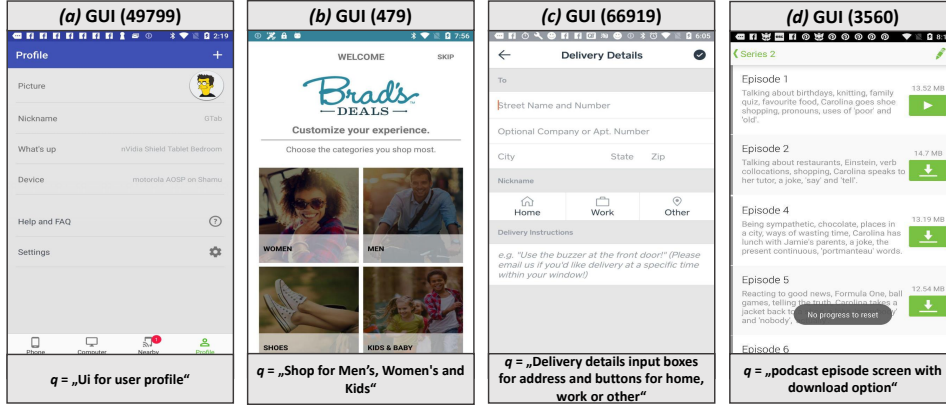


Figure 3.9: Four (a)-(d) example GUI-NLR-query pairs taken from the gold standard with the NLR queries gathered via crowdsourcing and the respective *Rico* GUI indices.

to the infeasibility of annotating the relevance of every GUI in *Rico* for each query, we decided to heuristically employ our three baseline models (*TF-IDF*, *BM₂₅* and *nBoW*) to retrieve the potentially relevant top-30 GUIs for each query. To retain a diverse and heterogeneous GUI collection for each query, we apply the pooling technique (Jones and Van Rijsbergen, 1976; Teufel, 2007; Tonon et al., 2015; Lipani, 2016) to the GUI retrieval results of the three models by iterating over them in an alternating fashion and adding the top-ranked GUI not yet contained in the pool to the GUI collection. Moreover, we directly added the reference GUI that we previously showed to workers during the NLR query harvesting to the pool of potentially relevant GUI documents. The pooling technique entails the disadvantage of potentially excluding some relevant documents in the collection. However, it represents a standard procedure for constructing large IR test collections by overcoming the infeasibility of annotating the large-scale corpus entirely.

To keep the data collection feasible while ensuring a sufficient amount of data for a valid retrieval evaluation, we randomly sampled 450 queries from our previously created query collection, computed the pooled top-30 GUIs for each query with the approach described previously and published another HIT on AMT that asked workers to provide relevance annotations. Due to the subjectivity of the relevance $\mathbf{R}$, each GUI was annotated by three different workers on a relevance scale of $\mathbf{R} = 0$ (*low relevance*), $\mathbf{R} = 1$ (*medium relevance*) and $\mathbf{R} = 2$ (*high relevance*). All 30 GUIs of an NLR query were annotated exactly three times, resulting in 90 GUI relevance annotations per NLR query by three distinct annotators. As guidance for the annotators, we provided multiple annotation examples for the relevance scale and included them in our accompanying repository. To assure high annotation quality, we restricted the task solely to workers that met multiple qualifications ($\#Approved-HITs \geq 1,000$, $Approval-Rate \geq 90\%$, Residence in an English-speaking country (*USA/GB/AU*)), as previous research found these metrics particularly effective for enhancing the annotation quality (Akkaya et al., 2010; Peer et al., 2014). In addition to these generic AMT platform requirements calculated based on the

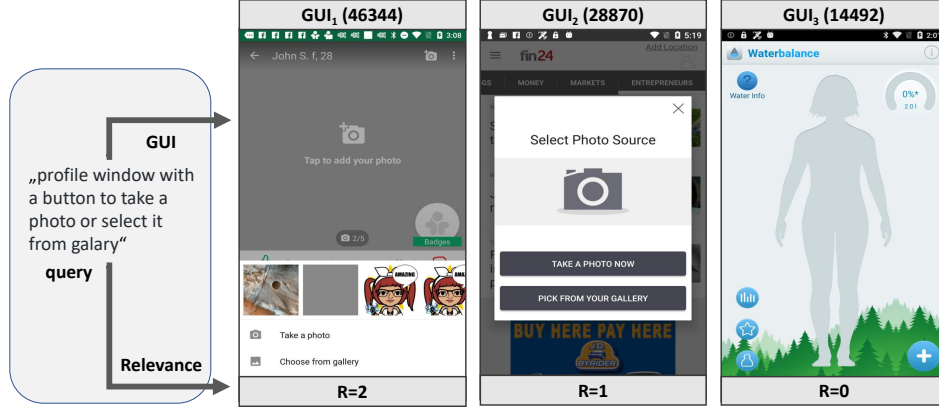


Figure 3.10: Example of an NLR query from the gold standard with three GUIs, each annotated with their relevance, according to the majority voting from crowdsourcing.

workers' history, we created a custom qualification test to assess the comprehension of the relevance criteria and the overall task of the annotators on a small set of GUI-NLR-query pairs, which simultaneously acted as a training phase. Workers were required to meet a certain performance threshold to be allowed to work on the annotation task. Furthermore, we informed the workers that submissions were rejected if a submission scored below $\frac{2}{3}$ agreement with the majority voting of a query. Overall, we gathered a total of 40,500 individual relevance annotations (for 450 NLR queries with each of the 30 GUIs annotated three times) from 49 unique workers.

Third, (iii) to obtain a high-quality gold standard from the previously harvested dataset, we run a pipeline of selection steps described subsequently. Due to the filtering, the final gold standard comprises a reduced set of 100 queries, each with their top-20 GUIs. To increase data quality, we removed GUIs with high annotation perplexity. In particular, we discarded GUIs with annotation ties or annotations that diverged to the extremes, for example, two votes for $\mathbf{R} = 0$ and one vote for $\mathbf{R} = 2$ or vice versa, since these annotations indicate high uncertainty in deciding for the ground truth. Annotators may have different opinions on relevance and are more or less strict in close annotation cases. Therefore, these annotation ties cannot be resolved and the respective GUIs are excluded. Afterwards, we set a suitable predefined ratio of different relevance scores per query: $14 \times \mathbf{R} = 0$, $3 \times \mathbf{R} = 1$ and $3 \times \mathbf{R} = 2$. This provides a typical relevance distribution with mainly non-relevant GUIs while ensuring a small number of relevant GUIs. For each query, we randomly sampled GUIs with the respective relevance scores according to the previously defined quantities. If the quantity for a relevance score could not be reached, we randomly sampled from the other scores to obtain 20 GUIs per query. However, we only kept queries that have a minimum of two $\mathbf{R} = 1$ GUIs, two $\mathbf{R} = 2$ GUIs and 20 GUIs remaining. To obtain the same amount of GUIs per query, we decided to set the cutoff at the top-20. In addition, we discarded queries where the reference GUI that was used to write the query did not receive a relevance score of

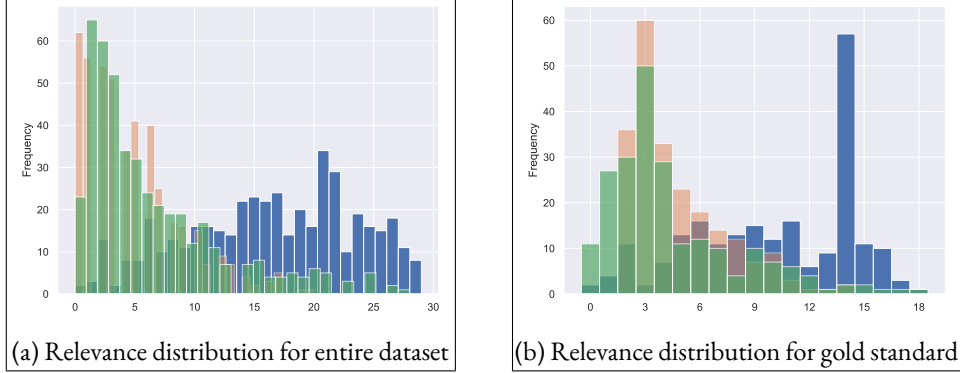


Figure 3.11: Distributions of the different relevance levels (*blue* = *low*, *yellow* = *medium*, *green* = *high*) for (a) the entire dataset ($k = 30$) and (b) the final gold standard ($k = 20$).

Variable	n	μ	σ	min	Q1	med.	Q3	max
(1) #Queries/Annotator	50	18.62	7.33	1	15.25	20.00	21.75	35
(2) #Annotations/Annotator	49	55.10	105.72	1	2.00	10.00	55.00	569
(3) #Words/Query	931	6.17	3.69	1	4.00	5.00	8.00	29
(4) #Components/GUI	931	17.52	13.75	1	7.00	13.00	24.00	104
(5) #Words/GUI	931	180.88	200.31	4	56.50	118.00	229.50	2464

Table 3.1: Summary statistics (*mean* μ , *standard deviation* σ , *minimum*, *first quartile* **Q1**, *median*, *third quartile* **Q3**, *maximum*) for the collected dataset: (1) number of queries written per annotator, (2) number of relevance annotations per annotator, (3) number of words per query, (4) number of components per GUI and (5) number of words per GUI.

$\mathbf{R} = 1$ or $\mathbf{R} = 2$ from the human relevance annotators, as a query sanity check. The rationale for discarding these queries is to ensure high quality in the written NLR queries. If the NLR queries do not match the shown reference GUIs (based on the relevance annotators' judgments), the query is potentially of low quality (e.g., the annotator that wrote the query did not understand the GUI, provided an ambiguous query, etc.). From the remaining query collection, we randomly sampled 100 queries to obtain the final gold standard. To illustrate the gold standard, an example NLR query with three GUIs and their respective ground truth relevance annotations is illustrated in Figure 3.10. Each of the GUIs comes with a different relevance score with respect to the NLR query. For example, GUI₁ is annotated with $\mathbf{R} = 2$ since it fully corresponds to the functionality requested in the query. However, GUI₂ only receives a relevance of $\mathbf{R} = 1$ since the GUI contains the requested photo selection options, but the functionality is not embedded in a user profile context. Finally, GUI₃ is annotated with a relevance of $\mathbf{R} = 0$ since it is unrelated to the query showing a user profile for daily water management.

To gain insights into the overall collected dataset and gold standard, we computed several summary statistics shown in Table 3.1. For example, annotators ($n=50$) wrote 18.62 ($\sigma=7.33$) NLR queries on average, while the other annotators ($n=49$) provided 55.1

($\sigma=105.72$) relevance annotations on average. In addition, the collected NLR queries consist of 6.17 ($\sigma=3.69$) words on average in our dataset, which indicates higher information need complexity and specificity for NLR-based GUI searches in comparison to typical search queries, where most of the queries consist of only up to three words (Phan et al., 2007). Moreover, sampled GUIs encompass 17.52 ($\sigma=13.75$) components on average, which provides a simplified proxy to GUI complexity. Furthermore, based on the applied GUI sampling approach, the distribution of included app categories is similar to the distribution of the original *Rico* dataset, which corresponds to high diversity (26 app categories). High lexical diversity is also achieved among the queries, as indicated by a very small *average pairwise Jaccard similarity* (Jaccard, 1901) (2.7%) over all queries in the gold standard. Figure 3.11 shows the relevance level distributions for the entire dataset (Figure 3.11a) and the final gold standard (Figure 3.11b), both illustrating typical distributions with only a few queries having large numbers of *high* (and *medium*) relevance GUIs, but usually larger numbers of *low* relevance GUIs. To assess the *Inter-Annotator Agreement (IAA)* of the gold standard, we computed *Krippendorff's α* (Krippendorff, 2011), a chance-corrected IAA metric supporting ordinal data (such as relevance levels). As a result, we obtain a score of $\alpha=0.67$, which indicates substantial agreement according to different score interpretation methods (Landis and Koch, 1977; Kraemer et al., 2012).

Ranking Model Parameters. To evaluate the various retrieval and ranking models in our experiments, we utilized TF-IDF, BM25 ($k_1=1.5$, $b=0.75$, $\epsilon=0.25$ as standard parameters (Manning et al., 2008)) and nBoW (using 300-dimensional *word2vec* embeddings (Mikolov et al., 2013b)) as baselines. The four considered AQE methods based on PRF-KLD are evaluated using BM25 as the base ranking model. In particular, we restricted the number of documents employed for expansion term computation to $k=10$ and set the number of added expansion terms to $n=10$. We used a similar setup for the *text-segment-wise* PRF-KLD variants, but restricted the number of expansion terms per text segment to $n_{ts}=2$. For brevity, we refer to the *text-segment-wise* PRF-KLD method as (*s*) and to the *weighted* variants as (*w*) in the evaluation results. For the BERT-LTR models, we adopted the pretrained large BERT-base version (Devlin et al., 2019) with a vector size of 768 and applied finetuning using the relevance annotations of the remaining 350 queries (9,495 instances) not contained in the gold standard. We employed the *TensorFlow*-Ranking BERT-Extension for training (Han et al., 2020) and trained each of the three model variants for 30,000 iterations (*learning-rate* = 1×10^{-5} and *batch-size* = 1). For the additional pretrained, semantic SBERT baseline (Reimers and Gurevych, 2019), we employed the full pretrained 768-dimensional model with *cosine similarity* to obtain a GUI ranking.

Evaluation Metrics. To measure the performance of the GUI ranking models, we computed several well-known IR metrics. First, (1) we computed the *Precision at rank k* ($P@k$) as the fraction of relevant GUI documents $|R_k|$ over the number of all retrieved GUI documents k . Second, (2) we computed the *Average Precision (AP)*, defined as the average of $P@k$ values at all relevant GUI document positions with rel_k being an

indicator for the relevance at position k and $|R|$ the number of relevant documents. In particular, the described $P@k$ and AP metrics are computed as shown in the following:

$$P@k = \frac{|R_k|}{k} \quad (3.4) \quad AP = \frac{\sum_{k=1}^n P@k \times rel_k}{|R|} \quad (3.5)$$

The AP is averaged over the dataset (in research often denoted as *Mean Average Precision (MAP)*). Third, (3) we computed the *Mean Reciprocal Rank (MRR)*, where $rank_i$ denotes the rank of the highest ranked relevant GUI document of the i -th query. In addition, (4) we computed the $HITS@k$, indicating whether at least one relevant GUI document is ranked among the top- k ranked documents. To compute these metrics (1)-(4) that require binary relevance annotations on the employed relevance scale, we binarize the relevance scores beforehand, thereby considering only *high* relevance scores (i.e. $\mathbf{R} = \mathbf{2}$) as relevant. In particular, the MRR and $HITS@k$ metrics are computed as follows:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i} \quad (3.6) \quad HITS@k = \begin{cases} 1 & |R_k| > 0 \\ 0 & |R_k| = 0 \end{cases} \quad (3.7)$$

with R_k denoting the set of relevant GUI documents up to rank k . In order to further take into account the full annotation scale and the ranking position of GUI documents with *high* and *medium* relevance (i.e. $\mathbf{R} = \mathbf{2}$ or $\mathbf{R} = \mathbf{1}$), we additionally considered (5) the *Discounted Cumulative Gain (DCG) at rank k ($DCG@k$)*. Relevant GUI documents ranked higher in the retrieval collection are more valuable, and the relevance score should accordingly be weighted more than the score of GUI documents at lower-ranked positions. Hence, in $DCG@k$, each relevance score is discounted by a weight (log-based smoothing), which is computed based on the ranking position and decreases with lower ranking positions. The $DCG@k$ is normalized by the $IDCG@k$ (*Ideal DCG@k*) to obtain (6) the *Normalized Discounted Cumulative Gain (NDCG) at rank k ($NDCG@k$)*, which is employed in our evaluation. The $IDCG@k$ is computed as the $DCG@k$ of relevance scores sorted in descending order (perfect ranking). To compute the final results, the metrics are averaged over all queries of the gold standard. Subsequently, the $DCG@k$ and $NDCG@k$ metrics are shown, with rel_i denoting the relevance score at the i -th rank:

$$DCG@k = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)} \quad (3.8) \quad NDCG@k = \frac{DCG@k}{IDCG@k} \quad (3.9)$$

To enable detection of statistically significant differences, we apply the non-parametric *Wilcoxon signed-rank test* (Woolson, 2007) with per-metric *Holm* correction across model comparisons to account for multiple pairwise model comparisons. Since BM25 is considered one of the strongest baseline IR models, we first compare BM25 to the other two baseline models TF-IDF and nBoW. Second, to determine any improvement of the AQE methods over BM25, we compare the four PRF-KLD variants against BM25. Third, to

evaluate whether the BERT-based models improve over the strong baseline BM₂₅, we compare the four BERT models against BM₂₅. Since SBERT acts as a pretrained, semantic baseline, we compare for differences between SBERT and the BERT-LTR variants.

3.3.2 RQ₂: Productivity of Rapid Prototyping

User Study Design. To measure potential productivity improvements of our data-driven GUI prototyping approach, we conducted a controlled experiment with 19 participants as a *within-subjects* study. In the experiment, we compared our approach to a traditional high-fidelity GUI prototyping approach. With regard to the discussed REI scenario, participants in the role of analysts were asked to create two different application prototypes (consisting of two GUIs each) on the basis of typical NLR requirements, using each approach separately to mitigate *carry-over* effects. To avoid bias from both the tasks and the ordering of the approaches, we considered all four combinations of the approaches A/B and the tasks 1/2 (A₁B₂/A₂B₁/B₁A₂/B₂A₁) and assigned the conditions to participants randomly while ensuring that the occurrence of conditions is evenly distributed. As the approach for comparison, we employed *Mockplus* (Mockplus) since it can be regarded as a representative tool for traditional GUI prototyping approaches, providing a restricted number of manually crafted GUI screens to reuse and enabling users to create GUIs by combining a large number of searchable GUI components in a graphical editor. Overall, we recruited 19 participants with technical backgrounds (BSc:7/MSc:8/PhD:4) mainly having *medium* to *high* experience in software development (M:3.52/SD:1.02) and mainly having *little* to *medium* GUI prototyping experience (M:2.42/SD:0.96), as self-reported by the study participants on a five-point Likert scale questionnaire. Most of the participants (13) reported that they have used a prototyping tool before, with the most frequently mentioned ones being general-purpose tools such as *PowerPoint* (9), *Photoshop* (2) and *Adobe XD* (2). Therefore, the participants are representative regarding the elicitation scenario, especially targeting support for inexperienced requirements analysts.

User Study Tasks. Both GUI prototyping tasks are based on real *Android* applications within the top-50 apps on *Google Play*, which are not included in the *Rico* GUI repository to avoid bias. The first task represents a *shopping browser* consisting of a start GUI asking the user to select from a list of countries (#GUI-comps: 25) and the actual main shopping browser GUI that allows users to search for websites and displays favorite websites (#GUI-comps: 33), among others. The second task represents a *hotel booking* application consisting of a GUI that allows users to enter search details (#GUI-comps: 25) and a GUI that shows a list of search results (#GUI-comps: 29). We omit the detailed task descriptions and NLR requirements for brevity. However, we provide all study instructions among the other publicly released materials in our accompanying repository. We decided to employ these GUIs in our experiments due to multiple reasons. First, (i) the GUIs contain a large number of different GUI components, thus making them difficult to create, especially for novice users. Second, (ii) both approaches *RaWi* and *Mockplus* cover the shopping and booking application domains with basic GUI prototypes allowing for a fair comparison.

Experimental Procedure. In the study, we asked the participants to create a GUI prototype using each approach separately. Depending on the assigned condition, the participant started with the first approach and task. Before working on the actual task, we showed a short tutorial video of the approach with a GUI prototyping example and allowed the participants as much time as they needed to test the approach. Afterwards, the actual task was conducted. With regard to the interactive GUI prototyping scenario with stakeholders for elicitation, the available time for prototyping is strictly limited. Therefore, we accordingly restricted the available time for creating the GUI prototypes to several minutes, with a maximum of seven minutes. To obtain a fine-grained productivity measurement, we captured screenshots of the current GUI prototype state after each minute starting from three minutes and thus collected five screenshots per GUI prototype. These screenshots formed the basis for our post-experiment data analysis. After completing the first task, we continued with the identical procedure for the second approach and task combination. Overall, we gathered 20 GUI screenshots per participant.

Evaluation Metrics. From the gathered screenshots, we computed multiple metrics. However, no prior research proposed metrics for assessing GUI prototyping productivity before. In particular, a closely related problem on GUI implementation effort estimation (Lo et al., 1996) identifies especially the number and type of GUI components as valuable factors for predicting the overall GUI implementation effort. In a similar fashion, GUI prototyping in the REI phase asks analysts to quickly select and arrange GUI components that properly reflect the requirements specified by the stakeholder. Here, one or more GUI components represent a particular functional requirement and, therefore, we consider the number of GUI components as a proxy for GUI prototyping productivity.

Thus, we first (a) count the number of *correct* GUI components (based on the GUI component type, e.g., using a *password field* instead of a plain *text field* for the functionality to enter passwords) available in the considered GUI prototype at time t that are appropriate for the given requirements as an approximation for productivity (denoted by $\#GUI-comps$). GUI components that overlapped the GUI screen or other GUI components were not included in the count. Detailed design decisions (e.g., placing a button bar on top or bottom or similar) are apparently neglected in this metric, since in the elicitation phase, the focus lies on quickly representing the main functionality and not the final GUI design. Different working styles and other relevant individual factors of participants that influence the outcome are considered by our *within-subject* design, comparing productivity only between the participants themselves. Second, (b) we count the number of GUI components as before, but data elements (e.g., *images* or *text*) are only counted multiple times if they contain realistic and diverse data (denoted by $\#GUI-comps-div$), accounting for data model fidelity (see McCurdy et al. (2006)). We considered computing this corrected count since having high content diversity through realistic data is important for more realistic high-fidelity GUI prototypes. Third, (c) we count the number of *incorrect* GUI components available at time t that were not included in the requirements (denoted by $\#GUI-comps-neg$). Finally, (d) we compute an adjusted count where including non-required components is punished by taking the count of

Table 3.2: Evaluation results overview of the different models on the NLR-based GUI retrieval gold standard using $P@k$ and $NDCG@k$ (**Bold** values indicate best metric score within result group and underlined values indicate best metric score across all groups).

	P@k				NDCG@k (N@k)			
	P@3	P@5	P@7	P@10	N@3	N@5	N@10	N@15
TF-IDF	22.3	20.4	18.1	17.5	32.9	33.9	39.5	48.0
BM25	30.3	27.6	24.6	22.6	42.6	44.1	51.5	57.9
nBoW	27.0	23.4	22.0	19.3	39.5	37.0	37.4	39.8
BM25	30.3	27.6	24.6	22.6	42.6	44.1	51.5	57.9
+PRF	31.3	26.6	25.9	23.6	43.2	44.3	52.0	58.4
+PRF (<i>s</i>)	31.7	28.0	26.0	23.5	44.1	46.2	53.6	60.4
+PRF (<i>w</i>)	32.0	27.6	24.4	23.1	43.9	41.7	42.1	44.6
+PRF (<i>sw</i>)	31.7	28.0	25.6	23.7	42.9	41.6	42.6	45.6
SBERT	34.3	31.4	29.4	26.7	48.1	51.1	61.1	66.7
BERT-LTR (<i>r</i>)	37.7	35.0	30.7	26.9	53.0	56.0	63.4	69.7
BERT-LTR (<i>z</i>)	40.0	34.0	30.4	28.1	54.3	55.6	63.6	70.1
BERT-LTR (<i>z</i>)	36.3	<u>35.4</u>	<u>31.7</u>	<u>28.7</u>	51.7	55.4	64.6	69.4

required GUI components (*a*) minus the count of GUI components not included in the requirements (*c*). We aim to compare the prototyping productivity between the two approaches at each time step *t*. To evaluate if the productivity is significantly different between approaches, we compute the *Wilcoxon signed-rank test* (Woolson, 2007) over all pairs of study tasks (*Holm*-corrected). We compare the productivity of each participant between both first GUIs of the tasks and both second GUIs of the tasks. Moreover, we compute *Cliff's delta* δ (Macbeth et al., 2011) to estimate the overall statistical effect size.

3.3.3 RQ₃: Perceived Usefulness

In addition to measuring the GUI prototyping productivity, we assessed the perceived usefulness of our GUI prototyping approach by the study participants. Using a five-point Likert scale, we asked participants (*a*) how much our approach supported them during the prototyping process, (*b*) how relevant the retrieved GUIs were on average, (*c*) how much the GUI retrieval results supported the users during the prototyping and (*d*) how much the GUI search results helped better visualize how the GUI could be prototyped. Moreover, we computed the *System Usability Scale (SUS)* (Brooke, 1996b), which is a validated, reliable and well-known measurement for system usability. To gain further insights, we asked the participants for free-form feedback to potentially discover current issues and interesting ideas for improvements to our novel GUI prototyping approach.

3.4 Results & Discussion

3.4.1 RQ₁: GUI Retrieval Performance

Table 3.2 shows the evaluation results of the different baseline ranking models, AQE techniques and BERT-based LTR models for the $P@k$ and $NDCG@k$ metrics, whereas Table

Table 3.3: Evaluation results overview of the different models on the NLR-based GUI retrieval gold standard using AP , MRR and $HITS@k$ (**Bold** values indicate best metric score within result group, underlined values indicate best metric score across all groups).

	AP	MRR	HITS@k (H@k)					
	AP	MRR	H@1	H@3	H@5	H@7	H@10	H@15
TF-IDF	33.1	45.1	32.0	46.0	58.0	65.0	76.0	91.0
BM25	41.3	52.0	37.0	60.0	69.0	76.0	86.0	93.0
nBoW	28.1	49.0	34.0	54.0	63.0	75.0	84.0	98.0
BM25	41.3	52.0	37.0	60.0	69.0	76.0	86.0	93.0
+PRF	41.9	50.5	37.0	58.0	68.0	78.0	85.0	93.0
+PRF (s)	42.7	53.2	38.0	61.0	70.0	78.0	88.0	96.0
+PRF (w)	32.5	52.3	38.0	58.0	70.0	72.0	86.0	93.0
+PRF (sw)	33.3	53.3	39.0	59.0	70.0	77.0	87.0	93.0
SBERT	45.4	56.0	37.0	68.0	76.0	88.0	96.0	99.0
BERT-LTR (r)	48.6	61.8	46.0	71.0	86.0	92.0	98.0	100.0
BERT-LTR (z)	50.1	63.1	44.0	75.0	91.0	96.0	98.0	100.0
BERT-LTR (β)	49.9	62.6	45.0	73.0	86.0	94.0	100.0	100.0

3.3 shows the evaluation results for the AP , MRR and $HITS@k$ metrics. Considering the three baseline ranking models, we can observe that BM25 outperforms the other two models substantially across most of the examined mean metric values, while statistically significant differences between BM25 and TF-IDF are observed for AP , $P@5,7,10$ and $NDCG@5,10,15$, and between BM25 and nBoW for AP and $NDCG@10,15$ (cf. Appendix B Tables B.1, B.2 and B.3). In particular, the MRR of the BM25 model indicates that the first relevant GUI appears at rank 1.92 on average over the gold standard. Moreover, the $HITS@5$ indicates that in 69% of the queries, the BM25 model ranked at least one relevant GUI among the top-5. Thus, BM25 seems to be an effective baseline model for NLR-based GUI retrieval. Comparing the plain TF-IDF and nBoW models, we can observe that especially the mean $P@k$ and $HITS@k$ as well as the $NDCG@k$ for small k are higher for the nBoW model, indicating that pretrained word embeddings can provide additional benefit for GUI retrieval from NLR-based searches.

In addition, both tables show the results of the four different AQE methods based on the PRF-KLD score using the strong baseline ranking model BM25 as the base ranker. The results show that the text-segment-wise PRF (s) method obtains the highest scores among most of the considered mean metric values and outperforms the BM25 base model across all mean metric values. Both weighted PRF-KLD variants can outperform the PRF (s) method for some individual metrics and mainly perform better for the binary metrics $P@k$ and $HITS@k$ compared to the BM25 base ranker. However, both are outperformed on the $NDCG@k$ metric that takes into account the entire relevance scale and additionally requires ranking GUIs with *medium* relevance at higher positions. Overall, the results indicate that AQE techniques in the form of PRF-KLD can improve

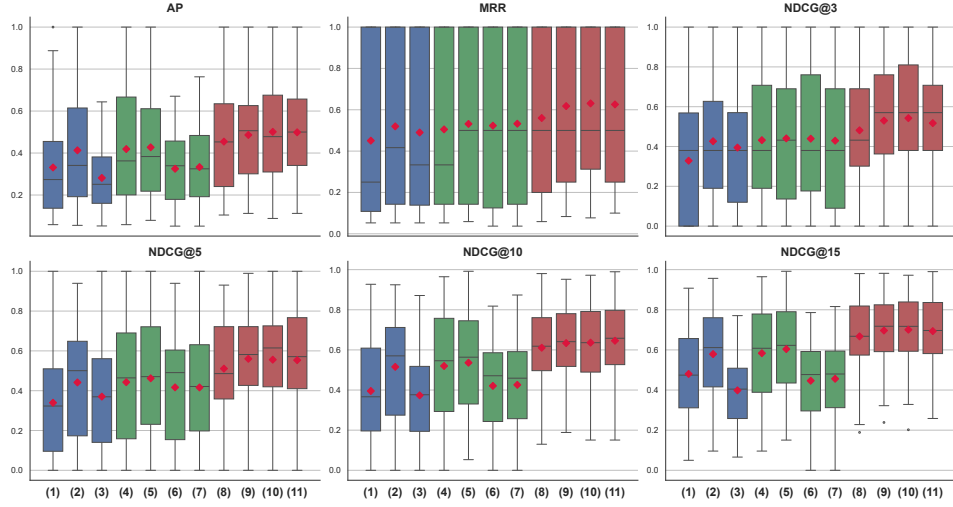


Figure 3.12: Multiple boxplots with AP , MRR and $NDCG@k$ metrics across models: (1) TF-IDF, (2) BM25, (3) nBoW, (4) PRF, (5) PRF (s), (6) PRF (w), (7) PRF (sw), (8) SBERT, (9) BERT-LTR (1), (10) BERT-LTR (2), (11) BERT-LTR (3), red diamonds repr. means.

the performance for NLR-based GUI retrieval compared to the baseline BM25 ranking model when considering mean metric values, especially the variant that takes the GUI-specific text segments into account. However, no statistically significant difference was observed (cf. Appendix B Tables B.1, B.2 and B.3). Additionally, we provide a more detailed per-query analysis of the AQE evaluation results in our accompanying repository. For example, the MRR and $P@5$ metrics were improved or at least as good in 69% and 77% of the queries with an average improvement of .19 and .07 compared to the base model BM25 by applying the PRF (s) method. In particular, for queries that have many relevant GUIs in the GUI repository and the initial retrieval results with the base model BM25 provide relevant and cohesive GUIs, the AQE method can improve the results. For instance, for the query "*a list of songs*" meaningful expansion words such as *artist*, *bookmark*, and *play* can be extracted, which strengthen the query. Figure 3.12 illustrates several boxplots for metrics of AP , MRR and $NDCG@k$.

Considering the results of the trained BERT-LTR models, we can observe that all three models consistently outperform the strong baseline BM25 across mean metric values, while statistically significant differences between BM25 and BERT-LTR (2) (*pairwise*) are observed for AP , $H@5,10$, $P@7,10$ and $NDCG@3,5,10,15$ (cf. Appendix B Tables B.1, B.2 and B.3). Between SBERT and BM25, $P@10$ and $HITS@10$ are significantly improved. In our experiments, the *pairwise* (BERT-LTR (2)) and *listwise* (BERT-LTR (3)) models appear to mainly perform best among the examined models based on the mean metric values. While the BERT-LTR (2) (*pairwise*) model consistently outperforms the SBERT model across all mean metric values, statistically significant differences are observed for $H@5$. Regarding the $NDCG@k$ metrics, the BERT-LTR models outperform the SBERT baseline substantially on the mean values, indicating that the models learned to rank

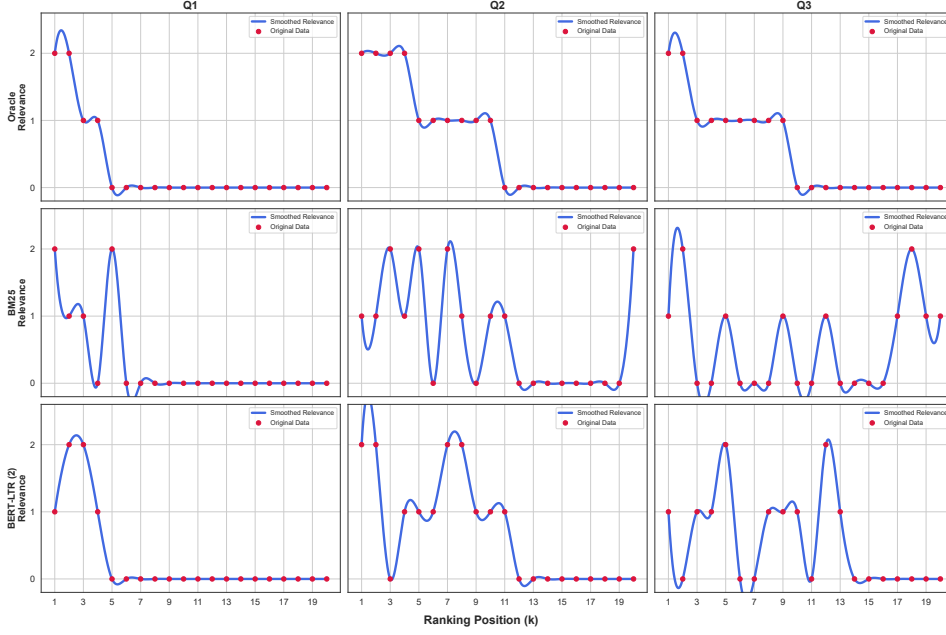


Figure 3.13: Multiple ranking plots showing the relevance of a GUI at position k for a particular model and query: Columns represent the same query, while rows represent the same model, enabling a comparison. The plot includes an *Oracle* model (the perfect ranking), BM25 and BERT-LTR (2). Q_1 =“screen with sample movies and information on recording with SnapMovie”, Q_2 =“profile window with a button to take a photo or select it from gallery [sic]” and Q_3 =“Weather details for city with option in Celsius and Fahrenheit”.

GUIs with both *medium* ($\mathbf{R}=1$) and *high* relevance ($\mathbf{R}=2$) higher in the overall ranking. Overall, the results indicate that the pretrained and finetuned BERT language model helps to better represent the semantics of the query and GUI documents and that, in combination with a LTR model trained specifically on the task of GUI ranking, it can be effectively applied to improve the NLR-based GUI ranking performance compared to the strong BM25 baseline. In particular, these specifically trained semantic models are better at bridging the gap between the language used in NLR queries and the GUI text representations. Figure 3.13 shows the ranking plots of an *Oracle* model (i.e. perfect ranking), BM25 and BERT-LTR (2) across three queries, where the x -axis represents the position in the ranking ($k=1-20$), while the y -axis shows the relevance level of the GUI at that rank position. In the illustrated examples, it can be observed that BERT-LTR (2) is able to rank GUIs with *medium* and *high* relevance levels at positions higher in the ranking, while *low* relevance GUIs are mostly placed at lower ranking positions. Moreover, Figure 3.14 shows two example queries taken from the gold standard with their respective top-5 GUIs as ranked by the trained BERT-LTR (2) model. Each GUI is annotated with its ground truth relevance. For the queries, we can observe that only a single GUI is annotated with a *low* relevance (due to an information overlay), whereas all other GUIs

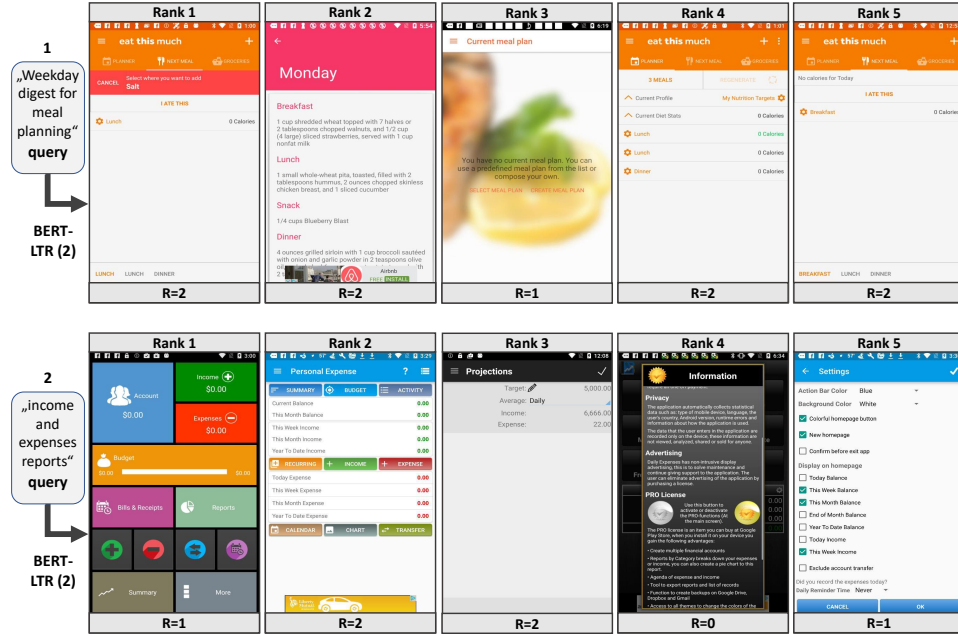


Figure 3.14: Example of two queries from the gold standard with the respective top-5 GUI rankings and their gold standard relevance scores, ranked by the BERT-LTR (2) model.

appear to have a *medium* to *high* relevance for the corresponding queries, which provides additional indication for the ranking strength of the finetuned BERT-LTR models.

Answer to RQ₁: The BERT-LTR models consistently outperform the strong baseline model BM25 across all mean metric values, while statistically significant differences between BM25 and BERT-LTR (2) (*pairwise*) are observed for AP , $H@5,10$, $P@7,10$ and $NDCG@3,5,10,15$ metrics. Moreover, the BERT-LTR models consistently outperform the SBERT model across mean metric values and statistically significant differences between BERT-LTR (2) (*pairwise*) and SBERT are observed for the $HITS@5$ metric.

3.4.2 RQ₂: Productivity of Rapid Prototyping

In Figure 3.15, we show the evaluation results of our controlled experiment. Each of the four boxplots (a)-(d) shows one of the four $\#GUI-comp$ metrics as a proxy for productivity as explained earlier. For the basic $\#GUI-comp$ count shown in Figure 3.15(a), we can observe that our approach outperforms the traditional approach substantially at each time step t (cf. Appendix B Tables B.4–B.9). The counts in both approaches increase with increasing t , but the differences between the two approaches become smaller with increasing t . Participants are often able to find suitable starting GUI screens via adequate searches in *RaWi* and thus often begin with higher counts followed by smaller adjustments (*adding* and *removing* components). In contrast, in the traditional approach, it was often

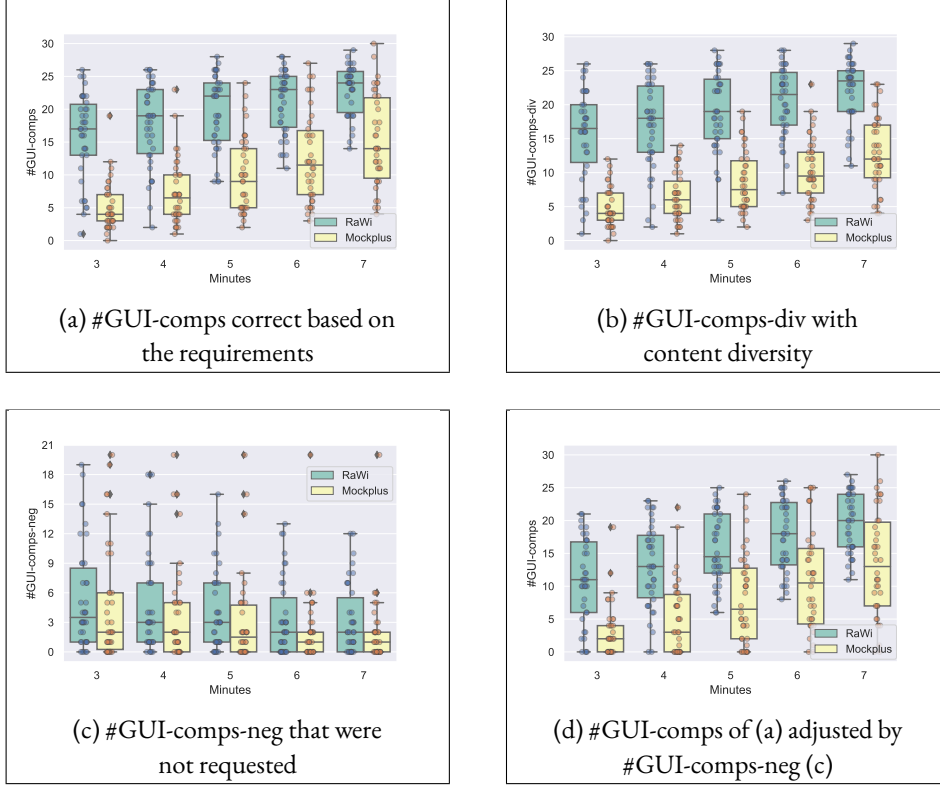


Figure 3.15: Evaluation results of our controlled experiment: each boxplot (a)-(d) shows one of the computed *#GUI-comp* metrics that the participants achieved with each rapid GUI prototyping approach *RaWi* and *Mockplus* at each minute t from minutes 3–7.

more difficult for participants to find an appropriate GUI at the beginning. However, repetitive component groups could be copied quickly after initial creation, which led to a strong increase in counts over time t . When working with *RaWi*, participants were challenged to extract good queries on the basis of the given NLR and select relevant starting GUIs. Here, participants were not always able to find optimal starting GUIs, but productivity improvements typically could still be observed in comparison to the traditional prototyping approach since *RaWi* simplified the task.

For the *#GUI-comp-div* counts that take into account the content diversity, shown in Figure 3.15(b), we can observe a behavior similar to that in (a), but the differences between the two approaches are apparently larger. GUIs retrieved with *RaWi* typically display already diverse and realistic data since they are derived from GUI screenshots of real applications. In contrast, participants often copied repetitive component groups and often were not able to provide diverse data in their GUI prototypes with the traditional approach. For the *#GUI-comp-neg* counts that measure the available but not requested components shown in Figure 3.15(c), we can observe that in both approaches the counts decrease over time t due to the removal of components available in the initial GUI screens

that were not required. However, at the same time, the GUI prototypes created with *RaWi* tend to contain more unrequested components. In *RaWi*, participants frequently found suitable screens in the beginning, yet often these GUIs included many additional components for other specific requirements that were not relevant for the requested prototype. However, we can observe a similar phenomenon in the traditional approach, where participants who started with unsuitable GUI screens obtained high counts indicated by the outliers. In particular, one participant used a GUI with many unrequested components and did not manage to remove them up to minute 7. Users of *RaWi* tend to focus on adding more requested functionality not yet contained in the starting GUIs and neglect removing unrequested components initially. This could potentially be due to the fact that the unrequested components in *RaWi* were possibly often perceived as less wrong (e.g., *a star rating for each of multiple search results*) compared to unrequested features in the templates in *Mockplus*, and therefore potentially neglected more initially.

For the adjusted $\#GUI-comp$ counts that represent the difference between the original count in (a) and (c) shown in Figure 3.15(d), we still can observe that our approach has higher counts at time t , but the differences become smaller when punishing the prototypes for containing unrequested components. In addition to the differences observed in the boxplots, the *Wilcoxon signed-rank test* results indicate that the counts as a proxy for productivity using our approach are statistically significantly larger at all considered time steps t ($p\text{-value} < 0.05$) and all considered counts (a), (b) and (d). Moreover, Cliff's δ indicates a *large* difference between counts of the approaches also in the cases where the difference appears smaller in the boxplots. Overall, the results indicate that our approach can improve the prototyping productivity compared to a traditional GUI prototyping approach and therefore provides a better applicability for interactive GUI prototyping.

Answer to RQ₂: Our prototyping approach, which integrates the effective NLR-based GUI retrieval techniques, significantly outperforms the traditional GUI prototyping approach *Mockplus* regarding the number of *correct* GUI components ($\#GUI-comp$), the data model diversity ($\#GUI-comp-div$) and the adjusted number of components.

3.4.3 RQ₃: Perceived Usefulness

Fig. 3.16 shows the evaluation results of the perceived usefulness of our GUI prototyping approach. The first question (a) regarding the support during the prototyping process received a high score ($M:3.73/SD:0.80$). The second question (b) regarding the perceived relevance of the search results received a similarly high score ($M:3.63/SD:0.68$). The third question (c) regarding the support of the results received an even higher score ($M:4.21/SD:0.78$). Finally, the fourth question (d) regarding the support of the search results for better visualization received the highest score ($M:4.47/SD:0.61$). Overall, we achieved a SUS of 81.57, which indicates that our approach has high usability compared to the average SUS of 68. Participants found our approach very easy to understand and use, even for inexperienced and novice users in GUI prototyping. In general, the participants reported further that they liked the large choice of available GUI screens to create a first prototype

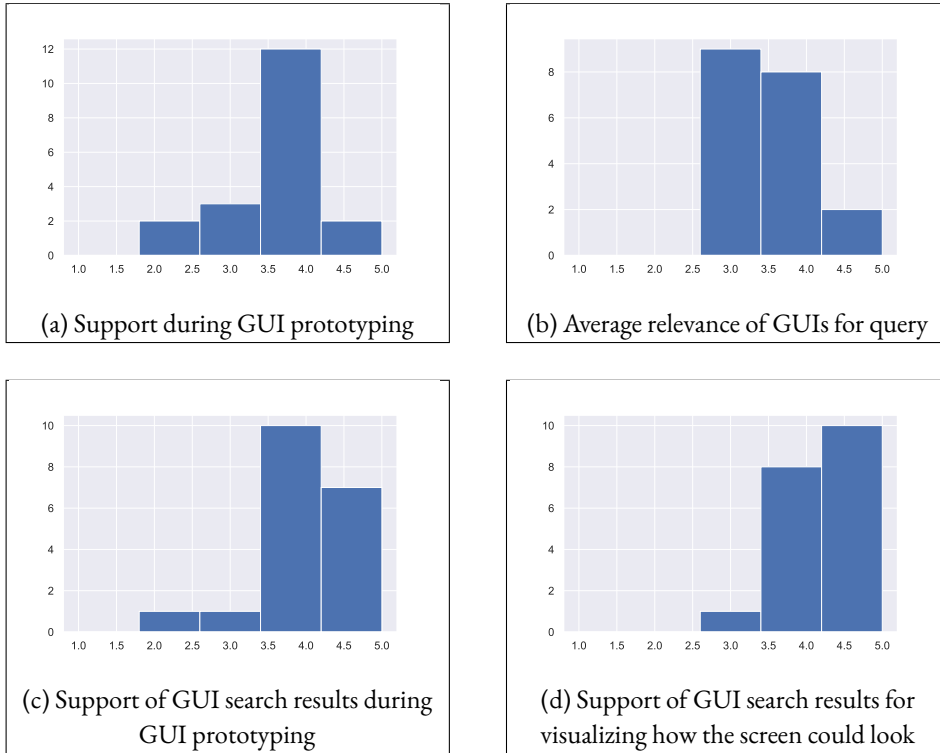


Figure 3.16: Four histograms showing the results of the four usability questions (a)-(d) regarding *RaWi* for rapid GUI prototyping as perceived by the user study participants.

quickly. Concerning improvements to the approach, participants mainly reported that the editor functionalities should be extended to enable more sophisticated GUI editing.

Answer to RQ₃: Our approach is perceived as useful, achieving a SUS of 81.57, which represents high usability. Moreover, the approach receives good scores regarding the support *RaWi* provides during prototyping, the relevance of retrieved GUIs, the support by the retrieval results and their help with visualization of the GUI prototype.

3.5 Threats to Validity

Internal Validity. First, we discuss threats to internal validity, which refers to potential bias present in our experimental evaluation. To reduce bias and subjectivity in the creation of the gold standard, we conducted several steps. Due to the absence of any prior accessible system that could be used to extract search queries for GUIs, we employed 50 workers from AMT to write queries on a random sample of the *Rico* GUIs, to reduce bias during the query creation. AMT provides a large number of workers with diverse demographics, suiting the need for including a large bandwidth of different ways of formulating queries,

use of language and vocabulary. However, there could be potential selection bias. For example, workers that are willing to work on the task may have a specific interest in apps and thus may have more knowledge on the topic compared to the average worker on AMT. Similarly, we asked 49 workers to provide relevance judgments and had each GUI annotated by three workers to reduce subjectivity. These workers were required to pass our task-specific qualification test, which served the purpose of increasing the relevance annotation quality. Final relevance annotations are based on majority voting and GUIs with high annotation perplexity were discarded due to the high uncertainty of the ground truth relevance for these instances. To obtain a high-quality gold standard, we applied several filters and randomly sampled from both the remaining queries and the GUIs to avoid selection bias. To reduce bias in our controlled experiment, we randomized the tasks and the ordering of the approaches. In addition, we ensured that the applications employed in the user study tasks are not included in *Rico* and both approaches basically support the domains of the selected prototyping tasks. Moreover, for each approach, we ensured that we conducted the identical experimental procedure.

External Validity. Second, we discuss threats to external validity, which refers to the generalizability of the results obtained in our experimental evaluation. Similarly, due to the absence of any prior system for GUI search that could be used as a resource for obtaining real-world queries, we employed GUIs taken from *Rico* for which to write queries. We already discussed the many reasons for employing *Rico* and emphasize that *Rico* is large-scale and covers many diverse domains, making it applicable in many scenarios. In addition to many domain-specific GUIs, *Rico* provides a plethora of domain-independent GUIs. Using GUIs from *Rico* as the basis for writing queries also assures that models could at least in principle retrieve a single relevant GUI for the query. We restricted our controlled experiment to two domains, but we hypothesize that we could obtain similar results for other domains covered by *Rico*. However, more user evaluations are required that include more diverse application domains to confirm the obtained results. Moreover, since *Rico* is a semi-automatic approach, it could be scaled up to harvest millions of GUIs, making our prototyping approach even more valuable for more domains.

3.6 Limitations

Our retrieval is currently restricted to the *Rico* dataset, which, however, is large-scale and covers many diverse domains already. To extend the GUI repository, other smaller GUI datasets (such as *ReDraw*) could be integrated. Since prototypes are created on the basis of GUIs from different real-world applications in *RaWi*, the design of the resulting application prototype may not be cohesive. However, the focus of our work lies on rapidly creating GUI prototypes that properly reflect FRs of stakeholders, neglecting the final GUI design. Since the implementation of our GUI prototyping approach is an early prototype, the editing functionality is currently restricted. However, we plan to extend our prototype to a fully-fledged graphical GUI editor with more editing functionality.

3.7 Related Work

GUI Retrieval *Guigle* (Bernal-Cárdenas et al., 2019) similarly exploits automatically crawled *Android* apps from *Google Play* (Moran et al., 2018) to index multiple parts from the crawled GUI hierarchy data such as the app name, text and type of GUI components, the screen color and employs a basic Boolean query language for relevant GUI retrieval. In contrast, our approach *RaWi* enables users to specify NLR-based searches to retrieve relevant GUIs and employs a more sophisticated ranking mechanism including BERT-based LTR models. In addition, *RaWi* enables users to directly employ retrieved GUIs to rapidly create GUI prototypes including the derivation of partly editable screens, whereas *Guigle* stops after GUI image retrieval. However, a direct comparison of the retrieval performance is not possible due to the unavailability of the implementation and the employment of a different, much smaller GUI dataset in *Guigle*. As stated in their paper, *Guigle* is built on top of the open-source retrieval framework *Lucene* (Lucene). The *Lucene* framework employs TF-IDF and BM25 retrieval methods, which are included in our evaluation experiments as well-known standard IR baselines. Therefore, although implementation details such as preprocessing and retrieval function parameters are not specified in *Guigle*, the presented IR baselines can be considered a good proxy for the retrieval methods employed in their approach. Moreover, to support further research and enable a direct comparison between approaches in the first place, we publicly provide the first high-quality gold standard for NLR-based GUI retrieval based on a state-of-the-art GUI repository in this work.

Gallery D.C. (Chen et al., 2019a) crawls a large number of real-world applications and automatically extracts GUI components such as various types of buttons from GUI screenshots and provides a multi-modal search supporting multiple dimensions such as width, height, main color and text for them. In addition, there is a plethora of GUI retrieval approaches that employ visual inputs such as screenshots, hand-drawn sketches or basic wireframes to retrieve similar GUIs, in contrast to our approach that focuses on NLR input. For example, Chen et al. (2020) train an image autoencoder to find relevant GUIs employing a basic wireframe of the GUI as their input. Similarly, *VINS* (Bunian et al., 2021) expects visual input either as a basic wireframe prototype or as a fully designed GUI image to retrieve similar GUIs using a multi-modal embedding network. *Swire* (Huang et al., 2019) also employs a visual embedding approach to find relevant GUI images on the basis of hand-drawn sketches provided by the user. *GUIFetch* (Behrang et al., 2018) requires an entire *Android* application sketch as input to retrieve similar apps from *GitHub*. *d.tour* (Ritchie et al., 2011) uses stylistic keywords and stylistic similarity to find similar website designs. By using a web design as input and conducting structure matching, *FaceOff* retrieves web GUI components fitting the design.

Another approach proposes a neural translator for transforming GUI images to GUI skeletons (Chen et al., 2018). *Screen2Vec* (Li et al., 2021) proposes a technique for learning multi-modal (textual content, visual design and layout patterns) GUI embeddings, which are useful for many GUI-related downstream tasks (e.g., retrieving similar GUIs using a GUI as the query). Due to the difference of the problem, these GUI embeddings cannot

be directly reused in our approach for comparison. The NLR query embeddings (e.g., based on BERT) and *Screen2Vec* embeddings represent two separate embedding spaces that require alignment in order to enable NLR-based GUI retrieval with *Screen2Vec* GUI embeddings. However, we included the same text-only SBERT baseline in our experiments, also used in their approach.

The *Screen2Words* (Wang et al., 2021) approach proposes an Encoder-Decoder architecture for translating GUIs into short text descriptions. The GUI encoder could potentially be employed to obtain GUI embeddings, but these embeddings similarly cannot be directly employed for retrieval based on NLR queries as described earlier. Considering general-purpose search engines (e.g., *Google*), a direct comparison with our approach is not meaningful since these engines cannot be evaluated on our gold standard. Especially due to the huge differences in the employed datasets (specialized *Rico* GUI dataset vs. general *Google* image dataset), it would be unclear whether differences between the retrieval performances are due to differences in the algorithms or due to the differences in the datasets. For example, since *Google* uses a general image dataset, many image results would be obtained that are not related to GUIs, compared to the *Rico* dataset specializing in *Android* GUIs. To make the comparison between these approaches meaningful, the algorithm would need to be evaluated on the new gold standard directly.

Program Code Retrieval Apart from GUI search approaches, retrieving code through NL queries has been studied extensively in research before (McMillan et al., 2011; Lv et al., 2015; Zhang et al., 2016; Gu et al., 2018b; Cambronero et al., 2019). These approaches range from the application of traditional IR algorithms to specifically developed semantic Deep Learning (DL) architectures to find relevant program code for a given NL query. More recently, the *CodeSearchNet* challenge (Husain et al., 2019) released a dataset of NL queries and expert relevance annotations of respective functions from various programming languages to provide a source for systematic evaluation of code search approaches. In contrast, we created the first GUI retrieval gold standard in this work to foster further research on retrieval methods specifically for NLR-based GUI search.

GUI Prototyping In addition, many traditional prototyping approaches exist and are employed by many designers, developers and analysts in their everyday work such as *Balsamiq* (Faranello, 2012), *Sketch* (Sketch), *Figma* (Figma) and *Mockplus* (Mockplus), among others. These approaches typically enable users to create prototypes in a graphical editor on the basis of a small number of hand-crafted templates and basic GUI components supporting low-fidelity or high-fidelity prototyping. Other approaches such as *UISKEI* (Segura et al., 2012) and *SketchiXML* (Coyette et al., 2006) attempt to automatically identify GUI components from hand-drawn sketches and generate reusable GUI representations. Recent work on GUI prototyping assistance such as *GUIComp* (Lee et al., 2020) supports novice users during the prototyping process via the recommendation of similar GUIs, provides various complexity metrics and visual attention maps based on the design currently created by the user. In contrast to these approaches, *RaWi* enables users to quickly retrieve matching GUIs based on NLR queries from a large-scale

GUI repository and automatically provides partly editable GUI screens to reuse for rapid, interactive GUI prototyping, thereby facilitating requirements elicitation and validation.

3.8 Conclusion

This work was driven by challenge **C1-I**: *How can we adapt and optimize text-based retrieval methods and techniques to enable more effective NLR-based GUI retrieval?* To answer this question, we presented *RaWi*, a data-driven GUI prototyping approach based on exploiting a large-scale GUI repository via effective NLR-based GUI retrieval methods and automatically deriving partly editable GUI screens for interactive GUI prototyping in the requirements elicitation phase. Particularly, we presented BERT-LTR models, which improved the NLR-based GUI ranking performance substantially over the strong text-based baseline model BM25. In addition, our approach is able to improve the prototyping productivity in comparison to a traditional GUI prototyping approach.

CHAPTER 4

GUI-RE-RANK: ENHANCING GUI RETRIEVAL WITH MULTI-MODAL LLM-BASED RERANKING

After introducing our first solution approach and evaluation for NLR-based GUI retrieval, we continue with challenge **C1**, namely, mapping NLR to GUI prototypes. In this chapter, we introduce a novel MLLM-based GUI reranking approach and improve on the prior results achieved with the BERT-LTR models. In particular, the work presented in this chapter is based on previously published research (Kolthoff et al., 2025a)¹. Again, our interactive prototype, source code, evaluation results and video are all publicly available².

Personal Contribution. I developed the idea and concept for the proposed approach, implemented the entire framework and tool prototype. The evaluation is based on the gold standard introduced in the previous chapter, which I employed to design the evaluation concept and conduct the data analysis. Finally, I wrote the entire manuscript.

4.1 Motivation

While we investigated NLR-based GUI retrieval methods that solely relied on textual representations of the GUI prototypes in the previous chapter and provided substantial effectiveness improvements based on the proposed BERT-LTR models over the strong BM25 baseline, this approach carries one major shortcoming. Representing GUI prototypes merely through text fragments extracted from various GUI hierarchy components (such as *displayed text of GUI components*, *Android* specifics including *activity names* and

¹This section is adapted from: **Kolthoff, Kristian**, Kretzer, Felix, Bartelt, Christian, Maedche, Alexander, and Ponzetto, Simone Paolo. GUI-ReRank: Enhancing GUI Retrieval with Multi-Modal LLM-based Reranking. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE, A*)*, Seoul, Republic of Korea, January 2026, pages 1–4. ACM. (in press)

²Materials for this chapter are available at <https://github.com/kristiankolthoff/GUI-ReRank>, <https://zenodo.org/records/16451923> and demo video at https://youtu.be/_7x9UCh82ug

component-wise *resource identifiers*, *semantic labels for icons*) provides only a heavily simplified, abstracted representation, neglecting crucial semantic information encompassed in the GUI prototypes in *Rico* (Deka et al., 2017). Particularly, these GUI prototypes are multi-dimensional objects, which contain most of their information in the screenshot images themselves. These prototype screenshots convey functional and non-functional information through displayed text (e.g., component-specific text), component types (e.g., *buttons*, *text fields*, *check boxes*), visual icons, component and group layouts, and the overall GUI design. Therefore, incorporating this important information while deciding on relevance given NLR has the potential to enhance the GUI retrieval and ranking effectiveness. While the GUI design plays a less significant role with respect to prototypes employed for REI and RVal, the remaining dimensions are crucial for determining relevance according to functional NLR. Moreover, the previous approach heavily relies on specific GUI hierarchy information, which might not be available for other GUI datasets. This restricts the generalizability of the approach and impairs the practical adoption. Although GUI design is of less importance for early requirements elicitation and validation purposes, it still represents an important aspect of GUI prototypes (NFR). To summarize, the following work is based on the leading research question of challenge **C1-2**: *How can the semantic representation gap between NLR (including NFR) and multi-dimensional GUI prototypes be reduced to enable more effective GUI ranking?*

To tackle this challenge and close the gap, in this work, we introduce *GUI-ReRank*, a novel framework that significantly advances the previous NLR-based GUI retrieval and reranking models by integrating the information encompassed in GUI screenshot images with powerful, state-of-the-art MLLMs. In particular, *GUI-ReRank* employs a two-stage retrieval and reranking approach, which introduces an embedding-based, constrained GUI retrieval architecture over multiple dimensions followed by an MLLM-based GUI reranking technique. *GUI-ReRank* enables more expressive and complex NLR queries including negation (i.e. *exclusion criteria/requirements*) across multiple customizable dimensions such as *domain*, *functionality*, *design* and *GUI components* via the advanced Natural Language Understanding (NLU) and image processing capabilities of modern MLLMs. To facilitate the adoption for various information needs and GUI datasets (e.g., *domain-*, *industry-* or *company-specific*), our approach offers a fully customizable GUI dataset annotation pipeline, which employs MLLMs to initially derive rich textual representations for each dimension of the GUI prototypes, enabling fast and computationally cheap multi-dimensional retrieval. To evaluate the effectiveness of the MLLM-based GUI reranking, we utilize the gold standard created in the previous chapter and additionally conduct a comprehensive cost analysis, showing the cost-effectiveness trade-offs.

Contributions. With this work, we make the following four research contributions:

- *MLLM-based GUI reranking approach*: we present a novel, MLLM-based GUI reranking approach that incorporates the information contained in the GUI images and significantly outperforms prior state-of-the-art models such as BERT-LTR.
- *Multi-dimensional embedding-based constrained GUI retrieval*: we propose a new

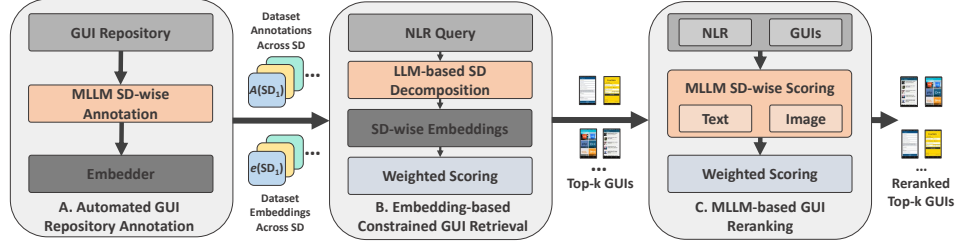


Figure 4.1: Overview of the *GUI-ReRank* approach with (A) the automated GUI repository annotation and embedding pipeline, (B) the embedding-based constrained GUI retrieval approach and (C) the MLLM-based GUI reranking via images or annotations.

approach to facilitate multi-dimensional constrained NLR-based GUI retrieval via pretrained embeddings and also enable fully customizable GUI search dimensions.

- *Detailed effectiveness and cost analysis:* we conduct a comprehensive evaluation of the proposed MLLM-based GUI reranking approaches by utilizing the previously created gold standard and investigate trade-offs between effectiveness and costs.
- *Rico GUI annotations and embeddings dataset:* we provide multi-dimensional text annotations for the *Rico* GUI dataset created by an MLLM and additionally their respective 3,072-dimensional embeddings to foster future GUI retrieval research.

Structure of the chapter The remainder of this chapter is structured as follows. In Section 4.2, we present the novel multi-dimensional embedding-based constrained GUI retrieval and MLLM-based GUI reranking approach. Section 4.3 introduces the research questions and evaluation setup, while Section 4.4 discusses the results. In Section 4.5, we present threats to validity, followed by limitations of our approach in Section 4.6. Finally, we provide an overview of related work in Section 4.7 and a conclusion in Section 4.8.

4.2 Approach: GUI-ReRank

GUI-ReRank encompasses three main components: (A) an MLLM-based, automated and customizable GUI repository annotation pipeline that creates textual GUI annotations and embeddings across predefined Search Dimensions (SDims), (B) an embedding-based constrained GUI retrieval approach, which leverages the previously created embeddings and enables multi-dimensional, complex NLR search queries (including negation) based on LLM-driven query decomposition and (C) an MLLM-based GUI reranking technique, which utilizes both *image* and *text* representations of the GUI prototypes to compute a refined ranking score according to the predefined SDims. Figure 4.1 shows an overview of *GUI-ReRank*, with the main components and their respective intermediate outputs.

4.2.1 Automated GUI Repository Annotation

Given a GUI repository as images and a collection of customizable SDims, *GUI-ReRank* employs a ZS-prompted MLLM to generate textual annotations for each GUI according

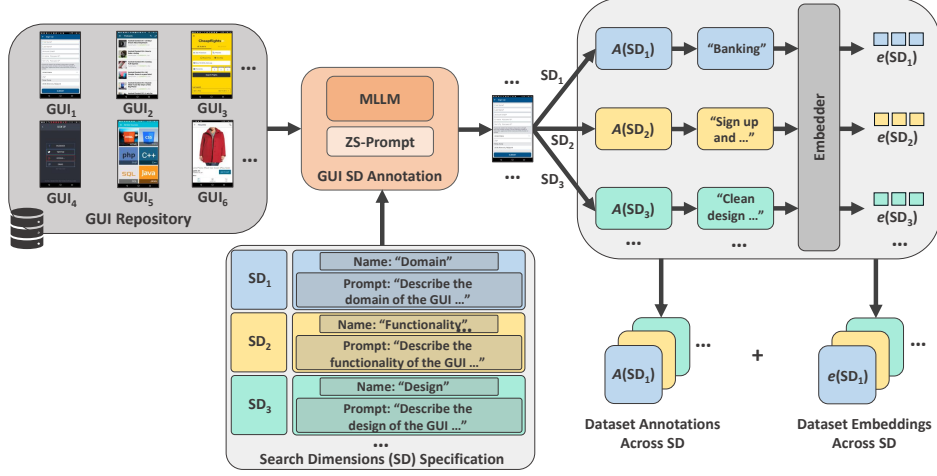


Figure 4.2: Overview of the automated GUI repository annotation pipeline, showing the input GUI repository (e.g., *Rico*), a collection of customizable SDims, the ZS-prompted MLLM which creates SDim-specific text annotations that are subsequently embedded.

to the SDims (Figure 4.2). Our framework provides a default collection of SDims, including *domain*, *functionality*, *design*, *GUI components*, and *displayed text*, which are widely applicable to various FR and NFR search needs. Particularly, to support REI and RVal, the default dimensions of *domain*, *functionality* and *GUI components* are valuable, since these directly relate to functional aspects of the GUI prototypes. In addition, users are enabled to modify or extend the SDims by specifying a name and a brief NL description of the SDim to address user-specific requirements for the search (e.g., *platform* or *accessibility*). Subsequently, each of the created annotations is processed by an embedding model to derive dense, SDim-specific embeddings. The generated text and embedding datasets provide the foundation for the following GUI retrieval and MLLM-based reranking approaches. To facilitate adoption, *GUI-ReRank* includes the well-known, large-scale GUI dataset *Rico* (Deka et al., 2017) ready to search, fully pre-annotated and embedded.

4.2.2 Embedding-based Constrained GUI Retrieval

To support complex semantic GUI searches with both positive and negative NLR constraints across various SDims, *GUI-ReRank* initially decomposes the input NLR by employing a ZS-prompted LLM, which extracts both positive and negative constraints or requirements across all predefined SDims (e.g., $SD_{Design}: pos^+(\text{"modern"}), neg^-(\text{"dark"})$ or $SD_{Functionality}: pos^+(\text{"Social login"}), neg^-(\text{"Google"})$). Figure 4.3 illustrates an overview of the approach. More formally, given SDims $SD = \{SD_1, \dots, SD_K\}$ and NLR, the LLM derives positive $P_k = \{p_{k,1}, \dots, p_{k,|P_k|}\}$ and negative $N_k = \{n_{k,1}, \dots, n_{k,|N_k|}\}$ SDim-wise constraints. Afterwards, the extracted constraints are embedded by the same embedding model which was utilized to create the GUI dataset embeddings, i.e. $e(p_{k,j}) = f_{emb}(p_{k,j})$ and $e(n_{k,j}) = f_{emb}(n_{k,j})$. We denote the

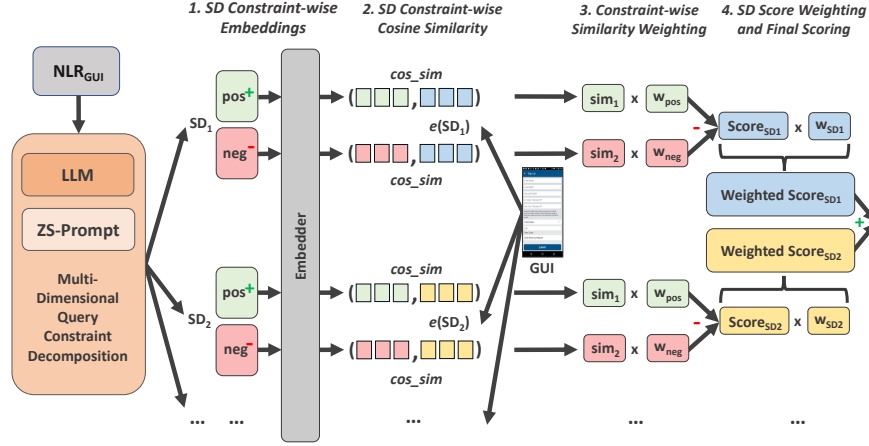


Figure 4.3: Overview of the embedding-based constrained GUI retrieval approach, showing (1) the decomposition of NLR into SDim-wise positive and negative constraints which are then embedded, (2) the computation of cosine similarity across SDims with the GUI embeddings, (3) similarity weighting and combination into single SDim-wise scores, and finally (4) the computation of the final score as a weighted sum of SDim-wise scores.

SDim-wise GUI embedding by $\mathbf{e}(G_{i,k}) = f_{\text{emb}}(G_{i,k})$. Subsequently, we compute the matching score as the *cosine similarity* between the positive and negative constraints for a particular SDim SD_k and the respective embeddings of GUI G_i as

$$s_{i,k,j}^+ = \text{cos_sim}(\mathbf{e}(p_{k,j}), \mathbf{e}(G_{i,k})) \quad (4.1)$$

$$s_{i,k,j}^- = \text{cos_sim}(\mathbf{e}(n_{k,j}), \mathbf{e}(G_{i,k})) \quad (4.2)$$

Particularly, $s_{i,k,j}^+$ represents a score of how closely the j -th positive constraint for SD_k matches the corresponding GUI embedding for the same SD_k . Next, we compute the average score for the positive and negative constraints, respectively, as follows

$$\text{sim}_{i,k}^+ = \frac{1}{|P_k|} \sum_{j=1}^{|P_k|} s_{i,k,j}^+ \quad \text{sim}_{i,k}^- = \frac{1}{|N_k|} \sum_{j=1}^{|N_k|} s_{i,k,j}^- \quad (4.3)$$

Afterwards, we compute the weighted sum of the overall positive and negative scores for each SD_k and GUI G_i as $\text{Score}_{i,k} = w_{\text{pos}} \text{sim}_{i,k}^+ - w_{\text{neg}} \text{sim}_{i,k}^-$, followed by weighting the score for the search dimension SD_k as $\text{WScore}_{i,k} = w_{SD_k} \text{Score}_{i,k}$. Finally, we compute the normalized sum across all weighted dimension scores to obtain the final score. This approach enables a fine-grained, weighted search across multiple dimensions.

4.2.3 Multi-Modal (M)LLM-based GUI Reranking

Given the top- k GUIs retrieved by the previously described model, we subsequently propose MLLM-based GUI reranking, which refines the top- k GUI ranking by evaluating

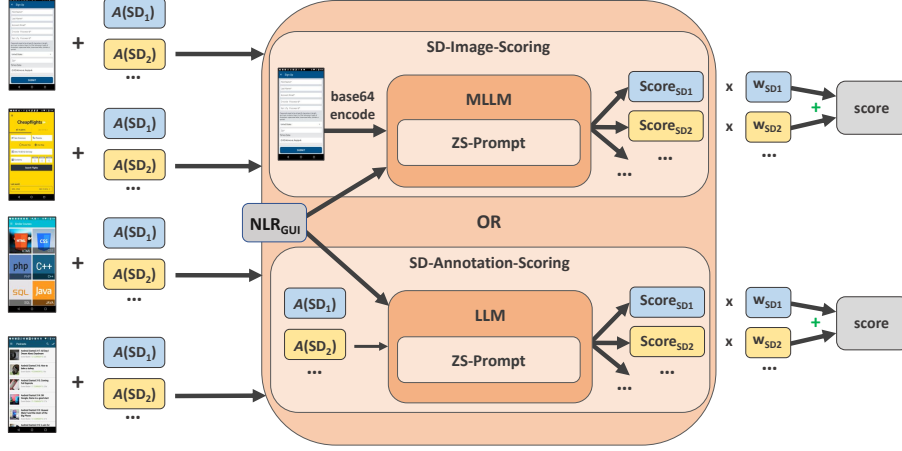


Figure 4.4: Overview of the MLLM-based GUI reranking showing both available variants: *image-based scoring*, which uses the *base64*-encoded image with the NLR for scoring, and *text-based scoring*, which uses the SDim-wise text annotations with the NLR for scoring.

either the GUI screenshot or the SDim-wise text annotations by using a ZS-prompted MLLM (Figure 4.4). For each SDim, the MLLM computes a score between 0 (*no match*) and 100 (*perfect match*), i.e. $RScore_{d,i} = MLLM(SD_d, NLR, G_i)$, which reflects the relevance of this dimension for the given NLR and the GUI G_i (represented either as an image or a text annotation). By allowing both image and detailed text annotations as input to the reranking model, *GUI-ReRank* offers the option for an effectiveness-cost trade-off. While image-based GUI reranking provides the most detailed and information-dense representation to the model for computing relevance, it also involves higher cost (i.e., more *computational resources*, *time* and *monetary cost*). On the contrary, text-based GUI reranking provides higher cost efficiency, but the input representation is still an abstraction, being less information-dense in comparison to the actual image. Similar to the previous embedding-based retrieval, the dimension-specific scores are weighted and a normalized sum is computed to obtain the final ranking score. We also provide a non-weighted variant, where NLR are matched against a single aggregated GUI summary.

4.2.4 Prototype Implementation

The *GUI-ReRank* prototype is implemented as a *Django* web application with an *HTML/CSS* and *JavaScript* frontend, which is combined with *MySQL* (MySQL, 2025) for data storage. To enable efficient and parallel processing of background tasks, for example, the GUI text annotation, embedding creation and batch-wise reranking of the top- k GUIs, we utilize the *Celery* framework (Celery, 2025) distributed across multiple worker nodes, combined with *Redis* (Redis, 2025) as the message broker. Our current implementation provides support for many popular MLLMs, including *GPT* (OpenAI, 2023) from *OpenAI* (OpenAI, 2025e,d), *Google Gemini* (Google, 2025b) and *Anthropic Claude Sonnet* (Anthropic, 2025b), and is easily extensible and customizable for additional MLLMs.

Table 4.1: Evaluation results overview of the different models on the NLR-based GUI ranking gold standard using $P@k$ and $NDCG@k$ (**Bold** values indicate best metric score within result group, underlined values indicate best metric score across all shown groups).

		P@k				NDCG@k (N@k)			
		P@3	P@5	P@7	P@10	N@3	N@5	N@10	N@15
BL	BERT-LTR-1	37.7	35.0	30.7	26.9	53.0	56.0	63.4	69.7
	BERT-LTR-2	40.0	34.0	30.4	28.1	54.3	55.6	63.6	70.1
	BERT-LTR-3	36.3	35.4	31.7	28.7	51.7	55.4	64.6	69.4
Text	GPT-4.1	72.0	55.2	45.1	34.9	84.1	82.3	86.6	89.1
	GPT-4.1 Mini	67.3	54.6	44.6	34.4	81.7	81.0	84.4	88.0
	GPT-4.1 Nano	55.0	46.0	39.0	30.4	71.4	71.2	76.0	81.0
	Gemini-2.5 Flash	66.0	51.8	43.1	34.1	78.4	77.9	82.6	86.3
	Gemini-2.5 Pro	69.0	54.0	44.1	34.1	82.6	80.5	84.0	87.9
	Claude Sonnet 3.7	65.0	52.6	43.7	34.0	78.2	77.9	83.0	86.6
	Claude Sonnet 4.0	66.3	51.4	43.9	34.6	81.8	79.1	84.8	88.1
	GPT-4.1	75.0	56.6	46.3	35.4	87.7	85.2	88.4	90.8
Image	GPT-4.1 Mini	71.7	54.0	44.1	35.1	85.0	82.0	86.8	89.5
	GPT-4.1 Nano	49.0	40.8	33.4	27.7	62.1	62.2	66.7	72.4
	Gemini-2.5 Flash	66.7	53.6	43.4	34.5	78.9	78.3	83.6	86.6
	Gemini-2.5 Pro	68.0	55.8	46.3	34.6	81.4	80.9	84.5	87.8
	Claude Sonnet 3.7	70.3	55.8	45.1	34.8	83.3	82.1	86.0	89.2
	Claude Sonnet 4.0	70.7	54.6	45.1	34.7	83.9	82.3	86.1	89.0
	GPT-4.1	75.0	56.6	46.3	35.4	87.7	85.2	88.4	90.8

4.3 Experimental Evaluation

In the following, we present our comprehensive evaluation of the proposed approach, which is based primarily on the NLR-based GUI retrieval and ranking gold standard. In particular, we provide the experimental setup for the following two research questions:

- **RQ₁:** *How effective are MLLM-based approaches for NLR-based GUI reranking?*
To answer this question, we utilize the NLR-based GUI retrieval and ranking gold standard created in the previous chapter, compute the same IR metrics and compare multiple variants of MLLM-based GUI reranking to previous baselines.
- **RQ₂:** *What are the effectiveness and cost trade-offs for MLLM-based GUI reranking?*
To answer this question, we track different cost metrics while running the experiments across models, including *token consumption*, *time* and *monetary cost*.

4.3.1 RQ₁: Effectiveness of MLLM-based GUI Reranking

We evaluated the effectiveness of the MLLM-based GUI reranking based on the gold standard developed in the previous chapter. To summarize, this benchmark comprises 100 NLR queries, each paired with 20 GUIs from *Rico* (Deka et al., 2017) and enables direct comparison with state-of-the-art BERT-LTR models (cf. Chapter 3). In addition,

Table 4.2: Evaluation results overview of the different models on the NLR-based GUI ranking gold standard using AP , MRR and $HITS@k$ (**Bold** values indicate best metric score within result group, underlined values indicate best metric score across all groups).

		AP	MRR	HITS@k (H@k)				
		AP	MRR	H@1	H@3	H@5	H@10	H@15
BL	BERT-LTR-1	48.6	61.8	46.0	71.0	86.0	98.0	<u>100.0</u>
	BERT-LTR-2	50.1	63.1	44.0	75.0	91.0	98.0	<u>100.0</u>
	BERT-LTR-3	49.9	62.6	45.0	73.0	86.0	<u>100.0</u>	<u>100.0</u>
Text	GPT-4.1	81.3	92.7	87.0	<u>100.0</u>	<u>100.0</u>	<u>100.0</u>	<u>100.0</u>
	GPT-4.1 Mini	79.7	92.3	87.0	98.0	<u>100.0</u>	<u>100.0</u>	<u>100.0</u>
	GPT-4.1 Nano	66.2	84.2	75.0	91.0	96.0	<u>100.0</u>	<u>100.0</u>
	Gemini-2.5 Flash	77.0	90.1	83.0	97.0	<u>100.0</u>	<u>100.0</u>	<u>100.0</u>
	Gemini-2.5 Pro	80.5	94.8	91.0	<u>100.0</u>	<u>100.0</u>	<u>100.0</u>	<u>100.0</u>
	Claude Sonnet 3.7	76.9	92.3	87.0	97.0	99.0	<u>100.0</u>	<u>100.0</u>
	Claude Sonnet 4.0	78.3	92.3	88.0	95.0	<u>100.0</u>	<u>100.0</u>	<u>100.0</u>
	GPT-4.1	84.0	92.8	87.0	99.0	<u>100.0</u>	<u>100.0</u>	<u>100.0</u>
	GPT-4.1 Mini	81.1	93.0	88.0	98.0	<u>100.0</u>	<u>100.0</u>	<u>100.0</u>
Image	GPT-4.1 Nano	58.2	79.0	66.0	91.0	99.0	<u>100.0</u>	<u>100.0</u>
	Gemini-2.5 Flash	77.5	89.7	83.0	96.0	99.0	<u>100.0</u>	<u>100.0</u>
	Gemini-2.5 Pro	79.8	90.8	85.0	96.0	99.0	<u>100.0</u>	<u>100.0</u>
	Claude Sonnet 3.7	81.2	91.9	86.0	97.0	<u>100.0</u>	<u>100.0</u>	<u>100.0</u>
	Claude Sonnet 4.0	80.7	91.8	85.0	99.0	<u>100.0</u>	<u>100.0</u>	<u>100.0</u>

we employed the same IR metrics utilized in the previous chapter (AP , MRR , $P@k$, $HITS@k$ and $NDCG@k$) to measure the ranking effectiveness. We used the non-weighted reranking and experimented with different models including three capacity variants of *GPT-4.1* (the *Original*, *Mini* and *Nano*) (OpenAI, 2025d), two variants of *Gemini-2.5* (*Flash* and *Pro*) (Google, 2025b,a) and two variants of *Claude Sonnet* (3.7 and 4) (Anthropic, 2025b,a). For determining statistically significant differences between models, we applied the *Wilcoxon signed-rank test* (Woolson, 2007) with per-metric *Holm* correction across model comparisons. In particular, we compare the three BERT-LTR models as baselines against the most powerful non-reasoning *GPT-4.1* model (according to different benchmarks at the time of release (OpenAI, 2025c)) in the *text* and *image* reranking variants. To generate the GUI text annotations, we used *GPT-4.1*. To investigate significant differences between *text* and *image* input, we compare the *GPT-4.1* model in both variants. Finally, to determine significant differences between different model capacities, we compare the full *GPT-4.1* model (*Original*) against its smaller variants (*Mini*, *Nano*).

4.3.2 RQ₂: MLLM-based Reranking Effectiveness-Cost Trade-offs

To evaluate the effectiveness-cost trade-offs associated with MLLM-based reranking, we tracked multiple metrics including *token consumption*, *time* and *monetary cost* during the

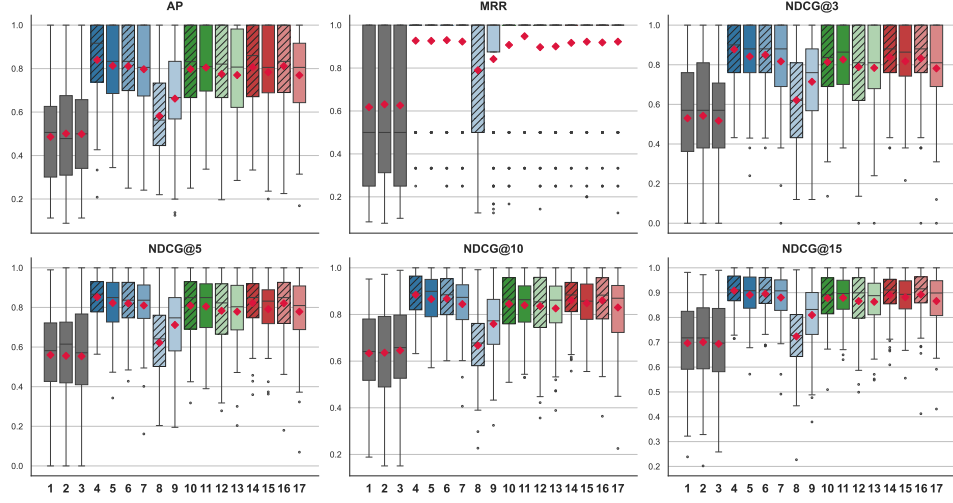


Figure 4.5: AP , MRR and $NDCG@k$ across models: (1-3) BERT-LTR (1-3), (4) *GPT-4.1 (I)*, (5) *GPT-4.1 (T)*, (6) *GPT-4.1 Mini (I)*, (7) *GPT-4.1 Mini (T)*, (8) *GPT-4.1 Nano (I)*, (9) *GPT-4.1 Nano (T)*, (10) *Gemini-2.5 Pro (I)*, (11) *Gemini-2.5 Pro (T)*, (12) *Gemini-2.5 Flash (I)*, (13) *Gemini-2.5 Flash (T)*, (14) *Claude Sonnet 4 (I)*, (15) *Claude Sonnet 4 (T)*, (16) *Claude Sonnet 3.7 (I)*, (17) *Claude Sonnet 3.7 (T)*, I=Image, T=Text, red diamonds=means.

execution of the experiments. We report the different values as averaged values obtained from the gold standard experiment runs and provide effectiveness(AP)-cost tradeoff plots.

4.4 Results & Discussion

4.4.1 RQ₁: Effectiveness of MLLM-based GUI Reranking

Table 4.1 and Table 4.2 show the ranking performance across the different IR metrics for BERT-LTR baselines (BL), text- and image-based MLLM approaches. As can be observed, all MLLM-based approaches (text and image variants) substantially outperform the BERT-LTR models in terms of mean metric values. For example, the AP achieved by the *pairwise* BERT-LTR (2) model is improved by 67.66% with the *GPT-4.1 (Image)* reranking. Moreover, the high $HITS@1$ for *GPT-4.1 (Image)* indicates that in 87% of cases, a relevant GUI is ranked at the top position, which represents an 89.13% improvement over the best BERT-LTR model. Statistically significant differences between all BERT-LTR models and *GPT-4.1* (both *Image* and *Text*) were observed across all metrics except $HITS@10,15$ (cf. Appendix C Tables C.1, C.2 and C.3). These results and the substantial improvements in mean metric values clearly illustrate the high effectiveness of the MLLM-based GUI reranking technique. Figure 4.5 illustrates multiple boxplots across the considered models as well as AP , MRR and $NDCG@k$ metrics.

Considering differences between input modalities by comparing text-based and image-based *GPT-4.1* models for GUI reranking, we can observe that the image variant consistently outperforms the text variant across mean $P@k$, $NDCG@k$, AP and MRR , while

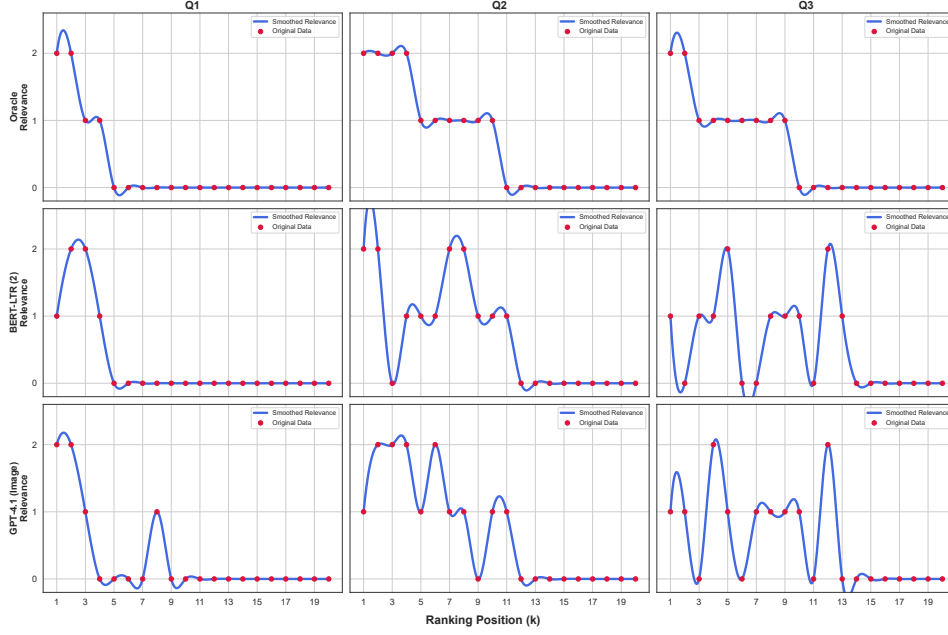


Figure 4.6: Ranking plots showing the relevance of a GUI at position k for a particular model and query: Columns represent the same query, while rows represent the same model, enabling a comparison. The plots include an *Oracle* model (the perfect ranking), BERT-LTR (2) and GPT-4.1 (Image). Q_1 =“screen with sample movies and information on recording with SnapMovie”, Q_2 =“profile window with a button to take a photo or select it from gallery [sic]” and Q_3 =“Weather details for city with option in Celsius and Fahrenheit”.

differences are statistically significant solely for the $NDCG@k$ metric (cf. Appendix C Tables C.1, C.2 and C.3). This illustrates that the higher information density of image-based GUI representations improves the effectiveness over text-based representations, even for mainly functional NLR (as in the gold standard). While most of the image-based models outperform their text-based counterparts across many mean metric values, we can observe some exceptions. Most notably, the image variant of *GPT-4.1 (Nano)* is substantially outperformed across all mean metric values (except $HITS@5$) by the corresponding text variant. This indicates that, while GUI screenshots carry higher information density, the image understanding capabilities of the MLLM are required to achieve a certain level of effectiveness. Otherwise the model is not capable of extracting relevant features from the GUI prototype image. In such a case, the model potentially can more easily extract relevant information from the detailed text annotations. Moreover, comparing differences between model capacities, we observed that the larger variant of *GPT-4.1* consistently and statistically significantly outperformed its respective counterparts (e.g., *Original* vs. *Mini*, *Original* vs. *Nano*, *Mini* vs. *Nano*) for both text and image variants across $NDCG@3,5,10,15$ (except for $NDCG@5$ between *Original* and *Mini*). Furthermore, statistically significant differences were also observed between both image and text

		#Tokens ($k=1$)		Cost (k)		Time (k)	
		Input	Output	100	500	100	500
Text	GPT-4.1	179.77	6.00	\$0.041	\$2.04	2.4s	12s
	GPT-4.1 Mini	179.77	6.00	\$0.008	\$0.041	2.6s	13s
	GPT-4.1 Nano	179.77	7.11	\$0.002	\$0.010	2s	10s
	Gemini-2.5 Flash	183.15	6.86	\$0.007	\$0.036	11s	55s
	Gemini-2.5 Pro	183.15	10.20	\$0.033	\$1.165	10.1s	50.5s
	Claude Sonnet 3.7	195.49	6.60	\$0.069	\$0.343	5.5s	27.5s
	Claude Sonnet 4.0	201.29	81.63	\$0.183	\$0.914	20.2s	101s
Image	GPT-4.1	1089.02	6.00	\$0.223	\$1.113	12.4s	62s
	GPT-4.1 Mini	2154.96	6.00	\$0.087	\$0.436	9s	45s
	GPT-4.1 Nano	3242.41	5.95	\$0.032	\$0.163	8s	40s
	Gemini-2.5 Flash	1088.37	6.34	\$0.034	\$0.171	12.2s	61s
	Gemini-2.5 Pro	1088.37	9.54	\$0.146	\$0.728	16.2s	81s
	Claude Sonnet 3.7	1439.31	6.60	\$0.442	\$2.208	42.8s	214s
	Claude Sonnet 4.0	1445.31	96.60	\$0.578	\$2.892	31s	155s

Table 4.3: *Token usage, cost and runtime* for the MLLM rerankers (means over gold standard ($\#Queries=100$), k = number of retrieved GUIs to rerank, $\#Celery\ workers = 10$).

variants (*Original* vs. *Nano* and *Mini* vs. *Nano*) for $P@3,5,7,10$, AP , MRR and $HITS@1$. Figure 4.6 shows multiple ranking plots for three queries and the *Oracle* (perfect ranking), BERT-LTR (2) and *GPT-4.1 (Image)*, which also indicates the improved performance of the MLLM-based GUI reranking, with more relevant GUIs at higher rank positions.

Answer to RQ₁: MLLM-based GUI reranking substantially improves the ranking performance compared to prior BERT-LTR baselines, statistically significant across all metrics (except $HITS@10,15$). The image variant consistently outperforms the text variant across mean values (except for *Nano*) and significantly for $NDCG@k$. The higher capacity models significantly outperformed the lower capacity models for $NDCG@k$.

4.4.2 RQ₂: MLLM-based Reranking Effectiveness-Cost Trade-offs

Table 4.3 shows multiple cost metrics associated with different MLLM-based GUI reranking approaches, including the *input* and *output tokens* required for reranking a single GUI ($k=1$), the *monetary Application Programming Interface (API) costs* according to the providers³ ($k=100$ and $k=500$) and the required time in seconds ($k=100$ and $k=500$, with 10 parallel *Celery* workers). As can be observed, when comparing the text-based MLLMs to their respective image-based variants model-wise, there is a substantial increase across all considered performance metrics. Therefore, while the image-based models outperform

³The monetary API costs are reported based on the *input* and *output token* price per 1M tokens as of December 2025 for *OpenAI* (OpenAI, 2025d), *Google* (Google, 2025b) and *Anthropic* (Anthropic, 2025a).

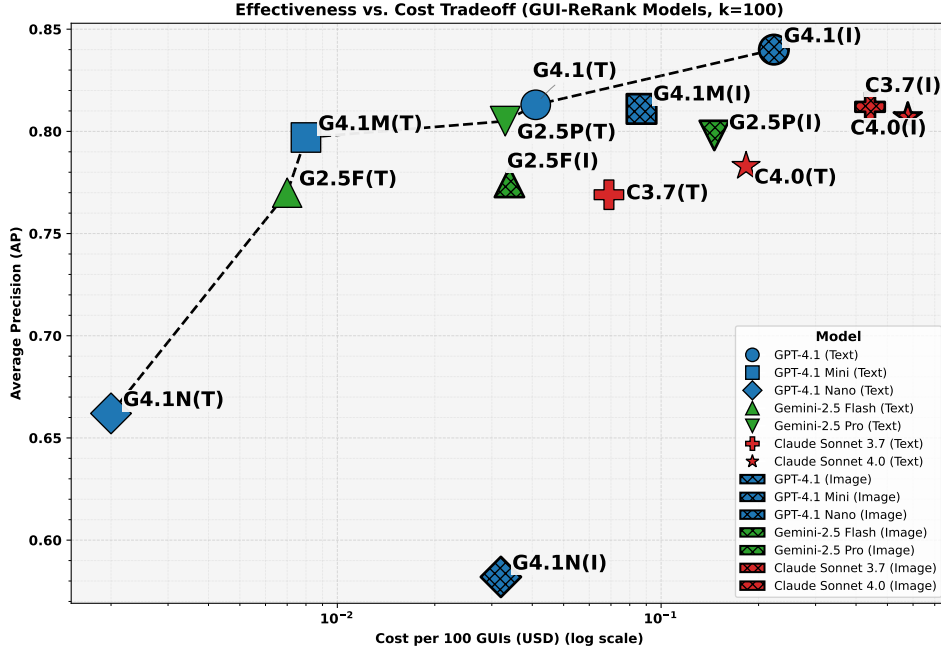


Figure 4.7: Effectiveness (as *AP*) and *monetary Application Programming Interface (API) cost* ($k=100$) (log scale) trade-off plot with *Pareto frontier*: all text-based *GPT-4.1* and *Gemini-2.5* models, as well as image-based *GPT-4.1* represent *Pareto-efficient* solutions.

the text-based models in reranking effectiveness, their application entails much higher cost. For example, the image-based *GPT-4.1* improves the mean *AP* by 3.32%, while the associated *API costs* ($k=100$) increase by 543.9%. This shows that substantial costs can be saved by applying text-based variants, while losing only little effectiveness according to the *AP*. Figures 4.7 and 4.8 show trade-off plots with the *Pareto frontier* (*Pareto-efficient* solutions), illustrating *AP* in relation to *API cost* ($k=100$) and time ($k=100$), respectively. When considering *API cost*, all text-based *GPT-4.1* and *Gemini-2.5* models lie on the *Pareto frontier*, and, additionally, *GPT-4.1* is the only image variant. This indicates that text inputs for MLLM-based GUI reranking offer a valuable effectiveness-cost trade-off. However, it should be noted that the employed gold standard focuses on functional NLR (the primary objective for *REI* and *RVal*). The gains could be larger in scenarios where non-functional NLR (*NFR*) (such as the GUI design) play a significant role, since the expressiveness of detailed GUI text annotations might be limited for these dimensions.

Answer to RQ₂: While image variants of MLLM-based GUI reranking outperform text variants in effectiveness, text variants are considerably more cost-efficient (e.g., *GPT-4.1 (Image)* improves mean *AP* over *GPT-4.1 (Text)* by 3.32% while entailing 543.9% higher *API costs* ($k=100$)). Moreover, all text-based *GPT-4.1* and *Gemini-2.5* models provide *Pareto-efficient* solutions for MLLM-based GUI reranking from NLR.

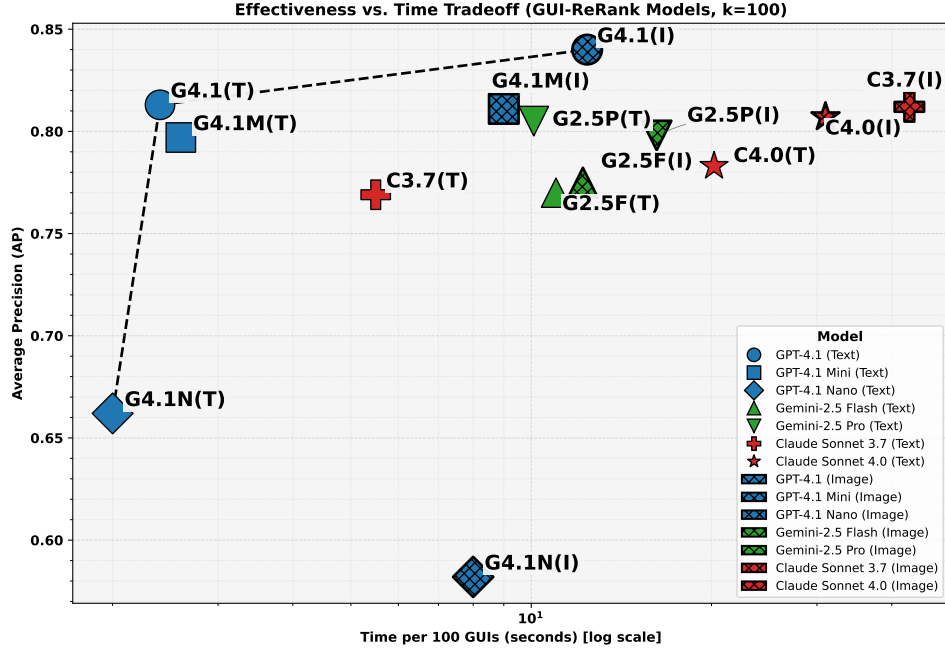


Figure 4.8: Effectiveness (as *AP*) and *time* required for reranking (in *seconds* ($k=100$ and $\#Celery\ workers=10$)) (log scale) trade-off plot with *Pareto frontier*: text-based *GPT-4.1* (*Original* and *Nano*) and image-based *GPT-4.1* models provide *Pareto-efficient* solutions.

4.5 Threats to Validity

Internal Validity. Threats related to internal validity mainly lie in the employed NLR-based GUI retrieval gold standard. Particularly, bias regarding the creation of queries, the pooling of potentially relevant GUIs, the subjectivity and inconsistency of relevance judgments are critical. However, we discussed mitigation strategies already (Section 3.5).

External Validity. Threats related to external validity again mainly are represented by the employed NLR-based GUI retrieval gold standard, including the generalizability of encompassed NLR queries, the utilized *Rico* GUI repository diversity and the population of crowdsourcing workers. Again, we discussed mitigation strategies before (Section 3.5).

4.6 Limitations

While the presented *GUI-ReRank* approach achieves high effectiveness across many evaluation metrics by utilizing MLLM-based GUI reranking, it also exhibits some limitations. First, even in low-temperature settings for non-reasoning MLLMs, the output might still encompass some variance and, therefore, reranking results might slightly vary between different runs. However, we manually evaluated multiple examples across runs and found few deviations in the GUI rankings. Second, employing MLLMs for GUI reranking

entails high cost in terms of monetary API cost, token consumption and time (latency). However, as we have shown in our experiments, text-based reranking offers high effectiveness with substantially reduced costs. Finally, while the multi-dimensional retrieval and reranking with *GUI-ReRank* is customizable, manually tuning for appropriate weights requires some expertise and experience to fully leverage the available search capabilities.

4.7 Related Work

As described earlier, many GUI retrieval approaches have been proposed in research before. For example, *GUIFetch* (Behrang et al., 2018) and *Swire* (Huang et al., 2019) both propose sketch-based GUI retrieval from large-scale GUI repositories by utilizing ML approaches. Similarly, *VINS* (Bunian et al., 2021) conducts wireframe-based GUI retrieval via neural image and GUI component embedding models. *Gallery DC* (Chen et al., 2019a) devises a GUI component retrieval approach, with components automatically extracted from a large-scale GUI collection. However, prior research neglected to investigate multi-dimensional and multi-modal NLR-based GUI retrieval and reranking with MLLMs. For a more detailed discussion of the related work on GUI retrieval, please refer to the previous chapter (see Section 3.7).

Moreover, utilizing LLMs for reranking text documents and passages has been investigated in research before. For example, *Pairwise Ranking Prompting (PRP)* (Qin et al., 2024) represents a simplified LLM-based text reranking approach by comparing document pairs. Mozafari and Jatowt (2025) conduct a comprehensive evaluation of text reranking approaches, showing that LLM reranking achieves higher effectiveness for familiar queries. Furthermore, *RagVL* (Chen et al., 2024) represents an instruction-tuned MLLM-based reranking approach for images, which enhances the image filtering and RAG capabilities. While prior research mainly focused on LLM-based reranking for text documents, MLLM-based reranking is increasingly explored in recent work. However, we were the first to propose and explore multi-dimensional MLLM-based GUI reranking.

4.8 Conclusion

The proposed approach in this chapter was driven by the challenge **C1-2**: *How can the semantic representation gap between NLR (including NFR) and multi-dimensional GUI prototypes be reduced to enable more effective GUI ranking?* To tackle this challenge, we proposed *GUI-ReRank*, a novel multi-dimensional embedding-based constrained GUI retrieval and MLLM-based GUI reranking approach from NLR. In particular, the MLLM-based GUI repository annotation pipeline provides multi-dimensional textual representations of GUIs for enhanced retrieval. Moreover, by employing MLLMs with advanced NLU and image understanding capabilities, the multi-dimensionality of GUIs can be more effectively reflected and evaluated for rank scoring. This stands in contrast to previous approaches, which relied solely on basic text fragments extracted from the GUI hierarchy data and thus neglected relevant information contained in the GUI images.

CHAPTER 5

SELF-ELICITATION OF REQUIREMENTS WITH AUTOMATED GUI PROTOTYPING

The first chapter of the thesis introduced novel approaches for NLR-based GUI ranking with a focus on BERT-LTR models, followed by the second chapter, which introduced MLLM-based GUI reranking, achieving state-of-the-art performance. In this chapter, we continue with challenge **C1**, namely, mapping NLR to GUI prototypes. While the previous two approaches focused on providing GUI prototyping support for the requirements analyst or prototype developer via retrieval and reranking techniques, this approach emphasizes enabling stakeholders themselves to conduct REI and RVal via automated GUI prototyping assistance, which integrates GUI and feature retrieval techniques. In particular, this work has been previously published (Kolthoff et al., 2024a)¹. Our interactive prototype, code, evaluation results, and demonstration video are publicly available².

Personal Contribution. The core idea and concept for this work were proposed by me. Moreover, I implemented all the techniques and the prototype. I designed the evaluation concept, the user study, and conducted the data analysis. Finally, I wrote the manuscript.

5.1 Motivation

The NLR-based GUI retrieval and reranking techniques proposed in the previous two chapters enable rapid mapping of NLR to relevant GUI prototypes. Moreover, these techniques were integrated into a rapid prototyping approach *RaWi*, which provides

¹This section is adapted from: **Kolthoff, Kristian**, Bartelt, Christian, Ponzetto, Simone Paolo, and Schneider, Kurt. Self-Elicitation of Requirements with Automated GUI Prototyping. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE, A*)*, Sacramento, CA, USA, October 2024, pages 2354–2357. ACM. Sections are directly reused with only minor adaptations (i.e. thesis Section 5.1 matches paper Section i)

²Materials for this chapter are available at <https://github.com/kristiankolthoff/SERGUI-Prototyping> and the prototype demonstration video at <https://youtu.be/pzAAB9Uht80>

capabilities for adapting the GUI prototype to particular requirements through a data-driven editor. This method substantially improves GUI prototyping productivity, as demonstrated through experimental evaluation, and focuses on facilitating the creation of GUI prototypes, while also assuming that users possess sufficient knowledge to conduct the GUI prototyping, elicitation and validation process. In particular, *RaWi* provides neither guidance nor a structured approach for the overall GUI prototyping process, which restricts its direct applicability to stakeholder-driven REI and RVal.

Prior research proposed various approaches aiming to simplify the GUI prototyping procedure. For instance, the *Fast Feedback* technique (Schneider, 2007) reduces the number of necessary elicitation sessions with stakeholders by providing tool-based support to rapidly and interactively create pen-and-paper prototypes combined with use cases, directly allowing stakeholder feedback to be incorporated. However, an experienced analyst is required to conduct the initial elicitation. In the REI approach proposed by Teixeira et al. (2014), analysts create initial GUI prototypes based on an initial meeting with stakeholders, subsequently enabling the stakeholders to access and modify the GUI prototype through a web-based GUI prototyping tool, but the approach provides no automatic guidance for prototyping and elicitation. Similarly, the *Graphical Requirements Collector* (Moore and Shipman, 2000) requires users to directly create their own GUI prototypes by constructing them bottom-up and augmenting them with textual requirements. This integrates stakeholders closely into the GUI prototyping process, but, due to the lack of guidance, requires them to have experience with the GUI prototyping process. Moreover, *LadderBot* (Rietz and Maedche, 2019) enables automatic dialogue-based elicitation with stakeholders by applying *laddering*, a structured interview technique. However, this automatic dialogue-based elicitation approach focuses solely on the collection of simple textual requirements and therefore cannot be applied to the automated initial elicitation with GUI prototyping. Hence, to summarize, the work presented in this chapter is based on the leading research question of challenge **C1-3**: *How can we provide automatic assistance for self-elicitation of requirements with GUI prototyping?*

To tackle this challenge, we propose *SERGUI*, a novel GUI prototyping approach that enables stakeholders to perform *SER* for interactive software systems through automated NLR-based GUI prototyping assistance. Furthermore, we provide a LLM-based contextualized GUI feature recommendation mechanism to promote the elicitation of requirements. The general notion of *SER* refers to guiding stakeholders to uncover their own requirements and has originally been introduced by *LadderBot* (Rietz and Maedche, 2019), thus reducing the effort of analysts in the initial REI phase, thereby closely integrating stakeholders into the REI process. By extending the notion of *SER* with automated GUI prototyping, we exploit the benefits of GUI prototypes as a requirements specification artifact, facilitating the obtainment of fast stakeholder feedback and achieving early clarification of requirements. *SERGUI* enables users to rapidly recombine and easily adapt existing GUIs from the large-scale GUI repository *Rico* (Deka et al., 2017) to incrementally build their own GUI prototypes, as shown in Figure 5.1. In *SERGUI*, stakeholders are guided through the GUI prototyping process by a rule-based dialogue assistant, which provides relevant GUI screens and proactively recommends contextually

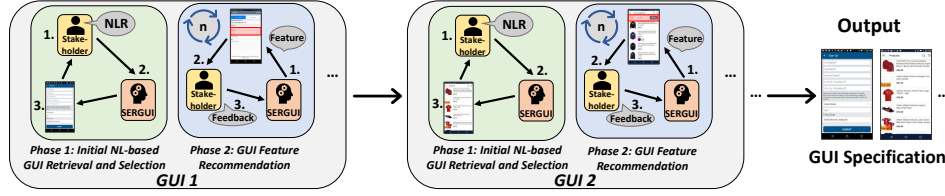


Figure 5.1: REL via interactive GUI prototyping assisted by our *SERGUI* approach enabling to rapidly map NLR into relevant GUI prototypes (*Phase 1*) followed by proactively stimulating REL and RVal via prompting-based GUI feature recommendations (*Phase 2*).

relevant GUI features. Therefore, *SERGUI* enables stakeholders to autonomously build their GUI prototypes requiring little prior GUI prototyping process experience. Overall, *SERGUI* produces a GUI prototype output in the form of selected GUIs and additional *aspect*-GUIs, which focus on and highlight particular GUI features (i.e. *aspects*). The produced GUI prototypes are intended to serve as an initial requirements specification artifact for analysts.

To evaluate *SERGUI*, we conducted a controlled user study. In particular, we asked 12 participants to create several GUI prototypes with our approach and evaluated the effectiveness of the GUI feature recommendation mechanism, the matching of features in GUIs for visualization, the GUI reranking performance and overall usability of the *SERGUI* approach. The results of the evaluation indicate that our approach can effectively support inexperienced users in rapidly creating GUI prototype specifications and is capable of recommending relevant features in combination with high usability (SUS).

Contributions. With this approach, we make the following research contributions:

- *Novel GUI prototyping assistance approach:* we present *SERGUI*, the first approach for *Self-Elicitation of Requirements (SER)* with GUIs enabled by an automatic GUI prototyping assistant to support fast feedback, driven by a NLR-based GUI retrieval and feature-based GUI reranking mechanism and the large-scale GUI repository *Rico*.
- *LLM-based GUI feature recommendation:* LLM-based contextualized GUI feature recommendation approach to proactively stimulate REL, integrated with the GUI repository *Rico* to facilitate the visualization of recommended GUI features for RVal.

5.2 Approach: SERGUI

This section delineates the *SERGUI* approach for enabling SER based on automated GUI prototyping. The traditional REL process with GUI prototyping typically consists of the following main phases: To begin with, an initial elicitation interview is conducted in a joint session between the stakeholder and the analyst, followed by initial requirements modeling (e.g., in the form of elementary *use cases* or *User Stories (USs)*). Afterwards, an iterative process is conducted in the steps of (a) requirements visualization in the form of

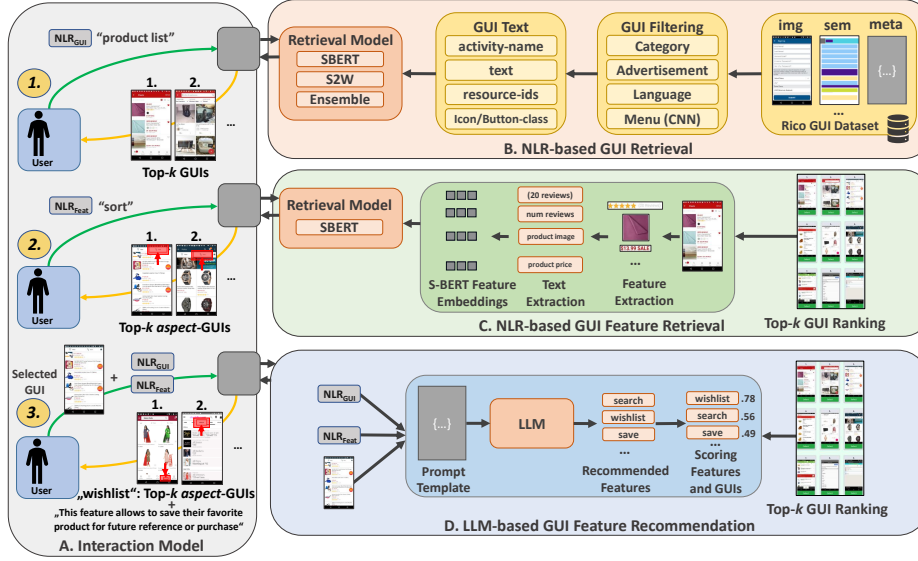


Figure 5.2: Overview of the *SERGUI* approach with (A) the interaction model showing the main interaction mechanisms, (B) NLR-based GUI retrieval approach (*Rico*), (C) the NLR-based GUI feature retrieval and (D) the LLM-based GUI feature recommendation.

GUI prototypes, (b) gathering stakeholder feedback related to the tangible GUI prototype and (c) the subsequent adaptation of the requirements and update of the respective GUI prototype. After several iterations, an adapted and validated GUI prototype is produced as output (see Chapter 2, Section 2.2.2, for GUI prototyping process model details). However, this procedure necessitates numerous synchronous meetings between stakeholders and requirements analysts, often with multiple weeks in between, potentially delaying the overall application development process (Schneider, 2007).

SERGUI is an approach to automating the initial REI phase by leveraging a comprehensive GUI repository in combination with a novel feature-based GUI reranking and GUI feature recommendation technique. Subsequently, an overview of our approach is given and depicted in Figure 5.2. *SERGUI* is divided into multiple components: First, (A) an interaction model encompassing the essential interaction mechanisms between the user and the automatic GUI prototyping assistance. Second, (B) a NLR-based GUI retrieval technique building on previously presented work (see Chapter 3). Third, (C) a NLR-based GUI feature retrieval mechanism. Fourth, (D) an LLM-based GUI feature recommendation mechanism to proactively suggest potentially relevant GUI features and simultaneously illustrate them by matching them to the GUIs from the top- k ranking.

5.2.1 Interaction Model

SERGUI is built as an interactive dialogue-based approach providing guidance through the GUI prototyping process, facilitating the close integration of stakeholders. The

interaction model encompasses three essential patterns that are repeated for each GUI in the application prototype. Initially, users can specify their NLR for a particular GUI (denoted by NLR_{GUI}), to which the approach responds with a top- k GUI ranking, as illustrated in Figure 5.2 *A1*. This ranking represents the best matches as computed by the NLR-based GUI retrieval model. The GUI ranking showcases many potentially relevant GUIs encompassing numerous variations, which already stimulates REI. Users can reformulate their NLR in case of an inadequate relevance of the ranked GUI.

Second, users are enabled to specify additional NLR for individual features (denoted by NLR_{Feat}), to which the approach responds with a top- k ranking of *aspect*-GUIs, as depicted in Figure 5.2 *A2*. An *aspect*-GUI is a GUI containing one particular highlighted GUI feature (i.e. *aspect*) that is considered relevant to the user. Therefore, the approach presents a ranking of matching GUIs potentially containing the formulated GUI feature. If the user finds a relevant *aspect*-GUI matching their needs, then it can be selected. Subsequently, the GUI ranking is recomputed based on the new aspects (potentially improving the ranking) and the *aspect*-GUI will be saved as part of the GUI prototype.

Third, when the user initially selects a GUI already satisfying many features plus the specified *aspect*-GUIs, then the approach utilizes the NLR_{GUI} , collection of NLR_{Feat} and the selected GUI to proactively recommend potentially relevant GUI features, as illustrated in Figure 5.2 *A3*. In an iterative fashion, the *SERGUI* approach will then present each GUI feature with a short textual explanation and the top- k ranking of potentially matching *aspect*-GUIs encompassing and visualizing the respective GUI feature. Similarly to before, a relevant *aspect*-GUI can directly be selected, which will then be added to the overall GUI prototype specification and exploited for GUI reranking. Moreover, if the GUI feature is relevant but no matching *aspect*-GUI could be retrieved, then the GUI feature will be added to the prototype specification in textual form. *SERGUI* allows users to walk through these three patterns and eventually, the overall GUI specification encompasses the finally selected GUI, potentially a collection of *aspect*-GUIs and a collection of textual GUI requirements. This GUI specification is then added to a simple linear application prototype, allowing users to briefly skip through the entire GUI prototype.

5.2.2 NLR-Based GUI Retrieval

In order to achieve NLR-based GUI retrieval, we mainly adopt the work presented in Chapter 3 and provide an extension for filtering and retrieval models. As before, the GUI retrieval mechanism exploits the GUI repository *Rico* (Deka et al., 2017). For *Rico*, the applied filtering steps and the extended GUI retrieval model, we provide a brief summary. Figure 5.3 depicts the overall procedure for the NLR-based GUI retrieval component.

Rico GUI Dataset. *Rico* (Deka et al., 2017) represents the most sophisticated GUI dataset for mobile applications currently available, compared to other research-based GUI datasets such as *ReDraw* (Moran et al., 2018) and *ERICA* (Deka et al., 2016). It is (i) the largest semi-automatically harvested GUI dataset (9,772 unique *Android* applications from 27 categories encompassing 72,219 GUIs), (ii) incorporates a wide range of

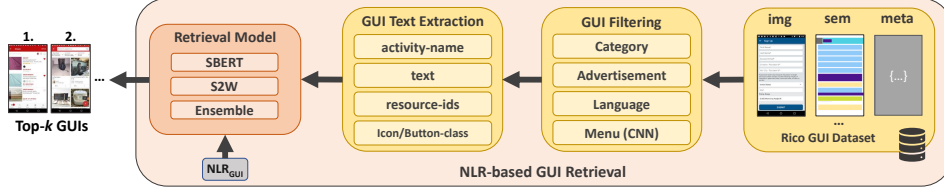


Figure 5.3: NLR-based GUI retrieval approach: *Rico* GUI dataset is utilized as the foundation, multiple GUI filtering steps are conducted, followed by the text extraction from the GUI hierarchy data and the retrieval model computes a score to produce top- k GUIs.

heterogeneous application domains and (iii) provides multiple artifacts including GUI screenshots, Extensible Markup Language (XML)-based GUI hierarchy and metadata. For NLR-based GUI ranking, we exploit the GUI hierarchy, extract and preprocess multiple text segments including the *activity name*, for each GUI component the *displayed text*, the *resource-id* as an internal name given by the developers and special semantic attributes included in the *Rico* GUI dataset such as *semantic button* and *icon categories*.

Dataset Filtering. To improve the overall quality of the GUI repository, we applied a filtering pipeline adopted from Chapter 3. This pipeline includes removal of GUIs from the *entertainment (game)* category, GUIs with *advertisement overlays* based on a heuristic and *non-English* GUIs utilizing a language detection framework (Shuyo, 2010). In addition, manual inspection of GUI rankings indicated that *Rico* contains a large portion of GUIs showing an opened side menu, providing no benefit for elicitation and additionally decreasing GUI retrieval effectiveness. To automatically remove these GUIs, we manually constructed a dataset consisting of GUIs with and without opened menus. Afterwards, we trained a *Convolutional Neural Network (CNN)* classifier (Gu et al., 2018a) (three *convolution* and *pooling* layers) and provided as input a combination of both the original GUI screenshot image and the semantic annotation image as grayscale variants. We trained the *CNN* model for 6 epochs (*adagrad* optimizer and *binary cross-entropy* as loss) and achieved a satisfying performance on a separate test set (*Precision*=0.9818 / *Recall*=0.7012). We focused on achieving high precision to avoid removal of adequate GUIs. We filtered 23,817 GUIs (9,363 by the *CNN*) resulting in 48,402 GUIs remaining.

Adapted GUI Retrieval Model. As a GUI retrieval approach, we adopt the strong pretrained embedding-based SBERT model presented in Chapter 3³. This model computes the ranking using *cosine similarity* between the embedded query $\mathbf{e}(NLR_{GUI}) = f_{\text{emb}}(NLR_{GUI})$ and embedded text representation of the GUI $\mathbf{e}(G_i) = f_{\text{emb}}(G_i)$ as

$$\mathbf{S}_1(NLR_{GUI}, G_i) = \text{cos_sim}(\mathbf{e}(NLR_{GUI}), \mathbf{e}(G_i)) \quad (5.1)$$

with $\mathbf{e}$ referring to the respective SBERT embedding. Moreover, we extend this GUI

³Note that the earlier presented BERT-LTR models are primarily reranking approaches, which cannot be utilized for initial GUI retrieval due to computational overhead and latency, whereas SBERT is suitable. In addition, *GUI-ReRank* was published after this work, but was presented earlier for coherence with *RaWi*.

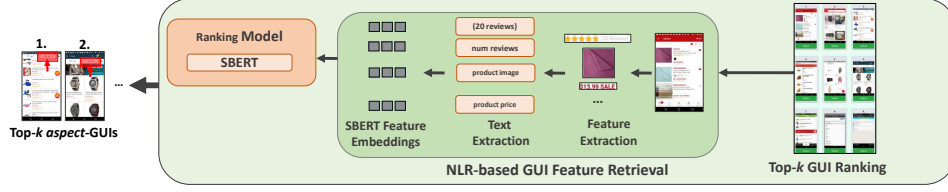


Figure 5.4: NLR-based GUI feature retrieval approach: starting from the top- k GUIs, individual GUI features are extracted (GUI components), followed by extracting text representations and then computing SBERT embeddings to produce feature rankings. Each of the features is represented as part of an *aspect*-GUI, highlighting its appearance.

ranking score by incorporating the *Screen2 Words* ($S2W$) dataset (Wang et al., 2021), adding another representation to the GUI prototypes to score against. $S2W$ represents a large collection of manually crafted high-level NL descriptions of *Rico* GUIs, providing five descriptions per GUI from different annotators. To effectively utilize the $S2W$ data for GUI G_i as $S2W(G_i) = \{d_{i,1}, \dots, d_{i,5}\}$, we similarly compute the average over the *cosine similarity* between the query embedding $\mathbf{e}(NLR_{GUI})$ and the embeddings of all five descriptions $d_{i,j}$, namely, $\mathbf{e}(d_{i,j}) = f_{\text{emb}}(d_{i,j})$ as

$$\mathbf{S}_2(NLR_{GUI}, G_i) = \frac{1}{5} \sum_{j=1}^5 \cos_sim(\mathbf{e}(NLR_{GUI}), \mathbf{e}(d_{i,j})) \quad (5.2)$$

By incorporating the average scores across five high-level descriptions from different annotators, we obtain a more robust retrieval score. Subsequently, we compute an ensemble between the two GUI scores to combine them as follows

$$\mathbf{S}(NLR_{GUI}, G_i) = \alpha \mathbf{S}_1(NLR_{GUI}, G_i) + (1 - \alpha) \mathbf{S}_2(NLR_{GUI}, G_i) \quad (5.3)$$

with $\alpha \in [0, 1]$. By creating an ensemble GUI ranking model using both the extracted GUI text representations and the high-level descriptions of $S2W$, we facilitate obtaining a more flexible and robust GUI ranking in contrast to utilizing a single representation only.

5.2.3 NLR-based GUI Feature Retrieval

To achieve NLR-based GUI feature retrieval, we employ the top- k GUIs representing potentially relevant context for matching the features, as depicted in Figure 5.4. Here, we restrict GUI features to individual GUI components. From the top- k GUIs, textual representations of the GUI components are extracted, including multiple texts for each component (such as *displayed text*, *resource-id* and *semantic classes*). The text representation of the j -th feature of GUI G_i is subsequently embedded with SBERT as $\mathbf{e}(G_{i,j}) = f_{\text{emb}}(G_{i,j})$ to compute the score against the corresponding NLR_{Feat} SBERT embedding $\mathbf{e}(NLR_{Feat}) = f_{\text{emb}}(NLR_{Feat})$ based on the *cosine similarity* as

$$\mathbf{S}_F(NLR_{Feat}, G_{i,j}) = \cos_sim(\mathbf{e}(NLR_{Feat}), \mathbf{e}(G_{i,j})) \quad (5.4)$$

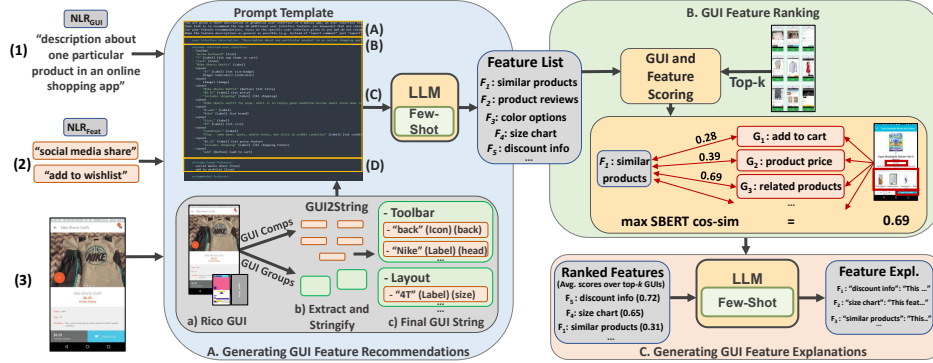


Figure 5.5: GUI feature recommendation approach overview of *SERGUI* with (A) generating LLM-based GUI feature recommendations via FS prompting, (B) scoring recommended features and top- k GUIs, and (C) generating LLM-based feature explanations.

5.2.4 LLM-based GUI Feature Recommendation

LLMs recently gained popularity due to their ability to rapidly learn and adapt to new tasks solely based on FS examples (Brown et al., 2020; Kojima et al., 2022). Particularly, these models can essentially be adapted to a plethora of specific tasks through a technique called *prompting* (Liu et al., 2023). In prompting, solely a textual instruction for the task and several examples of input and expected output are provided to the LLM (see Chapter 2 Section 2.3.4). Due to the vast amount of knowledge (including technical and domain knowledge) embodied in the LLMs and its accessibility via prompting, exploiting these models within *SERGUI* holds large potential to facilitate automatic elicitation. Subsequently, we introduce our novel approach for LLM-based GUI feature recommendation. An overview of the approach architecture is illustrated in Figure 5.5.

Generating GUI Feature Recommendations. Within the *SERGUI* approach, the GUI feature recommendations are based on the context of (1) the initial textual requirements for the GUI denoted by NLR_{GUI} , (2) a collection of already textually specified features by the user as NLR_{Feat} and (3) an initially selected GUI from the top- k GUI ranking. With these contextual inputs, we fill in a *prompt template* encompassing the following sections. First, (A) we provide the *task instructions* asking the model to recommend the top-30 GUI features given the described context. We emphasize that the model should thoroughly examine the provided GUI and features to avoid recommending already included features. Moreover, we instruct the model to focus on recommending features for the particular GUI at hand instead of more general application-level features (e.g., *help*, *user support*). Next, (B) the template displays the initial requirements as NLR_{GUI} . Third, (C) the initially selected GUI is provided to the model⁴. Since the original XML-based GUI hierarchy contains a large amount of information, often

⁴Note that at the time of implementing the approach, more powerful MLLMs, which have highly effective image understanding capabilities, were not readily available yet (such as *GPT-4o* (OpenAI, 2025e)).

spanning several thousand lines, transforming the GUI hierarchy into a more abstract and focused representation is necessary. This both helps to reduce consumed context length in the LLM and enables the model to focus on the most important aspects. To this end, we first produce a string representation for each GUI component in the following abstract format, accompanied by three concrete examples (*Label*, *Button* and *Text Input*):

"uicomp-text" (uicomp-type) (uicomp-name)
"+7.10" (Label) (price Change TV)
"Install App" (Button) (native Ad Call To)
"Example: 'New York'" (Text Input) (location)

Additionally, we extract grouping components from the semantic annotations of *Rico* (Deka et al., 2017) including types such as *List Item*, *Card* and *Toolbar*, among others. To also find groups for remaining components not grouped by the previous groups, we further extracted layout groups from the original GUI hierarchy by matching them to the GUI components. Finally, we construct the GUI representation as two-level bullet points, the outer level being the layout groups and the inner level being their respective GUI components. Before producing the string representation, the layout groups are sorted based on their bounds from top-left to bottom-right and similarly the GUI components within each group. Fourth, (D) the already specified NLR_{Feat} is added to the template as additional bullet points. Following the FS technique for ICL, we provided several recommendation examples as additional context for each GUI feature recommendation.

GUI Feature Ranking. To compute a GUI feature ranking, each of the recommended features is matched against the features in each GUI in the top- k GUI ranking. To obtain a confidence score that a GUI G_i contains a predicted feature (NLR_{Feat}) and to determine which feature within the GUI matches best (as *aspect-GUI*), we compute

$$\mathbf{S}_g(NLR_{Feat}, G_i) = \max_j \mathbf{S}_F(NLR_{Feat}, G_{i,j}) \quad (5.5)$$

where $G_{i,j}$ refers to feature j of GUI G_i and $\mathbf{S}_F$ uses *cosine similarity*, as illustrated in Figure 5.5. Additionally, for each recommended feature, we compute a feature score as

$$\mathbf{S}_{pf}(NLR_{Feat}) = \frac{1}{k} \sum_{i=1}^k \mathbf{S}_g(NLR_{Feat}, G_i) \quad (5.6)$$

over the top- k GUI ranking to estimate the feature coverage among top- k GUIs. Predicted features are then recommended, sorted based on $\mathbf{S}_{pf}$ and shown as the top- k *aspect-GUIs*.

Generating GUI Feature Explanations. To convey the meaning of a predicted GUI feature, we already present users with the top- k *aspect-GUI* ranking as a visualization (as a form of validation), sorted by the score $\mathbf{S}_g$. Additionally, we employ a second FS prompt and instruct an LLM to generate a short textual explanation for each GUI feature, as illustrated in Figure 5.5. This explanation is then utilized in the dialogue with the user.

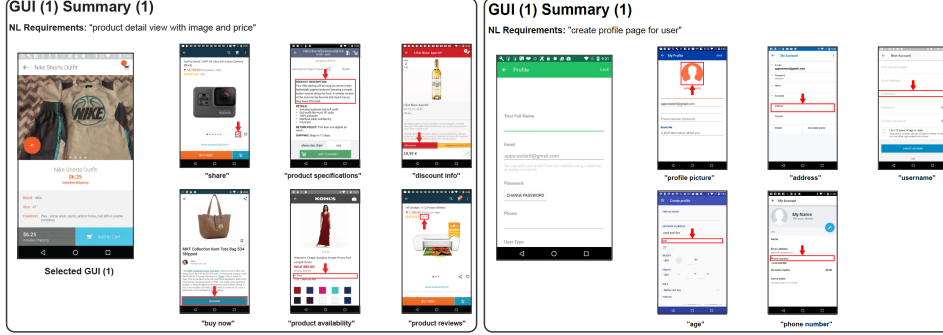


Figure 5.6: Example output of *SERGUI* showing two GUI summaries each with the selected GUI and several *aspect*-GUIs for a *product detail view* and a *create profile page*.

5.2.5 Feature-based GUI Reranking

Based on positive feedback from the user by selecting an *aspect*-GUI from the top- k *aspect*-GUI ranking, we can compute a reranking of the GUIs. Users often explore no more than the top-20 or top-30 GUIs in the ranking, missing potentially relevant GUI features. By reranking the top- k GUIs based on additionally selected GUI features, users potentially find more relevant GUIs at the top of the GUI ranking or they can keep their original choice. We compute the reranking $\mathbf{S}_{RR}$ of GUI G_i with $\beta \in [0, 1]$ as follows

$$\mathbf{S}_{RR}(G_i) = \beta \mathbf{S}(NLR_{GUI}, G_i) + (1 - \beta) \frac{1}{|F|} \sum_{NLR_{Feat} \in F} \mathbf{S}_g(NLR_{Feat}, G_i) \quad (5.7)$$

The reranking score $\mathbf{S}_{RR}(G_i)$ is an ensemble between the initial GUI ranking based on NLR_{GUI} and the normalized sum over all feature scores $\mathbf{S}_g(NLR_{Feat}, G_i)$. Only feature requirements NLR_{Feat} are added to the set F if the user selected one GUI from the top- k *aspect*-GUI ranking. Next, the original $\mathbf{S}_g(NLR_{Feat}, G_i)$ scores are adapted as follows: The score of the selected GUI will be set to 1, scores of GUIs ranked higher than the selected GUI will be set to 0 and the scores of GUIs ranked lower are unchanged.

5.2.6 GUI Prototype Output

Since *SERGUI* is based on fixed *Rico* GUIs, users potentially are not able to find a GUI perfectly fitting their requirements. Hence, we introduced the idea of *aspect*-GUIs, enabling users to choose a single already well-matching GUI (supported by feature recommendations and reranking) plus a collection of *aspect*-GUIs, which contain only a single aspect relevant for the user. At the end of the GUI prototyping process, *SERGUI* produces an application summary including (i) a visualization of the overall linear application wireframe and (ii) for each GUI the main selected GUI, the collection of *aspect*-GUIs as illustrated in Figure 5.6 and an additional collection of textual requirements, representing

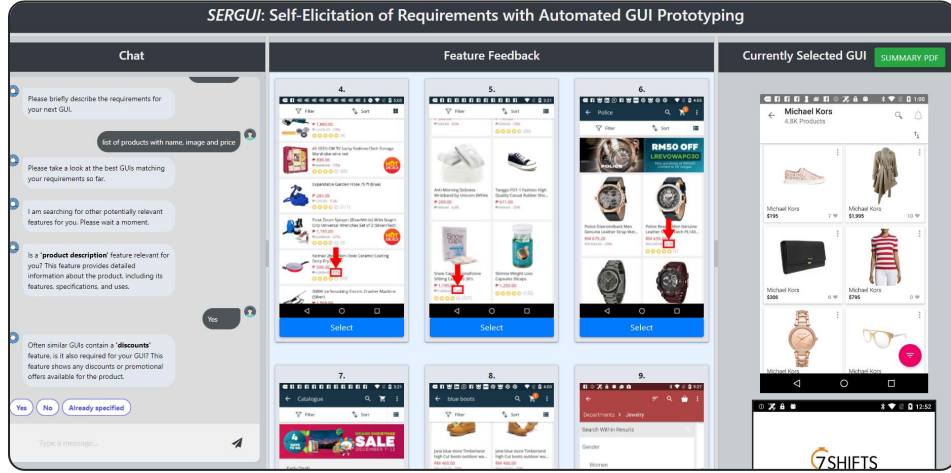


Figure 5.7: Web-based prototype implementation of the approach with a *chat-section*, *workbench* and *GUI prototype preview*, currently showing the top- k *aspect-GUI* ranking.

recommended features that were relevant but no relevant *aspect-GUI* was found. This GUI prototype can then be employed by analysts as a starting point for further elicitation.

5.2.7 Prototype Implementation

We implemented the prototype of the presented *SERGUI* approach as a web application using HTML, CSS and JS. The application consists of a *chat-section*, a *workbench* for showing the top- k GUIs or *aspect-GUIs*, respectively. The implementation provides a *GUI prototype preview*. Figure 5.7 illustrates *SERGUI* with a top- k *aspect-GUI* ranking.

5.3 Experimental Evaluation

In this section, we delineate the design of our experiments to evaluate *SERGUI*. The main goal of our experimental evaluation is to assess the overall capabilities of the approach to stimulate REI and facilitate SER. This is divided into (i) assessing the relevance of the GUI feature recommendations as a means to stimulate the automatic elicitation of interesting features with users, (ii) measuring the relevance of the ranked *aspect-GUIs* on the basis of NLR, (iii) assessing the performance of the feature-based GUI reranking and (iv) measuring the overall perceived usefulness and gaining additional user insights. The first two points evaluate our second contribution, while the last two points evaluate our first contribution. To this end, we formulate the following four research questions:

- **RQ₁**: *How relevant are the contextualized LLM-based GUI feature recommendations?* To answer this question, we conducted a user study where participants employed the approach to create several prototypes and provided relevance annotations for the recommended GUI features. Given these relevance annotations, we computed several standard binary IR metrics including *AP*, *MRR*, and *P@k*.

- **RQ₂:** *How relevant are the visualizations of the matched GUI features (i.e. aspect-GUIs)?* To answer this question, we collected binary relevance annotations during the user study and computed standard IR metrics, including *MRR* and *HITS@k*.
- **RQ₃:** *Can users find more relevant GUIs at the top ranks through the feature-based GUI reranking?* To answer this question, we compared the ranks of the selected GUI prototype before and after the reranking based on the relevance annotations of recommended features and report how often users found better matches.
- **RQ₄:** *Do users perceive SERGUI as useful?* To answer this question, we asked participants of the user study several questions on a five-point Likert scale questionnaire regarding the perceived usefulness of the approach and report the *SUS*.

5.3.1 User Study

User Study Design. To evaluate our approach, we conducted a user study and asked participants to employ *SERGUI* to create several GUI prototypes. In particular, we recruited 12 participants possessing technical backgrounds (BSc.:7/MSc.:5). In addition, the recruited participants had medium to high experience in software development (Mean:3.50/SD:0.90) and mostly low experience in GUI prototyping (Mean:2.16/SD:0.83). These demographics were collected as self-reported by the participants on a five-point Likert scale. Moreover, a large fraction of participants (75%) reported that they had never utilized a GUI prototyping tool. The remainder of participants reported that they previously had used two different GUI prototyping tools, either *MockPlus* or *Wireframe*. In particular, we recruited participants with low experience in GUI prototyping, matching the characteristics of the envisioned target audience of our GUI prototyping approach *SERGUI*. Furthermore, the user study encompassed three GUI prototyping tasks and each participant worked on two of them. Each combination of tasks was employed and the occurrence was distributed evenly and randomly among the participants. In summary, the 12 participants created 72 GUIs distributed among 24 applications, which provided the foundation for evaluating the proposed *SERGUI* approach and answering the RQs.

User Study Tasks. The user study consisted of three applications from diverse domains (*shopping*, *news* and *social*), each encompassing three GUIs, resulting in nine different GUIs overall. Each application was presented to the participants as a low-fidelity GUI prototype, containing solely a minimal collection of features to enable participants to recognize the notion of the GUI and overall application. On this basis, participants were asked to create the respective GUI prototypes utilizing *SERGUI* and extending them with typical and meaningful additional features not contained in the rudimentary mockup provided to them. The first task *Task-1* is a *shopping* app encompassing the following GUIs: **(1-1)** an overview of products, each containing an image, name and price as features, **(1-2)** a subsequent product detail view with image, name, price and description and **(1-3)** a checkout page with one item (image, name and price), total cost and a confirmation button. Second, *Task-2* is a *news* app consisting of **(2-1)** a list of news, each with image

and short heading, **(2-2)** a single news article with image, heading and description and **(2-3)** a comment section for the news article, each comment consisting of a user image, name and the text content of the comment. Third, *Task-3* is a *social* app containing more general GUIs, in particular, **(3-1)** a user registration form with name, gender and birthday inputs accompanied by a submit button, **(3-2)** a user profile overview with image, name and age and **(3-3)** a collection of contacts, each displaying a contact name and number.

Experimental Procedure. Participants were first asked to read a user study instructions document, explaining their role and providing an example prototyping task and solution. Afterwards, a brief demonstration of *SERGUI* was given (always same for each participant), which was followed by the participants completing their first and second task. After each GUI prototype was completed, a summary (in the form of a Portable Document Format (PDF)) produced by the tool was saved. Simultaneously, the interaction and feedback data provided by participants while completing their prototyping tasks were saved to a database. The user study instructions, tasks, created GUI prototype summaries and datasets are all publicly available at our accompanying *GitHub* repository.

5.3.2 RQ₁: Feature Recommendation Relevance

To answer RQ₁, we evaluated the ability of *SERGUI* to recommend meaningful features. On the basis of the initial NLR_{GUI} , already specified features NLR_{Feat} and the initially selected GUI, we recommended the top-10 features as predicted by our approach to the participants and the respective top-15 ranked *aspect*-GUIs. In *SERGUI*, participants can provide positive feedback on a recommended GUI feature either by selecting an *aspect*-GUI or by answering with "yes". Negative feedback could be provided either by selecting "no" or "already specified", meaning that this GUI feature was already included either in the selected GUI, NLR_{Feat} or in earlier recommendations. We transformed the iterative feedback into a relevance array and computed AP , MRR and $P@k$ (see Chapter 3 Section 3.3.1). To conduct the experiments, we used the most recent *GPT-4* model⁵ (OpenAI, 2023) (8,192 token context length, *temperature*=0, *top_p*=1 (enabling nucleus sampling), accessed in July, 2023) as the LLM and employed the top-300 GUIs from the initial GUI ranking to compute the GUI feature ranking. Finally, we used the most recent and largest available SBERT model (768-dimensional embeddings) (Reimers and Gurevych, 2019).

5.3.3 RQ₂: GUI Feature Retrieval Relevance

To answer RQ₂, we evaluated the ability of *SERGUI* to retrieve *aspect*-GUIs based on textual requirements of the features. Basically, we employed the identical setup as described for RQ₁ (top-15 *aspect*-GUIs) and for each relevant feature (either answered with *yes* or by picking an *aspect*-GUI), we employed the rank of the selected *aspect*-GUI to compute respective ranking metrics. If none was picked, it counted zero towards the

⁵Note that at the time of implementing the approach, *GPT-4* was recently published and achieved state-of-the-art results across many tasks (OpenAI, 2023). Therefore, we decided to employ this model.

metric. Since we only have at most one relevant *aspect*-GUI, we computed the MRR and $HITS@k = 1$ if $|R_k| > 0$, and 0 otherwise, with R_k representing the set of relevant *aspect*-GUIs positioned up to rank k . We computed these metrics for both the GUI features actively searched by participants and the LLM-based recommended features.

5.3.4 RQ₃: GUI Reranking Performance

To answer RQ₃, we evaluated the reranking performance of the proposed feature-based GUI reranking mechanism on the basis of the feature feedback provided by the participants. After completing the feedback on GUI feature recommendations, participants were shown an updated GUI ranking. Participants could either keep their original choice or, in case of a better matching GUI including more of the features marked as relevant, they could also select a new main GUI. In cases where participants picked a better matching GUI, we computed the reranking metric for the picked GUI as follows:

$$RR(G_i) = R_{initial}(G_i) - R_{updated}(G_i) \quad (5.8)$$

with $R(G_i)$ being the rank of the GUI G_i . Since participants can only choose a single GUI in *SERGUI*, we cannot evaluate the entire top- k reranking performance. Thus, we focus on the single selected GUI. For the initial GUI ranking score $\mathbf{S}(G_i)$ based on the SBERT and $S2W$ ensemble, we employed the average ($\alpha = 0.5$). Similarly, for the GUI reranking score $\mathbf{S}_{RR}(G_i)$, we also computed the corresponding score average ($\beta = 0.5$).

5.3.5 RQ₄: Perceived Usefulness

To answer RQ₄, we evaluated the overall perceived usefulness of our approach. On the basis of a five-point Likert scale, we asked participants (a) how much *SERGUI* supported them during the GUI prototyping process, (b) how relevant the shown GUIs were on average for their initial requirements, (c) how relevant the recommended GUI features were on average, (d) how relevant the visualizations of GUI features (i.e. *aspect*-GUIs) were on average, (e) how much the feature recommendations helped them add relevant features that they did not think about initially and (f) how much the visualizations of feature recommendations helped them better understand the GUI feature. Furthermore, we employed the System Usability Scale (SUS) (Brooke, 1996b) as a reliable, validated measurement of system usability. In order to gain further insights, we gathered additional free-form feedback from participants regarding potential improvements for *SERGUI*.

5.4 Results & Discussion

5.4.1 RQ₁: Feature Recommendation Relevance

Table 5.1 illustrates the evaluation results for RQ₁, showing the AP , MRR and $P@k$ values for each GUI individually and on average across all GUIs. Moreover, Figure 5.8 shows multiple boxplots. An average AP of 74.1 indicates a strong feature recommendation performance as approximately three out of four features are marked as relevant on average.

Table 5.1: Evaluation results overview of LLM-based GUI feature recommendation per GUI and on average using AP , MRR and $P@k$ (**Bold** values indicate best metric score within result group and underlined values indicate best metric score across all groups).

	AP	MRR	P@k					
	AP	MRR	P@1	P@2	P@3	P@5	P@7	P@10
GUI 1-1	76.4	91.7	87.5	68.8	62.5	67.5	67.9	56.2
GUI 1-2	82.4	100.0	100.0	87.5	83.3	72.5	71.4	72.5
GUI 1-3	80.4	93.8	87.5	81.2	83.3	72.5	64.3	60.0
GUI 2-1	81.1	87.5	75.0	75.0	70.8	75.0	76.8	77.5
GUI 2-2	76.3	79.2	62.5	62.5	66.7	77.5	73.2	70.0
GUI 2-3	69.9	62.9	37.5	56.2	66.7	70.0	62.5	63.8
GUI 3-1	71.2	79.2	62.5	62.5	70.8	65.0	60.7	62.5
GUI 3-2	60.0	65.8	50.0	43.8	58.3	55.0	51.8	53.8
GUI 3-3	69.2	74.4	62.5	62.5	58.3	60.0	57.1	57.5
Avg.	74.1	81.6	69.4	66.7	69.0	68.3	65.1	63.8

Likewise, the MRR of 81.6 shows that the first relevant feature occurs at rank 1.22 on average, indicating that the top-features are relevant. In addition, the $P@k$ values also indicate a substantial recommendation performance ranging from 69.1 ($P@1$) to 63.8 ($P@10$). This also shows that the feature scoring mechanism accordingly ranks more relevant GUI features higher as intended. From the features marked as non-relevant, 26.05% were marked as *already specified*. In these cases, the subsequent root causes could be identified and are discussed subsequently.

First, the model itself sometimes predicted similar features, for example, for GUI (1-2) the features *product details* and *product description* were recommended often. Through the explanations, the model might differentiate moderately between these two features (e.g. more general details in contrast to specifications such as weight, dimensions, etc.), but participants often identified them as identical. Another similar example from GUI (1-3) is represented by the features *coupon code* and *gift card*. Second, some recommended features were already encompassed in the initial GUI. Often, these features were not sufficiently represented in the *Rico* GUI hierarchy data and subsequently in the generated GUI abstraction. For example, *shipping information* on GUI (1-2) was often encapsulated in an image without a *resource-id*. Hence, the feature was only encoded as an image component without further semantic information.

In other cases, the feature was hard to identify for the model as being encoded as more complex semantics. For example, for task GUI (1-2) some selected GUIs contained a *market price* and a *shop price* (which was cheaper) and therefore participants marked the recommendation *discount* as already specified. Similarly, for GUI (2-3) often the feature *comment threading* was predicted, which typically is already included by most selected GUIs. However, this feature is also more difficult to recognize. Finally, other features marked as non-relevant often included general features such as *help* or *customer*

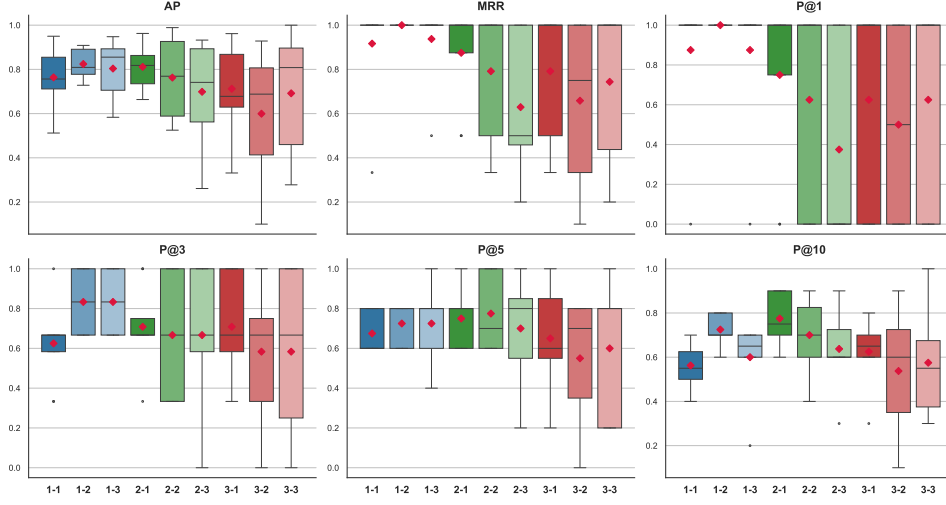


Figure 5.8: Boxplots with AP , MRR and $P@k$ across *shopping app* (1-1, 1-2, 1-3) (blue), *news app* (2-1, 2-2, 2-3) (green) and finally the *social app* (3-1, 3-2, 3-3) (red), diamonds=means.

support, which might be meaningful for the overall app but are not specific enough for the particular GUI. Figure 5.9 depicts two examples of NLR_{GUI} with selected GUIs and their feature predictions and top-three *aspect*-GUI matches. The two illustrated examples represent two GUIs from the user study tasks (*Rico* dataset (Deka et al., 2017)), namely, GUI (2-3), a *comment section* of a news article page and GUI (1-2), a *product details view*.

Answer to RQ₁: *SERGUI* benefits from a strong LLM-based GUI feature recommendation performance, indicated by both a mean AP of 74.1 and an MRR of 81.6.

5.4.2 RQ₂: GUI Feature Retrieval Relevance

Table 5.2 depicts the evaluation results for RQ₂, showing the MRR and $HITS@k$ values for each GUI separately and averaged across all GUIs. An MRR of 39.0 shows that the first relevant *aspect*-GUI appears at rank 2.56 on average, indicating a substantial performance in finding relevant *aspect*-GUI matches. Moreover, an average $HITS@1$ of 27.0 represents that on average in 27% a relevant *aspect*-GUI could be found at the top-1 position. A $HITS@5$ of 52.7 indicates that in 52.7% of the cases a relevant *aspect*-GUI could be found within the top-5 ranking positions. Finally, a $HITS@15$ of 69.1 represents that on average our matching approach could find a relevant *aspect*-GUI in 69.1% within the top-15 *aspect*-GUI ranking. Therefore, for a large number of recommended features, our feature matching approach is capable of finding a relevant *aspect*-GUI visualizing the GUI feature. Additionally, for active feature searches our approach performs similarly with an overall better coverage ($MRR = 26.2$ and $HITS@15 = 78.6$). Figure 5.9 illustrates the recommended GUI features and their respective top selected *aspect*-GUIs.

For example, for GUI (2-3) "*comment section*" the matching of recommended features performs well and includes features such as *share comment*, enabling sharing on social

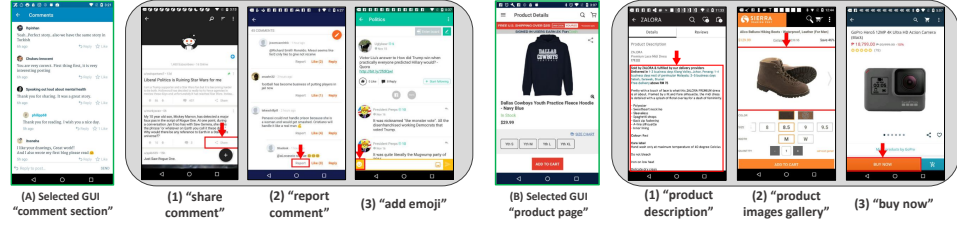


Figure 5.9: Two GUI feature recommendations and their top-three *aspect*-GUI matches: (A) "comment section" with recommended GUI features *share*, *report* and *add emoji* and (B) "product page" with features *product description*, *product image gallery* and *buy now*.

media, *report comment* to directly report a single comment and an *add emoji* feature enabling users to add an emoji to their comment. Likewise, example GUI (1-2) "product page" contains interesting top selected *aspect*-GUI matches encompassing features such as *product description* providing detailed descriptions of the product, *product image gallery* enabling the users to quickly browse multiple images of the product (both matched by their *resource-ids*) and a *buy now* component, enabling fast purchasing without checkout. If *aspect*-GUIs could not be matched, the root causes were identified.

First, sometimes the feature recommendation model generated over-specified text descriptions of the features, e.g., including non-valuable domain words. For example, instead of generating *rating* only, sometimes *product rating* was generated or instead of solely generating *source*, sometimes *news source* was generated by the LLM. These additional unspecific domain words often led to less precise matches. Additionally, the matching performance decreased with increasing feature recommendation rank, since the feature ranking is based on the matching scores. The initial NLR_{GUI} also has a high impact on the matching performance. For example, when the initial top- k GUI ranking is heterogeneous (e.g., containing both *product overviews* and *product detail views*), then it also becomes more difficult to properly match the features with SBERT. Overall, we obtained a good matching performance of the *aspect*-GUIs as indicated by the metrics.

Answer to RQ₂: *SERGUI* is able to retrieve relevant *aspect*-GUIs for visualization with a good performance, indicated by an *MRR* of 39.0 and an *HITS@15* of 69.1.

5.4.3 RQ₃: GUI Reranking Performance

Based on the positive feature feedback (selecting an *aspect*-GUI), we computed a reranking of the GUIs. As per the design of *SERGUI*, participants could initially select a GUI and then were presented with the feature recommendations. Afterwards, participants could either choose a new GUI from the updated ranking or keep their previous choice. In 68.05% of the cases, the participants selected a better matching GUI from the GUI reranking. For these cases, the reranking score is +61.89 ranks on average (SD:79.82/Min:-26/Max:335), indicating that the feature-based reranking method can substantially improve the ranks of relevant GUIs. We only computed the reranking score for cases where participants re-

Table 5.2: Evaluation results overview of our GUI feature (*aspect*-GUI) retrieval approach for each GUI and on average using *MRR* and *HITS@k* (**Bold** values indicate best metric score within result group, underlined values indicate best metric score across all groups).

	MRR	HITS@k						
	MRR	H@1	H@2	H@3	H@5	H@7	H@10	H@15
GUI 1-1	35.0	20.0	35.6	42.2	55.6	60.0	66.7	75.6
GUI 1-2	41.5	27.6	43.1	51.7	56.9	65.5	67.2	72.4
GUI 1-3	28.9	16.7	25.0	31.2	43.8	54.2	60.4	66.7
GUI 2-1	37.7	24.2	41.9	43.5	46.8	54.8	62.9	75.8
GUI 2-2	36.7	26.8	35.7	46.4	48.2	55.4	57.1	58.9
GUI 2-3	30.8	25.5	31.4	31.4	35.3	39.2	45.1	47.1
GUI 3-1	63.7	52.0	66.0	74.0	78.0	84.0	84.0	86.0
GUI 3-2	46.1	27.9	39.5	65.1	76.7	83.7	83.7	86.0
GUI 3-3	30.1	21.7	30.4	37.0	37.0	41.3	45.7	54.3
Avg.	39.0	27.0	39.0	46.8	52.7	59.5	63.4	69.1

lected the GUI, due to the fact that participants simply could keep their previous choices if no better GUI could be found. In addition, by comparing the *MRR* of the individual *SBERT* ($MRR = 17.2$) and *S2W* ($MRR = 21.0$) retrieval models with the ensemble model ($MRR = 42.9$), we can observe that the ensemble considerably improves the GUI ranking performance of each separate model. Therefore, participants often picked initial GUIs only from the very top ranks (rank 2.38 on average) and rarely examined more than the top-20 GUIs in the overall ranking. Achieving an average reranking score of +61.89 thus could help participants find better matching GUIs at the very top of the GUI reranking.

Answer to RQ3: GUI feature feedback could be employed to compute a reranking score resulting in 68.05% re-selections and +61.89 ranks improvement on average.

5.4.4 RQ4: Perceived Usefulness

Fig. 5.10 depicts the evaluation results of the perceived usefulness of *SERGUI* as reported by the participants. First, (a) the support provided by *SERGUI* during the prototyping process received a high score ($M:4.25/SD:0.75$). Second, (b) the average relevance of the shown GUIs for the initial requirements received a similar high score ($M:4.16/SD:0.71$). Third, (c) the average relevance of the recommended GUI features also received a comparably high score ($M:3.83/SD:0.93$). Fourth, (d) the average relevance of the visualizations of GUI features received a medium score ($M:2.9/SD:0.51$). Fifth, (e) how much the feature recommendations helped them add relevant features they did not think about initially also received a high score ($M:4.25/SD:0.96$). Lastly, (f) how much the visualizations of the recommended features helped them better understand the feature received a medium score ($M:3.16/SD:1.26$). In summary, *SERGUI* accomplished a SUS of 84.16, indicating

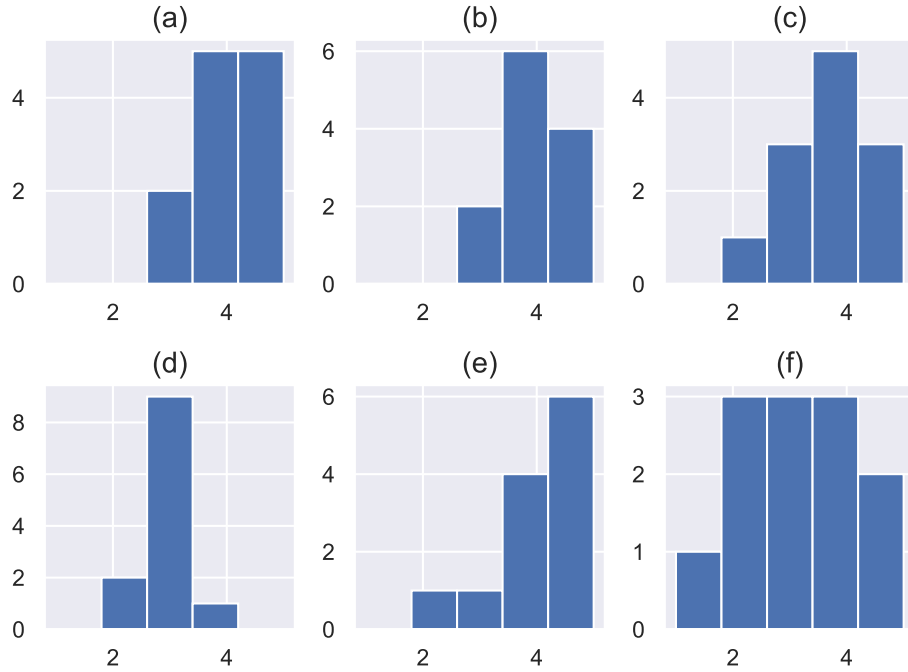


Figure 5.10: Six histograms showing the evaluation results for the posed usability questions with reference to employing *SERGUI* for GUI prototyping as perceived by participants.

that our approach possesses high perceived usability particularly in comparison with the average SUS of 68. In general, participants reported that they found it very easy to get started quickly with *SERGUI* and they required little to no prior knowledge to be able to utilize our approach. Moreover, participants mostly found the feature recommendations helpful to integrate new relevant and interesting features into their prototype, which they were not aware of initially. For example, one participant reported specifically that a *notification bell* and a *subscription* feature in the context of the *social* app was stimulated to be included by our recommendation approach. Regarding improvement directions for the proposed approach, participants mostly reported that the respective GUI feature visualizations sometimes did not match the corresponding recommended GUI features.

Answer to RQ₄: *SERGUI* showcases a high perceived usability as indicated by a high SUS of 84.16, substantially higher than the average SUS value of 68 (Brooke, 1996b).

5.5 Threats to Validity

Internal Validity. To ensure internal validity, we conducted the experimental procedure including instructions, tasks and a tutorial in the same manner for each participant. To avoid bias from the tasks, we solely provided low-fidelity mockups of the GUIs. This ensures that participants have to formulate their own textual requirements for the GUIs

and features. The annotation of relevance for selected GUIs, feature recommendations and *aspect*-GUIs is inherently subjective. To mitigate potential bias from the annotation procedure, we therefore conducted the experiments with 12 different participants.

External Validity. To improve the generalizability of our evaluation, we included nine different GUIs from three diverse domains. Since *Rico* encompasses many diverse domains and GUIs, we hypothesize that similar results could be obtained for other well-covered domains in *Rico*. However, to increase the generalizability further, more evaluation should be conducted to confirm the findings. To improve external validity with regard to the user study participants, we included users with low GUI prototyping experience.

5.6 Limitations

Our approach possesses notable limitations which provide the foundation for future work. Due to the data-driven nature of our approach, the potentially supported domains and applications are constrained. *SERGUI* cannot support the SER process for very special and custom GUIs. However, the GUI repository *Rico* already encompasses many diverse domains with a large number of GUIs, enabling versatile application scenarios. Similarly, *GPT-4* has been trained on highly diverse text data (Brown et al., 2020; OpenAI, 2023), encoding knowledge from a plethora of domains.

Another limitation of our approach lies in the current inflexibility and the determined process flow of the interaction model. Therefore, enabling more flexibility with reference to, for example, interweaving more closely the active search for own features and getting features recommended could enhance the overall usability. Furthermore, the currently possible feedback on proposed GUI features is restricted to either including or excluding the feature in a binary fashion. This is sufficient in many cases, but there might be situations for which a fine-grained NL response is more appropriate. For example, in cases where including a *payment methods* feature is relevant, customers would potentially provide more detailed feedback directly such as the specific payment methods that should be supported. By integrating LLMs, with their impressive NLU capabilities, more closely into the process, such NL feedback could be processed additionally.

Moreover, the LLM sometimes recommends similar features with different granularity, e.g., *payment methods* and *PayPal* or semantically related features such as *add contact* and *delete contact* at different ranks. Grouping these hierarchical and semantically related features could also improve the usability. Furthermore, the sometimes heterogeneous matching of *aspect*-GUIs might mislead users. Improving the feature matching performance could solve this limitation, e.g., by optimizing a Deep Learning (DL) model specifically for this task. Finally, the current implementation of the tool misses standard features (e.g., reverting actions in case of user errors) and solely shows the initially selected GUI and the current app prototype. Extending the view of the current GUI to include all chosen *aspect*-GUIs and text features could further facilitate utilizing the novel approach.

5.7 Related Work

In Chapter 3 of this thesis, we introduced *RaWi*, proposing NLR-based GUI retrieval approaches. In that work, the focus lies on investigating and evaluating various methods for NLR-based GUI retrieval and providing support for the analyst in a REI scenario. With our proposed *SERGUI* approach, we extend this work to enable SER with GUIs for users. In particular, we additionally propose an automatic GUI feature recommendation approach, the visualization of recommended features, feature search and a feature-based GUI reranking technique. Our approach is integrated into a user-friendly tool utilizing chat-based guidance, requiring no prior prototyping knowledge or experience. Furthermore, our approach integrates the selected GUIs and chosen GUI features into a novel GUI prototyping artifact that can be utilized by analysts.

Another similar approach for NL-based GUI retrieval is *Guigle* (Bernal-Cárdenas et al., 2019), exploiting *Android* apps automatically crawled from *Google Play* and providing search functionality based on simple retrieval models from *Lucene* (Lucene). In contrast, our *SERGUI* approach utilizes more sophisticated embedding-based GUI ranking and reranking models plus GUI feature recommendation and matching techniques. Moreover, *Guigle* can merely be employed by designers or analysts, but cannot be utilized for SER with stakeholders. Many other GUI retrieval approaches utilizing visual input have been proposed before. For example, *Swire* (Huang et al., 2019) employs visual embeddings for retrieving GUIs based on hand-drawn sketches, *GUIFetch* (Behrang et al., 2018) enables retrieval of entire applications on the foundation of an entire *Android* application sketch, and *VINS* (Bunian et al., 2021) provides GUI retrieval by utilizing either a simple wireframe prototype or a fully designed GUI as input. Finally, *Gallery D.C.* (Chen et al., 2019a) represents an approach for providing multi-faceted (e.g., *width*, *height* and *color*) search functionality for designers to enable the retrieval of individual GUI components such as buttons. In contrast, *SERGUI* utilizes an embedding-based GUI component ranking to enable semantic feature search and is capable of matching a variety of GUI component types.

Prior feature recommendation approaches as presented in (Chen et al., 2019b; Wang et al., 2022b) mainly provide the ability to predict potentially relevant features on the basis of a high-fidelity GUI as input. These bottom-up feature recommendation approaches merely extract simple features from the GUIs restricted to clickable components (mainly *buttons* and *icons*). In contrast, the feature recommendation problem in the context of *SERGUI* is substantially different, involving multiple inputs such as fine-grained unstructured NLR for the GUIs, a collection of actively specified individual features as NLR and an initially selected GUI. Moreover, our top-down LLM-based feature recommendation approach exploits the vast knowledge embedded in LLMs and enables the recommendation of GUI features that the mentioned works are not capable of, e.g., simple features such as a *comment count* or *promotion badge* and semantically more complex features such as *comment threading* or *related articles*. Further, we match recommended features with the GUIs from the top-*k* GUI ranking to create *aspect-GUIs*, enabling visualizations of features and integrate the recommendations into the whole, automated SER approach.

5.8 Conclusion

This work was driven by challenge **C1-3**: *How can we provide automatic assistance for self-elicitation of requirements with GUI prototyping?* To answer this question, in this work, we proposed *SERGUI* as an initial approach towards enabling SER with GUI prototyping. *SERGUI* allows users to rapidly and independently create their own GUI specifications, requiring little prototyping experience through multiple interaction mechanisms. The evaluation indicates that *SERGUI* is able to effectively support users during the GUI prototyping process in terms of providing relevant GUIs from NLR, recommending relevant GUI features with visualizations and achieving high perceived usability (SUS).

Part II

PROTOTYPING: GUI GENERATION VIA NATURAL LANGUAGE

CHAPTER 6

TAILORING LLMs TO PROTOTYPING: ZERO-SHOT GUI GENERATION WITH RETRIEVAL-AUGMENTATION AND SELF-CRITIQUE PROMPTING

After presenting multiple NLR-based GUI retrieval and reranking solution approaches for the first challenge (C1) to rapidly map NLR to GUI prototypes, we provide additional novel solution approaches through LLM-based GUI generation. In this chapter, we introduce several ZS prompting approaches to efficiently adapt pretrained LLMs for the problem of GUI generation from NLR, requiring no training or finetuning. The following sections are based on previous work (Kolthoff et al., 2024b)¹. The source code, datasets and experimental resources are all publicly available to foster future research².

Personal Contribution. I ideated and created the concept for the work. While I created the final implementation for the prompting techniques and framework, *Lennart Fiebig* created the initial implementation draft for the self-critique technique. *Felix Kretzer* and I contributed equally to the evaluation design, while the data analysis was conducted by *Felix Kretzer*. The manuscript was mainly written by me, while *Felix Kretzer* wrote the *GUI description dataset* section and created the result tables and figures.

¹This section is adapted from: **Kolthoff, Kristian**^{*}, Kretzer, Felix^{*}, Fiebig, Lennart, Maedche, Alexander, Ponzetto, Simone Paolo, and Bartelt, Christian. Zero-Shot Prompting Approaches for LLM-based Graphical User Interface Generation. arXiv preprint arXiv:2412.11328, 2024. The paper is currently under review for the 2026 ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE). Sections are directly reused with adaptations (i.e. thesis Section 6.i matches paper Section i+1 starting from the *Approach* section 6.2, since the *Background* section from the original paper has been removed. See Chapter 2 regarding background details.

²Materials for this chapter, including source code, datasets and experimental resources, are available at our corresponding *GitHub* repository <https://github.com/kristiankolthoff/ZS-Prompting>

6.1 Motivation

We previously presented NLR-based GUI retrieval techniques, including BERT-LTR models, improved the GUI reranking effectiveness with MLLM-based reranking and proposed an approach for SER with automated GUI prototyping that integrates GUI retrieval techniques. While GUI retrieval enables rapid mapping of NLR to corresponding GUI prototypes, these approaches have one major shortcoming. Practically, retrieval methods are bound to the available GUI datasets with static GUI prototypes, which restricts support for arbitrary NLR input and GUI prototype customizability. While the proposed data-driven GUI prototyping editor *RaWi* simplifies the process, it still requires effort to create novel GUI prototypes adapted to custom NLR that are not directly supported by an individual GUI prototype. Furthermore, pretrained LLMs have gained popularity over recent years, showing impressive performance across many NLP and NLU tasks. In particular, LLMs possess high effectiveness for code generation (Jiang et al., 2024). Hence, adapting LLMs for the problem of NLR-based GUI generation offers significant potential. Furthermore, recent trends such as *prompting-based coding* (Sapkota et al., 2025; Ray, 2025) highlight the increasing need for automated GUI generation. In prompting-based coding, users experiment with design and functionality through iterative and conversational interactions. Since this approach often attracts participants who lack advanced prototyping skills (Sapkota et al., 2025; Ray, 2025), enhancing LLM-based GUI generation to convert NL descriptions directly into high-fidelity GUI prototypes is particularly valuable.

Recent research proposed several approaches for automatically generating GUI prototypes. For example, *Instigator* (Brie et al., 2023) is based on *minGPT* (minGPT - GitHub Repository) trained on a vast number of crawled web pages to produce low-fidelity GUI layouts based on text descriptions and user-selected GUI component types. An alternative to training a task-specific GPT model from scratch involves fine-tuning a pretrained LLM (Feng et al., 2023), leveraging the *Rico* GUI repository (Deka et al., 2017). However, both approaches not only require resource-intensive training but are also limited to producing low-fidelity layouts in a Domain-Specific Language (DSL), which is hard to integrate into practical GUI prototyping workflows or link to individual prototyping tools. Furthermore, *MAxPrototyper* (Yuan et al., 2024) enables the creation of GUI prototypes by prompting LLMs. However, instead of relying solely on NLR as input, their approach requires a fully developed GUI layout, generates a proprietary DSL and neglects creating actual GUI prototype functionality. Therefore, the work presented in this chapter is driven by the leading research question of challenge **Cr-4**: *How can we efficiently optimize LLMs for more effective GUI prototype generation?*

While existing research has focused mainly on resource-intensive training and fine-tuning to generate low-fidelity GUI prototypes, a comprehensive investigation into less resource-intensive approaches based on ZS prompting techniques for generating high-fidelity GUI prototypes is currently lacking. In order to address this research gap, we explore the potential and effectiveness of different ZS prompting approaches for generating high-fidelity GUI prototypes from NLR in HTML/CSS, as shown in Figure 6.1.

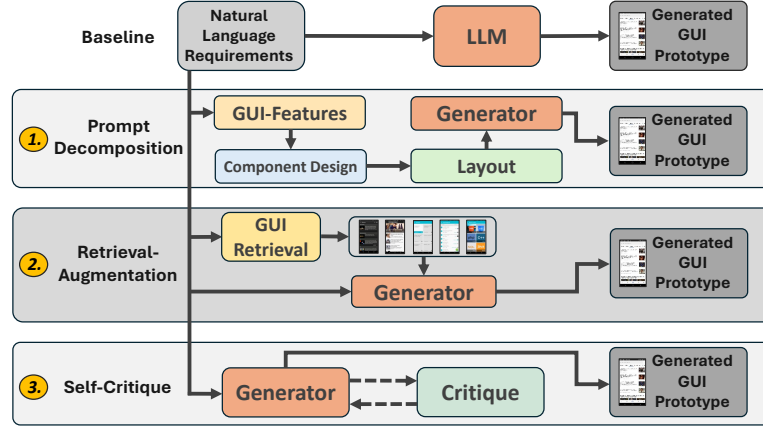


Figure 6.1: ZS prompting approaches for GUI generation from NLR: (1) Prompt Decomposition (PD), Retrieval-Augmented Generation (RAG) and Self-Critique (SC).

Our focus lies particularly on ZS prompting because (i) training or fine-tuning LLMs is resource-intensive and (ii) FS prompting struggles with very large context windows required for ICL (Dong et al., 2022; Li et al., 2024), which would be necessary for GUI generation, since high-quality GUIs typically consist of thousands of tokens. In particular, we propose Retrieval-Augmented GUI Generation (RAGG), which combines the advantages of GUI retrieval for rapid access to vast prototyping knowledge embodied in large GUI repositories with the reasoning and adaptation capabilities of LLMs. To this end, we utilize the presented GUI retrieval and reranking approaches. Moreover, we investigate Prompt Decomposition (PD) (Khot et al., 2022) for GUI generation, enabling the LLM to generate meaningful intermediate reasoning outputs, instead of directly generating low-level HTML/CSS from high-level NLR. This approach more closely follows a human expert process and ensures that computational capabilities of the LLM are used more effectively. Finally, we investigate Self-Critique (SC) (Saunders et al., 2022) for GUI generation, employing the LLM itself in a GUI prototyping and feedback loop.

To assess the proposed approaches, we conducted an extensive evaluation consisting of over 20,400 GUI annotations from 101 crowd-workers with UI/UX experience regarding the accuracy and self-reported assessments of generated GUIs. The results suggest that especially the RAGG and SC prompting approaches can substantially enhance GUI prototypes across multiple evaluation metrics in comparison with multiple ZS baselines.

Contributions. With this work, we make the following two research contributions:

- *Retrieval-Augmented GUI Generation (RAGG) and Self-Critique (SC)*: we present the first pipeline that retrieves example GUIs from a large-scale GUI repository, applies advanced MLLM-based reranking techniques to them, extracts their feature and layout knowledge, and conditions an LLM to generate executable, high-fidelity HTML/CSS

prototypes. Moreover, we provide the first systematic adaptation of Prompt Decomposition (PD) and SC prompting techniques for end-to-end NLR-based GUI generation.

- *Large-scale, reproducible evaluation and insights:* we conduct a large-scale human study with 20,400 annotations from 101 UI/UX practitioners over 50 GUI generation tasks and release code, prompts and data and analyze common LLM-induced GUI defects.

6.2 Approach

We present the different ZS prompting methods for generating GUI prototypes. First, we introduce the baselines to benchmark the more comprehensive approaches. Next, we present PD, RAGG and SC techniques for GUI generation, as illustrated in Figure 6.2.

6.2.1 Baselines: ZS-Instruction and ZS-CoT

As our prompting baselines, we first employ a ZS instruction prompt, including a clear description of the base task to create a mobile page in HTML/CSS according to the provided NLR. We selected HTML/CSS as the target language to generate due to its widespread use as a GUI description language, support for interaction and extensive pretraining of LLMs on this language. To improve the alignment of LLM responses for user interaction, LLMs are often optimized using *Reinforcement Learning with Human Feedback (RLHF)* (Ouyang et al., 2022), leading to LLMs providing explanations and a structured representation of the response. In particular, the LLMs for GUI generation have a tendency to provide explanations and separate the HTML/CSS into multiple markdown blocks. To avoid this, we additionally instruct the LLM to directly output the non-separated code without explanation. In addition, we employ a ZS-CoT prompt (Wei et al., 2022) as our second baseline, instructing the model to provide a self-created step-by-step reasoning sequence with intermediate computation steps, instead of directly mapping high-level NLR to low-level HTML/CSS, which is effective for several tasks.

6.2.2 Prompt Decomposition (PD) for GUI Generation

Regarding the notion of enabling the model to perform step-by-step reasoning for generating GUIs instead of directly outputting code, we extend this idea by decomposing the instruction into multiple separate tasks and prompts (Khot et al., 2022) for GUI generation, which also follows a human expert approach more closely, as shown in Figure 6.2 (A). Usually, REI takes place initially, where human experts utilize knowledge and previous experience to gain deeper insights into the requirements.

Therefore, in the first step, we employ a feature extraction prompt, asking the LLM to provide a more detailed collection of GUI features with textual descriptions based on the high-level input. Second, we instruct the LLM to derive implementation and design ideas for each of the features, i.e. which GUI components and libraries to employ for implementing the feature. Third, we instruct the LLM to provide an overall layout structure and page design given the previously generated feature collection. With each step of

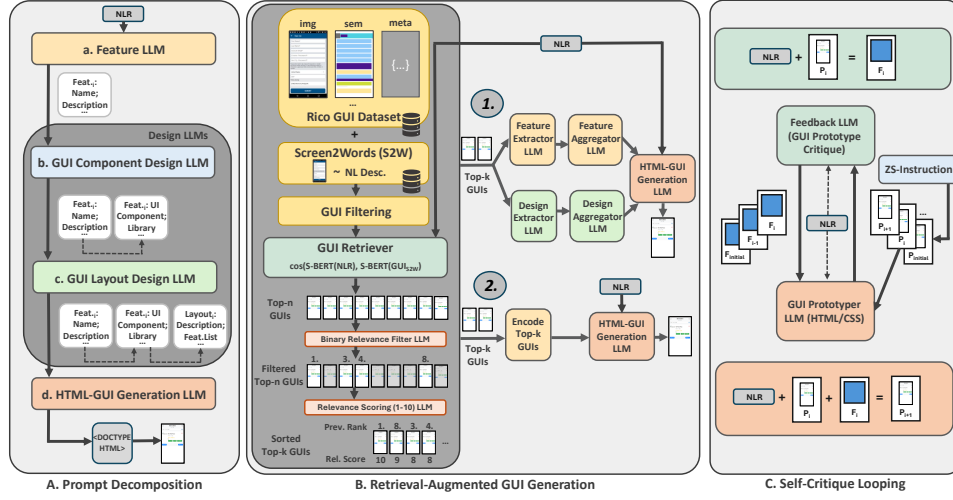


Figure 6.2: Overview of ZS prompting approaches for generating GUI prototypes from NLR: (A) Prompt Decomposition (PD), which divides the generation task into smaller sub-tasks, (B) Retrieval-Augmented GUI Generation (RAGG), which exploits the *Rico* GUI dataset and previously developed GUI retrieval and reranking techniques to augment the prompt context with GUIs relevant to the provided NLR, and (C) Self-Critique (SC), which employs generation-and-critique loops to iteratively improve the GUI prototypes.

the decomposed prompt pipeline, the initial more implicit NLR is extended by more and more information to form a more explicit representation of the GUI generation problem. Subsequently, this representation serves as the input to the final LLM which is tasked with the translation of the entire text specification of the GUI into HTML/CSS. In addition to decomposing the task into subtasks to provide more computational capabilities for each step, we also created a second variant of the described process by combining all instructions into a single prompt. This represents a custom ZS-CoT prompt with predefined domain-specific reasoning steps. The stepwise process mirrors human expert reasoning, encouraging logical and contextually grounded responses. As the code is generated from an explicit, incrementally refined GUI prototype specification, the likelihood of propagating inaccurate information into the final prototype is substantially reduced.

6.2.3 Retrieval-Augmented GUI Generation (RAGG)

The notion behind RAGG lies in integrating LLMs, possessing impressive text generation and reasoning capabilities, with the advantage of retrieval approaches — namely, providing rapid access to a vast number of potentially relevant documents for effectively completing a generation task, which achieves state-of-the-art results for many NLP problems, especially knowledge-intensive tasks (Weston et al., 2018; Lewis et al., 2020; Li et al., 2022). Usually, the retrieved documents are provided as part of the prompting context in a ZS setting, enabling the LLM to complete tasks requiring knowledge which has

not implicitly been stored in the LLM during the pretraining phase. We adopt this idea and propose RAGG by leveraging the vast GUI prototyping knowledge embodied in a large-scale GUI repository for LLM-based GUI generation. Subsequently, we briefly delineate the employed GUI repository and its preprocessing, a novel LLM-based GUI reranking and filtering approach to ensure high relevance of retrieved GUIs, and our prompting approaches to integrate knowledge embodied in the GUI screens for GUI generation. An overview of the proposed RAGG approach is provided in Figure 6.2 (B).

GUI Retrieval Approach To enable RAGG, we employ the previously developed NLR-based GUI retrieval approach. Particularly, the approach relies on the filtered *Rico* GUI repository (Deka et al., 2017) and computes ranking scores over the dataset with SBERT by matching NLR against the *S2W* data (see Chapter 5 Section 5.2.2 for details).

Adapted GUI Reranking. As shown in Figure 6.2 (B), we first retrieve the top- n relevant GUIs with SBERT, then apply a binary relevance filter using ZS prompting. In contrast to *GUI-ReRank* (see Chapter 4), we initially conduct binary relevance filtering to ensure that only highly relevant GUI prototypes are provided to the generation context. We *base64*-encode each top- n GUI screenshot and provide it to an MLLM as part of the prompt, then instruct the MLLM to decide whether the GUI is relevant given the requirements. To ensure high precision (thus avoiding inputting FPs), we instructed the model to be critical and rather vote for non-relevant in cases of ambiguity. The binary decisions obtained from the LLM were observed to be consistent across multiple runs. Since the SBERT ranks of GUIs do not necessarily correspond with fine-grained relevance, we run a second-level ZS prompt instructing the LLM to provide a finer-grained relevance score (from 1 (*not relevant*) to 10 (*relevant*)) and sort the binary-filtered GUIs. To avoid inconsistencies in the relevance scores (which is a finer-grained task compared to binary relevance and potentially more challenging), we sampled the LLM multiple times and computed the average and standard deviation over the obtained relevance scores, which indicated consistent ratings. The filtered and sorted GUI screenshots are used as input to the generation, ensuring that solely relevant GUIs are provided in the MLLM context.

Prompting Approaches (GUI-enriched Context). We propose two distinct approaches for utilizing the retrieved GUIs for LLM-based GUI generation in combination with the requirements, as shown in Figure 6.2 (B). First, (1) we employ a ZS prompt for each GUI by *base64*-encoding the screenshot in the prompt context of an MLLM and instructing the model to extract a feature collection from the GUI. Afterwards, we employ a second ZS prompt instructing the model to summarize and aggregate the k feature collections of the top- k GUIs. Semantically similar features should be aggregated and ranked higher in the collection based on their frequency. In addition, we employ a similar pipeline to extract design and layout for each of the top- k GUIs and subsequently aggregate them. Aggregated feature and design collections are then provided in a ZS prompt instructing the model to generate HTML/CSS using the NLR and the provided collections. Second, (2) we directly *base64*-encode the top- k GUIs in the context of a ZS

prompt instructing the MLLM to implicitly use the provided GUIs as inspiration for features, design, and layout while generating HTML/CSS for the prototype from NLR.

6.2.4 Self-Critique (SC) Looping for GUI Generation

SC aims at enhancing the effectiveness of LLM task solving by employing the LLM itself to provide feedback on an LLM-generated solution (i.e. criticize one’s own prior output) (Saunders et al., 2022). Particularly, the effectiveness improves when providing critiques compared to directly ZS tasking the LLM to refine the response. For enhancing ZS-based GUI generation, we adapt the concept and propose SC looping for GUI generation. An overview of the approach is illustrated in Figure 6.2 (C). Our self-critique approach consists of two main components. First, (1) a feedback MLLM (i.e. GUI prototype critique) that utilizes the NLR and GUI prototype $\mathbf{P}_i$ to generate feedback (i.e. critique) $\mathbf{F}_i$. This feedback MLLM is based on a ZS prompt, instructing the model to provide feedback on additional relevant GUI features that the current prototype neglects, improvements to the implementation of the current features and improvements to the design and layout of the overall prototype. In addition, we instruct the MLLM to solely focus on textual descriptions and avoid providing code. Second, (2) a GUI prototyping LLM that utilizes the requirements, prior GUI prototype $\mathbf{P}_i$ and the respective critique $\mathbf{F}_i$ to generate the subsequent GUI prototype $\mathbf{P}_{i+1}$. In the prompt, we instruct the model to improve the current GUI prototype by integrating the provided feedback and respond with revised HTML/CSS code. To summarize, the self-critique approach employs two components:

$$\text{Feedback-MLLM: } \mathbf{F}_i = \mathbf{MLLM}(\mathbf{P}_i, \text{NLR}) \quad (6.1)$$

$$\text{Prototyper-LLM: } \mathbf{P}_{i+1} = \mathbf{LLM}(\mathbf{P}_i, \text{NLR}, \mathbf{F}_i) \quad (6.2)$$

This approach requires an initial prototype $\mathbf{P}_0$ in HTML/CSS to be able to provide the initial critique. Therefore, we provide the ZS instruction prototype as a starting point (i.e. the ZS baseline prompt) to further refine the generated prototypes over k iterations.

6.2.5 LLM-based GUI Content Generation

To further improve the high-fidelity GUI prototypes, we propose a novel LLM-based GUI content generation pipeline illustrated in Figure 6.3. Often, generated GUIs from the proposed ZS approaches do not include realistic data (or sufficient example data at all) and contain generic content³. To enhance the realism and the appearance of the GUI prototypes using content, we first provide the base HTML/CSS to an LLM and instruct it to extend the prototype with realistic data and add *id* attributes to each `<img>` tag (e.g., *add list items, related products*, etc.). Subsequently, we instruct another LLM to extract a collection of all images contained in the HTML and provide more detailed descriptions. We utilize an LLM for this step instead of directly extracting the `<img>` tags with a library to enable the LLM to exploit the context to create better descriptions. These

³At the time of implementing the proposed approach, we utilized the most advanced MLLM available from *OpenAI*, namely *GPT-4o* (Hurst et al., 2024), which is an extension over prior *GPT-4* (OpenAI, 2023).

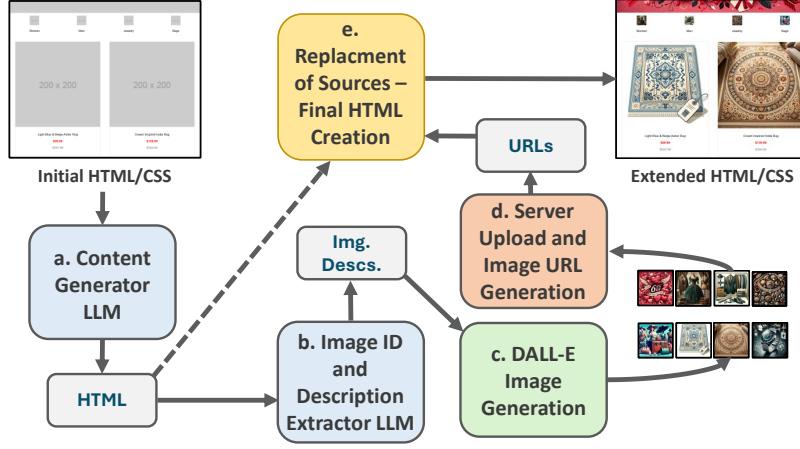


Figure 6.3: Overview of the LLM-based content generation pipeline, which augments the initial prototype with content, generates images with *DALL-E-3* and integrates them.

descriptions are then employed to generate corresponding images with *DALL-E-3* (Betker et al., 2023). The generated images are uploaded to a server, and the Uniform Resource Locators (URLs) are incorporated into the HTML/CSS by matching the extracted *ids*.

6.3 Experimental Evaluation

This section presents the design and methodology of our experimental evaluation. The main goal of our evaluation is to assess the effectiveness of the different prompting approaches for GUI generation. Hence, we state the following four research questions:

- **RQ₁**: *Which LLM prompting approach is most effective for generating GUIs from NLR?* We compare different prompting strategies to determine which yields the most accurate and user-aligned GUI prototypes. Effectiveness is assessed through human ratings on multiple qualitative criteria for the generated prototypes. In addition, we investigate the token consumption of the prompting techniques.
- **RQ₂**: *How does the number of examples in RAGG impact its effectiveness?* This question examines whether providing more GUI examples enhances the quality of LLM-generated GUIs for RAGG. We evaluate this by varying the number of input examples and analyzing changes in human ratings for the quality of GUI prototypes.
- **RQ₃**: *How does the number of loops in the SC for GUI generation impact its effectiveness?* We assess if iterative SC cycles lead to incremental improvements in generated GUIs. The impact is measured by comparing the quality of GUIs after multiple SC iterations.
- **RQ₄**: *How does LLM-based content generation for GUIs influence the overall GUI quality?* We explore whether automatically generated content increases the realism

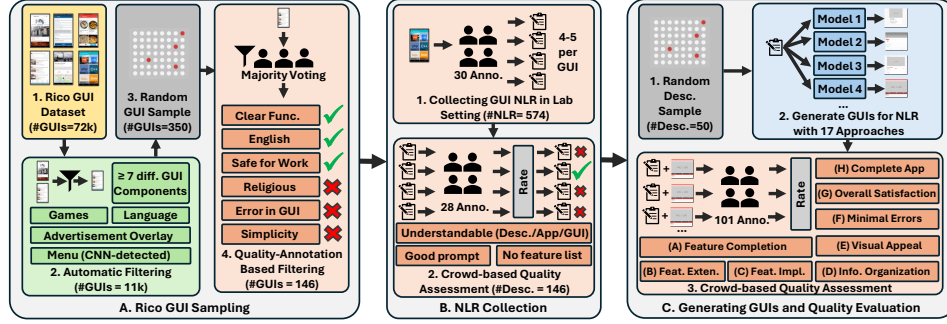


Figure 6.4: Overview of the evaluation: (A) Sampling and filtering GUIs from the *Rico* GUI dataset, (B) collecting and assessing NLR for corresponding GUIs, and (C) generating and evaluating GUIs using multiple models and their crowd-based quality assessment.

and quality of GUI prototypes, as perceived by human annotators. In particular, we compare user ratings of GUIs with and without LLM-generated content and images.

6.3.1 GUI Requirements Dataset

In this section, we present the conducted procedure to collect the GUI NLR dataset to evaluate the research questions, as illustrated in Figure 6.4. First (1), *Rico* (Deka et al., 2017) GUIs were sampled, for which requirements were collected in a lab-based setting in the following step (2) and then evaluated by crowd-workers to increase data quality (3).

Rico GUI Sampling. A GUI prototype selection was required before participants could create NLR for them. Again, we used the established *Rico* GUI dataset (Deka et al., 2017). To build the collection of *Rico* GUIs serving as a foundation for the following data collection, we performed multiple steps to increase the quality and diversity of GUIs. First, we applied basic GUI filtering (see Chapter 5 Section 5.2.2). To ensure the inclusion of feature-wise more comprehensive GUIs (to increase the difficulty for the GUI generation), we removed GUIs with fewer than seven different GUI component types. Then, three research assistants rated each GUI for six predefined categories (*Clear Funct.*, *English*, *Safe for Work*, *Errors*, *Religious Content*, *Simplicity*), generating 1,050 ratings in total. Research assistants were provided with clear instructions, as well as positive and negative examples for each of the dimensions. On average, each GUI received three sets of ratings. These categories were selected to ensure ethical and practical suitability: *Safe for Work* and *Religious Content* excluded problematic or sensitive screens, while *Errors* addressed technical issues common in *Rico* GUIs (Leiva et al., 2020). *English* and *Clear Functionality* ensured participants could understand the GUI without extra context, and *Simplicity* helped us focus on more challenging, feature-rich GUIs for robust evaluation. Participants rated each criterion on a binary *yes/no* basis. We decided to use majority voting for all aspects (i.e., 2/3 participants had to approve the respective category), except for the *Safe for Work* category. Here, a GUI was already screened out if this category was rated accordingly by one of the three annotators. Consequently, 146 GUIs were admitted.

Collecting NLR for GUI Prototypes. We recruited 30 participants (21 male, nine female), with an average age of 23.67 years ($\sigma = 3.25$), from a university panel to create NLR of the previously randomly sampled and filtered GUIs that later served as the basis for our automated GUI prototype generation. We chose a lab-based setting instead of sourcing crowd-workers for this step to ensure that participants were not using LLMs to generate the GUI prototype NLR, since recent work (Veselovsky et al., 2023) has shown that crowd-workers broadly use LLMs for text-generation tasks. Each participant created 20 GUI prototype NLR. 600 GUI prototype NLR were created in total, with an average of 4.11 NLR for each of the 146 individual GUIs. We then had a research assistant examine the created GUI prototype NLR to identify NLR we would have to sort out (e.g., because of sensitive content or personal data). Consequently, 26 NLR were finally sorted out. Our final collection amounted to 574 GUI NLR. Next, data quality assurance followed.

Evaluating NLR for GUI Prototypes. Subsequently, we asked crowd-workers to rate the created GUI NLR, filtering for quality and ensuring understandability. Additionally, we expected the GUI NLR to not solely represent a listing of single features since such a granular feature level is usually unavailable when generating first prototypes. We also required NLR that allow understanding of not only the individual GUI page (essential functionality), but also the overall app. To rate the previous GUI NLR, we invited 30 crowd-workers from *Prolific* (Palan and Schitter, 2018), requiring *English skills (B2)* and *experience in UI/UX design*. We excluded two participants for failing our attention checks, leaving us with 28 crowd-workers (20 male, seven female, and one non-binary). Participants had an average age of 30.50 ($\sigma = 7.13$), self-reported 3.79 ($\sigma = 3.55$) years of experience creating and 2.71 years ($\sigma = 3.08$) of experience evaluating visual design.

Crowd-workers rated 574 GUI NLR for 146 GUIs (on average, 3.93 NLR for one GUI). Participants, on average, created 1.95 ratings per GUI NLR on 9-point Likert scales (including *Description Understandability*, *App Purpose Understandability*, *GUI description mainly a list of features*, *GUI description a prompt for Gen AI to create a first GUI draft*, *Understandable overall app’s purpose*, *Understandable individual GUI screen*). To select a single suitable NLR per GUI, we created weighted averages of ratings for each created GUI NLR. For each GUI, we then selected the NLR which was rated the highest. From the 146 high-quality GUI NLR with an average of 18.77 ($\sigma = 8.18$) words per NLR, we randomly sampled ten examples as our validation dataset, which we used to conduct internal experiments. Next, we randomly sampled 50 NLR from the remaining examples to obtain our final test dataset. For the feasibility of conducting the expensive and labor-intensive annotation of generated GUIs by crowd-workers with UI/UX experience, we restricted the test set to 50 examples, while still encompassing diverse NLR for GUIs.

6.3.2 RQ₁: Effectiveness of Prompting for GUI Generation

Annotation of Generated GUI Prototypes. The evaluation of the approaches focuses on assessing the created GUIs. We utilized working prototypes in HTML/CSS for this assessment, and humans evaluated the GUIs based on multiple metrics given the

Table 6.1: Eight Likert-scale items and corresponding self-reported assessments (A)—(H) employed to evaluate GUI prototypes generated by different ZS prompting approaches.

ID	Likert-scale question	Assessment
A	This GUI prototype fulfills all functions defined in the GUI task description.	<i>Feature completion</i>
B	The GUI prototype’s feature set is extensive, clearly exceeding the GUI task description by implementing additional useful functions.	<i>Feature extensiveness</i>
C	The features were perfectly implemented for the given GUI prototyping task.	<i>Feature implementation</i>
D	The organization of information on the GUI prototype page is clear.	<i>Information organization</i>
E	The visual design of the GUI prototype is appealing.	<i>Visual appeal</i>
F	This GUI prototype has only minimal errors.	<i>Minimal errors</i>
G	Overall, I am satisfied with this GUI prototype.	<i>Overall satisfaction</i>
H	The presented GUI prototype looks like a screen from a complete app.	<i>Complete app</i>

GUI NLR, as illustrated in Figure 6.4 (C). For this purpose, we hired crowd-workers with UI/UX experience on the crowd-working platform *Prolific* (Prolific Academic Ltd.). Participants on *Prolific* offer better annotation quality in comparison to other crowd-working platforms (Douglas et al., 2023) and were required to have a high *approval rate* ($> 99\%$), *minimum number of previously completed tasks* (> 30), *experience in UI/UX design* (self-reported), and *English skills*.

Out of 126 survey submissions, we meticulously screened out 25 participants, ensuring a high-quality dataset (eight participants did not complete the survey, and 17 participants failed one or more attention checks). The following data is reported for the 101 participants (73 male, 27 female, and one non-binary) accepted to our study. On average, participants reported an age of 30.86 years ($\sigma = 9.21$), 4.60 years of experience creating visual design ($\sigma = 4.61$), and 3.58 years of experience evaluating visual design ($\sigma = 3.72$). 49% of our participants reported *somewhat high* to *very high* (85% *medium* or higher) knowledge of creating GUI prototypes and 39% *somewhat high* to *very high* (78% *medium* or higher) skills in creating GUI prototypes.

We conducted a comprehensive evaluation of three baseline models: ZS Instruction, ZS-CoT and ZS-CoT with content generation, alongside 14 prompting-based approaches. The latter include PD variants (ZS, ZS-CoT and ZS-CoT with content generation), RAGG with direct encoding for different numbers of retrieved examples ($k = 1, 3, 5, 7$), RAGG with content generation ($k = 3$), and RAGG with explicit feature and layout extraction ($k = 3$). Additionally, we evaluated SC models with varying numbers of

critique loops ($k = 1, 2, 3, 4$) and SC with content generation ($k = 3$). For each of these 17 models, we generated interactive GUIs for all 50 sampled GUI NLR. Each GUI was then annotated three times using eight metrics, resulting in 20,400 ($17 \times 50 \times 3 \times 8$) annotations. For conducting the annotation, participants first received a detailed explanation followed by comprehension checks. Participants were presented with the respective GUI NLR, eight Likert scale items and a link to the interactive GUI prototype. We randomized the GUI selection and order in which GUIs were shown to each participant while ensuring that no participant received multiple GUIs generated from the same GUI NLR (but different model) so that similar GUIs would not bias participants. Since each crowd-worker received 25.5 GUIs on average generated from 17 models (based on 50 GUI NLR), each crowd-worker was randomly assigned a few GUIs generated from the same model, but not based on the same NLR. This rigorous evaluation allows us to systematically compare the strengths and limitations of each prompting-based approach and its parametrization for prototype generation via NLR.

Evaluation Metrics. To assess the quality of prototypes, we evaluated GUIs based on eight measures. Table 6.1 presents the measured Likert items (9-point scale from *Strongly Disagree* to *Strongly Agree*) including items regarding the features and overall satisfaction.

Model Setup. We used the most recent *GPT-4o*⁴ model (128k token context length, accessed in July, 2024), which is a multi-modal extension of the previous *GPT-4* model (OpenAI, 2023). We employed a consistent temperature of $t = .50$ for all models to balance creativity and consistency in the generated outputs (same for RQ2-4), which is an important factor for creative tasks such as GUI generation. To compare the models for significant differences, we conducted two analyses. The first analysis included the two ZS baselines, two PD models and SC ($k = 4$) on the full dataset of 50 NLR. The second analysis additionally included RAGG ($k = 7$) and was conducted on a reduced (*cleaned*) dataset of 15 NLR, since at least 7 GUIs could be retrieved for only 15 NLR. For both analyses, we then conducted the following procedure: to obtain paired data across models, we first aggregated (median) over multiple annotators per same GUI and prompting method (ensures paired observations across models). Subsequently, we initially computed *Friedman tests* (Sheldon et al., 1996) per metric to identify whether there are any significant differences between the set of models. For metrics with significant *Friedman tests*, we computed post-hoc paired *Wilcoxon signed-rank tests* (two-sided) (Woolson, 2007) to identify significant differences between individual models per metric and finally applied *Holm* correction across model comparisons per evaluation metric.

6.3.3 RQ₂: Effectiveness of Example Number on RAGG

To evaluate the influence of the number k of retrieved GUIs for RAGG, we employed the approach with direct encoding of top- k GUIs with a varying number of GUIs ($k = 1, 3, 5, 7$). We restricted the maximum number tested in this setup, since only a smaller

⁴At the time of implementing the proposed approach, we utilized the most advanced MLLM available from OpenAI, namely *GPT-4o* (Hurst et al., 2024), which is an extension over prior *GPT-4* (OpenAI, 2023).

fraction of the entire dataset would be influenced by it (only 30% of the top-20 retrieval sets of the test set contain seven or more relevant GUIs after applying MLLM-based filtering and reranking). As in RQ₁, we computed the results on the *cleaned* dataset (15 NLR) and computed *Wilcoxon signed-rank tests* with *Holm* correction within metrics.

6.3.4 RQ₃: Effectiveness of Number of Loops in Self-Critique (SC)

To evaluate the impact of the number of loops in the SC approach on the effectiveness of GUI generation, we employed the ZS instruction output as initial input and the identical *GPT-4o* configuration as described before (see RQ₁ for details). Then, we evaluated this SC model with five different settings ($k = 0, 1, 2, 3, 4$) of loop numbers. We computed *Wilcoxon signed-rank tests* with *Holm* correction within metrics on the full NLR dataset.

6.3.5 RQ₄: Effectiveness of GUI Content Generation

To evaluate the influence of the GUI content generation approach on the effectiveness of generated GUI prototypes, we employed two identical prompting methods *with* and *without* content generation, allowing us to measure statistically significant differences with the *Wilcoxon signed-rank test*. We employed (i) the ZS-CoT model for the baseline prompting approach, (ii) the decomposed prompting approach (PD-ZS), (iii) RAGG _{$k=3$} with direct encoding and (iv) SC _{$k=3$} with ZS instruction output as the initial prototype.

6.4 Results & Discussion

6.4.1 RQ₁: Effectiveness of Prompting for GUI Generation

To assess the effectiveness of our approaches for GUI generation, we evaluated the GUI annotations collected from crowd-workers. Figure 6.5 presents boxplots for the ratings across the eight different metrics considered. Subsequently, we briefly summarize the results for the baselines (ZS, ZS-CoT), PD, RAGG ($k = 7$) and SC (loop number $k = 4$) approaches (the two analyses), not considering GUI content generation.

PD models, particularly when using the sequential prompts variant (PD-ZS), significantly outperformed the baseline in (B) *Feature Extensiveness*. However, both PD approaches did not show statistically significant improvements over the baselines across other metrics such as (D) *Information Organization* (cf. Appendix D Table D.3).

RAGG significantly surpassed at least one baseline on multiple metrics, with higher ratings overall (see Figure 6.5), though differences in (C) *Feature Implementation*, (D) *Information Organization*, and (F) *Minimal Errors* were not significant. Given that RAGG-generated GUIs are more comprehensive, showing statistically significant improvements across (A) *Feature Completion*, (B) *Feature Extensiveness*, (E) *Visual Appeal*, (G) *Overall Satisfaction*, and (H) *Complete App*, maintaining performance on (C) *Feature Implementation*, (D) *Information Organization*, and (F) *Minimal Errors* at least on par with the baselines is notable given the increased complexity (cf. Appendix D Table D.4).

SC models outperformed baselines and PD across nearly all criteria, except for (D) *Information Organization* and (F) *Minimal Errors* (and (H) *Complete App* for PD-ZS). In

6. ZERO-SHOT GUI GENERATION WITH PROMPTING APPROACHES

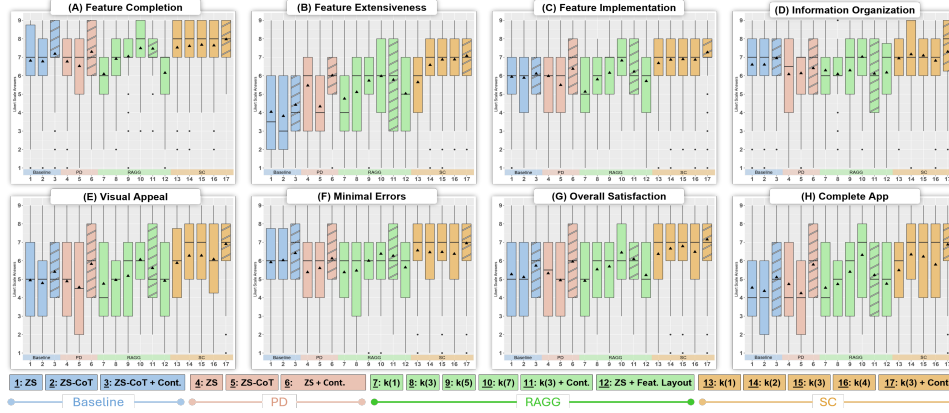


Figure 6.5: Crowd-workers’ GUI ratings across eight different GUI quality metrics and prompting approaches: baseline (1-3, blue), Prompt Decomposition (PD) (4-6, red), Retrieval-Augmented GUI Generation (RAGG) (7-12, green, *cleaned set*), Self-Critique (SC) (13-17, orange). Hatched bars repr. models using content generation, triangles means.

particular, SC achieved significantly higher ratings for (A) *Feature Completion*, (B) *Feature Extensiveness*, (C) *Feature Implementation*, (E) *Visual Appeal*, (G) *Overall Satisfaction*, and (H) *Complete App* (except for PD-ZS) (cf. Appendix D Table D.3). Between SC and RAGG models, we could not observe significant differences, except for (H) *Complete App*. Notably, for this metric, RAGG ($k = 7$) achieved higher ratings overall compared to the SC ($k = 4$) approach (cf. Appendix D Table D.4).

RAGG might not improve over SC in our experiments largely because LLMs might already encode substantial latent GUI prototyping knowledge from pretraining, particularly in general domains. The SC process effectively unlocks and applies this expertise, enabling the model to refine its outputs without relying on external examples. Therefore, RAGG may offer limited gains when the intrinsic knowledge of the LLM is already sufficient. However, the improvements of RAGG over the baseline indicate that LLMs can leverage contextual GUI information when required, suggesting that retrieval-based augmentation may be especially valuable in niche or less familiar domains, where the pre-existing knowledge of the LLM is less comprehensive.

Figure 6.6 illustrates the mean input and output token usage and the respective standard deviation for each prompting approach (for SC the token usage per individual round). For RAGG, input token consumption increases with the number of retrieved examples, while output tokens remain relatively stable, reflecting that more context is provided to the LLM, but the generated amount of tokens for the GUI prototypes does not grow proportionally. In contrast, both input and output tokens increase with additional SC iterations, as each round involves generating and critiquing new prototypes. In the baseline models, most tokens are output tokens because little prompting context is provided and the response is dominated by the HTML/CSS code. Although SC and RAGG approaches consume more tokens than the baseline and PD, the absolute token

6. ZERO-SHOT GUI GENERATION WITH PROMPTING APPROACHES

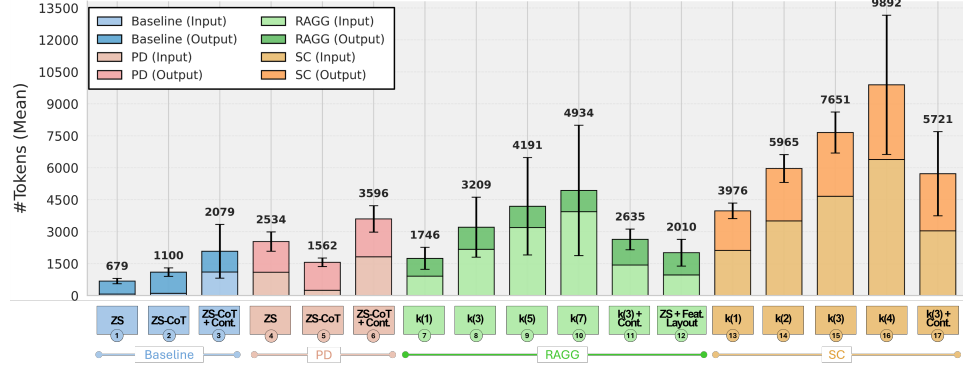


Figure 6.6: Mean input and output token usage and standard deviation per model configuration. Bars are stacked by input (*lighter shade*) and output (*darker shade*) tokens, grouped by method (*Baseline*, *PD*, *RAGG*, *SC*). Error bars indicate one standard deviation for the total token count. Exact total mean tokens per prompting approach above bars.

usage remains moderate and the associated costs are negligible in practice, especially since most performance gains for SC are achieved after the first iteration. This suggests that even advanced prompting strategies can be employed effectively and cost-efficiently in real-world GUI prototyping workflows, providing higher quality of generated GUIs.

To better understand the defects of LLM-generated GUI prototypes, we manually investigated a sample of GUIs with the lowest (*F*) *Minimal Errors* scores. The identified defects can be structured into three potentially non-exhaustive categories. First, (*i*) defects related to the *layout* occurred frequently. This includes *misalignment of GUI components* (e.g., between labels and their respective check boxes), *component overlapping* (e.g., labels and radio buttons or icons with a search bar), *overflow of content* (e.g., images and text that exceed their grouping components) and *underflow of content* (e.g., list items that only span half of the container width). Second, (*ii*) defects related to *function* included *misinterpretation of requirements* (e.g., a navigation app provides buttons to navigate through the app), *missing features* (e.g., upcoming shows not represented in a calendar although requirement explicitly stated in the description, no header or footer), *unusual implementation of features* (e.g., implementing a save or share functionality with text buttons instead of more commonly used icons) and *broken features* (e.g., broken image or icon source links). Third, (*iii*) defects related to inconsistent styling of the GUI (e.g., inconsistent color schema or layouts). Figure 6.7 illustrates two requirements taken from our test set and their respective generated GUI prototypes for models from each group.

Answer to RQ₁: SC significantly outperforms the baselines and PD across most considered evaluation metrics. Furthermore, RAGG models with seven examples ($k = 7$) showed statistically significant improvements over at least one baseline in most metrics, but no statistically significant differences were observed between both RAGG ($k = 7$) and SC ($k = 4$) techniques across all metrics, except for (*H*) *Complete App*.

6. ZERO-SHOT GUI GENERATION WITH PROMPTING APPROACHES

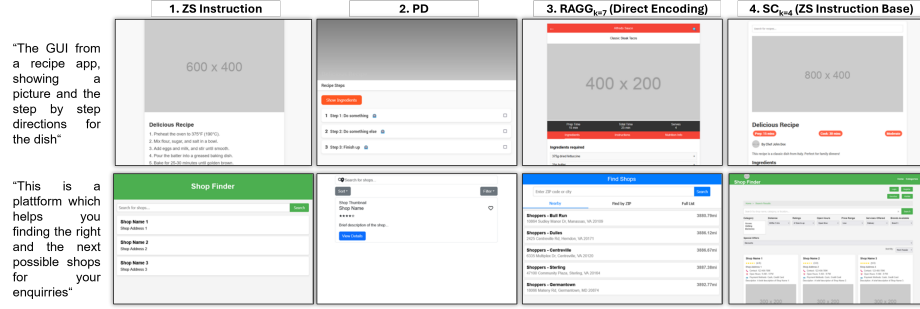


Figure 6.7: Two example requirements from the evaluation dataset with generated GUI prototypes for (1) ZS instruction, (2) PD, (3) $RAGG_{k=7}$ and (4) $SC_{k=4}$ illustrating an overall quality improvement towards the more comprehensive prompting approaches.

6.4.2 RQ₂: Effectiveness of Example Number on RAGG

Considering the number of GUI examples in RAGG, we can observe a significant improvement between $k = 1$ and $k = 5$ for (A) *Feature Completion*, but no significant improvement over the remaining metrics (cf. Appendix D Table D.5). This suggests that the LLM can effectively utilize the examples to extract meaningful functionality relevant to the given requirements. Moreover, considering $k = 1$ and $k = 7$, we can observe statistically significant improvements for (A) *Feature Completion*, (C) *Feature Implementation*, (E) *Visual Appeal*, and (H) *Complete App*. This indicates that even more examples help the LLM not only to improve the feature-related aspects more ((A), (C), (H)), but also non-functional aspects of the prototype, such as the GUI design (E). In addition, considering the increase of k , the mean ratings increased for each subsequent step (except for (D) *Information Organization*), which suggests that – while mostly not significant – more examples potentially contribute to enhancing the overall GUI quality.

Answer to RQ₂: A larger number of GUI examples in the prompt context for RAGG leads to increased mean ratings across metrics (except (D) *Information Organization*), while significance is mainly observed between $k = 1$ and $k = 7$ for (A) *Feature Completion*, (C) *Feature Implementation*, (E) *Visual Appeal*, and (H) *Complete App*.

6.4.3 RQ₃: Effectiveness of Number of Loops in SC

In the first iteration ($k = 1$, with $k = 0$ representing our baseline), the improvements are significant for all metrics except (D) *Information Organization* and (F) *Minimal Errors* (cf. Appendix D Table D.6). These results show that both feature-related metrics and visual aspects are improved, while achieving better (D) *Information Organization* and (F) *Minimal Errors* might be more difficult with an increasingly comprehensive feature collection. In addition, by comparing $k = 2, 3$ with $k = 1$, we could observe a significant improvement for (B) *Feature Extensiveness*, (E) *Visual Appeal*, and (H) *Complete App*. This indicates that the SC model performs plenty of improvements from the initial

prototype to the first iteration and afterwards mainly new features are added, the visual design is improved and the GUI is extended for more *(H) Complete App* (e.g., adding navigation). This could potentially be improved by dynamically focusing the feedback MLLM more on currently underdeveloped parts of the GUI. Overall, we recommend a single SC iteration ($k = 1$) as the default, considering the increasing token consumption.

Answer to RQ₃: The first round ($k = 1$) of SC leads to significant improvements in most aspects of GUI quality, while further iterations ($k > 1$) mainly enhance *(B) Feature Extensiveness*, *(E) Visual Appeal*, and *(H) Complete App*. Most gains from SC are achieved early, with diminishing returns later (token consum. can be kept small).

6.4.4 RQ₄: Effectiveness of GUI Content Generation

Content generation showed a statistically significant improvement for ZS-CoT across multiple metrics, except for *(C) Feature Implementation* and *(D) Information Organization* (cf. Appendix D Table D.7). Similarly, significant improvements are observed for the PD-ZS method, with exceptions of *(B) Feature Extensiveness*, *(C) Feature Implementation*, and *(D) Information Organization*. These results indicate that both methods benefit from the content generation approach. While no statistically significant differences were observed between the RAGG and SC (except for *(H) Complete App*) methods, the mean metric ratings are consistently higher for the content variant across all considered metrics, as shown in Figure 6.5. The results suggest that content generation mainly benefits the simpler methods, while RAGG already benefits from multiple relevant GUI examples to improve its content and SC potentially enhances its content by its internal critique.

Answer to RQ₄: LLM-based GUI content generation significantly enhances ZS-CoT and PD-ZS across metrics (except for *(B) Feature Extensiveness*, *(C) Feature Implementation*, and *(D) Information Organization*). RAGG and SC achieve consistently higher mean ratings, but differences are not significant. These findings highlight the value of adding realistic, context-aware content and images for better GUI prototype quality.

6.5 Threats to Validity

Internal Validity. One potential threat to internal validity is the relationship between measured items. In particular, a larger number of features in the generated prototypes might increase the difficulty of properly organizing the information, increase the number of feature implementation errors and increase the difficulty of creating appealing visual designs. Since the simpler baseline methods usually only implement a small number of features and the other metrics are not normalized on feature count or complexity, this could potentially favor the simpler methods in the mentioned items. Another threat is the selection of participants to evaluate the generated GUI prototypes. To reduce bias, we solely included participants with self-reported experience in GUI design, included only participants that were fluent in English, had at least 30 prior submissions, and had

an approval rate of over 99%. In addition, recent research showed that crowdworkers on *Prolific* create better data quality in comparison to other popular crowdworking platforms (Douglas et al., 2023). Furthermore, we excluded annotations from crowdworkers that failed one or more of overall five attention checks, to improve data annotation quality.

External Validity. One threat to external validity is the description dataset that we employed in our experiments. Since the descriptions are based on the *Rico* GUI dataset, the generalizability of our findings might be restricted. For example, GUIs from special domains which are rare might not be included in the *Rico* dataset. For these rare GUIs, the descriptions we could obtain from users might also contain more ambiguity or obscurity. To ensure a representative sample of functionally more complex GUIs employed for creating the descriptions, we filtered the GUIs by containing at least seven unique GUI component types and then applied random sampling. Furthermore, we asked annotators to detect functionally simple GUIs (e.g., *login*), which we then excluded by majority voting. This increased the inclusion of more complex GUIs, increasing the difficulty of the GUI generation task. In addition, for more than 20% of the descriptions, the retrieval approach was not able to find relevant GUIs, showing the complexity of descriptions and the inclusion of rare GUIs. Another threat to external validity is that we employed only a single LLM in our experiments. However, at the time of evaluation, this model achieved state-of-the-art performance on many benchmarks, meaning that the baseline capabilities of the vanilla LLM were already very strong. As a result, demonstrating further improvements with shown ZS techniques may be more challenging, and observed gains should be interpreted in the context of an already high-performing baseline LLM.

6.6 Limitations

While we presented several advantages and discussed the effectiveness of our proposed ZS prompting approaches for GUI generation, they also possess several limitations. First, the LLM-based GUI reranking approaches significantly outperform state-of-the-art methods (see Chapter 4), but still produce FPs in some cases. Although the FPs are usually not entirely irrelevant to the posed problem, these false instances might still negatively influence the GUI generation effectiveness of the RAGG approach. Therefore, RAGG is highly dependent on the retrieval method and underlying GUI repository. For requirements for which neither the retrieval model nor the MLLM-based filtering and reranking approach is able to find relevant GUIs or the GUI repository lacks relevant GUIs overall, the GUI generation falls back to the baseline ZS instruction approach. In addition, RAGG would potentially work most effectively if the knowledge embodied in the external GUI repository and internally stored in the LLM is complementary to each other. Due to the fact that GUI retrieval on *Rico* as well as the LLM are both data-driven models, the distribution of seen GUI knowledge could be similar. However, in a more particular domain with custom characteristics of the GUI prototypes, which the LLM probably will less likely have stored, the RAGG approach might be able to outperform LLM-only methods. While SC shows high effectiveness for generating GUIs as well, it

consumes a larger amount of tokens while providing the critique and re-generating the prototype compared to baselines. This could potentially be enhanced in the future by a more sophisticated re-generation approach, e.g., updating only affected GUI segments.

6.7 Related Work

Earlier research proposed several ML-based GUI generation approaches. *Variational transformer networks* (Arroyo et al., 2021) and *LayoutTransformer* utilizing *self-attention* (Gupta et al., 2021) have been proposed to generate high-level layouts, including basic low-fidelity GUI layouts with abstract GUI component rectangles. In line with previous approaches, mixing *transformer encoder-decoder* with *Gaussian mixture models* (Bishop, 1994) for generating abstract low-fidelity GUIs from text has been proposed before (Huang et al., 2021). While these approaches can provide coarse design guidance, the generated layouts lack functional and design details compared to the interactive prototypes generated by our approach. Moreover, *GUIGAN* (Zhao et al., 2021) utilizes *Generative Adversarial Networks (GANs)* (Goodfellow et al., 2020) to generate GUI prototypes based on GUI subtrees, i.e. image-based excerpts extracted from *Rico* GUIs. While the GUI prototypes generated by their approach possess a realistic appearance, they are image-based and therefore not interactive, which restricts the usefulness compared to the GUI prototypes generated by our approach.

More recent approaches focused on LLMs for generating GUI prototypes of different fidelity. To generate low-fidelity GUI layouts based on high-level textual layout descriptions and a collection of selected GUI component types, *Instigator* (Brie et al., 2023) trains a full *minGPT* (minGPT - GitHub Repository) model from scratch exploiting a large-scale repository of automatically scraped web pages, which are then transformed to low-fidelity layouts for training. Instead of training a task-specific GPT model from scratch, another approach has been proposed which finetunes a pretrained LLM for generating GUIs exploiting the *Rico* GUI repository (Feng et al., 2023). While both approaches generate GUI layouts or prototypes from text, they require resource-intensive training, create low-fidelity prototypes and generate merely DSLs.

Similarly, *MAxPrototyper* (Yuan et al., 2024) generates a custom DSL to represent GUI prototypes from short text descriptions and a prespecified GUI layout using an LLM. However, their approach focuses mainly on generating GUI content in the form of text and images matching the provided text description. *UIDiffuser* (Wei et al., 2023) utilizes *stable diffusion* (Rombach et al., 2022) to directly generate GUI prototype images from short text descriptions. While this approach can provide coarse design inspirations, the generated images lack clarity and do not represent functioning GUI components.

6.8 Conclusion

The proposed approaches in this chapter were driven by the challenge **C1-4**: *How can we efficiently optimize LLMs for more effective GUI prototype generation?* To tackle this

challenge, we introduced several ZS prompting methods for high-fidelity GUI generation. Based on 20,400 expert annotations, SC and RAGG mostly outperformed ZS baselines. RAGG benefited from more retrieved examples, and SC delivered most of its gains after a single iteration, with both methods incurring moderate token costs. However, RAGG would perform most effectively when the LLM knowledge and GUI dataset are more complementary to each other. LLM-based content generation further improved perceived *visual quality*, *completeness*, and *overall satisfaction*, underscoring that ZS prompting is a practical alternative to training or finetuning for generating interactive GUI prototypes.

CHAPTER 7

CLOSING THE LOOP BETWEEN USER STORIES AND GUI PROTOTYPES: AN LLM-BASED ASSISTANT FOR CROSS-FUNCTIONAL INTEGRATION IN SOFTWARE DEVELOPMENT

After presenting NLR-based GUI retrieval and reranking approaches in the first part of the thesis, followed by ZS-based prompting approaches for efficiently adapting LLMs for NLR-based GUI generation at the beginning of the second part, we continue with challenge **C1**. In particular, we provide a novel approach for efficiently adapting LLMs to generate fully editable GUI prototypes in proprietary representations, which the LLMs have not been pretrained on. The following sections are based on previously published work (Kretzer et al., 2025)¹. Our prompts and demonstration video are publicly available².

Personal Contribution. *Felix Kretzer* and I contributed equally to the ideation and creation of the concept for the work. While I implemented the LLM-based approaches

¹This section is adapted from: Kretzer, Felix^{*}, **Kolthoff, Kristian^{*}**, Bartelt, Christian, Ponzetto, Simone Paolo, and Maedche, Alexander. Closing the Loop Between User Stories and GUI Prototypes: An LLM-based Assistant for Cross-Functional Integration in Software Development. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI, A*)*, Yokohama, Japan, April 2025, pages 1–19. ACM. ^{*}Authors contributed equally. Sections are rearranged and directly reused with only minor adaptations.

²Materials for this chapter are available at <https://github.com/kristiankolthoff/Closing-the-Loop-US-GUI> and the respective demo video at <https://youtu.be/QQd007gJLHQs>

and backend functionality, *Felix Kretzer* implemented the GUI and frontend of the approach. *Felix Kretzer* and I contributed equally to the evaluation design, while *Felix Kretzer* conducted the evaluation and analysis of results. I wrote the section on LLM-based approaches (and created the architecture overview figure) and related work, while *Felix Kretzer* wrote the rest of the original paper and created respective figures and plots.

7.1 Motivation

While NLR-based GUI generation methods with LLMs from the previous chapter are effective in creating GUI prototypes of high quality in common representations, on which the LLMs are pretrained, such as the investigated HTML/CSS, these representations restrict direct editing and customization via commonly employed visual GUI prototyping editors, which enable direct manipulation of visual prototype components. Therefore, recent LLM-based approaches for GUI generation may be effective, but are currently not integrated into typical development workflows³.

In practice, UI/UX designers create prototypes based on requirements provided in different forms. For example, requirements are sometimes only discussed verbally and then quickly turned into paper prototypes. However, a prominent approach is to explicitly articulate requirements, for example, as User Stories (USs) (Cohn, 2004). Requirements are sometimes formalized after building initial GUI prototypes, leading to ambiguity during early GUI prototyping stages. Software developers typically work with GUI prototypes and formalized requirements, where changes during the technical implementation can create a synchronization effort for both the formalized requirements and GUI prototypes. We found that updating requirements in prototypes can be overlooked, making the implemented version the de facto latest version.

Moreover, the creation of GUI prototypes typically necessitates the collaboration of UI/UX designers, requirements analysts (e.g., product owners), and software developers. In practice, SD teams often encounter different tools that are only partially integrated (Palani et al., 2022). Although efforts are being made to integrate workflows (e.g., *Figma's* developer mode (Figma, Inc., 2024)), core processes are still entirely separate. Furthermore, requirements are subject to continuous adaptation and extension during the development (Debnath et al., 2021a). Changing requirements leading to extensive communication effort has been described in the literature, and was one of the most frequently mentioned topics during interviews conducted as part of this study. Due to the sustained modification of requirements, not only are synchronization efforts between the roles in the development teams additionally increased, but also the effort to continuously update the respective GUI prototypes. To summarize, LLM-based approaches are effective in generating GUI representations that are not suited for visual prototype editing and are not well integrated into typical GUI prototyping workflows. Thus, the work in this chapter is driven by the leading research question of challenge **C1-5**: *How can LLMs be efficiently adapted to generate editable GUI prototype representations from NLR?*

³At the time of writing the paper, to the best of our knowledge, we were the first to investigate the integration of LLMs into typical GUI prototyping workflows via a novel RAG-based integration approach.

To tackle this challenge, we devise a novel two-stage RAG approach, which enables the efficient integration of proprietary GUI component libraries (such as *Material Design* (Google LLC, 2025)) into LLM-based GUI generation, thereby allowing the generation of proprietary GUI representations. In particular, we provide an implementation of the approach for the popular GUI component library *Material Design* (Google LLC, 2025) within the well-known prototyping editor *Figma* (Figma). Moreover, we propose a novel *Figma* plugin that integrates the novel RAG-based approach for generating proprietary GUI prototypes. In addition, it closely integrates requirements specifications (e.g., US collections) and the corresponding GUI prototype implementations through several LLM-based matching techniques. Finally, we evaluated the proprietary GUI representation generation approach by crowdsourcing-based annotation of generated artifacts and assessed the effectiveness of our overall approach through a user study, indicating that our approach is able to effectively support users during the GUI prototyping process.

Contributions. With this work, we make the following three research contributions:

- *Two-stage RAG approach for adapting LLMs to proprietary GUI representations:* we present a two-stage RAG approach, which enables the efficient integration of proprietary GUI component libraries for generating proprietary GUI prototype representations. In addition, we provide an implementation of this approach for the popular *Material Design* (Google LLC, 2025) GUI component library within *Figma* (Figma).
- *Novel Figma plugin closely integrating LLMs into GUI prototyping:* we present a novel plugin for *Figma* with role-specific functionalities. The interface for UI/UX designers integrates an approach within a prototyping environment (e.g., *Figma*) that utilizes a state-of-the-art LLM to (i) assess whether a requirement is completed, (ii) identify GUI components completing the requirement (i.e. GUI component and NLR matching) and (iii) generate recommendations for UI/UX components fulfilling a given requirement.
- *Comprehensive evaluation and insights:* we conduct an empirical evaluation of the functionalities of our proposed GUI prototyping assistant for UI/UX designers and the resulting prototype quality and US completion.

7.2 Approach

We first present the LLM-based approaches utilized in the novel *Figma* plugin, with a focus on the two-stage RAG approach for proprietary GUI representation generation. Subsequently, we provide an overview of the functionality of the developed *Figma* plugin.

7.2.1 LLM-Based Approaches

Our underlying LLM framework is composed of several components. An overview of the approach is illustrated in Figure 7.1. In alignment with the figure, we describe the steps

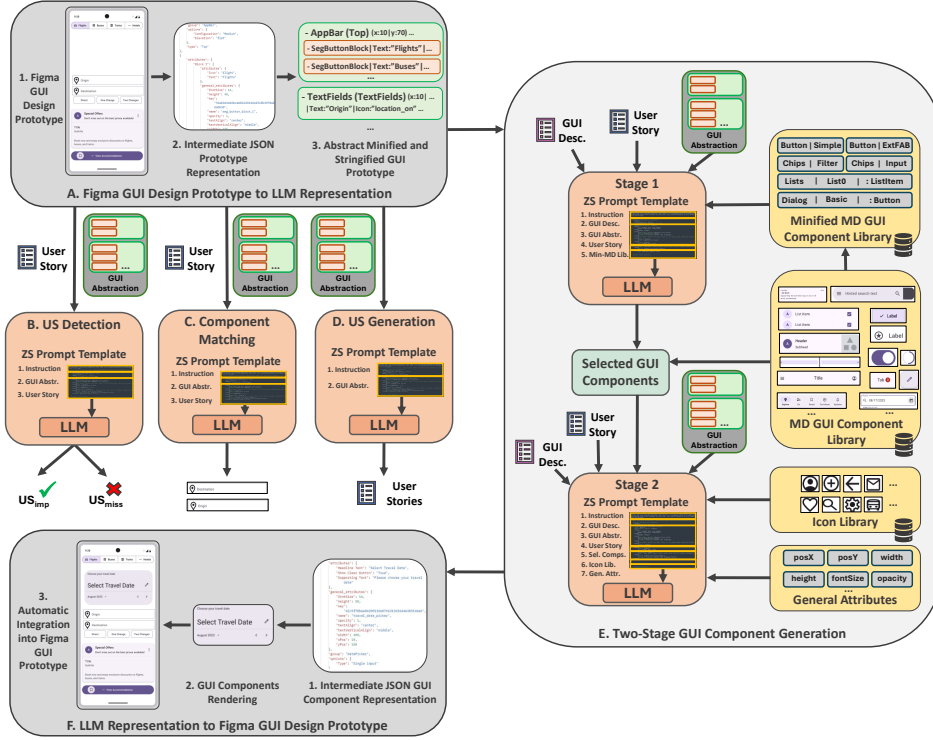


Figure 7.1: Overview of the LLM framework for automatically detecting US implementation, matching GUI components to US, generating US for GUI prototypes and performing two-stage GUI component generation for US including the transformation of GUI prototypes in *Figma* to a compressed representation for inputting into the LLM and rendering of generated GUI components within the prototype. The figure contains *Material 3 Design Kit* (Google LLC, 2025) components from *Google*, under *CC BY 4.0*.

of the approach sequentially in a top-down manner. Since US represent an important way of specifying requirements, our approach focuses on NLR in the form of US. First, (A) we propose a component transforming the GUI prototype represented by *Material Design* components in *Figma* to an abstract and compressed string representation as the input to the LLM. Second, (B) the US detection component consists of a ZS-prompted LLM for deciding whether a provided US is implemented in the GUI prototype, which is especially helpful for the prototype developers under rapidly, continuously changing and updating requirements, ensuring that all requirements are represented in the prototype. Third, (C) the US matching component enables the coupling of GUI components corresponding to a given US, which provides traceability from the NLR to the particular implementation within the GUI prototype. In addition, (D) the US generation component for automatically creating USs for the GUI prototype, ensuring the availability of NLR for each implemented feature in the GUI prototype. Fifth, (E) the two-stage GUI component generation approach enables the rapid creation of implementations

for a provided US. Finally, (*F*) the component transforms the generated intermediate prototype representation back to a rendered GUI component and places it appropriately in the GUI prototype within *Figma*. All detailed prompts used in this work can be found in our supplementary materials. Subsequently, we present each of the shown components of our LLM framework for supporting the GUI prototyping process in more detail.

GUI Prototype Representation (A). Our approach focuses on GUI prototypes created within the popular prototyping tool *Figma* (Figma) and supports prototypes implemented with the *Material Design* (Google LLC, 2025) component library. This extensive component library encompasses over 85 distinct GUI components ranging from elementary individual components such as *Checkbox*, *Slider*, and *Button* to more intricate GUI elements assembled from multiple individual GUI components such as *Dialogues*, *List-Item*, *Search-Bar*, and *Card*. Each library GUI component possesses numerous configuration options including, for example, *Icon* and *Main-Text*, whereas the assembled components encompass several sub-components in a multi-level fashion. To further enhance the versatility of supported GUI components, we incorporated additional more generic component types (e.g., *Label*, *Image-Placeholder*, *Rectangle*, and *Icon*). To enable the previously discussed LLM-based assistance, the prototype initially needs to be transformed to an abstract and minified string representation to provide an efficient and effective input to the LLM. An intermediate JSON representation of the prototype is reduced to the abstract variant, ensuring not only substantial increases in token efficiency, but also the removal of details unnecessary for the proposed tasks to potentially increase effectiveness. The abstract string representation is constructed as a multi-level bullet point list, with each GUI component being represented as an individual item using an abstract pattern, providing basic information of the component such as the *group*, *type*, *position*, and *size* as well as a list of component-specific attributes represented as

```
component-group (component-type) (position) (size) |attribute
name:"attribute value" | (id=#)
```

This representation includes all necessary information for the problems considered in our approach. For example, a concrete instantiation of this pattern for a *Button* could be

```
Button (SimpleButton) (x:25|y:790) (width:359|height:54)
|Icon:"add"|Label
Text:"Search"|Style:"Filled"|State:"Enabled"|Show
Icon:"True"| (id=27)
```

Multiple levels are introduced in the bullet point list when assembled elements contain sub-components and form nested structures to ensure and retain the appropriate grouping of the components, closely reflecting the grouping of the prototype in *Figma*.

Implementation Detection (B). The second component within our approach is represented by the automatic implementation detection of NLR within a GUI prototype

in *Figma*. However, the core ideas behind this approach and evaluation results are presented in Chapter 9 (Part III of the thesis), as part of the MLLM-based verification⁴.

GUI Component Matching (C). Similarly as above, the third component in our approach is represented by the automatic component matching of NLR within a prototype in *Figma*. However, the core ideas behind this approach and evaluation results are presented in Chapter 9 (Part III of the thesis), as part of the MLLM-based verification⁵.

User Story Generation (D). To enable the creation of USs from a fraction of or an entire GUI prototype currently not covered by any US within the requirements collection, we also employ ZS prompting with an LLM. In the corresponding prompt template, we (i) clearly instructed the model to extract all present USs with their respective GUI components and (ii) provided the abstract GUI representation with GUI component identifiers. In particular, the LLM is tasked with creating a JSON, providing multiple objects with the US text and a list of corresponding GUI components as attributes.

GUI Component Generation (E). In addition to the detection, matching, and US generation methods, we propose to facilitate the prototyping process by enabling the contextualized GUI component generation for a given US based on the current GUI prototype in proprietary representations, thereby creating editable prototypes in visual prototyping environments. To tackle the challenge of generating GUI component recommendations for a given US, we employed another ZS prompting approach utilizing an LLM. While the LLM is pretrained on large numbers of text corpora from the web and potentially includes various information about *Material Design* utilized in our approach, the LLM lacks the specific *Material Design* component library and configuration options employed in our approach. Therefore, we manually constructed the comprehensive component library as input to the model. However, to increase token efficiency for the LLM, we propose a two-stage RAG-based GUI component generation method.

As depicted in Figure 7.1E, we first derive a minified *Material Design* component library containing only the information about available component types and sub-component references. Afterwards, we construct our first stage ZS prompt template by instructing the LLM to select all required components from the minified library and encompassing (i) the US to generate the implementation for, (ii) the abstract GUI representation, (iii) a brief textual description of the main functionality of the GUI, and (iv) the minified *Material Design* component library. Afterwards, we construct the second stage ZS prompt template instructing the LLM to generate the intermediate GUI component representation

⁴Note that the LLM-based implementation detection approach has been published as part of a previous work (Kolthoff et al., 2024c). However, in the chapter, the focus lies on LLM-based generation of proprietary GUI representations and integration of these LLM-based components into a GUI prototyping assistance plugin for *Figma*. For coherence, the detection approach with evaluation will be presented in Chapter 9.

⁵Note that the LLM-based component matching approach has been published as part of a previous work (Kolthoff et al., 2024c). However, in the chapter, the focus lies on LLM-based generation of proprietary GUI representations and integration of these LLM-based components into a GUI prototyping assistance plugin for *Figma*. For coherence, the detection approach with evaluation will be presented in Chapter 9.

by utilizing the same information as before, but instead of the minified *Material Design* library, we additionally provide (i) the full specifications from the *Material Design* GUI component library retrieved for the components selected by the LLM, (ii) an icon library, and (iii) specifications for general attributes that are shared among all GUI components (such as *positionX*, *positionY*, *width*, and *height*).

By conducting this two-stage RAG approach, we avoid inputting the entire large *Material Design* component library for each generation and instead reduce it to a minimal representation. To ensure correctness of the generated component specifications of the first and second stages, we set the temperature of the LLM to zero for a more probable and deterministic output. Moreover, we implemented an automatic verification of the generated specifications by matching them against the *Material Design* specifications. The component position and size are generated by the LLM via the general attributes and influenced by the components and their positions in the current prototype. Finally, the intermediate representation is rendered within our *Figma* plugin and the generated GUI component can directly be integrated into the existing GUI prototype design, including the automatic positioning of the component within the GUI prototype. Although the focus of our approach lies on generating functional GUI prototypes, the LLM is also able to generate different component styles (e.g., different buttons such as *IconButton* or *FloatingActionButton*), since different styles are represented as different components in the *Material Design* specification. In the future, we plan to integrate more fine-grained styling options (e.g., *background color*, *font color*, *font style*) by extending the general attributes. With this functionality, custom *corporate identity* such as a color palette or special fonts could be incorporated into the generation process by enabling users to provide textual style requirements in addition to the textual functional requirements.

LLM Configuration. As the LLM in our approach, we use the most recent *GPT-4o* model⁶ (Hurst et al., 2024) with 128k token context (*accessed in August 2024*), which represents an advancement over *GPT-4* (OpenAI, 2023). We decided on *GPT-4o* since it provides state-of-the-art performance across many NLU and NLP tasks (OpenAI, 2024).

7.2.2 GUI Prototyping Plugin (Control)

Figure 7.2 shows the *Figma* plugin of our proposed approach. In particular, we developed two variants of our GUI prototyping assistant to evaluate its effectiveness in our lab study: a *control* and a *treatment* configuration. The *treatment* configuration builds on the baseline functionality of the *control* version by introducing advanced features enabling LLM-based automation. In the following, we describe the shared functionality of the *control* configuration, followed by the unique capabilities of the *treatment* configuration. Table 7.1 provides a comparative overview of these differences. The following section outlines the baseline functionality of the GUI prototyping assistant, as implemented in the *control* configuration. These core features are also present in the *treatment* configuration.

⁶At the time of implementing the proposed approach, we utilized the most advanced MLLM available from *OpenAI*, namely *GPT-4o* (Hurst et al., 2024), which is an extension over *GPT-4* (OpenAI, 2023).

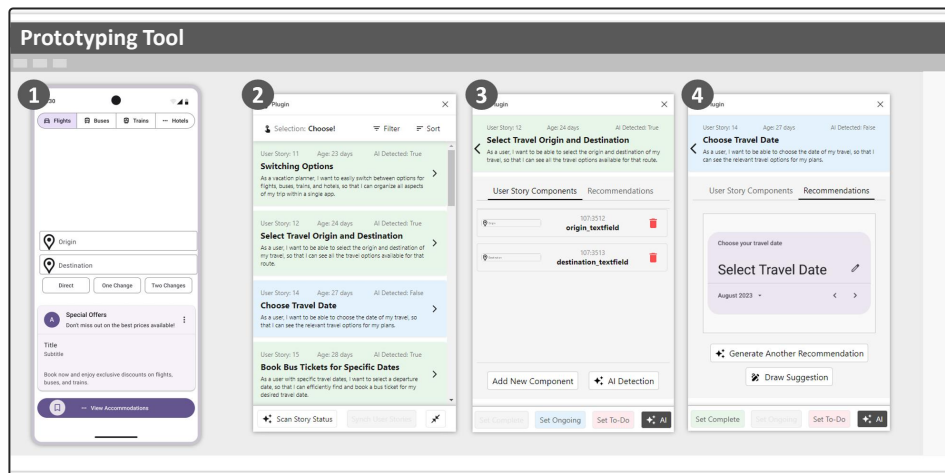


Figure 7.2: GUI prototype (1) and three views (2–4) of our assistant for GUI prototype designers integrated as a plugin into a prototyping tool. Our assistant displays USs (2) imported from collaboration tools (e.g., JIRA) for prototype designers to reference while working. The assistant can automatically detect (bottom bar in 3 and 4) whether a US is implemented in the GUI prototype, can detect GUI components completing a US (3) and can generate and draw GUI components fulfilling a given US (4). Figure contains *Material 3 Design Kit* (Google LLC, 2025) components from *Google*, under *CC BY 4.0*.

Tool Integration. Our assistant directly integrates into the popular prototyping tool *Figma*. We based the size of the plugin on popular plugins for *Figma* and integrated a minimization function ($C6$ and $T6$ in Figure 7.3), allowing the plugin to shrink and display only numerical values for *open*, *ongoing*, and *completed* USs to optimize screen space. This feature is particularly useful during tasks such as graphical finetuning of design components, where workspace is a priority (see $C6$ and $T6$ in Figure 7.3). With the assistant’s integration as a plugin, we aim at supporting UI/UX designers, product owners, and software developers within tools of practice. By facilitating tasks in tools like *Figma* such as synthesizing textual descriptions (i.e. GUI prototype requirements) into GUI components, and picking and contextualizing fitting GUI components, we aim at allowing UI/UX designers to focus more on the creative and conceptual aspects of their work, such as sketching rough designs and crafting the overall prototype user experience.

Listing User Stories in Figma. The assistant lists USs directly in the plugin, with their status visually highlighted as *completed* (green), *ongoing* (blue), or still a *to-do* (red). By default, all USs for all app screens currently open in *Figma* are displayed. If a single frame (i.e., the screen of an app) is selected (selection indicator: $C1$ and $T1$ in Figure 7.3), only the USs associated with that frame are shown in the plugin. Additional information, such as the age and identifier of USs, is displayed alongside status indicators. Users can update the status of a US (*completed*, *ongoing*, or *to-do*) directly in the plugin ($C3$ and $T3$

7. CLOSING THE LOOP BETWEEN USER STORIES AND GUI PROTOTYPES

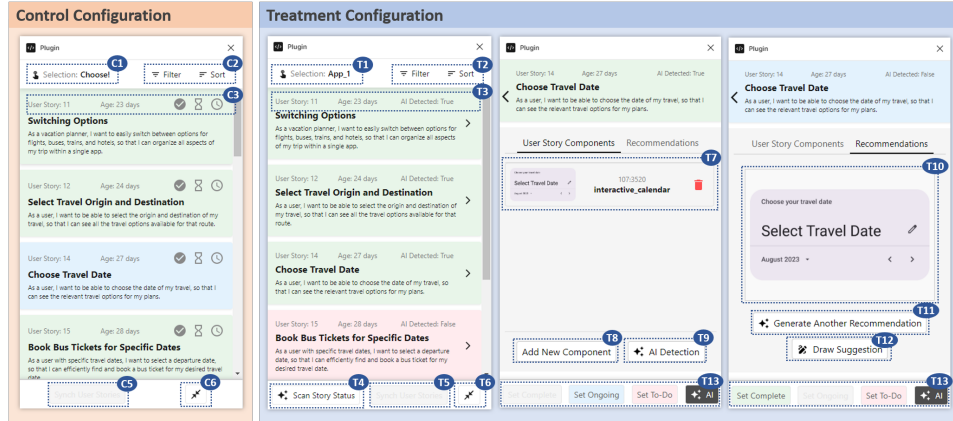


Figure 7.3: Assistant for GUI prototype designers in the *control* (left side) configuration and *treatment* (right side) configuration. In both configurations, the assistant shows USs based on a selected frame in the prototyping tool ($C1/T1$), allows to filter and sort USs ($C2/T2$), can synchronize the USs with a database ($T4$) and be minimized ($C6/T6$). USs can be marked *completed*, *ongoing* or as a *to-do* ($C3$ and $T13$). In the *treatment* configuration, the assistant can identify components completing a US ($T7$), manually added ($T8$) or identified using an LLM ($T9$). Additionally, an LLM can detect the completion status of a US ($T13$), can recommend GUI components completing a US ($T11$ and $T10$) and draw them into the GUI prototype ($T12$). Figure contains *Material 3 Design Kit* (Google LLC, 2025) components from *Google*, utilized under the *CC BY 4.0 license* as previously.

in Figure 7.3). Based on the interviews, we implemented filtering (based on *State* and *AI Detected* in the *treatment* configuration) and sorting (based on *age* and *state* of the US).

Change Intervention. One identified design rationale guiding our implementation is the assistant’s ability to deal with new and constantly changing requirements. To fulfill this rationale, we developed a feature synchronizing USs (e.g., with an external requirements database). For the evaluation conducted in this paper, we developed an intervention, as shown in Figure 7.4. As part of this intervention, users were notified of updates after a specified interval, when the “*Sync User Stories*” button became active, simulating a synchronization with US databases (such as *JIRA*). Clicking the button led to the new or updated USs being displayed in the plugin and highlighted with red dots.

7.2.3 GUI Prototyping Plugin (Treatment)

The *treatment* configuration builds upon the baseline features of the *control* configuration by incorporating advanced functionalities guided by specific design rationales (from formative interviews). In this section, we describe the unique features implemented exclusively in the *treatment* configuration, as illustrated on the right side of Figure 7.3.

Table 7.1: Comparison of assistant’s features in *control* and *treatment* configurations.

Feature	Control Configuration	Treatment Configuration
Tool Integration	Plugin integrated in Figma	Plugin integrated in Figma
User Story Listing	Lists all user stories	Lists all user stories
Change Intervention	Synchronizes user story updates	Synchronizes user story updates
User Story State Mang. / Detection	Manual	Manual and LLM-based
User Story Matching	Not available	LLM-based matching of GUI components to user stories
GUI Component Generation	Manual	LLM-based GUI components generation allowing user control over suggestions

User Story Detection. The assistant can automatically detect whether a US has already been implemented in a GUI prototype. Leveraging our LLM-based detection approach, the method automatically analyzes the implementation status of the US in a selected prototype. Users can apply this feature at two levels: individually for specific USs (T_3 in Figure 7.3) or collectively for all stories in a frame (T_4 in Figure 7.3). Detected implementation states are visually indicated by color changes. After clicking “*Scan Story Status*” (T_4 in Figure 7.3), discrepancies between user-assigned and LLM-detected states are flagged for review in a dialog box, where users can confirm or override the LLM’s suggestions. To ensure transparency, each US in the list displays a label indicating whether the completion state was assigned by the user or detected by the LLM (e.g., “*AI Detected: True/False*,” see T_3 in Figure 7.3). By automating detection, this feature aims at minimizing manual effort and ensuring a reliable overview of the GUI prototype implementation progress.

User Story Matching. For our assistant, we also implemented the function to automatically recognize which components of a GUI prototype in *Figma* fulfill a US, enabling the traceability of requirements within the implemented GUI prototype. This feature is shown in the third screen in Figure 7.3. Matched components are displayed in a dedicated field (T_7 in Figure 7.3), showing their names as assigned in *Figma* alongside an isolated image of each component. Users can manage these associations by adding components manually (“*Add New Component*,” T_8), using the LLM to detect components (“*AI Detection*,” T_9) or by removing incorrectly associated components. The objective of this

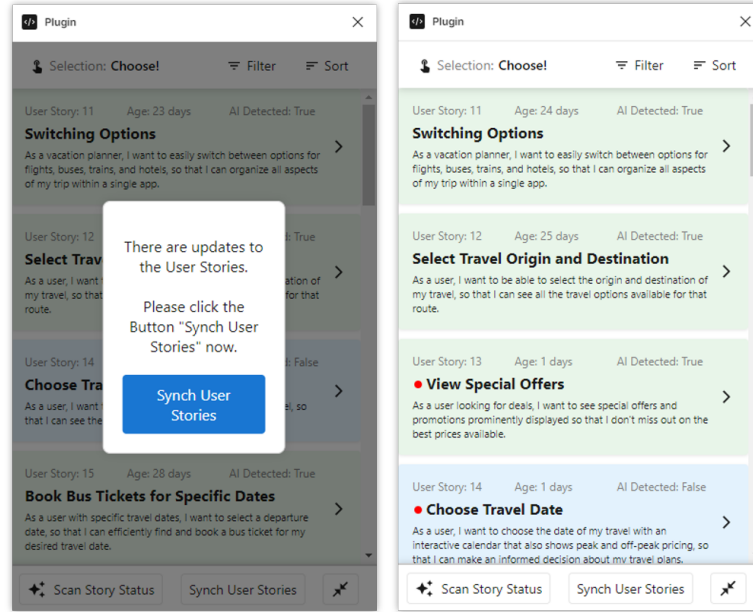


Figure 7.4: New and changed USs intervention shown to participants after 30 minutes in both the *treatment* and *control* configuration in lab experiments (shown: *treatment*). Participants were shown a pop-up message and later red dots next to updated USs, mimicking requirements updates and synchronization with collaboration tools (e.g., *JIRA*).

feature is to provide users with a better understanding of the components influencing the decision of the LLM for automatically assessing the US implementation status and to enable the direct interlinking between US and their counterparts in GUI prototypes. Thereby, it provides transparency regarding the features that still necessitate attention and enables the verification of whether the appropriate GUI components have been utilized.

GUI (Component) Generation. Our assistant also allows users to generate components based on USs in proprietary GUI representations, which can be directly integrated into *Figma*. This feature is shown in Figure 7.3 in the *treatment* screens (fourth screen from the left). To ensure that users retain control over the decisions of the assistant and decide for themselves whether LLM-based component proposals are to be incorporated, recommendations are first previewed in the assistant (*T10* in Figure 7.3). Users can then decide whether they want to use the button “Generate Another Recommendation” (*T11* in Figure 7.3) to generate another recommendation or use the button “Draw Suggestion” (*T12* in Figure 7.3) to insert these components as already correctly placed GUI components in *Figma*. In this case, correctly contextualized and parameterized material design assets, material icons, labels, or rectangles are drawn into the existing GUI prototype. Our assistant utilizes the existing design as a context for correct dimensioning and positioning.

7.3 Experimental Evaluation

In this chapter, we present the underlying methodology of our evaluation studies. In particular, we first focused on measuring the ability of our approach to create relevant GUI components matching the NLR. Subsequently, we focused on the question if our LLM-based assistant improves the effectiveness of GUI prototyping. In addition to the resulting quality of the GUI prototypes, we were also interested in the subjective satisfaction of the participants and the usability of our assistant. Finally, we focused on the question of how effective our approach is in generating USs from components of GUI prototypes. Overall, we posed the following research questions for our evaluation:

- **RQ₁:** *How effective are LLM-based approaches for generating GUI prototype components in proprietary representations completing a US?* To answer this question, we used a previously created and published dataset encompassing USs and corresponding GUI screens. With our two-stage RAG approach, we generated implementations for each US and annotated their quality with *Prolific* crowdworkers.
- **RQ₂:** *How does our LLM-based assistant influence GUI prototype quality and US completion?* To answer this question, we conducted a *between-subjects* user study, asking participants to create several GUI prototypes in the *control* or *treatment* configuration of our approach and evaluated them with *Prolific* crowdworkers.
- **RQ₃:** *How does our LLM-based assistant influence perceived user experience?* To answer this question, participants were asked to answer multiple questionnaires after finishing the user study tasks. In particular, we measured common usability metrics including *SUS*, *NASA-TLX*, and *CSI* from participants.
- **RQ₄:** *How effective are LLM-based approaches for generating user stories from components of GUI prototypes?* To answer this question, we employed a random sample of GUI screens (dataset from *RQ₁*) and generated USs with the LLM-based approach. Afterwards, we evaluated the created USs with *Prolific* crowdworkers.

In the following, we will describe the underlying methods, procedures, and evaluation datasets employed to provide answers to the research questions articulated above by a series of evaluation studies. In Figure 7.5, we present an overview of our three evaluations, investigating *RQ₁*, *RQ₂* together with *RQ₃* through a user study and finally *RQ₄*.

7.3.1 RQ₁: GUI Component Generation

First, we present our evaluation which examines the extent to which the recommendations generated by our LLM-based approach are suitable for fulfilling the corresponding USs.

Procedure. In order to measure the extent to which the GUI component generations in proprietary representations, created by our two-stage RAG approach, match their corresponding USs, we employed a previously created and published dataset, which

7. CLOSING THE LOOP BETWEEN USER STORIES AND GUI PROTOTYPES

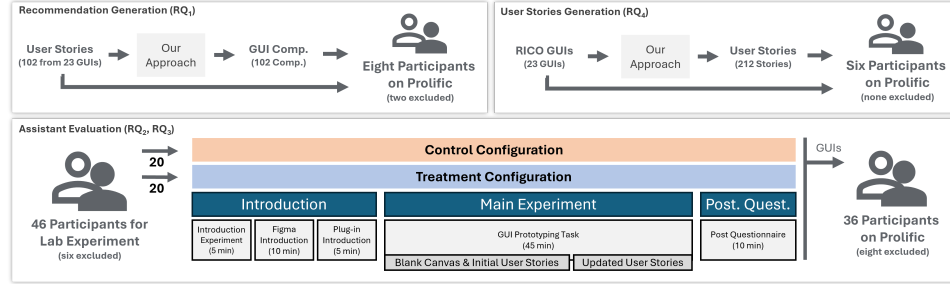


Figure 7.5: Overview of experiment procedures for evaluating the RAG-based GUI component generation in proprietary representations (RQ_1), the assistant (RQ_2 and RQ_3), and the US generation (RQ_4). For RQ_1 , GUI components were generated with our approach from USs and evaluated by crowdworkers on *Prolific*. For RQ_2 and RQ_3 , participants were recruited from a student panel, randomly assigned to either the *control* or *treatment* group to create GUI prototypes. They were then evaluated by a second set of participants sourced through *Prolific* who provided prototype annotations. For RQ_4 , USs were generated with our approach and evaluated by crowdworkers on *Prolific*.

provides a US collection combined with GUI screens from *Rico*⁷. Recommendations were then generated using our approach with the assistant in the prototyping tool, rendered as images and presented to crowdworkers on *Prolific* (Prolific Academic Ltd.; Douglas et al., 2023) for evaluation. As there was no existing *Figma* context in this setup (no previous GUI prototype for which more recommendations are generated), a short, high-level text description as context created by students was utilized in addition to the USs (e.g., “a screen from a travel app showing flight search results”). Created GUI components were presented to the crowdworkers as they were created by the novel LLM-based assistant.

Participants. In particular, we invited eight crowdworkers from *Prolific* with self-reported UI/UX experience, more than 30 *previous submissions* and a *high approval rate* (>99%). No crowdworker participated in any of our other studies. We had to exclude two participants based on failing our attention checks. The remaining participants (4 male, 2 female with an average age of 35.7 years) had, on average, 6.7 years ($\sigma = 5.59$) of experience in creating and 3.7 ($\sigma = 3.40$) years of experience in visual design evaluation (GUI prototyping). We did not collect information about participants’ LLM or generative AI experience, as such expertise was not required for evaluating the GUI components and revealing that the GUI components were LLM-generated could have introduced bias.

Data Collection. The crowdworkers were shown the GUI components in this survey as a picture in combination with the respective US. The crowdworkers were then asked whether the GUI components fully meet the functional requirements described in the

⁷This dataset was created and previously published in prior work (Kolthoff et al., 2024c) for evaluating the component detection and matching components and will be discussed in more detail in Chapter 9.

US, the GUI excerpt contains all the necessary components (e.g., *buttons*, *text input fields*) to fulfill the US and whether textual descriptions within the GUI excerpt (e.g., *labels*, *instructions*, *messages*) are clear and appropriate for fulfilling the US (9-point Likert scale).

7.3.2 RQ₂, RQ₃: GUI Prototyping Assistant Evaluation

To evaluate our assistant, we conducted a lab experiment in which participants were asked to create GUI prototypes based on predefined USs in a controlled environment. Next, we evaluated the resulting GUI prototypes with crowdworkers. We explicitly decided against a remote structure (e.g., with crowdworkers) to better control boundary conditions (such as the context and environment as well as processing time for the GUI prototyping tasks).

Procedure. We created two versions of the assistant (*control* and *treatment*) and evaluated both in a *between-subjects* design experiment. For this purpose, GUI prototypes were created by participants in a lab from predefined USs. USs were derived from the publicly available US dataset (same dataset as used in RQ₁) to employ high-quality USs that have already been evaluated in previous studies. While the USs are focused on single GUIs and do not span across multiple GUIs of an entire app, the dataset contains multiple coherent USs describing different functionalities of a GUI screen. The participants in the lab had help from one or the other version of the assistant (*control* or *treatment*). The GUI prototypes created were then evaluated by crowdworkers with UI/UX experience sourced on *Prolific* (Prolific Academic Ltd.; Douglas et al., 2023). In the following, we present insights into the lab experiment and crowdworking related procedures.

Lab Experiment. In the 75-minute laboratory study, after agreeing to the data collection, participants read a briefing on the study task (5 min), watched a video explaining the GUI prototyping tool used (10 min) and a video demonstrating the assistant implemented as a plugin for the GUI prototyping tool (5 min). Since our assistant in the *treatment* configuration generated *Material Design* components, we therefore decided to give both groups an introduction to *Figma*, including the use of *Material Design* components. Then, participants started the GUI prototyping task (45 min) and finished the experiment with a post-hoc questionnaire (10 min). Participants worked with mobile screen templates and had pre-loaded icons and *Material Design* assets available. For the GUI prototyping task, the laboratory study participants were challenged with creating up to three GUI prototypes based on USs. For each GUI prototype, there were eight USs to complete. After 30 minutes, there were updates for two existing USs and two new USs were added to the assistant. The update was intended to simulate changed requirements, as is usual in practice. Participants were instructed to start with the first GUI prototype and its first eight USs for this GUI prototype and continue with the second and third GUI prototypes only when the previous prototype was completed. For our *control* group, which used the assistant without generative component creation, producing three prototypes as part of the study task is clearly extensive. We deliberately opted for an extensive task so that no participant would finish early, even in the *treatment* with generative component creation, and would continue to prototype in the 45-minute GUI prototyping phase. We

measured the load in pretests to find the right amount of GUI prototyping tasks. The study included attention and comprehension checks. The study design was carefully evaluated with the university’s Institutional Review Board (IRB).

Evaluation of GUI Prototypes. We conducted an evaluation study on *Prolific* with crowdworkers to evaluate the created GUI prototypes. We chose *Prolific* since comparative studies indicated that crowdworkers on *Prolific* produce higher data quality in comparison to other crowdworking platforms (Douglas et al., 2023). Participants from *Prolific* received a questionnaire to evaluate the GUI prototypes created in the lab sessions. After consenting to data processing, participants received a study description and evaluated 20 GUI prototypes each, which were randomly drawn. On average, each participant received 13 GUI prototypes for the first GUI prototyping task (five for the second and two for the last GUI prototyping task) created with the assistant in the *control* or *treatment* configuration, respectively. The study lasted 65 minutes on average and included six attention checks. Furthermore, the study design was again carefully evaluated with the university’s Institutional Review Board (IRB).

Participants After describing the user study procedure, in the following, we describe the participants of both the lab study and the evaluation of the created GUI prototypes.

Lab Experiment. We recruited 46 participants from a university panel for the lab study who were randomly assigned to the *control* group (23 participants) or *treatment* group (23 participants). In both groups, three participants had to be eliminated (one technical problem, three times failure to complete attention checks, two failures to meet the 45-minute GUI prototyping task time) ex-post. Eventually, 20 participants were admitted to the study in both groups. The participants in the *control* group (14 male, 6 female) were, on average, 25.40 years ($\sigma = 4.65$) old. The participants in the *treatment* group (12 male, 8 female) were, on average, 23.85 years ($\sigma = 2.67$) old. Participants in both groups studied, on average, a little over four years and had the same average experience with creating and evaluating visual design (such as GUI prototypes).

Evaluation of GUI Prototypes. On *Prolific*, we originally invited 36 participants with self-reported UI/UX experience, more than 30 *previous submissions* on *Prolific*, and a *high approval rate* ($>99\%$). Submissions from eight participants were excluded for failing one or multiple attention checks. Only data from the remaining 28 (22 male, 6 female) participants were considered. *Prolific* participants were 28.25 years old ($\sigma = 6.78$) and had 4.25 years ($\sigma = 5.87$) of experience creating visual design and 3.86 ($\sigma = 4.04$) years experience evaluating visual design on average. We did not collect information about participants’ LLM experience, as such expertise was not required for evaluating the GUI prototypes and revealing that the GUI prototypes’ creation was partially assisted by our LLM-based assistant could have introduced bias.

Data Collection We collected a range of data, both in the lab setting and on *Prolific*.

Lab Experiment. During the lab experiment, we logged the usage data of the assistant (navigation in the assistant, clicks and web-calls in the *treatment* group) and the GUI

prototypes generated during the sessions. In addition, the participants filled out a post-hoc questionnaire in which task load with the *National Aeronautics and Space Administration – Task Load Index (NASA-TLX) Raw* (Hart, 2006), *System Usability Scale (SUS)* (Brooke, 1996a), and *Creativity Support Index (CSI)* (Carroll et al., 2009) were recorded. We asked for further Likert items (e.g., ease of use and effectiveness of the assistant) and surveyed subjective use of the features (detection of US completion, recognition of the GUI components of a fulfilled US, and recommendation of GUI components) in the *treatment*, and in both groups positive and negative aspects of the assistant via open text.

Evaluation of resulting GUI Prototypes. For each GUI and US shown, the crowdworkers assessed the extent to which the GUI fully meets the functional requirements described in the corresponding US, whether the GUI contains all the necessary GUI components (e.g., *buttons, text input fields*) to fulfill the US, and whether text within the GUI (e.g., *labels, instructions, messages*) is clear and appropriate for fulfilling the US. Additionally, they rated each GUI for overall consistency with USs, visually appealing design, clear information organization, intuitive interaction, whether the GUI looks like an app page, minimal prototype errors, and overall satisfaction with the GUI prototype.

7.3.3 RQ₄: User Stories Generation

We evaluated the extent to which our approach can create USs from existing prototypes.

Procedure. To evaluate the ability to create suitable USs, research assistants recreated *Rico* GUIs (Deka et al., 2017) for the prototyping tool. 23 GUIs were randomly chosen based on the GUIs employed in the previously described dataset (same dataset as used in RQ₁). We then utilized these GUI prototypes to create LLM-based USs for the GUI components using our proposed assistant. Created USs were presented to the crowdworkers as they were generated, i.e., without editing. These were then evaluated by *Prolific* crowdworkers with UI/UX experience. Each crowdworker rated ten prototypes and the corresponding USs. On average, 9.2 USs were created by the LLM for each GUI.

Participants. We recruited six crowdworkers through *Prolific* (Prolific Academic Ltd.; Douglas et al., 2023), applying the same selection criteria as in the lab study. None of the crowdworkers had participated in any of our previous studies. No participant failed any of our attention checks. The final sample (6 male, average age of 33.83 years) had an average of 6.00 ($\sigma = 4.12$) years of experience in creating and 4.33 ($\sigma = 2.98$) years of experience in visual design evaluation (GUI prototyping). Once more, we did not inquire about participants' LLM experience, since such was not required for assessing the USs.

Data Collection. The USs created and GUI prototypes were presented to crowdworkers for comparison. Next, they assessed the USs in combination with the prototype for the following four items: (i) "*The user story accurately describes functionality found in the presented GUI.*", (ii) "*The user story is written with sufficient clarity and precision.*", (iii) "*The user story is specific enough to describe a particular feature of the presented GUI.*", finally (iv) "*The level of detail in this user story meets the standards in a professional project.*"

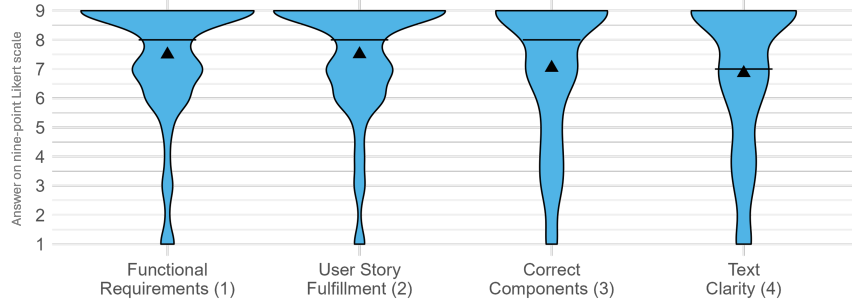


Figure 7.6: Crowdworker ratings (9-point Likert scale) for components generated by our LLM-based approach based on over 100 USs. Crowdworkers rated (1) functional requirements fulfillment, (2) US fulfillment, (3) component correctness and (4) text clarity for USs and derived GUI components. Triangles represent means, black lines medians.

7.4 Results

In this section, we provide the results on the capabilities of our LLM-based approach to create GUI components based on USs through measuring how well the generated components fulfill the USs (RQ_1). Afterwards, we present the results of the lab experiment using the assistant, including the evaluation of the lab results on *Prolific* (RQ_2 , RQ_3). In addition, we provide the results of the evaluation for automatically creating USs (RQ_4).

7.4.1 RQ_1 : GUI Component Generation

To maximize the diversity of our requirements, we took 102 USs from 27 different GUIs in the publicly available dataset (for details, see Section 7.3.1) and generated components for each US using our LLM-based approach. These were presented individually (i.e. as single components with the USs) to crowdworkers on a basic schematic mobile app outline. We collected 40 pairs of US component ratings from each of the six crowdworkers, resulting in an average of 2.35 ratings per US and GUI component.

Crowdworkers rated USs and generated GUI components to assess whether the components (1) allow the user to perform the task as specified by the US, (2) all necessary components for fulfilling the US are presented, (3) appropriate components were chosen, and (4) text of components was clear and appropriate. All ratings were obtained on nine-point Likert scales (1: *Strongly Disagree*, 3: *Disagree*, 5: *Neutral*, 7: *Agree*, 9: *Strongly Agree*). Figure 7.6 shows violin plots for each of the four assessed aspects. All four aspects were rated high, with mean (median) values of 7.5 (8) for functional requirement fulfillment, 7.51 (8) for US fulfillment, 7.04 (8) for component correctness, and 6.86 (7) for textual clarity. In addition, to evaluate the efficiency benefits, we compared the proposed two-stage RAG-based GUI component generation approach against a similar one-stage GUI component generation ZS prompt, which always incorporates the full *Material Design* component library. Over the 102 USs dataset, the two-stage GUI generation approach accomplished a considerable 64.35% reduction of the consumed

7. CLOSING THE LOOP BETWEEN USER STORIES AND GUI PROTOTYPES

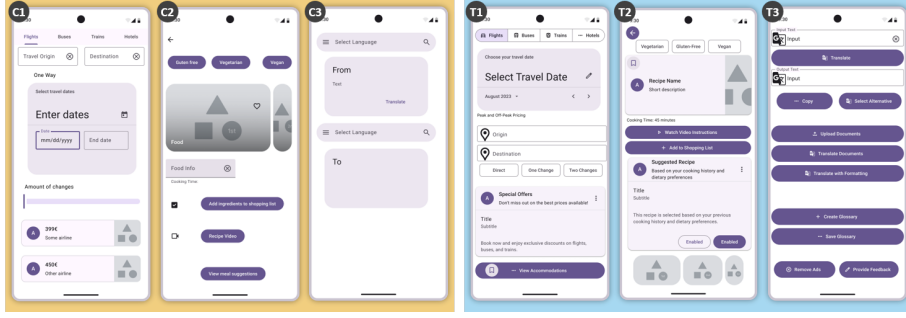


Figure 7.7: Examples of GUI prototypes created in the lab sessions with the *control* (orange, left) and *treatment* (blue, right) configuration of the assistant for all three tasks: (1) travel booking app, (2) recipe app and (3) translation app (Control: C1 — C3, Treatment: T1 — T3). Material 3 Design Kit (Google LLC, 2025) components, Google, CC BY 4.0.

input tokens (Mean $\#One\text{-}Stage\text{-}Tokens=8559.49$ |Mean $\#Two\text{-}Stage\text{-}Tokens=3050.78$) and a significant 60.82% reduction of the total consumed or generated tokens (Mean $\#One\text{-}Stage\text{-}Tokens=9359.44$ |Mean $\#Two\text{-}Stage\text{-}Tokens=3666.54$)⁸. These evaluation results clearly highlight the strong token consumption efficiency improvements by using RAG.

Answer to RQ₁: Our RAG-based approach for generating GUI components in proprietary representations achieves high effectiveness, as indicated by the high annotation ratings received for the generated GUI components. The two-stage RAG approach significantly reduces the mean total (input) token consumption by 60.82% (64.35%).

7.4.2 RQ₂, RQ₃: GUI Prototyping Assistant Evaluation

Our analysis of the data collected in the lab experiment is twofold. Firstly, we examine the completion of USs and the quality of the GUI prototypes created by study participants, as assessed by crowdworkers on *Prolific*. Afterwards, we report the self-assessments of the lab participants, including both qualitative and quantitative aspects. In addition, we evaluated the usage data of the assistant, highlighting which features and functions were used by the participants. Figure 7.7 shows three GUI prototypes created by participants from the *control* and *treatment* groups for the first, second, and third prototyping tasks.

User Story Completion and Prototype Quality. The results for US completion and overall GUI prototype quality are based on data collected from crowdworkers on *Prolific*. The results for evaluating the completion of the USs are shown in Figure 7.8. The participants in the lab study were tasked with creating three app prototypes based on ten USs each. The participants were told to start with the first app and move on to

⁸At the time of writing the original manuscript, this reduction in token consumption led to a cost reduction from \$0.052 to \$0.024 per generated GUI component using the *GPT-4o* model (OpenAI, 2024).

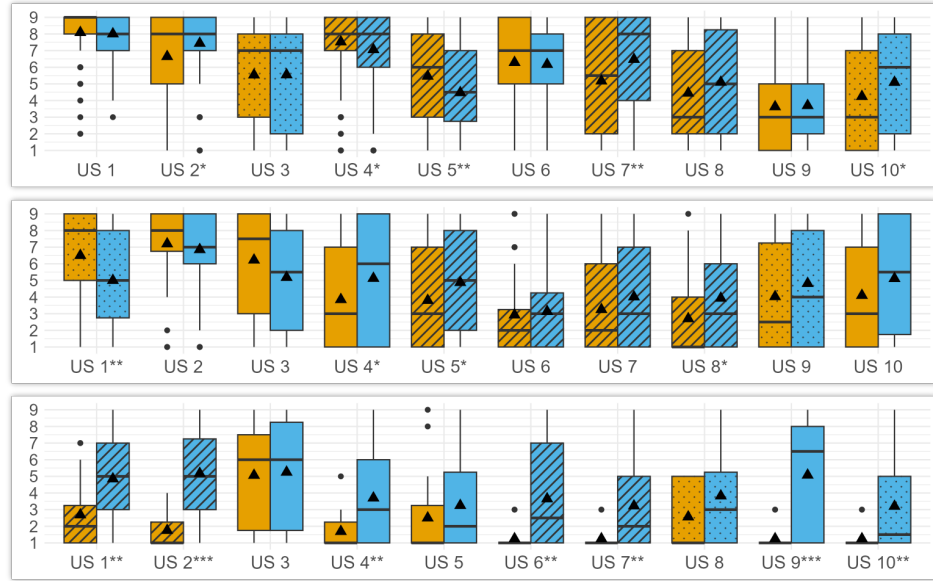


Figure 7.8: Boxplots showing US completion as rated by crowdworkers for the three experiment apps (*top*: first app, *middle*: second app, *bottom*: third app) using the *control* (orange) or *treatment* (blue) configurations. For each app, ten USs were to be created (US 1 to US 10). Dotted (new US) and dashed (extended US) bars represent USs with an update intervention after 30 minutes. Triangles represent means. One, two, and three stars represent significance (<0.05 , <0.01 , <0.001) for *two-sided Wilcoxon rank-sum tests*.

the next one once the USs had been completed. As a result, a different number of GUI prototypes were created for each of the apps in the *control* group and the *treatment* group. In the *control* group, all 20 participants created a first app, five created a second app, and two created a third one. In the *treatment* group, 19 participants created the first app, 11 created the second one, and five participants created the third one. On average, 1.35 apps were created in the *control* group and 1.75 apps in the *treatment* group. To analyze the degree of fulfillment of the USs, we solely investigated apps that participants had worked on. Apps that were not started were excluded from the evaluation in *Prolific* (i.e. we did not count USs for apps that were not started as unfulfilled). An app was classified as started when changes were made to the schematic app background in *Figma* and GUI components were added (simply moving a schematic app background in *Figma* would not count as starting the US).

For the first app, for six (three significant)⁹ USs, the *treatment* group produced GUI

⁹Participants in the *treatment* group started with just the schematic app background without any components, but could quickly use the recommendation feature to generate components. To give participants in the *control* group an idea of how to pick and adjust *Material Design* components, the introduction video demoed the use of an app bar demonstrating the creation of the first US. This component was also already drawn to the schematic app background in *Figma* when participants started. Since this component completes the first US, technically participants in the *control* group did not complete the first US by themselves.

7. CLOSING THE LOOP BETWEEN USER STORIES AND GUI PROTOTYPES

Table 7.2: Means of crowdworkers' ratings (Likert scale 1—9) of GUI prototypes for *control* and *treatment* (*Mann-Whitney-U-Tests*). Crowdworkers were asked if (1) *the GUI design is consistent with the overall USs*, (2) *whether the visual GUI design is appealing*, (3) *organization of information is clear*, (4) *the GUI allows for intuitive interaction*, (5) *the GUI prototype looks like a screen from a complete app*, (6) *the GUI prototype only has minimal errors*, and finally (7) *whether crowdworkers were satisfied with the GUI prototype*.

	Consistent Design (1)	Appealing Design (2)	Information Organisation (3)	Intuitive Interaction (4)	Screen from Complete App (5)	Minimal Errors (6)	Overall Satisfied (7)
Control	4.583	4.454	4.820	4.528	3.465	4.190	3.819
Treatment	5.046	4.586	5.025	4.761	3.614	4.225	4.254
<i>p</i> -value	0.0151*	0.5105	0.2600	0.1849	0.5540	0.7485	0.0423*

prototypes with a higher rating for US completion in contrast to four (two significant) USs in the *control* configuration that led to higher US completion. For the second app, seven (three significant) USs using the *treatment* and three (one significant) USs with the *control* configuration were rated higher. Finally, for the third and last app, all ten (seven significant) USs with the *treatment* configuration were rated as more complete in contrast to the *control* group.

For the two other aspects that were evaluated by crowdworkers for each US, the following picture emerged: For the selection of the right components to fulfill the respective US, the *control* group was rated higher on average for nine USs, and the GUIs created with the *treatment* configuration for 21 cases. Regarding the selection of correct descriptions (e.g., *displayed texts* or *messages*), the results of the *control* group were rated higher for eight USs and the results of the *treatment* group for 22 USs.

Not all USs were available to participants from the start. In our intervention, the USs were updated after 30 minutes of processing. Two USs were made available for the first time as new USs (*dotted* in Figure 7.8). For two USs, an update of the previous USs was displayed that expanded the scope of the previous US (*dashed* in Figure 7.8). In Figure 7.8, stories 4/5 and stories 7/8 represent pairs in the first app, in which the second US represents the update. For app two (app three), these are 5/6 (1/2) and 7/8 (6/7). Congruent with our expectations, the updated USs were, on average, rated as less complete in contrast to their initial versions, since the update of USs merely extended the first version of the US. In our discussion (Section 7.5.2), we provide insight into examples and discuss cases where the *control* results were assessed as more complete.

In addition to completing the USs, crowdworkers also independently evaluated whether (1) *the GUI design is consistent with all USs for this GUI prototype*, (2) *the visual GUI design is appealing*, (3) *organization of information is clear*, (4) *the GUI prototype allows for intuitive interaction*, (5) *the GUI prototype looks like a screen from a complete app*, (6) *the GUI prototype has only minimal errors*, and (7) *crowdworkers were satisfied with*

Table 7.3: Rating means and respective p -values (*Mann-Whitney-U-Tests, one-sided*) for seven questions answered by the participants in lab experiment using Likert scales. Questions for (1) *satisfaction with final designs*, (2) *ease of use and navigation of plugin*, (3) *confidence in creating professional GUI designs*, (4) *efficiency in design process*, (5) *future use consideration*, (6) *impact on design quality*, and (7) *difficulty in creating GUI prototypes*.

	Satisf. Designs (1)	Ease of Use (2)	Confidence Creating Prof. GUIs (3)	Efficiency in Design Proc. (4)	Future Use (5)	Impact Design Quality (6)	Difficulty Creating GUI Prototypes (7)
Likert scale	Strongly Disagree (1) – Strongly Agree (9)				Definitely not (1) – Definitely yes (5)	Not at all (1) – Extremely (5)	Very easy (1) – Very difficult (5)
Control	3.00	6.20	4.35	5.35	3.25	2.90	3.75
Treatment	3.75	6.30	5.65	6.85	3.80	3.50	3.45
<i>p</i> -values	0.2543	0.4400	0.0277*	0.0573	0.0392*	0.0462*	0.8408

the GUI prototype for each of the created GUIs. Results are shown in Table 7.2. For each aspect, means in the *treatment* configuration were considerably higher, but solely the two metrics (1) *Consistent Design* and (3) *Overall Satisfaction* are statistically significant.

Answer to RQ₂: Our approach mainly outperformed the *control* configuration on US completion and GUI prototype quality. Participants completed more prototypes with our approach (1.75 vs. 1.35 on average), especially for the third prototype where 7 (out of 10) USs are rated as significantly higher for completion. Moreover, for perceived GUI quality, our approach outperformed the *control* configuration across all mean metric values and is significant for (1) *Consistent Design* and (3) *Overall Satisfaction*.

Participants' Perceptions. We asked our participants qualitatively and quantitatively how they rated using the plugin in the *control* and *treatment* groups. Table 7.3 shows the results for seven Likert scale items. The *control* group achieved a lower mean value for each aspect, except for (7) *Difficulty Creating GUI Prototypes*, where a higher value is associated with greater perceived difficulties. Particularly noteworthy are the significantly better values of the *treatment* for (3) *Confidence Creating Professional GUI Prototypes*, (5) *Consideration for Future Use*, and (6) *Impact on Design Quality*.

We also measured the task load via the NASA-TLX (raw) questionnaire, as perceived by the user study participants. Figure 7.9 shows the values for the task load. On average, the *treatment* configuration was rated lower for *Mental Demand*, *Physical Demand*, *Temporal Demand*, *Necessary Effort*, and *Frustration Level*. The value for *Own Performance* proved to be significantly higher for the *treatment* configuration. The measured SUS (Brooke, 1996a) and CSI (Carroll et al., 2009) showed no significant difference. However,

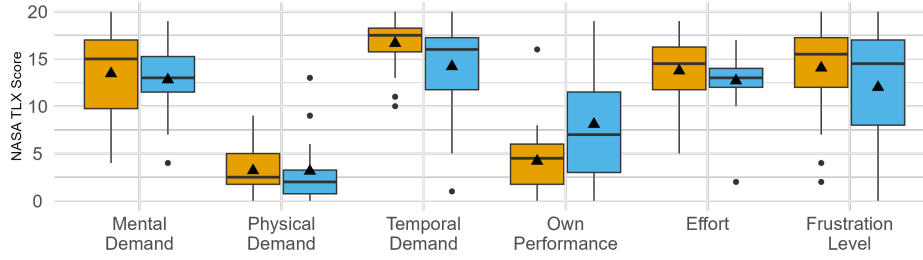


Figure 7.9: Results of NASA-TLX (raw) (Hart, 2006) for participants in *control* (orange) and *treatment* (blue). *Own Performance* significant (5% level). Triangles represent means.

for both scales the *treatment* configuration rated higher than the *control* configuration. For SUS, means were 52.13 for the *control* and 55.50 for the *treatment* configuration, reflecting a steep learning curve of participants using *Figma* and GUI component libraries in *Figma* (Material Design). Considering the CSI, means were 47.63 and 56.13, respectively.

Answer to RQ₃: Participants rated all perception items higher for our approach on average, except for (7) *Difficulty Creating GUI Prototypes*, for which lower values are considered to be better. Items (3) *Confidence Creating Professional GUI Prototypes*, (5) *Consideration for Future Use*, and the (6) *Impact on Design Quality* are significantly better for our approach. Moreover, our approach received better mean ratings for the NASA-TLX scores, with significant improvement for *Own Performance*. Finally, our approach received a SUS score of 55.50 (vs. 52.13) and an CSI score of 56.13 (vs. 47.63).

7.4.3 RQ₄: User Stories Generation

To investigate the effectiveness of our LLM-based approach to create USs from GUIs, we created 212 USs from 23 *Rico* (Deka et al., 2017) GUI prototypes directly in *Figma* with our prototyping assistant (therefore, an average of 9.22 USs per GUI prototype). Both were rated by our *Prolific* experts on Likert scales (1: *Strongly Disagree*, 3: *Disagree*, 5: *Neutral*, 7: *Agree*, 9: *Strongly Agree*). Participants rated the US while being shown the GUI prototype for (1) *the user story being found in the GUI prototype*, as well as (2) *clarity and precision*, (3) *specificity*, and (4) *professionalism*. Figure 7.10 shows violin plots for each assessed aspect for all USs. Means (medians) for each of the four questions were 6.70 (7), 6.63 (7), 6.53 (7), and 6.02 (6). However, while the total average and median values were high (medians of "Agree" for the first three questions in the Likert scales), there was considerable variance of the ratings for the USs derived from individual GUIs. Therefore, we included US ratings for four particularly selected GUIs in the subsequent Figure 7.10.

Answer to RQ₄: The USs generated by our LLM-based approach from GUI prototypes achieved positive ratings across the four different considered metrics (mean scores from 6.02—6.70), but with substantial variance across individual GUI prototypes.

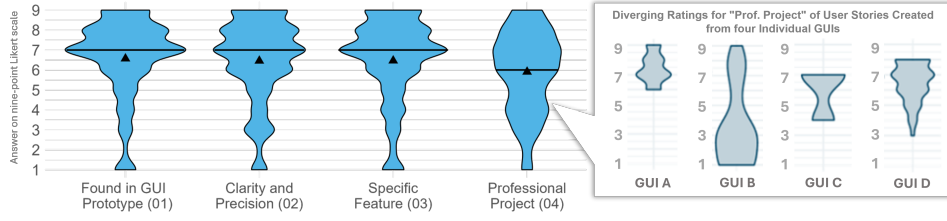


Figure 7.10: **Left:** Violin plots for crowdworkers' rating of generated USs from GUI prototypes for (1) *USs accurately describing functions of the GUI*, (2) *USs being precise*, (3) *for an identifiable feature*, and (4) *USs formulated as in professional projects*. Triangles represent means, lines medians. **Right:** Plots for crowdworkers' rating of generated USs from GUI prototypes for the fourth question, GUI prototypes (A—D) and US pairings.

7.5 Discussion

Using our assistant in the *treatment* configuration, participants created GUI components rated by crowdworkers as, on average, completing the USs to a higher degree. Nevertheless, some USs were rated as better completed in the *control* configuration. Subsequently, we discuss some of the results and put them into context. In addition, we discuss recommendations for the design of automated LLM-based support systems for GUI prototyping.

7.5.1 RQ₁: Effectiveness of Generating Prototype Components

In our first study, we evaluated the ability of our LLM-based approach to generate GUI prototype components, which received high ratings from crowdworkers on functional requirements fulfillment, US fulfillment, and component correctness (three times median of eight and one median of seven on a nine-point Likert scale). These results indicate that our approach successfully produced components that effectively fulfill functional requirements and align with USs. This supports the thesis that LLM-based methods can assist early-stage prototyping by generating functional and relevant elements with minimal manual intervention. However, slightly lower ratings for textual clarity suggest room for improvement in how text is generated for GUI components. We propose two potential avenues for refinement: (i) technical improvements, such as domain-specific FS prompting or finetuning, and (ii) procedural improvements, such as integrating human feedback loops into the creation process. The efficiency gains from the two-stage RAG approach hint at feasibility in large-scale design workflows where cost is a bottleneck.

7.5.2 RQ₂, RQ₃: LLM-Based Assistant Impact on GUI Quality

Our investigation into how our LLM-based assistant affects the quality of GUI prototypes, US completion, and perceived User Experience (UX) showed that experts rated GUI prototypes created with the LLM-based assistant as having a higher quality, especially in selecting the right components and utilizing correct descriptions. USs were also rated as completed to a higher degree for prototypes created using the LLM-based assistant (23

7. CLOSING THE LOOP BETWEEN USER STORIES AND GUI PROTOTYPES

User Story 3: As a user, I want to be able to save a particular meal for later, so that I don't have to search for it again and can just find it on my favourites page.

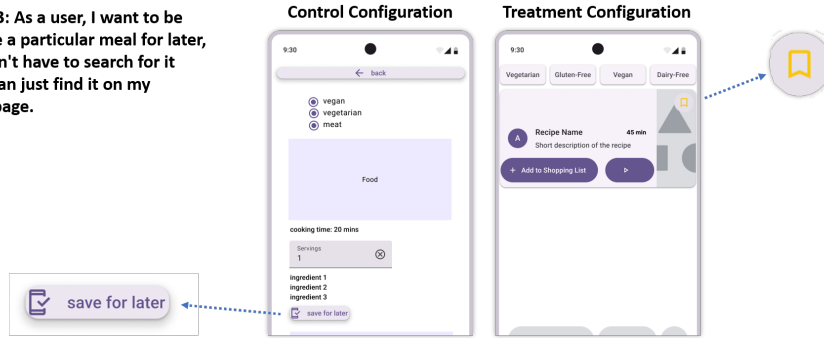


Figure 7.11: Example of an instance where a US (taken from the *second* app and *third* US of the user study) was rated more complete in the *control* group. The *control* group tended to use buttons with a clear call to action (e.g. “*save for later*” here), whereas in the *treatment* group our approach almost always created *bookmark* icon buttons, representing a concise but more implicit implementation of this particular US. Figure contains *Material 3 Design Kit* (Google LLC, 2025) components from *Google*, used under *CC BY 4.0 license*.

cases for the *treatment* versus seven cases for the *control* group). These results suggest that the assistant effectively supports task completion and improves the overall quality of GUI prototypes. As expected, participants who used the LLM-based assistant started or completed more app screens as part of the study tasks. The automatic generation of GUI components likely contributed to this efficiency benefit and enabled faster task progression. Participants also reported a higher perceived UX when using the LLM-based assistant, such as significantly higher ratings for *future use*, *perceived impact on design quality*, and *confidence in creating professional GUIs*. While NASA-TLX scores for mental and temporal demands, effort, and frustration were lower for the *treatment* group, these differences were not significant. However, both groups experienced high cognitive and temporal demands potentially due to the extent of the tasks (e.g., creating three GUIs, each with ten USs). This intentional aspect of the study design was finetuned in pre-tests to ensure that the *treatment* group supported by LLM-based GUI component generation would not run out of tasks.

When analyzing the cases in which crowdworkers rated GUI prototypes created in the *control* group as completing the US to a higher degree in comparison to the *treatment* group, it is noticeable that the presence of a text label often played a crucial role. Figure 7.11 illustrates such an example where a button with a text label (“*save for later*”) was created for the US by a participant in the *control* group (*left*). At the same time, an icon button with a *bookmark* icon was used in the *treatment* group (*right*). Almost every GUI prototype in the *treatment* group shows a *bookmark* icon for this US and our tests have shown that our LLM-based approach generates *bookmark* icon buttons almost consistently for this US.

This motivates two discussion points. Firstly, the extent to which crowdworkers were influenced by explicit textual mentions in the components, especially when comparing

them with aesthetically styled components such as the illustrated icon button. Secondly, how the temperature setting in our LLM-based approach led to the generation of similar recommendations when the solution space was somewhat limited and participants clicked the *"Generate Another Recommendation"* button. We recommend balancing the generation of more diverse additional recommendations for further studies. Here, multiple possible solutions come to mind: evaluating the extent to which the temperature can be changed in further iterations of generating the same component, enabling the approach to generate multiple implementations of the same US while instructing it to produce distinct variants or allowing users to provide further LLM instructions in addition to the US. Future users could restrict the next version of the same component generation by specifying instructions such as *"include a label in the next generation"*. While participants expressed great appreciation for the automated generation of components, e.g. P12: *"Straightforward to use and it gave good advice most of the time, excellent starting point for most stories"*, P19: *"Really good recommendations."*, P10: *"So much work was reduced"*, there were improvements mentioned in regard to the discussed variations of recommendations. Participants, such as P7: *"[I] would have liked to have had an option to adjust the task that is sent to the AI"*, frequently mentioned the desire for more variation in the recommendations. In fact, this suggestion was among the most mentioned improvements by participants (only second to the time it took the LLM to generate recommendations).

7.5.3 RQ₄: Effectiveness of Generating US from Components

In the third study, we investigated the effectiveness of our approach for automatically deriving USs from GUI prototypes. Overall, crowdworkers rated the USs created by the approach highly on relevance, clarity, precision, and specificity (all with a median of seven on a nine-point Likert scale), indicating that the generated stories generally align well with the functionality and purpose of the GUIs. However, ratings for professionalism (six-point median) were slightly lower, raising questions about, e.g., the tone and depth of the generated USs. While the median across all ratings and underlying prototypes is high, it is worth looking at the ratings of USs derived from individual GUIs. In the results in Figure 7.10, we have intentionally shown ratings of USs that emerged from four individual GUI prototypes. Here, the presented GUIs resulted in variations in the ratings, showing that the quality of the USs created varies depending on the underlying GUI. This variability may stem from differences in GUI complexity, ambiguity, or the LLMs' ability to generate USs for specific tasks, which were more likely to be included in pretraining. Despite these limitations, the findings underscore the potential of LLM-based tools to support early-stage design workflows. Integrating such tools into iterative design processes could reduce manual effort, enabling focus on higher-level challenges.

7.5.4 Generating GUI Prototypes from USs or Vice-Versa?

Our results show that our LLM-based approach enables the effective creation of GUI prototypes (components), mainly desired by UI/UX designers and partly by product owners, and effectively derives USs from GUI prototypes, which product owners mainly

desired in our interviews. Readers may wonder that it is certainly not possible to do both simultaneously and might sense a certain chicken-and-egg problem. Based on our interviews, we identified two options for integrating both features. We see our assistant as a mediator for both roles through the two described features. While product owners define USs, the GUI prototyping assistant could generate GUI components in parallel. Vice versa, the UI/UX designer could create design drafts from which the assistant derives USs for the product manager. In addition, GUI prototypes are often created in an iterative process, so it is conceivable that product owners formulate initial USs and the assistant creates a design proposal, which UI/UX designers subsequently refine. The created refinement is then used to generate more precise USs. Therefore, we regard the features of automatic GUI creation and US creation as one of the leading collaboration features.

7.6 Threats to Validity

In the following, we provide a collection of threats to internal validity, such as selection biases, and external validity, such as threats to generalizability. Furthermore, we briefly explain how they may influence the results of our paper and applied mitigation strategies.

Internal Validity. *Measuring Treatment Completeness.* We evaluated several aspects such as the detection, matching, and component generation mechanisms as well as the overall design of the proposed plugin within a user study. Therefore, the contribution of each of these aspects to the quality of the created GUI prototypes is not entirely clear. To address this issue, we initially conducted the evaluation of LLM-based generation of GUI components based on USs ex ante and showed its effectiveness, indicating that a large contribution to the improvement of the quality of the GUI prototypes and the improvements in terms of efficiency are mainly due to the automatic GUI generation.

External Validity. *Artificial Lab Setting.* The user study was conducted in an artificial lab setting including a less representative population of participants as a proxy for UI/UX designers regarding the group (i.e. undergraduate and graduate students) as well as the young age distribution (25.4 years on average). However, we primarily utilized this conducted user study to evaluate various usability aspects associated with the proposed approach. To address the cross-functional nature of GUI prototyping, we conducted interviews with stakeholders from different roles (UX, Product Owner, SD) to ensure that the assistant’s design rationales reflect the perspectives of diverse stakeholders. To strengthen the evaluation of the quality of generated USs and generated GUI components, we conducted an additional annotation of the generated artifacts with self-reported UI/UX professionals on *Prolific* (Prolific Academic Ltd.). The obtained results show the generation effectiveness. Nonetheless, we did not evaluate the assistant in a co-design situation involving multiple stakeholders at the same time, which represents an important direction for future work to explore its potential in collaborative prototyping contexts.

Artificial Functional User Stories. To evaluate our proposed approach, we heavily relied on the publicly available US dataset (see Chapter 9 for details), which is the only dataset available combining USs with GUI prototypes. These USs solely focus on functional aspects of a single GUI prototype. In more natural settings, USs often span several GUI prototypes as well as encompass non-functional aspects. However, in our work, we decided to initially focus on functional USs to reduce complexity, yet still provide valuable support and facilitate the creation of GUI prototypes. In addition, the employed US dataset is validated and filtered for quality (see Chapter 9 for details).

High-Level GUI Descriptions for Component Generation. To provide additional context for the generation of GUI components as part of its evaluation, we employed additional high-level GUI descriptions as an input to the ZS prompt for the LLM. Moreover, in the user study, the context of the current GUI prototype status could meaningfully be utilized by the LLM, for example, to adapt the generated GUI components to the GUI prototype and to predict accurate positioning of the GUI components. While these brief GUI descriptions can easily be obtained in practical GUI prototyping settings, in the conducted evaluation they were created by a research assistant and evaluated for accuracy and correctness by multiple paper authors.

Participant Interviews. There is a potential sample bias through the recruitment of UI/UX professionals, product owners, and software developers for the initial interviews, which may involve limited diversity in terms of company size, geographic location, and experience. However, we attempted to include multiple participants for each of the roles.

7.7 Limitations

Mobile GUIs and Limited Component Library. Since our approach focuses solely on mobile GUIs, other GUI types with varying characteristics (e.g., screen size, domains, etc.) are disregarded. In addition, we heavily relied on mobile GUIs taken from the *Rico* dataset (Deka et al., 2017), which has a restricted scope and usefulness. However, the *Rico* dataset is the largest publicly available GUI dataset and encompasses GUIs from over 27 different domains. In the future, we plan to extend our approach to more diverse GUI types. In addition, the considered GUI component library is restricted in size and configuration complexity. To enable meaningful support, we already included over 89 different *Material Design* GUI components and 100 icons. However, we plan to further expand both libraries to additionally enhance the support capabilities.

Static GUI Prototypes. In this work, we proposed a support approach for creating single static GUI prototypes, and the resulting format does not allow for interaction. Usually, the interactions considered in practice span multiple GUIs and maintaining visual design and interaction consistency across screens within an entire application is of high importance. In order to improve the usefulness and application scenarios of the proposed approach, we plan to further extend our LLM-based approaches and plugin to additionally enable support for USs spanning across several GUI prototype screens.

Functional User Stories. Since the current approach solely provides support for functional USs, the application scenarios are limited. However, particularly for creating initial GUI prototypes for rapidly obtaining stakeholder feedback regarding the functionality, our approach can be applied. In such an initial elicitation scenario, where the GUI prototypes act as a functional requirements communication artifact, the detailed design aspects are of less significance. For future work, we plan to extend the support to also include non-functional USs, for example, including styling aspects such as coloring and corporate design, among others. However, styling aspects of GUI components could also be integrated by replacing the component library within our approach. Therefore, the assistant could be extended to enable providing textual prompts for stylistic or design requirements. To increase flexibility, another extension could be that the assistant provides multiple variants of the generated component.

Handover, Versioning and Commenting. One additional limitation is that we could only thoroughly implement selected collaboration requirements for the version of the assistant in this paper. In the interviews, both UI/UX designers and product owners described the versioning of USs, commenting on USs combined with their components in the GUI prototype, and an improved handover (e.g., of parameters to software developers) compared to the existing functionalities in *Figma* as further requirements. Versioning was addressed in our intervention, but the implementation of our approach in its current form is limited to creating new components and cannot adapt existing components. We plan to address this in the future. Commenting functionality was not implemented, but as *Figma* already supports commenting components, it is feasible to link USs to components and leverage this feature. Finally, the extraction and handover of data to developers were not included, but tools like *Figma* already provide solutions for this.

7.8 Related Work

To support the GUI prototyping process, different approaches have been proposed in research. For example, *DesignScope* (O'Donovan et al., 2014; O'Donovan et al., 2015) facilitates the design process by actively recommending layout refinements and design suggestions. Moreover, *SketchPlorer* (Todi et al., 2016) represents a sketching approach that integrates an ad-hoc layout optimizer to rapidly provide layout improvement suggestions to users. In addition, *GUIComp* (Lee et al., 2020) provides assistance for novices during GUI prototyping through a multi-faceted support system including the retrieval of similar GUIs from the *Rico* dataset (Deka et al., 2017). *GUIComp* offers a visualization of several GUI complexity metrics such as GUI component alignment, balance, and density, and an attention map for the created GUI prototype. In contrast, our approach targets rapid generation of GUI prototypes (components) based on fine-grained USs that can instantly be integrated into the current working GUI prototype in proprietary representations, enabling visual editing within the well-known visual editor *Figma*. We close this loop and provide a tight integration of user requirements (in the form of USs) and resulting GUI prototypes. To ensure consistency between user requirements and GUI prototypes, an ontology-based approach was proposed before (Rocha Silva et al.,

2019). However, this approach necessitates the availability of a respective ontology for the domain and enables mere verification of the implementation of requirements and thus cannot actively provide support during the creation of the GUI prototype. In contrast, with our research, we proposed an LLM-based approach for rapidly generating the implementations for USs in an editable form as well as an integration of the LLM-based assistant in the form of a fully-fledged plugin in *Figma*.

Moreover, multiple approaches for generating GUI prototypes from text descriptions have been proposed in research before. For example, *Instigator* (Brie et al., 2023) trains a full *minGPT* (minGPT - GitHub Repository) model from scratch exploiting a large-scale repository of web pages to enable training of a task-specific GPT model. Furthermore, approaches such as *MAxPrototyper* (Yuan et al., 2024), *UIDiffuser* (Wei et al., 2023), and finetuning an LLM (Feng et al., 2023) have been proposed before (see Chapter 6 Section 6.7 for details). In contrast, our approach enables the generation of proprietary GUI representations through a novel two-stage RAG approach, which efficiently incorporates external GUI component libraries and closely integrates LLMs into prototyping tools.

7.9 Conclusion

The proposed approaches in this chapter were driven by the challenge **C1-5**: *How can LLMs be efficiently adapted to generate editable GUI prototype representations from NLR?* To tackle this challenge, in this chapter, we proposed a novel two-stage RAG approach which enables the efficient generation of proprietary GUI representations and integration into well-known prototyping tools to generate visually editable GUI prototypes. In addition, we integrated other LLM-based approaches such as automatic detection of US implementation, matching of requirements to their implementation in the GUI prototype, and generation of USs from prototypes. Our evaluation indicates that the LLM-based approach can successfully generate GUI component recommendations from USs. Moreover, we present a novel LLM-based assistant as a plugin for *Figma* with role-specific functionalities. In particular, we found that participants using our assistant achieved higher US completion. Our assistant also led to significantly higher satisfaction of experts with the created GUI prototype designs. Using the US detection, matching, and automated GUI (component) generation, our participants reported lower task load values, significantly higher confidence in creating professionally appearing GUI prototypes, high interest in future use of the assistant, and a higher impact on design quality. Inspired by real-world requirements from product owners collected in our interviews, we successfully expanded the approach to derive USs directly from GUI prototypes that can then be directly refined by product owners. In our evaluation, large parts of the USs were rated as specific, explicit, and derived from identifiable GUI components. Our results show that LLM-based US detection, matching, and the automated GUI (component) generation can be integrated directly into *Figma* providing a resource-efficient prototyping assistant to increase US completion and facilitate the integration of USs and GUI prototypes.

CHAPTER 8

GUIDE: LLM-DRIVEN GUI GENERATION DECOMPOSITION FOR AUTOMATED PROTOTYPING

In the previous chapter, we presented a novel two-stage RAG approach tackling challenge **Cr-5**, which enables the efficient integration of external GUI component libraries (such as *Material Design*) into LLMs to generate proprietary GUI prototype representations and thereby create visually editable prototypes (inside the *Figma* prototyping environment). While this approach requires a fine-grained requirements collection, such detailed individual requirements are often absent especially in the early stages of elicitation. Therefore, we build on the core RAG approach presented in the previous chapter, but extend it to enable the specification of high-level GUI prototype requirements. The work presented in this chapter has been previously published (Kolthoff et al., 2025c)¹. Our source code, prototype (*Figma* plugin), datasets, and demonstration video are all publicly available².

Personal Contribution. *Felix Kretzer* and I contributed equally to the ideation and creation of the concept for the work. While I implemented the LLM-based approaches and backend functionality, *Felix Kretzer* implemented the GUI and frontend of the approach. *Felix Kretzer* and I contributed equally to the evaluation design, while *Felix Kretzer* conducted the evaluation and analysis of results. *Felix Kretzer* wrote the section on the experimental evaluation setup and results (and created the evaluation results table), while I wrote the rest of the original manuscript and created respective figures and plots.

¹This section is adapted from: **Kolthoff, Kristian***, Kretzer, Felix*, Bartelt, Christian, Maedche, Alexander, and Ponzetto, Simone Paolo. GUIDE: LLM-Driven GUI Generation Decomposition for Automated Prototyping. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion) (ICSE, A*)*, Ottawa, ON, Canada, April 2025, pages 1–4. IEEE. *Authors contributed equally. Sections are rearranged and directly reused with only minor adaptations.

²Materials for this chapter are available at <https://github.com/kristiankolthoff/GUIDE-Prototyping> and the respective demonstration video at <https://youtu.be/C9RbhMxqpTU>

8.1 Motivation

With the approaches presented in the previous chapters, we introduced techniques for challenge **C1**, namely, enabling rapid mapping of NLR to GUI prototypes. The ZS prompting-based methods for efficiently adapting LLMs to more effectively generate GUI prototypes (see Chapter 6) provide a substantial performance. However, as discussed in the previous chapter, there is currently a gap between LLMs generating GUI code as text and established GUI prototyping workflows, namely visual editors such as *Figma* (Figma, Inc., 2024) widely employed by practitioners. Specifically, because LLMs are primarily designed for text generation, their outputs are not easily visualizable, posing a challenge for their deep integration into GUI prototyping. Nevertheless, these LLMs are pretrained on large amounts of HTML/CSS data, enabling them to generate prototype code that can be rendered in web browsers. However, modifications to GUI prototypes are generally performed by the prototype developers visually rather than through direct code manipulation. Moreover, when minor changes to generated GUI prototypes are requested by the user, LLMs typically inefficiently regenerate the entire GUI prototype. While we provided initial solution approaches in the previous chapter, these approaches assume the availability of a low-level, fine-grained requirements collection, which is often absent in early elicitation stages. More often, solely high-level, abstract NLR are available. Therefore, the work in this chapter is still driven by the leading research question of challenge **C1-5**, with the focus being on high-level, abstract NLR in comparison to the fine-grained requirements as shown in the previous chapter: *How can LLMs be efficiently adapted to generate editable GUI prototype representations from NLR?*

To close this gap, in this work, we introduce *GUIDE*, a novel LLM-driven GUI generation decomposition approach, which is tightly integrated into the popular prototyping tool *Figma*. Our approach derives low-level GUI features from high-level NLR provided by users and directly generates a corresponding editable GUI prototype using the popular *Material Design* component library (Google LLC, 2025) by adopting the RAG-based approach developed in the previous chapter. *GUIDE* facilitates the efficient modification of existing features or the addition of new ones within the GUI prototype, reducing the need to regenerate the entire prototype. Thus, this integration combines the LLM generation capabilities with the advantages of traditional visual GUI prototyping, enabling both more effective (GUI quality) and efficient (token consump.) GUI prototyping. Our approach can help prototype developers to rapidly and effectively create prototypes based on high-level NLR. To evaluate our approach, we conducted a user study and asked participants to create GUI prototypes *with* and *without* *GUIDE* and assessed the quality of created prototypes with crowdworkers from *Prolific* (Prolific Academic Ltd.; Douglas et al., 2023). Our evaluation shows that the prototypes generated with our approach obtain significantly higher scores across all considered evaluation (GUI quality) metrics.

Contributions. With this work, we make the following main research contribution:

- *High-level NLR to editable GUI prototypes*: we extend our previous RAG-based approach — which enables the efficient integration of external GUI component libraries

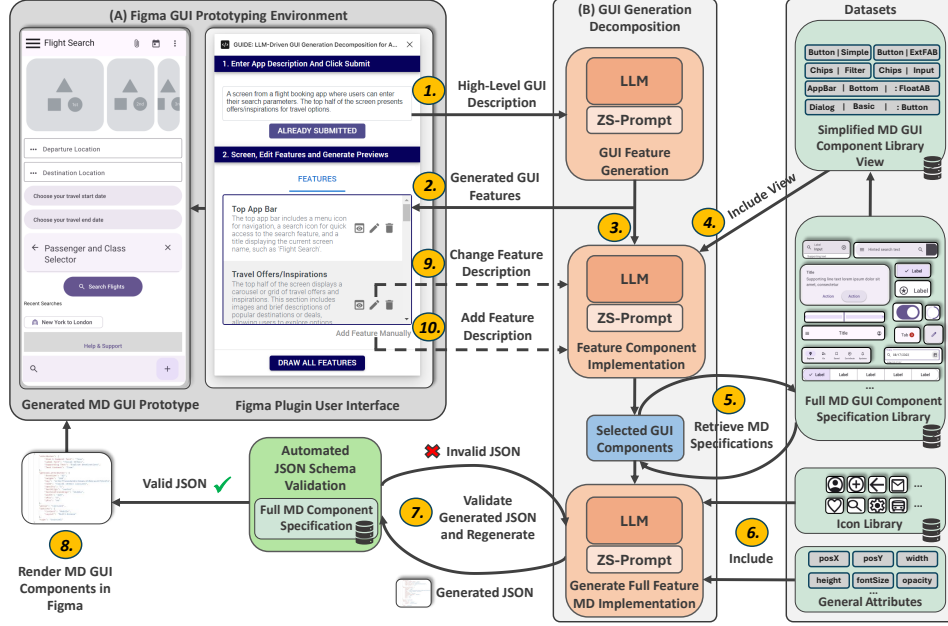


Figure 8.1: Overview of the *GUIDE* architecture with (A) *Figma* plugin and (B) GUI generation decomposition with a two-stage RAG approach and a GUI component library.

to generate proprietary GUI prototype representations — by an LLM-based approach that initially decomposes high-level NLR into low-level GUI features. In addition, we implement a tool prototype of our proposed approach as a novel *Figma* plugin.

- *Comprehensive user study*: we conduct an experimental evaluation of our approach by comparing the quality of created GUI prototypes *with* and *without* our approach through a user study, showing significant prototype quality improvements for *GUIDE*.

8.2 Approach: GUIDE

GUIDE is composed of two main components including (A) the *Figma* plugin enabling users to interact with our approach via an easy-to-use interface and (B) the GUI generation decomposition, which is an LLM-based approach for (i) decomposing the high-level NLR provided by the user into a fine-grained GUI feature collection, (ii) selecting relevant combinations of GUI component types for each GUI feature from a prespecified GUI component library, and (iii) generating fully specified GUI component implementations (RAG approach). An overview of the *GUIDE* architecture is illustrated in Figure 8.1.

8.2.1 Figma Plugin User Interface

Initially, prototype developers can provide their short and high-level NLR for the GUI prototypes they are tasked with creating directly into the user interface of *GUIDE*, which

is implemented as a plugin within *Figma* (Figma). Subsequently, *GUIDE* derives a fine-grained GUI feature collection displayed in the plugin with a name and a short textual description of the feature. Our approach helps users to rapidly screen the generated feature list, directly edit the feature description, and delete or add additional features manually, controlling GUI generation. *GUIDE* enables users to directly render the implementation of the suggested features as editable GUI components within *Figma*.

8.2.2 GUI Generation Decomposition

To improve the generated GUI prototypes and enhance control over the generation process, we decompose the GUI generation task into multiple smaller steps, similar to Prompt Decomposition (PD) (Khot et al., 2022). As research has shown before, this can not only improve the effectiveness of the LLM in completing complex tasks, but also follow a more human-like approach to creating GUI prototypes (Khot et al., 2022).

GUI Feature Generation. Based on the high-level NLR provided by the user, we employ an LLM in a ZS prompting approach (Radford et al., 2019) to generate the corresponding GUI feature collection. In particular, we instruct the LLM to focus solely on recommending functional features that can be represented visually in a GUI prototype, therefore neglecting NFR (such as *responsive design*, *accessibility*). In addition, we provide a JSON schema to the LLM including a name and a short description of the requirements and instruct the model to directly generate the respective JSON format. This representation can then easily be consumed by the frontend and subsequent steps.

Retrieval-Augmented GUI Feature Generation. After generating a feature collection, we decompose the subsequent generation of the respective feature implementations as GUI components within *Figma* into two separate steps, mainly adopted from the two-stage RAG approach from the previous chapter (see Chapter 7) and briefly summarized. Within *GUIDE*, we integrated the popular *Material Design (MD)* component library (Google LLC, 2025) to enable the generation of respective editable GUI prototypes. In particular, the library consists of over 59 different components, ranging from simple individual components (e.g., *Button*, *Checkbox*, and *Label*) to more complex component groups (e.g., *Search Bar*, *Dialog*, and *Top App Bar*). Although LLMs are trained on massive amounts of textual data, the specifically employed component library with the particular configuration options and syntax cannot directly be generated by the LLM. Since the full JSON specification of each GUI component encompasses many different properties and typically only a small number of components are relevant for implementing a feature, incorporating the entire specification into the GUI generation as part of a ZS prompt for an LLM would be inefficient.

Therefore, we utilize the RAG (Lewis et al., 2020) approach for the implementation of GUI features presented in the previous chapter. In particular, we first automatically derive a simplified component library view from the full specification library (only including the component group (e.g., *Button*) and component type (e.g., *FloatingActionButton*)

information), significantly reducing the number of required tokens (reduction of approximately 60% of consumed tokens, see Chapter 7 for details). Afterwards, we construct a ZS prompt that incorporates the simplified library in the context and instructs the LLM to select the relevant component types to implement the respective GUI feature. Subsequently, we employ the selected GUI components to retrieve the full JSON specifications of the relevant GUI components and utilize them in a second ZS prompt that finally generates the actual GUI feature implementation with *Material Design* components. In addition, we provide an icon collection and a set of general attributes (e.g., *posX*, *posY*, *width*, and *height*) that each component possesses in the generation process. The LLM is instructed to generate the full implementation for each GUI feature using a prespecified JSON format, which can be used to render the components in *Figma*. To improve the reliability of the JSON generation, we additionally conduct an automated validation of the generated JSON format by comparing it to the respective JSON schema for correctness.

8.2.3 Prototype Implementation

The *GUIDE* prototype is implemented as a *Figma* plugin using *TypeScript* for the frontend. To enable the integration of the LLM into generating prototypes in *Figma*, we created a proprietary JSON format that can be interpreted both by the LLM and the plugin for rendering. The backend of *GUIDE* is implemented as a *Python*-based *Quart* app running within a highly scalable *unicorn/nginx* server, providing a REST API for the described functionality. For the LLM, we employed the most recent *GPT-4o* model³ from *OpenAI* (128k token context, accessed in October 2024), representing the multimodel extension of their prior *GPT-4* model with state-of-the-art performance (OpenAI, 2023).

8.3 Experimental Evaluation

In the following, we describe the setup of our evaluation. Particularly, the evaluation consists of a small *between-subjects* lab-based study in which participants created GUI prototypes with (*treatment*) and without (*control*) our assistant based on NLR. These GUI prototypes were later evaluated by crowdworkers with UI/UX experience on *Prolific*. In particular, we pose the following research question in our experimental evaluation:

- **RQ₁:** *How effective are LLM-based approaches for generating editable GUI prototypes from high-level NLR?* To answer this question, we conducted an *between-subjects* user study in which participants created GUI prototypes *with* or *without* our *GUIDE* GUI prototyping assistance approach support. Subsequently, we evaluated the quality of created GUI prototypes with crowdworkers from *Prolific*.

8.3.1 RQ₁: Effectiveness of GUI Prototype Generation

User Study Procedure. Participants started the first sub-study with a questionnaire and watching an initial video introduction on the prototyping tool, including the com-

³At the time of implementing the proposed approach, we utilized the most advanced MLLM available from *OpenAI*, namely *GPT-4o* (Hurst et al., 2024), which is an extension over *GPT-4* (OpenAI, 2023).

Table 8.1: Median, mean, and results of *Wilcoxon rank-sum tests (two-sided)* for GUI ratings on Likert scales (*Strongly Disagree (1)* to *Strongly Agree (9)*) for created GUIs from the *control* and *treatment* prototyping groups. Crowdworkers on *Prolific* rated GUIs on: (1) *meets requirements from description*, (2) *utilizes correct components*, (3) *texts fit to GUI description*, (4) *GUI design consistent with GUI description*, and (5) *appealing GUI design*.

	Measure	Requi. (1)	Comp. (2)	Text (3)	Consist. (4)	Appeal. (5)
Control	Median	5	5	6	5	4
	Mean	4.746	4.769	5.015	5.000	4.354
Treatment	Median	8	7	7	7	7
	Mean	7.393	7.107	6.713	7.260	6.127
Wilcoxon	<i>p</i> -value	<0.001*	<0.001*	<0.001*	<0.001*	<0.001*
	<i>r</i> -value	0.545	0.470	0.371	0.480	0.376
	CI	[-3, -2]	[-3, -2]	[-2, -1]	[-3, -2]	[-2, -1]

ponent library *Material Design 3* (Google LLC, 2025), which was the component library used to create GUI prototypes by both groups. The *treatment* group watched an introduction to the use of our assistant, including the core functions: (i) allowing the assistant to create GUI features based on the NLR, (ii) editing existing features and adding custom features, (iii) previewing the generation, and (iv) drawing the generation in the prototyping tool. The participants then conducted a 45-minute GUI prototyping phase and were instructed to create four GUI prototypes for four NLR. After the prototyping phase, the *treatment* group answered multiple questions about using the assistant in the survey.

User Study Participants. We recruited 11 participants (3 female, 8 male) from a university pool to create GUI prototypes in a lab environment. On average, participants were 26.64 years old ($\sigma = 5.68$), studied for 5.05 years ($\sigma = 1.44$), and were randomly assigned to either the *control* group (six participants) or the *treatment* group (five participants).

GUI Prototype Evaluation Procedure. To evaluate the GUI prototypes created in our experiment across quality metrics (9-point Likert scale), we asked crowdworkers with UI/UX experience. In an annotation task, crowdworkers each evaluated ten randomly drawn GUI prototypes. Crowdworkers on *Prolific* rated GUIs on (1) *meets requirements from description*, (2) *utilizes correct components*, (3) *texts fit to GUI description*, (4) *GUI design consistent with GUI description*, (5) *appealing GUI design*, (6) *clear information organization*, (7) *intuitive interactions*, (8) *minimal errors*, and (9) *overall satisfaction*.

GUI Prototype Evaluation Participants. We invited 30 participants through *Prolific* with UI/UX design experience, a high approval rate (>99%) on *Prolific*, and certain

Table 8.2: Median, mean, and results of *Wilcoxon rank-sum tests (two-sided)* for GUI ratings on Likert scales (*Strongly Disagree (1)* to *Strongly Agree (9)*) for created GUIs from the *control* and *treatment* groups. Crowdworkers rated GUIs on: (6) *clear information organization*, (7) *intuitive interactions*, (8) *minimal errors*, and (9) *overall satisfaction*.

	Measure	Inf. Org. (6)	Intui. (7)	Errors (8)	Satisf. (9)
Control	Median	6	5	4	3
	Mean	5.154	4.762	4.392	4.015
Treatment	Median	7	7	7	6.5
	Mean	6.973	6.967	6.227	6.187
Wilcoxon	<i>p</i> -value	<0.001*	<0.001*	<0.001*	<0.001*
	<i>r</i> -value	0.397	0.464	0.368	0.431
	CI	[-2, -1]	[-3, -2]	[-3, -1]	[-3, -2]

language skills (*English* and *German*, since some GUI prototypes contained German texts). We excluded two participants for failing at least one attention check in our study, which we included to increase data annotation quality. Hence, we eventually considered data from 28 participants (12 female, 1 diverse, 15 male). Participants had an average age of 33.21 years ($\sigma = 8.23$), 7.36 ($\sigma = 7.71$) years of experience creating designs and 6.46 ($\sigma = 6.97$) years of experience in judging design (spent 16 minutes (mean) on our survey).

8.4 Results & Discussion

8.4.1 RQ₁: Effectiveness of GUI Prototype Generation

Subsequently, we briefly present the results of both sub-studies. Although both groups had 45 minutes to create the GUI prototypes and received the identical short NLR, participants started to work on significantly more GUI requirements while using *GUIDE*, with 3.2 GUI prototypes per participant commenced in the *treatment* group (16 GUIs with six participants in total) compared to 2.33 per participant (14 GUIs with five participants in total) in the *control* group. Table 8.1 and Table 8.2 show the medians, mean values, and the results of *Wilcoxon rank-sum tests* for the crowdworker ratings, while Figure 8.2 illustrates boxplots across all considered evaluation metrics. For each of the asked questions, the mean and median ratings were higher for the *treatment* group. Moreover, the *Wilcoxon rank-sum tests (two-sided)* were significant for each of the evaluated questions.

For example, considering whether the created GUI prototypes meet the specified requirements (1), we observe a substantial improvement for *GUIDE* with a median (mean) of 8 (7.393) compared to the *control* with 5 (4.746). In addition, considering whether the correct GUI components are utilized to implement the requested functionality (2), we observe again a substantial improvement in the *treatment* group, with a median (mean)

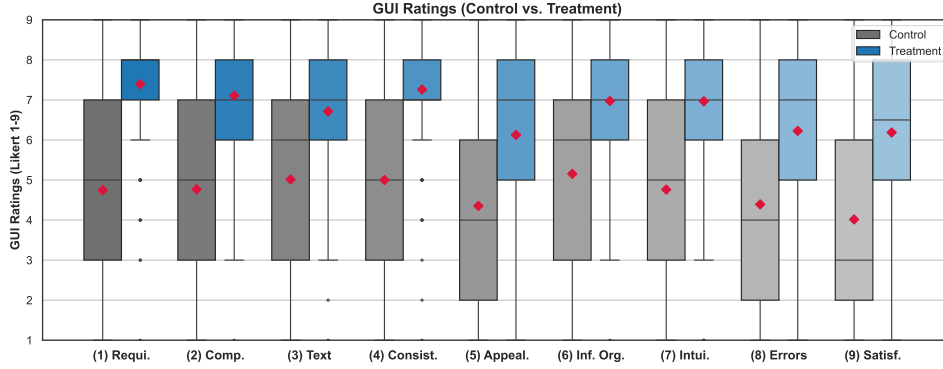


Figure 8.2: Multiple boxplots for *control* (gray) and *treatment* (blue) across nine GUI metrics (Likert-scale items (1–9)): (1) *meets requirements from description*, (2) *utilizes correct components*, (3) *texts fit to GUI description*, (4) *GUI design consistent with GUI description*, and (5) *appealing GUI design*, (6) *clear information organization*, (7) *intuitive interactions*, (8) *minimal errors*, and finally (9) *overall satisfaction*, diamonds repr. means.

of 7 (7.107) in contrast to 5 (4.769) in the *control*. These two evaluation metrics focusing on the functional aspects of the prototypes are particularly important for early elicitation phases. This indicates a clear advantage and suitability of our approach in REI scenarios. Furthermore, GUI prototypes created by participants using *GUIDE* encompassed substantially fewer errors (8) and design-related metrics also improved significantly (such as (4), (5)). Overall, the results indicate that *GUIDE* effectively generates editable prototypes.

Answer to RQ₁: Our *GUIDE* approach significantly outperforms the *control* group without the GUI prototyping assistance across all nine considered GUI quality metrics.

8.5 Threats to Validity

Internal Validity. One potential threat to internal validity is the subjectivity of GUI prototype quality annotations, which might be perceived differently by various human annotators. To mitigate this subjectivity and bias that could potentially be introduced, we invited 28 participants for annotation from *Prolific* with multiple exclusion criteria to increase quality. On average, each GUI was annotated 9.3 times by distinct participants.

External Validity. One potential threat to external validity is the restriction of the experiments to *Figma*, *Material Design*, and the consideration of only four different GUI prototypes, which might limit the generalizability. However, *Figma* represents a widely employed and state-of-the-art prototyping tool, and other prototyping environments and component libraries typically possess similar editing functionality and complexity. To increase diversity in our dataset, we included GUI prototypes from different domains.

8.6 Limitations

The current implementation is limited to the popular *Figma* prototyping platform and optimized for the *Material Design* component library. To increase adoption of the approach, more prototyping tools and component libraries could be integrated. Moreover, our approach currently provides support with a focus on individual GUI prototypes. In the future, this could be extended to enable support for entire applications, including interactions and complex requirements dependencies between several GUIs.

8.7 Related Work

To support the GUI prototyping process, several approaches have been proposed before. For example, *DesignScope* (O'Donovan et al., 2014; O'Donovan et al., 2015), *SketchPlorer* (Todi et al., 2016), and *GUIComp* (Lee et al., 2020) have been proposed. However, we discussed these approaches already in the previous chapter (see Chapter 7 Section 7.8 for details). In addition, multiple approaches have been proposed before to automatically generate GUIs from text descriptions. Similarly, we also already discussed these GUI generation approaches in an earlier chapter of the thesis (see Section 6.7 for more details).

8.8 Conclusion

The approach proposed in this chapter was driven by the challenge **C1-5**: *How can LLMs be efficiently adapted to generate editable GUI prototype representations from NLR?* Specifically, we investigated the generation of editable GUI prototypes from high-level, abstract NLR, whereas we already presented a solution approach for low-level, fine-grained requirements in the previous chapter. To tackle this specific challenge, we proposed *GUIDE*, an approach combining the ZS GUI generation capabilities of recent LLMs with the advantages of traditional GUI prototyping tools such as *Figma* by extending the RAG approach in the previous chapter by deriving low-level features via an LLM initially. This integration facilitates the creation of high-quality GUI prototypes and provides effective support for prototype developers. Our evaluation shows the effectiveness of our approach, which enables users to create significantly better prototypes across all metrics.

Part III

VERIFICATION: (M)LLM-BASED APPROACHES

CHAPTER 9

INTERLINKING GUIs AND USER STORIES: A SEMI-AUTOMATIC LLM-BASED APPROACH

In the first two parts of the thesis, we provided solution approaches for challenge **C1**, namely, to rapidly map NLR to GUI prototypes. In particular, we investigated retrieval-based methods and LLM-based generation approaches. In the third part of the thesis, we focus on complementary challenge **C2**, namely, automated verification of NLR in GUI applications. Specifically, in this chapter, we conduct a feasibility study of automated LLM-based verification approaches by focusing on simplified, static GUIs without dynamic behavior. The work presented in this chapter has been previously published (Kolthoff et al., 2024c)¹. The source code, datasets, and results are all publicly available².

Personal Contribution. *Felix Kretzer* and I contributed equally to the ideation and creation of the concept for the work. I implemented the LLM-based automation approaches. *Felix Kretzer* and I contributed equally to the evaluation design, while *Felix Kretzer* conducted the lab-based data collection and I conducted the evaluation and analysis of results. *Felix Kretzer* wrote the section on data collection, threats to validity, and half of the introduction. I wrote the remainder and created figures, tables, and plots.

9.1 Motivation

While previous chapters focused on rapidly mapping NLR to GUI prototypes, which represents an important activity for REI and RVal, the verification of implemented GUI applications also plays a significant role. Quickly and effectively communicating require-

¹This section is adapted from: **Kolthoff, Kristian***, Kretzer, Felix*, Bartelt, Christian, Maedche, Alexander, and Ponzetto, Simone Paolo. Interlinking User Stories and GUI Prototyping: A Semi-Automatic LLM-based Approach. In *Proceedings of the 32nd IEEE International Requirements Engineering Conference (RE, A)*, Reykjavik, Iceland, June 2024, pages 380–388. IEEE. *Authors contributed equally. Sections are rearranged and directly reused with adaptations.

²Materials are available at <https://github.com/kristiankolthoff/IUS-GUI-Prototyping>

ments and mitigating NL ambiguity are the main goals of employing GUI prototypes. When implementing the actual software system, verification ensures the conformance of the system with its specification. One of the most commonly used verification techniques is testing, i.e. executing the software system with specific inputs and under certain conditions, then comparing its behavior and outputs against expected results according to the requirements specification. However, testing represents a time-consuming and effort-demanding activity (Myers et al., 2004). Moreover, GUI testing represents a particularly resource-intensive procedure, due to its special characteristics, such as graphical input and output, and the dynamic visual behavior of GUI applications (Memon, 2001, 2007).

To mitigate the time-consuming characteristic of testing, many approaches have been proposed in research before. Specifically, automated GUI testing approaches have evolved from automated test scripts (Xie and Memon, 2007), random exploration (Mao et al., 2016), and model-based techniques (Zeng et al., 2016; Gu et al., 2019) to more scripted bug replay methods (Gomez et al., 2013; Feng and Chen, 2022). Although these methods enhance the testing efficiency, they suffer from limited test coverage and brittleness, arising from the dynamic and semantic complexity of GUI applications. Moreover, these testing techniques primarily focus on identifying bugs (i.e., crashing the application), rather than verifying the functional correctness against NLR. One major reason for the absence of automation methods for this particular scenario is represented by the high complexity and required NLU capabilities of processing NLR in combination with complex and dynamic GUI applications. While older NLP approaches did not have sufficient capabilities, the advent of LLMs provides the potential to address this gap. Therefore, the work presented in this chapter is driven by the leading research question of challenge **C2-1**: *How can NLR-based verification be automated for simplified, static GUI representations as a foundation for more complex GUI verification tasks?*

To tackle this challenge, in this chapter, we conduct an initial feasibility study of employing LLMs for automated verification of GUIs. In particular, we evaluate the abilities of different LLM-based ZS, FS, and CoT prompting approaches to (i) identify whether NLR (in the form of USs) are implemented in a GUI and (ii) which GUI components implement the requirements. While verification is usually conducted for implemented, complex, and highly dynamic GUI applications at later development stages, we initially focus on a simplified scenario, in which GUIs are static (no dynamic behavior) to evaluate the general feasibility of employing LLMs. To this end, we utilize the *Rico* GUI dataset (Deka et al., 2017), which encompasses individual GUIs screens from fully implemented applications, represented by comprehensive structural GUI component hierarchy data. Furthermore, the methods developed in this chapter have already been employed in our *Figma* plugin (see Chapter 7) in the context of prototyping (identifying implementation, matching GUI components). As we have seen, employing these methods to enable conformance checking already at design time provides benefits, especially due to the large number of rapidly changing requirements in practice (Debnath et al., 2021a).

Contributions. With this work, we make the following three research contributions:

- *Novel LLM-based approaches for static GUI verification from NLR*: we propose ZS, FS,

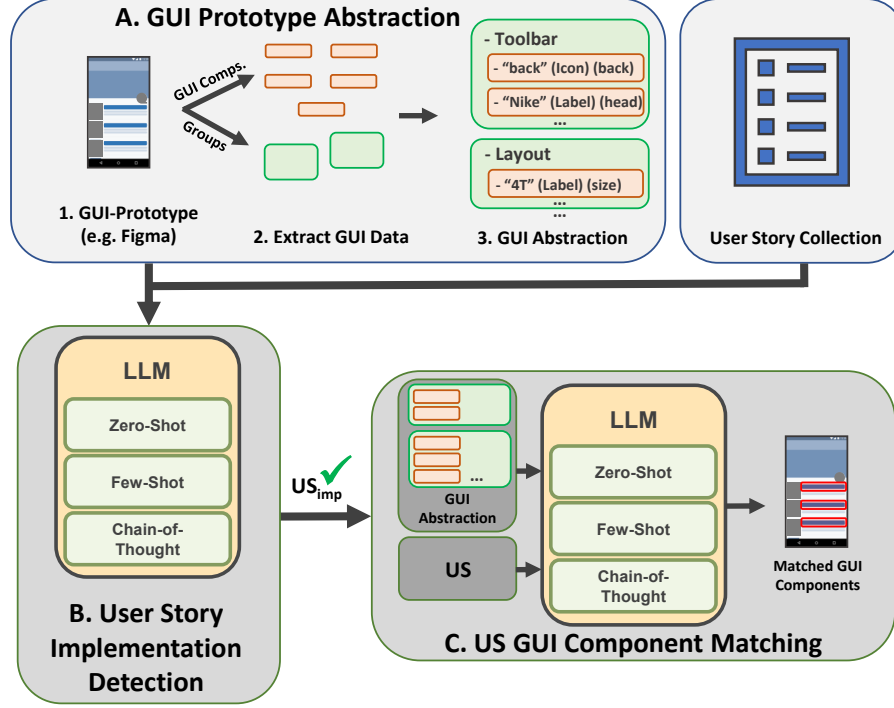


Figure 9.1: Overview of our approach consisting of (A) creating a textual representation for *Rico* GUIs to feed into an LLM, (B) detecting the implementation of a US in the GUI via LLM-based prompting and (C) matching corresponding GUI components to the US.

and CoT prompting approaches for enabling verification of static GUIs from NLR. Specifically, we conduct this work as a first feasibility study and employ abstracted, simplified GUI representation from the *Rico* dataset with detailed GUI hierarchy data.

- *Interlinked User Story (US) and GUI components dataset:* we create a novel dataset through a lab-based study with human annotators, which represents the first dataset that combines USs and GUIs from *Rico* (matched to individual GUI components).
- *Comprehensive evaluation and insights:* with the created GUI-USs dataset, we conduct a comprehensive experimental evaluation of the proposed LLM-based approaches and conduct an error analysis to provide further insights into the LLM-based predictions.

9.2 Approach

Our approach is divided into several main components, as shown in Figure 9.1. First, (A) a GUI prototype abstraction component to transform the structural *Rico* GUI hierarchy data to a textual representation, which can be fed into an LLM, (B) an NLR (here US) verification component that utilizes the GUI abstraction and an US collection in an LLM-based approach to classify whether USs are already implemented in the given GUI, (C)

an additional component matching approach that identifies the GUI components that implement the US. Next, we provide detailed descriptions for each of the components.

9.2.1 GUI Prototype Abstraction

As an input to the previously mentioned LLM-based methods, the *Rico* (Deka et al., 2017) GUI needs to be transformed to an abstract textual representation to be fed into an LLM³. In particular, we adopt the method already presented earlier (see Chapter 5 Section 5.2.4). For each extracted GUI component, we then create a textual representation using the following abstract pattern, followed by three examples created from respective *Rico* GUIs:

```
"uicomp-text" (uicomp-type) (uicomp-name)
"+7.10" (Label) (price Change TV)
"Install App" (Button) (native Ad Call To)
"Example: 'New York'" (Text Input) (location)
```

Specifically, the *uicomp-text* refers to the displayed text of the component, the **uicomp-type** refers to the basic GUI component type (e.g., *Label*, *Button*, *Checkbox*), and the *uicomp-name* refers to the internal name of the component (e.g., provided by developers). In the absence of any of the properties, the respective field is left empty. This textual representation of the GUI components encompasses relevant information from a functional perspective. Moreover, we derive clustering elements from the semantic annotations of *Rico*, which incorporate categories such as *List Item*, *Card*, and *Toolbar*, among others. To identify clusters for the remaining components not encompassed by the preceding groups, we further extracted layout clusters from the original GUI hierarchy by aligning them with the GUI components. Subsequently, we create the GUI representation as a two-tier bullet-point list, with the outer tier representing the layout groups and the inner tier denoting their corresponding GUI components. Prior to generating the string representation, the layout groups are arranged based on their boundaries from the top-left to the bottom-right and in a similar fashion, the GUI components within each group are organized to resemble the original *Rico* GUI component structure closely.

9.2.2 User Story Implementation Detection

To tackle the problem of identifying whether a US is implemented in a GUI prototype, we propose several LLM-based methods and approach it as a binary classification problem. Given the extensive knowledge encapsulated within LLMs and their ICL capabilities, the application for the detection and matching problems (verification) at hand is promising.

Zero-Shot (ZS) Prompting. In particular, we adopt the ZS prompting method (Kojima et al., 2022) by creating a prompting template divided into (i) a task instruction providing clear guidelines for the model, (ii) the US to validate, followed by (iii) the

³Note that at the time of implementing the approach, more powerful MLLMs, which have highly effective image understanding capabilities, were not readily available yet (such as *GPT-4o* (OpenAI, 2023e)).

generated *Rico* GUI text representation. We instruct the model to predict a single token for the classification, and we extract the log probabilities for both labels, which provides a US ranking mechanism. In particular, the extracted probability can be employed to estimate the certainty of the classification. This probability can be further exploited to rank the user stories from high to low probabilities (e.g., for later visualization to users).

Few-Shot (FS) Prompting. In addition, we adopt the FS prompting method (Brown et al., 2020) often showing enhanced performance for various tasks by providing examples. In FS prompting, we basically follow the ZS pattern, but we additionally provide several *input-output* pairs to guide the model for the specific task. For our experimental evaluation, we created multiple FS prompting templates (each encompassing varying examples).

Chain-of-Thought (CoT) Prompting. Moreover, we adopt the CoT prompting method (Wei et al., 2022), in which the LLM is instructed to create multiple intermediate reasoning steps before generating a prediction. Thus, we instruct the model to first generate an explanation providing reasoning whether the US is implemented. In addition, this provides an interpretable explanation that can further be used for later error analysis.

9.2.3 User Story GUI Component Matching

In addition to solely predicting the coverage of a US in a *Rico* GUI, we further investigate the task of extracting all relevant components from the GUI that are required to fulfill the US. This represents a natural extension of the previous task, enabling direct interlinking of the US with its respective GUI components and gaining deeper insights into the LLM predictions. Similarly to the previous task, we adopt ZS, FS, and CoT prompting models for this matching task. However, we extend the GUI abstraction by adding identifiers to each GUI component and correspondingly adapt the instructions and prompting templates to enable the LLM to output a parsable collection of the component identifiers. We evaluate ZS_A (plain task desc.) and ZS_B (focus on extracting all relevant components).

9.3 Experimental Evaluation

This section delineates the design of our evaluation of the proposed approach. In particular, we focus the evaluation on two main aspects of the approach including the US implementation detection and GUI component matching methods. For enabling this evaluation, we constructed a gold standard of USs with corresponding GUI component annotations. To this end, we formulate the subsequent two research questions:

- **RQ₁:** *How effective are LLM-based approaches for detecting US implementation in static GUIs?* To answer this question, we employ the created gold standard and create a test dataset for the binary classification problem. To construct negative instances, we remove the GUI components for the respective US and for positive instances, we simply keep them. To assess the effectiveness of the approaches, we compute standard classification metrics including *precision*, *recall*, and *F1-measure*.

- **RQ₂:** *How effective are LLM-based approaches for extracting GUI components fulfilling a US from static GUIs?* To answer this question, we use the gold standard and compare the extracted collection of GUI components by the LLM approaches with the reference collection. Again, we compute *precision*, *recall*, and *F1-measure*.

9.3.1 Data Collection

To evaluate our approach, a dataset of GUIs and associated USs was required. While there are established datasets of GUIs available (e.g., *Rico* (Deka et al., 2017)), there exists no dataset combining GUIs with USs. Therefore, we decided to collect USs for existing GUIs from the *Rico* dataset. Subsequently, we introduce the collection and preprocessing of the dataset by presenting existing GUIs to annotators writing USs and matching them.

GUI Sample. In order to get a broad selection of different *Rico* GUIs, our initial GUI sample was randomly drawn from ten different domains. Following our exclusion criteria, we then selected valid GUIs from our random sample. We decided ex-ante to exclude interfaces with *non-English text*, *personal data displayed*, *overlays* (such as *pop-ups*) shown, *components without annotations in the Rico dataset*, *trivial GUIs* (e.g., simple log-in screens resulting in the same repetitive USs), and to exclude interfaces with *unclear functionality*. Our final sample included 59 GUIs from the domains: *Shopping* (8), *Health & Fitness* (11), *Education* (5), *News* (4), *Sports* (5), *Travel* (6), *Books* (5), *Music* (6), *Finance* (4), and *Food & Drink* (5). We additionally created GUI versions where each component was annotated with a number so that participants could associate their USs with ≥ 1 GUI components.

Procedure and Survey. USs were collected using a questionnaire. The participants were presented with information about the study, data protection, and conditions of participation. They then learned how to write functional USs. Learning content was supported with examples of USs and concluded with comprehension checks. Participants had two chances for each comprehension check before a screenout took place. Participants then created USs for nine consecutively shown GUIs. We instructed participants to create three to five functional USs per GUI describing features already implemented and provided a USs template for guidance. After creating USs for nine GUIs, the final task was to specify for each US which components of the GUI belong to the respective US. The participants were shown the identical nine GUIs one after the other with the already created USs, but the GUIs now contained numbers that identified each component.

Participants. We selected 8 participants (2 female, 6 male) from a student pool. Participants were on average 24.3 years old, had 1.1 years of experience creating and 1.0 years of experience in evaluating visual design. Each participant generated 4.5 USs/GUI (mean).

Data Processing. Overall, the participants created 327 USs, with duplicate USs and varying quality (e.g., despite explicit instructions, some USs were written for *not implemented* features). In order to obtain a usable dataset, the USs were cleaned up. Therefore,

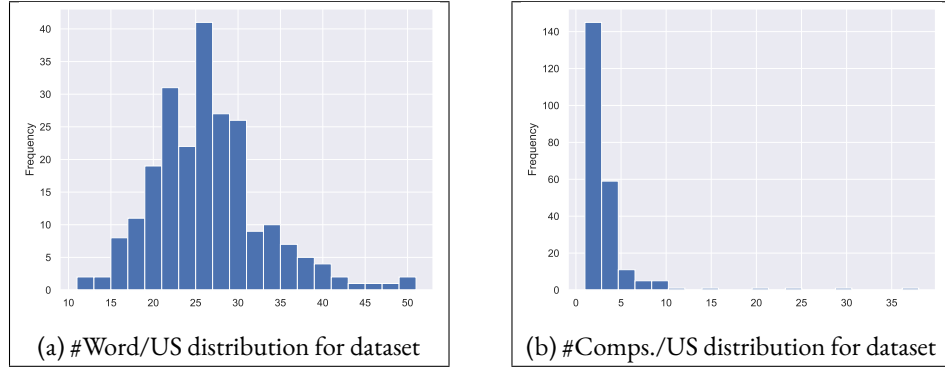


Figure 9.2: Distributions for the created dataset combining USs with GUIs. In particular, each US is represented by a one-to-many mapping (≥ 1) to respective GUI components.

Variable	n	μ	σ	min	Q1	med.	Q3	max
(1) #Words/User Story	231	26.08	6.56	11.00	22.00	25.00	29.00	51.00
(3) #Components/User Story	231	2.93	4.01	1.00	1.00	2.00	3.00	38.00
(2) #User Stories/GUI	59	3.92	1.38	1.00	3.00	4.00	5.00	7.00

Table 9.1: Summary statistics (*mean μ , standard deviation σ , minimum, first quartile Q_1 , median, third quartile Q_3 , maximum*) for the US dataset: (1) number of words per US, (2) number of corresponding GUI components per US, and (3) number of USs per GUI.

two paper authors labeled each US independently. For this purpose, the first step was to determine whether the US *a*) fully meets the requirements, *b*) contains one or more errors, or *c*) is a duplicate of a previous US. After the separate labeling, the IAA was calculated (Cohen’s Kappa $\kappa = .548$). After resolving disputes, 231 USs (with their respective GUIs) were included in the final data set, whereas 96 USs (16 duplicates, 80 different exclusion criteria) were not considered further. The applied exclusion criteria are provided in our accompanying materials. Figure 9.2(a) shows the distribution of the number of words per US, indicating complex requirements due to many words included per US (26.08 on average). Moreover, Figure 9.2(b) shows the distribution of the number of components matched to respective USs. In addition, Table 9.1 provides summary statistics across these two aspects and the number of USs per GUI in the cleaned dataset. Furthermore, Figure 9.3 illustrates multiple example annotations taken from the gold standard, with *Rico* GUIs, the USs written by participants, and corresponding GUI component annotations.

9.3.2 RQ₁: User Story Implementation Detection

To answer RQ₁, we evaluated the ability of various LLM-based prompting techniques to predict whether a US is contained in a GUI. To this end, we created a gold standard based on the collected USs and GUI annotation pairs. First, we randomly selected five GUIs comprising 21 USs to be employed as examples for the FS prompting approach. The remaining 210 US-GUI-pairs form the basis for the gold standard. This procedure ensures the avoidance of overlapping GUI abstraction data between gold standard and FS

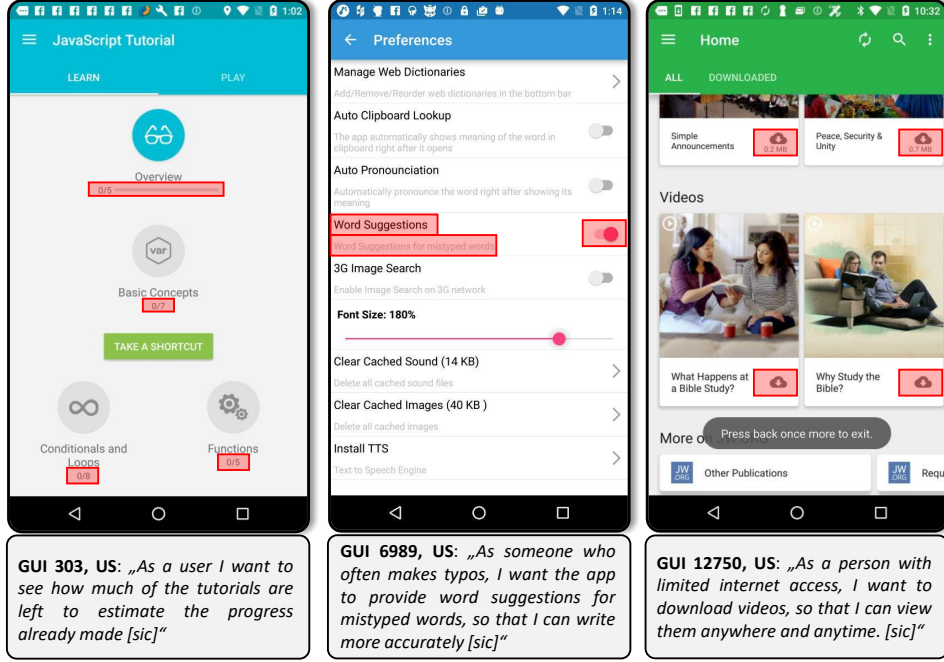


Figure 9.3: Three *Rico* (Deka et al., 2017) GUI examples from our gold standard with their respective USs and visually marked GUI components associated with the requirements.

examples, thereby avoiding bias. Next, we randomly assigned half of the examples (105) to the class *Implemented* (*i*) and the remainder the class *Not-Implemented* (*o*). While the GUI data for the first class remains unchanged, in the GUI abstraction of the second class (i.e. *Not-Implemented* (*o*)), the paired GUI component annotations were removed. For example, consider the second GUI of Figure 9.3. For the shown US, the respective GUI components associated with the US according to the gold standard are marked in the GUI. To include such an US as a negative example in the gold standard, we would remove the respective GUI components from the GUI abstraction (two *labels* and a *checkbox*). Moreover, we compute *precision* (*P*), *recall* (*R*), *F1-measure* (*F1*), and *accuracy* (*A*) as

$$P = \frac{TP}{TP + FP} \quad (9.1) \quad R = \frac{TP}{TP + FN} \quad (9.2)$$

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R} \quad (9.3) \quad A = \frac{TP + TN}{N} \quad (9.4)$$

based on True Positive (TP), False Positive (FP), True Negative (TN), and False Negative (FN) instances, denoting $N = TP + TN + FP + FN$. To conduct the experiments, we employed the most recent *GPT-4* model⁴ (OpenAI, 2023) (8,192 token context length,

⁴Note that at the time of implementing the approach, more powerful MLLMs, which have highly effective image understanding capabilities, were not readily available yet (such as *GPT-4o* (OpenAI, 2023e)).

Table 9.2: Evaluation results of LLM-based approaches (ZS, FS (#*examples*), and CoT (*temperature*)) for detecting US implementation in *Rico* GUIs (binary classification) showing *Precision* (P), *Recall* (R), and *F1-measure* ($F1$) for both classes and *Accuracy* (A).

	US Implemented (i)			Not Implemented (o)			A
	P ₁	R ₁	F1 ₁	P ₀	R ₀	F1 ₀	
Zero-Shot	83.0	88.6	85.7	87.8	81.9	84.7	85.2
Few-Shot₅	82.9	87.6	85.2	86.9	81.9	84.3	84.8
Few-Shot₁₀	81.8	85.7	83.7	85.0	81.0	82.9	83.3
CoT_{t=0.0}	90.0	68.6	77.8	74.6	92.4	82.6	80.5
CoT_{t=0.5}	88.8	67.6	76.8	73.8	91.4	81.7	79.5
CoT_{t=1.0}	88.8	75.2	81.4	78.5	90.5	84.1	82.9
CoT_{t=1.3}	81.2	74.3	77.6	76.3	82.9	79.5	78.6

temperature=0, accessed in February 2024) as our base LLM. For the FS prompting, we evaluated one model with five (FS₅) and another with ten examples (FS₁₀), respectively. We evaluated four CoT models with varying *temperature*, based on the idea that with varying *temperature*, we restrict the model to provide more or less diverse explanations.

9.3.3 RQ₂: User Story GUI Component Matching

To answer RQ₂, we evaluated the ability of several LLM-based prompting techniques to extract all GUI components relevant for fulfilling a given US. Therefore, we employed the same gold standard as previously described. However, we used the original unchanged abstraction for each of the 210 GUIs with the labels being the annotated GUI component identifiers from the gold standard. Precisely, as the input, the model received the GUI abstraction (each GUI component marked by a numerical identifier) and a US to predict a set of GUI component IDs relevant for the US. Next, we compared the set of extracted GUI component identifiers with the gold standard set of identifiers and computed P , R , and $F1$ measure. We computed these metrics for each example in the dataset and averaged them over the gold standard to obtain *macro* values. Therefore, *macro* values represent the average of each metric over the gold standard. Although metric values for two US examples might be equal, the absolute number of correct or erroneous classifications might differ significantly between US depending on the GUI component set length. For example, the second US example from Figure 9.3 is only associated with three respective GUI components, whereas another US from the gold standard about visualizing an overview of the daily nutrients consumed daily (see gold standard example 78 in our supplementary materials) has 20 associated GUI components. To take into account these set length differences across the gold standard and thus counter potential evaluation result inaccuracies introduced by set length, we additionally constructed binary prediction arrays to compute respective *micro* values possessing correct weights, i.e. double GUI component set length leads to double the influence on the final metric result. To conduct the experiments, we utilized the identical setup of LLMs as described previously for RQ₁.

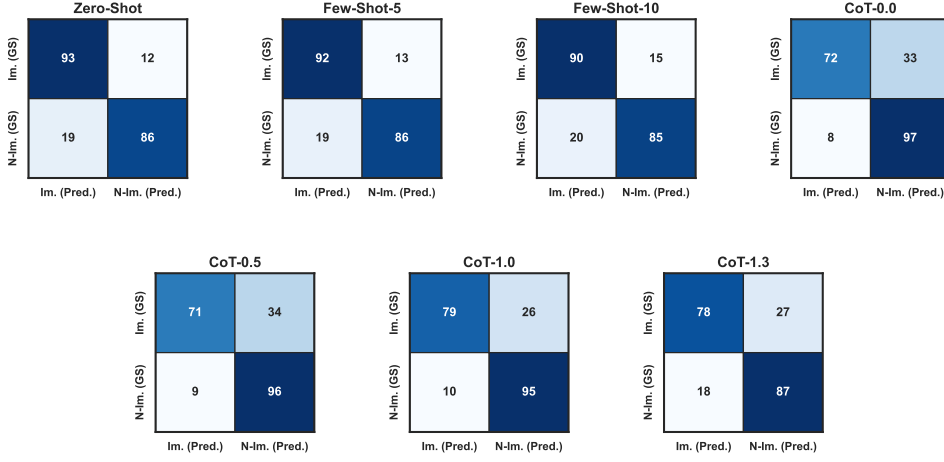


Figure 9.4: Confusion matrices for LLM-based approaches (ZS, FS ($\#examples$), and CoT ($temperature$)) for detecting US implementation in *Rico* GUIs (as binary classification).

9.4 Results & Discussion

9.4.1 RQ₁: User Story Implementation Detection

Table 9.2 illustrates the evaluation results for RQ₁, showing the P , R , and $F1$ metrics for both classes and the accuracy A over the created gold standard. Moreover, we provide the confusion matrices for each prompting approach in Figure 9.4. First, we can observe a substantially high absolute performance across all of the investigated prompting methods indicated by, for example, accuracy scores of 85.2 (ZS), 84.8 (FS₅), and 82.9 (CoT_{t=1.0}). This indicates that LLMs are capable of effectively processing the semantics of the created GUI abstraction and matching it to the semantics of the functionality encompassed in the USs. These high metric values indicate that LLMs can produce promising results for the approach. Although the ZS method seems to perform best overall according to mean values, the respective *Holm*-corrected pairwise *McNemar tests* (Adedokun and Burgess, 2012; Lachenbruch, 2014) between each of the prompting methods indicate no statistically significant differences (see Appendix E Table E.1). Moreover, the CoT methods apparently tend to be more restrictive about predicting that a US is implemented, as indicated by the highest P_1 and R_0 values. In contrast, the ZS and FS methods appear to be more balanced among the different metrics and both classes.

To enhance the understanding of the misclassifications made by the LLM, we conducted an error analysis and investigated the FP and FN instances. For the FP instances, the main root cause for misclassifications appears to be a semantic misinterpretation of GUI components with reference to the US by the LLM. For example, for a US that describes providing addresses of the nearest stores the LLM identifies the component "*SAN FRANCISCO, Store #6498*" (**Button**) (storelocator address line1) as fulfilling the US, although this component merely provides the city name and the detailed address fields were absent, as shown in Figure 9.5(A). Similarly, for the US to enable/disable

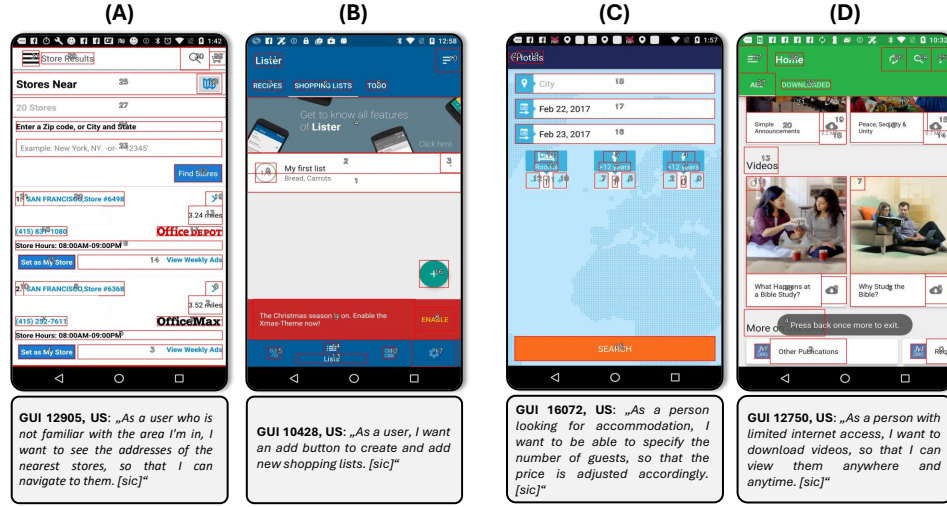


Figure 9.5: Four example *Rico* GUIs taken from the gold standard with their respective US. (A) represents a FP and (B) represents a FN instance for US implementation detection, while (C) represents a FN and (D) represents a FN/FP instance for the US matching task.

the ability to mark days as complete within a settings GUI of a fitness app, the LLM identified the GUI component “check” (**Icon**) (done) as fulfilling the US. However, this component is located in the GUI toolbar and refers to saving the overall settings.

For the FN instances, we identified several similar main root causes. Often, detailed semantic information about the functionality of the GUI components might be absent in the created GUI abstraction, resulting in the LLM being restrictive about positively identifying the US as fulfilled. For example, a US requiring a donation button to easily donate money could not be detected, since the GUI component was implemented as an image without any further textual description. Hence, the image information is not accessible to the model. Similarly, a US to add new shopping lists to a collection of lists could not be identified since the description of the GUI component “add” (**Icon**) (overview viewpager fab) (Figure 9.5(B)) is general and ambiguous for detection. These errors are mainly driven by the simplified GUI representation of used *Rico* GUIs.

Answer to RQ₁: LLM-based prompting approaches are effective to detect US implementation in static GUIs, achieving high *precision* (83.0 for ZS), *recall* (88.6 for ZS), and *F1-measure* (85.7 for ZS), while no significant pairwise differences were observed.

9.4.2 RQ₂: User Story GUI Component Matching

Table 9.3 illustrates the evaluation results for RQ₂, showing the *Macro* and *Micro P*, *R*, and *F1* metrics. Moreover, Figure 9.6 shows boxplots for all prompting methods and across the *Macro* metrics, while Figure 9.7 depicts the confusion matrices. Overall, the

Table 9.3: Evaluation results of LLM-based approaches (ZS, FS (#*examples*), and CoT (*temperature*)) for matching GUI components to USs in *Rico* GUIs with *Precision* (*P*), *Recall* (*R*) and *F1-measure* (*F1*) averaged over all instances (*Macro*) or size-adjusted (*Micro*).

	Macro			Micro		
	P	R	F1	P	R	F1
Zero-Shot_A	78.4	81.9	75.5	62.0	75.5	68.1
Zero-Shot_B	71.8	90.3	74.3	50.2	85.8	63.3
Few-Shot₅	85.0	75.5	76.5	67.7	64.3	65.9
CoT_{t=0.0}	75.8	80.6	72.7	55.7	73.1	63.2
CoT_{t=0.5}	72.1	78.8	68.8	49.2	71.0	58.1
CoT_{t=1.0}	69.8	80.9	69.0	53.4	74.6	62.2
CoT_{t=1.3}	67.7	74.0	65.4	56.2	64.6	60.1

results indicate that the models achieve a moderate to good performance in matching GUI components to USs shown by, for example, *Micro-F1* scores of 68.1 (ZS_A), 65.9 (FS₅), and 63.2 (CoT_{t=0.0}). Although a direct comparison with the results of the task discussed in RQ₁ is difficult due to the difference in datasets, still the matching task can be seen as an extension of the classification task, since classification models probably perform similar computations (e.g., as indicated by explanations from CoT models) as part of their reasoning sequence. As can be observed, the performance difference of the tasks indicates that the matching task is significantly more difficult for the LLMs. However, we argue that the obtained results are promising due to the model being capable of extracting the majority of GUI components correctly, shown by the metric values. As indicated by the *Holm-corrected Wilcoxon signed-rank test* (Woolson, 2007) between the *Macro-F1* scores of the methods, all CoT prompting methods (except $t = 0.0$) are significantly outperformed by both ZS and FS approaches, whereas the differences of the ZS and FS methods are insignificant (see Appendix E Table E.2). In addition, the ZS_B prompting method has substantially better mean *R* values compared to ZS_A (and vice versa for *P* values), indicating that the prompt extension in ZS_B optimizes the model for not missing component matches. Figure 9.3 shows two example GUIs from the gold standard (*left* and *center* GUIs) and highlighted GUI component matches as extracted by the LLM.

Moreover, we investigated *low P* and/or *low R* instances to improve the understanding of errors made by the LLM. For the cases of *low P*, the LLM often extracted wrong GUI components that were semantically related. For example, for the US to specify the number of guests in a hotel search GUI, the models also erroneously extract components for the number of rooms (see Figure 9.5(C)). For the instances possessing *low R*, similar to the misclassifications discussed earlier, we identified missing or ambiguous descriptions of GUI components as a main cause. For example, for a US to be able to download videos to watch them offline, the GUI components offering download functionality were represented as **(Image)** (document). Although the succeeding GUI component “0.7 MB” **(Label)** (document size) could act as a hint, the naming as a *document* and the

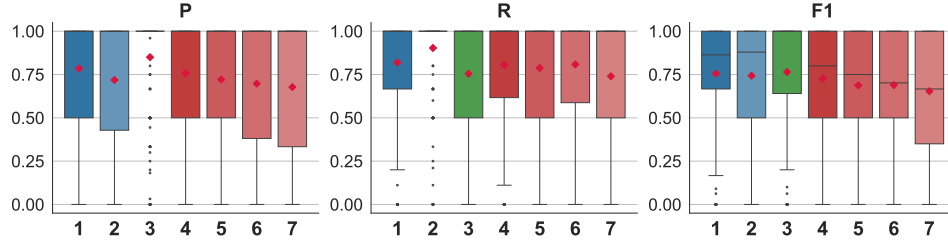


Figure 9.6: Boxplots for *Macro Precision* (P), *Recall* (R), and *F1-measure* (F_1) across methods, with ZS (*blue*), FS (*green*), and CoT (*red*), in particular: (1) ZS_A , (2) ZS_B , (3) FS_5 , (4) $CoT_{t=0.0}$, (5) $CoT_{t=0.5}$, (6) $CoT_{t=1.0}$ and (7) $CoT_{t=1.3}$, diamonds repr. means.

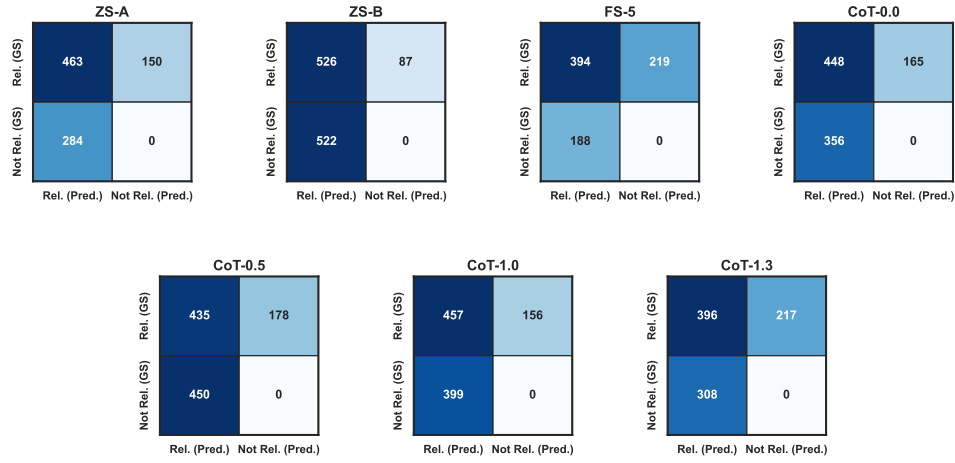


Figure 9.7: Confusion matrices for LLM-based approaches (ZS, FS (#examples), and CoT (temperature)) for matching GUI components to USs within their respective *Rico* GUIs. Lower right cell (*Not. Rel. (GS)* and *Not. Rel. (Pred.)*) always zero since TNs were not computed (reference for comparison was the GS set instead of all *Rico* GUI components).

component being marked as an image introduce ambiguity (see Figure 9.5(D)). Finally, some cases possess ambiguity that is difficult to resolve without the stakeholder. For example, for the US requesting to see an overview of the course (see Figure 9.3), the LLM extracts all listed lessons, whereas the annotation marks only the overview course item.

Answer to RQ₂: LLM-based prompting approaches are effective for matching US to their respective GUI components within static GUIs, achieving a moderate to good performance with *Macro precision* (78.4 for ZS_A), *recall* (81.9 for ZS_A), and *F1-measure* (75.5 for ZS_A). Significant differences were observed between ZS-CoT and FS-CoT.

9.5 Threats to Validity

Internal Validity The *Rico* GUI dataset is widely adopted in research and thus indicates suitability for GUI-based evaluation tasks. Nevertheless, the dataset is influenced

by the collection methods. Particularly, *Rico* GUIs present a sample of free *Android* applications from *Google Play*. However, to avoid bias, we selected GUIs from a wide range of different domains and the initial sample was randomly drawn. Additionally, there exists no dataset combining GUIs with USs. We therefore decided to collect USs for existing GUIs in a lab-based annotation study, which might introduce bias and subjectivity from participants. However, we provided clear instructions and training for writing high-quality USs and collected a large number of USs. The exhaustiveness of USs per GUI in our data set may be limited, as a fixed number of USs were written by eight participants for the respective GUIs. Additional bias might have been introduced by the authors' evaluation of collected USs. However, we conducted independent labeling and calculated IAA. Afterwards, we discussed and resolved conflicts to reduce potential bias.

External Validity Since we restricted the evaluation to *Rico* GUIs with a particular representation, the evaluation results could vary for different GUI datasets and software platforms. However, *Rico* represents the largest and most diverse mobile GUIs dataset available for research. In addition, we evaluated only a single LLM, which potentially restricts generalizability to other LLMs. Nevertheless, we employed a state-of-the-art LLM (at the time of implementing the approach) and tested different prompting methods.

9.6 Limitations

This work was a feasibility study, investigating the effectiveness of LLM-based approaches for verifying the implementation of NLR (particularly USs) in simplified, static GUIs. While the representation of *Rico* GUIs in this work clearly represents a limitation, it confirmed that the profound NLU capabilities of LLMs can be effectively employed to identify the implementation of USs in GUIs and match the relevant GUI components.

9.7 Related Work

Previous research investigated automated consistency or conformance checking of requirements on GUIs. For example, Rocha Silva et al. (2019) present a Behavior-Driven Development (BDD)-based approach that enables the automated testing of USs on interactive web GUIs with browser automation tools. However, their approach relies on manually crafted ontologies, which are time-consuming and effort-demanding to create. Furthermore, they rely on manually mapping abstract GUI components to concrete GUI identifiers, which introduces brittleness and is effortful to maintain. In contrast, we investigated leveraging the powerful NLU capabilities of modern LLMs via adopting multiple prompting-based approaches, which require no manual finetuning and can effectively process complex GUI context.

Furthermore, Mukasa and Kaindl (2008) proposed the Requirements Specification Language (RSL), which integrates requirements (e.g., in the form of NLR) and GUI specifications into a conceptually coherent representation. Their specification approach

enables requirements and GUI specifications to evolve simultaneously, closely bound together. Therefore, the developed system facilitates the implementation of correct FR while also supporting high usability. While the proposed RSL provides a novel approach for facilitating the simultaneous evolution of requirements and their respective GUI specifications, the approach is unable to automatically provide consistency checking and requires effort-demanding modeling activities.

Moslehi et al. (2020) propose a feature localization approach that enables automatic traceability between individual GUI features of a software system and the corresponding source code. In particular, they rely on crowd-based screencasts for software systems as input, apply Latent Dirichlet Allocation (LDA)-based mining approaches, and locate relevant code artifacts. While their approach also provides a mapping between GUI elements and related artifacts, their focus lies on source code, while we focus on NLR.

9.8 Conclusion

The work presented in this chapter was driven by challenge **C2-I**: *How can NLR-based verification be automated for simplified, static GUI representations as a foundation for more complex GUI verification tasks?* To tackle this question, we conducted a feasibility study by employing LLM-based approaches for automatically detecting whether NLR (particularly in the form of USs) are implemented in a GUI and subsequently matching the relevant GUI components. While the approach relied on simplified, static *Rico* GUI representations, our evaluation indicates high effectiveness for both tasks, showcasing the large potential of LLM-based approaches for automatic verification of NLR in GUIs.

CHAPTER 10

GUISPECTOR: AN MLLM AGENT FRAMEWORK FOR AUTOMATED VERIFICATION OF NATURAL LANGUAGE REQUIREMENTS IN GUI APPLICATIONS

The previous chapter introduced an initial solution approach for challenge **C2**, namely, the automated verification of NLR in GUIs. While the previous approach was conducted as a feasibility study to investigate the potential of LLM-based approaches for verifying NLR in GUIs with static, simplified GUI representations, in this chapter, we propose a novel approach for the automated verification of GUI applications using an MLLM-based CUA against NLR. The work presented in this chapter has been accepted for publication and is currently in press (Kolthoff et al., 2025b)¹. Our source code, interactive prototype, evaluation datasets, and tool demonstration video are all publicly available².

Personal Contribution. I developed the initial idea and concept for the proposed approach, while *Felix Kretzer* and I conducted discussions to refine the concept. Moreover, I implemented the entire framework and tool prototype. *Felix Kretzer* and I contributed equally to the evaluation design, while *Felix Kretzer* created the dataset code and conducted the data collection. *Felix Kretzer* and I both contributed equally to the manual annotation of the dataset. Furthermore, I conducted the data analysis and computed the

¹This section is adapted from: **Kolthoff, Kristian***, Kretzer, Felix*, Bartelt, Christian, Maedche, Alexander, and Ponzetto, Simone Paolo. GUIspector: An MLLM Agent Framework for Automated Verification of Natural Language Requirements in GUI Prototypes. In *Proceedings of the 48th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, April 2026, 1-4, Rio de Janeiro, Brazil. ACM. (in press) (arXiv preprint arXiv:2510.04791). *Authors contributed equally.

²Materials for this chapter, including source code and evaluation datasets, are available at <https://github.com/kristiankolthoff/GUISpector> and demo video at <https://youtu.be/JByYF6BNQeE>

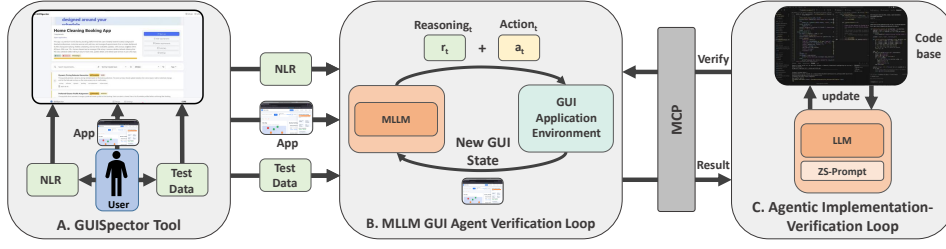


Figure 10.1: Overview of *GUIInspector* consisting of (A) our user-friendly tool, which enables to provide unstructured NLR, the GUI application for verification and test data, (B) the core MLLM CUA verification loop, which iteratively employs an MLLM to predict next actions, executes them in GUI environments and obtains new GUI states, and (C) an agentic implementation-verification loop, which enables LLM programming agents to run GUI verifications in *GUIInspector* via Model Context Protocol (MCP) interface.

results. While *Felix Kretzer* wrote the experimental setup and results sections, I wrote the rest of the original manuscript and contributed all respective tables, plots, and figures.

10.1 Motivation

The first two parts of the thesis focused on enabling rapid GUI prototyping from NLR by proposing GUI retrieval and generation techniques. Consequently, REI and RVal activities are facilitated, ensuring that the correct requirements are captured according to the actual needs of stakeholders. Subsequently, verification ensures that the implemented software system adheres to the created requirements specification, which is often conducted via time-consuming and effort-demanding testing of the system (Myers et al., 2004). While the previous chapter introduced LLM-based approaches for verifying NLR in GUIs and demonstrated high effectiveness, limitations arise from employing static, simplified *Rico* (Deka et al., 2017) GUI representations, which differ from real-world, complex, and highly dynamic GUI applications.

As discussed in the previous chapter, many approaches have been proposed in research before to improve testing activities. For example, these include automated test scripts (Xie and Memon, 2007), random exploration (Mao et al., 2016), model-based techniques (Zeng et al., 2016; Gu et al., 2019), and scripted bug replay methods (Gomez et al., 2013; Feng and Chen, 2022). While these approaches enhance the efficiency and coverage of GUI testing, they are restricted to recognizing runtime failures (system crashes) and are therefore incapable of directly verifying the correctness of NLR in GUI applications. In addition, these methods require manual tuning or cannot fully capture the complexity and dynamics of GUI applications. More recently, LLM-based approaches gained traction for more sophisticated automated bug detection in GUIs. For example, LLM-based approaches enable the generation of tests from NL descriptions (Feng et al., 2024, 2025). However, these methods still rely on textual abstractions of GUI hierarchies (similar to our

proposed approaches in Chapter 9), which renders them unable to fully assess dynamic, visual aspects of GUI applications and verify NFR and fine-grained FR. Therefore, the work in this chapter is driven by the leading research question of challenge **C2-2**: *How can NLR-based verification be automated for highly dynamic, fully-fledged GUI applications?*

To tackle this challenge and close the gap, in this chapter, we introduce *GUISpector*, a novel MLLM-based CUA for automated verification of GUI applications from NLR, as shown in Figure 10.1. In particular, the core of *GUISpector* is the MLLM CUA verification loop (see Figure 10.1(B)), which iteratively enables the MLLM to provide reasoning and predict next actions, executes them in GUI environments, and obtains new GUI states. Given NLR, the agent derives meaningful action sequences to support the verification of the requirements, thereby creating meaningful test trajectories and enabling detailed summarization of executed tests. These summaries and test results can be accessed by users from our user-friendly *GUISpector* tool (see Figure 10.1(A)) and employed in a closed loop to feed back to LLM-based programming agents. Moreover, users can provide NLR, the GUI application to test, and test data as input to our tool. Since LLM-based programming agents become more popular, *GUISpector* provides an MCP implementation to enable closed agentic implementation-verification loops (see Figure 10.1(C)), enabling agents to run automated GUI verifications after implementing changes in the codebase. Our evaluation across five GUI applications and 150 requirements with 450 AC shows the effectiveness of the approach, achieving highly accurate automated verification scores.

Contributions. With this approach, we make the following research contributions:

- *Novel MLLM-based CUA approach for automated verification of highly dynamic GUI applications from NLR*: we present *GUISpector*, a novel MLLM-based CUA approach for enabling automatic verification from NLR in GUI applications by iteratively predicting next actions, executing them and obtaining updated GUI states.
- *Tool prototype and MCP interface*: we integrate the MLLM-based CUA verification approach into a novel user-friendly tool, which enables rapid adoption by practitioners and provides detailed test summaries. In addition, we provide an MCP implementation, enabling LLM-based programming agents to access *GUISpector*.
- *Comprehensive evaluation and insights*: we create a novel dataset, which integrates web-based GUI applications with 150 detailed NLR and 450 AC. Moreover, we conduct a comprehensive experimental evaluation of the capabilities of the MLLM CUA approach, showing high effectiveness, and investigate generated test trajectories.

10.2 Approach: *GUISpector*

GUISpector is composed of three main components: (A) a human-in-the-verification-loop, which enables users to set up and monitor verification runs, provide unstructured NLR for verification, the corresponding web-based GUI application, and optional test input data. Moreover, (B) the MLLM-based CUA verification loop, which employs

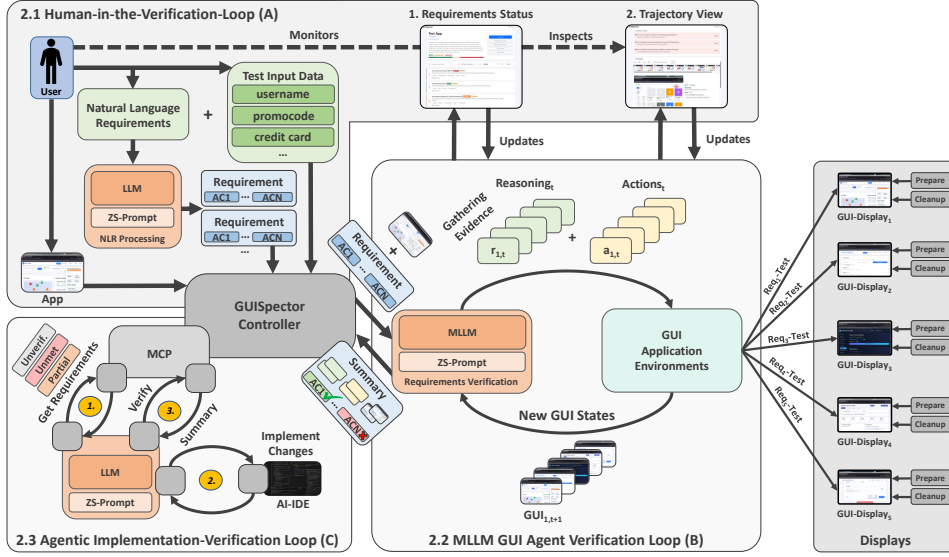


Figure 10.2: Overview of the *GUIInspector* architecture: (A) human-in-the-verification-loop, (B) MLLM CUA GUI verification loop with reasoning, actions and GUI state trajectories, and (C) agentic implementation-verification loop for LLM-based programming agents.

the MLLM to conduct reasoning and predict actions given the current GUI state and trajectory, run actions on dedicated GUI environments, and obtain updated GUI states to close the loop. Finally, (C) *GUIInspector* provides an MCP implementation, enabling fully autonomous agentic implementation-verification loops for LLM-based programming agents. An overview of the architecture details of *GUIInspector* is illustrated in Figure 10.2.

10.2.1 Human-in-the-Verification-Loop

To enable users to effectively run verifications with *GUIInspector*, we provide a user-friendly tool prototype and GUI. Specifically, users begin by specifying the entry point (URL) of the web application to run verification on, followed by providing structured or unstructured NLR. Next, the NLR are processed by employing a ZS-based LLM that creates a structured representation of the NLR. This acts as a generic NL interface to *GUIInspector*, and NLR can be provided in any form. In addition, the LLM can automatically infer missing details (such as low-level AC for high-level NLR in the context of the application) if necessary. After successfully processing the requirements, *GUIInspector* shows a structured collection of them as an overview, including their verification status after a run has been completed, as depicted in Figure 10.3. Furthermore, the verification trajectories created by the MLLM-based CUA of each run within each requirement can be inspected in our GUI, combined with a detailed breakdown of AC fulfillment status. Moreover, the agent gathers evidence for each AC decision and generates an actionable summary. Overall, users are enabled to create, organize, and manage individual verification setups.

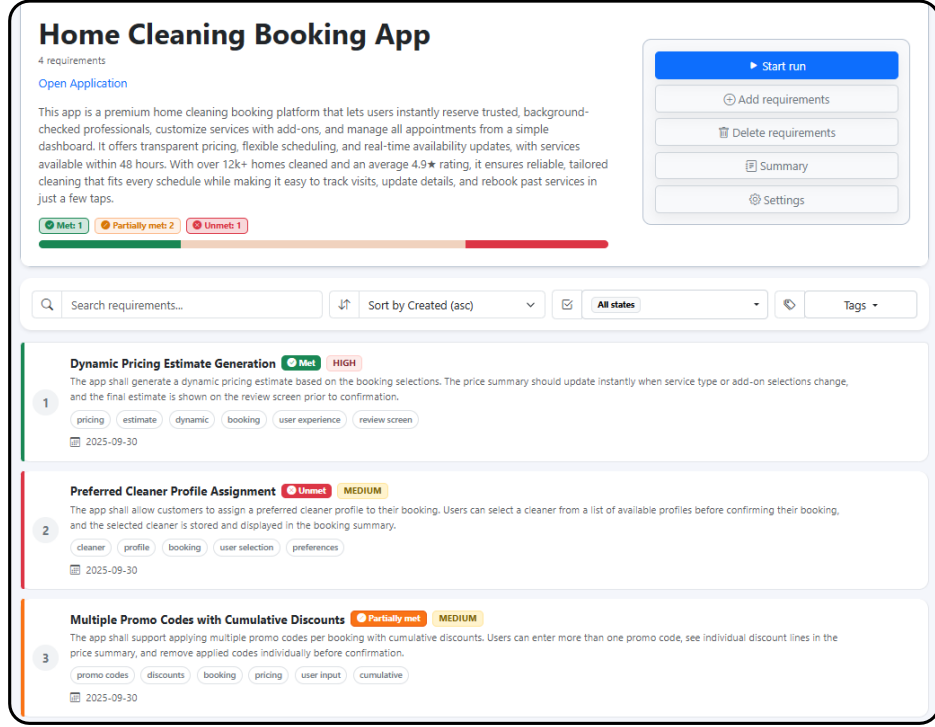


Figure 10.3: Tool prototype of *GUIInspector* showing the user-friendly GUI with a collection of *met*, *unmet*, and *partially met* NLR. Moreover, *GUIInspector* provides options to start verification runs across all listed requirements, add unstructured requirements via an LLM-based interface, delete requirements, obtain run summaries, and update settings.

10.2.2 MLLM GUI Agent Verification Loop

The central component of the *GUIInspector* approach is the MLLM CUA GUI verification loop, which adapts a pretrained MLLM that has additionally been finetuned for effective computer use (understanding GUI screens, recognizing GUI components, and predicting parametrized GUI actions for solving an interaction task) to autonomously run verification of GUI applications given NLR. Specifically, we adapt the model by employing a ZS prompt, clearly instructing the model to autonomously verify the implementation of requirements in GUI applications, providing the application entry point (i.e. start URL) and expecting the model to run meaningful interactions necessary for verification. We particularly instruct the model to employ a minimal interaction sequence to optimize testing efficiency and integrate user-provided test data if necessary. Moreover, we instruct the MLLM to verify each AC for a requirement individually, gather specific evidence during the runs, and thus justify its decisions. In particular, for each verification run, the MLLM receives the requirement to verify, accompanied by AC, necessary test input data (such as *login data*, *coupon codes*, or *user profile data*), the current GUI state represented as a screenshot, and the previous trajectory for improved contextual awareness. Based

on this task and context representation, the MLLM subsequently provides reasoning and predicts the most likely next action to verify an AC. Specifically, the employed CUA supports all standard actions including *click*, *scroll*, *type*, *keypress*, *drag*, *double click*, *move*, and *wait*. In addition to the action type, the agent also predicts the different action parameters (such as x , y , and *button type* for the *click* action, and *text* for the *type* action). Subsequently, the selected parametrized action is executed in a dedicated environment running the actual GUI application, and the updated GUI state is reported back to the MLLM, closing the loop. This iterative procedure creates a trajectory encompassing GUI states (i.e. screenshots), reasoning, and parametrized actions ($GUI_1, r_1, a_1, \dots, GUI_n, r_n, a_n$). Finally, the MLLM is instructed to generate a JSON output encompassing a detailed summary of the verification run, explanations, the predicted status of each AC (*met* or *unmet*), and gathered evidence for the decision. The requirement status (*met*, *unmet*, or *partially met*) is then computed from the individual AC states. In particular, the requirement status is set to *met* (*unmet*) if all of the corresponding AC states are *met* (*unmet*). Otherwise, the status is set to *partially met* (at least one *met* and one *unmet* AC). The generated summaries are especially helpful to manually employ as feedback to developers or LLM-based programming agents. To avoid contaminating GUI states and introducing errors, the environments are cleaned up before and after each verification.

10.2.3 Agentic Implementation-Verification Loop

While the previously introduced MLLM-based CUA verification loop in combination with the *GUISpector* tool interface enables users to rapidly run autonomous verification of their GUI applications, *GUISpector* additionally supports a fully automatic mode integrated into LLM-based programming agents. More recently, LLM-powered Integrated Development Environments (IDEs) such as *Cursor* (Anysphere, 2025) and *Copilot* (GitHub, 2025) gained popularity, focusing on agentic programming support by enabling developers to easily provide relevant parts of their codebases to LLMs to implement changes or entirely new functionality for increasing developer productivity.

To facilitate seamless integration of the verification capabilities of *GUISpector* into LLM-based programming agents, we implemented an MCP (Hou et al., 2025) server to allow external agents to interact with the verification API. Specifically, for different verification setups created by users through the *GUISpector* tool, these programming agents can retrieve requirements collections with their accompanying AC, their current verification states, and start implementing patches for violated requirements. Moreover, the LLM agent can actively start verification runs for selected requirements, for example, when they are not verified yet or after a patch was implemented for a violated requirement. Subsequently, the programming agent receives the previously discussed detailed summaries with evidence gathered during the verification run, which can be employed to inform the agent to update the codebase again and potentially increase its implementation effectiveness (particularly in cases of *unmet* or *partially met* requirements).

This enables a fully autonomous, iterative agent mode: the LLM-based programming agent retrieves a requirement, implements patches if necessary, runs verification, and

either implements further changes if the requirement status is not identified as *met* during the verification, or continues the same process for the next requirement. We created a ZS prompt that can be employed inside LLM-powered IDEs or in general for LLM-based programming agents, which instructs the LLM to follow the described autonomous implementation-verification loop, providing a novel approach for integrating automatic verification with requirements-driven, LLM-based development of GUI applications.

10.2.4 Prototype Implementation

GUISpector is implemented as a *Django* web application with an *HTML/CSS/JS* frontend, combined with *MySQL* (MySQL, 2025) for data storage. To enable efficient and parallel processing of background tasks (such as the MLLM-based CUA verification runs), we utilize the *Celery* framework (Celery, 2025) distributed among worker nodes in combination with the message broker *Redis* (Redis, 2025). Since *GUISpector* requires dedicated environments to simultaneously run verification of GUI applications, we additionally implemented a *display resource manager* based on *Redis*, which enables the dynamic allocation of *Xfce* desktop environments deployed on *Xvfb* virtual displays. To enable execution of the parametrized GUI actions predicted by the MLLM, we utilize *xdotool* for user interaction simulation. For running the MLLM-based CUA, we experimented with the *OpenAI CUA* approach (OpenAI, 2025b). However, other agents can be easily integrated into the proposed framework. For other LLM-based tasks in *GUISpector* (processing of NLR), our prototype currently supports *OpenAI GPT* (OpenAI, 2023), *Google Gemini* (Team et al., 2023), and also *Anthropic Claude Sonnet* (Anthropic, 2025a).

10.3 Experimental Evaluation

In the following, we present the comprehensive evaluation of our approach, which is based on a novel dataset combining web-based GUI applications and fine-grained NLR. Particularly, we provide the experimental setup for the following two research questions:

- **RQ₁:** *How effective is the MLLM-based CUA approach for automated verification of highly dynamic GUI applications based on NLR?* To answer this question, we create a novel dataset of web-based GUI applications combined with *met*, *unmet*, and *partially met* NLR. To evaluate the effectiveness of the approach, we compute standard classification metrics including *precision*, *recall*, and *F1-measure* per class.
- **RQ₂:** *What are the costs associated with an MLLM-based CUA approach for automated verification of highly dynamic GUI applications based on NLR?* To answer this question, we track the costs (*token consum.*, *steps*, *time*, *monetary cost*).

10.3.1 RQ₁: Effectiveness of MLLM CUA for GUI Verification

Gold Standard. To evaluate the ability of the proposed approach to recognize requirements fulfillment, we first created a new dataset, due to the lack of available datasets

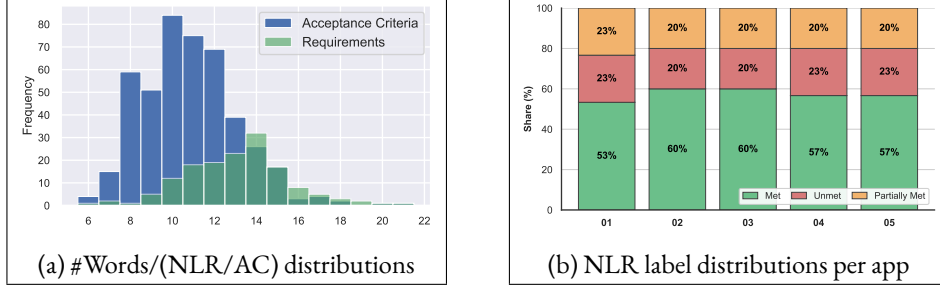


Figure 10.4: Distributions for the created dataset combining USs with GUIs. In particular, each US is represented by a one-to-many mapping (≥ 1) to respective GUI components.

Variable	n	μ	σ	min	Q1	med.	Q3	max
(1) #Words/Requirement	150	13.11	2.51	6.00	11.00	13.00	14.00	21.00
(2) #Words/Acceptance Criterion	450	10.83	2.30	6.00	9.00	11.00	12.00	21.00

Table 10.1: Summary statistics (*mean* μ , *standard deviation* σ , *minimum*, *first quartile* **Q1**, *median*, *third quartile* **Q3**, *maximum*) for the novel gold standard considering the following aspects: (1) number of words per requirement, (2) number of words per AC.

of GUIs paired with requirements. Using *Codex* (OpenAI, 2025a), we generated 30 functional NLR with three AC each for five domain-diverse applications (*Park-and-Pay*, *Budget Tracker*, *Recipe Generator*, *Fitness Quests*, *Cleaning Booking*). Each requirement was also randomly assigned a status (*met*, *unmet*, and *partially met*), with the target distribution being set to 18 *met*, six *unmet*, and six *partially met* requirements (30 per app). In a second step, *Codex* was instructed to create the corresponding apps (single *HTML* files with local state, no external Content Delivery Networks (CDNs)/APIs, including all *CSS* and *JS*) and ensure the respective requirements statuses. All employed prompts, NLR, AC, and apps are available in our accompanying repository.

We then evaluated whether the generated apps conformed to the specifications, especially considering the different requirements statuses. Two authors independently labeled 450 AC as *met* or *unmet* by running the applications and conducting manual testing. Annotations were independent but not equally blinded: one annotator had not seen the detailed specification; the other had prior exposure but did not consult it during annotation. As described before, requirement statuses were derived from AC labels (all AC *met* = Req. *met*, all AC *unmet* = Req. *unmet*, otherwise = Req. *partially met*).

Before resolving disagreements, we measured IAA across 150 requirements (5 apps $\times$ 30 requirements) for three raters (*LLM-Specification*, *Annotator 1*, *Annotator 2*), indicating very high agreement (*Krippendorff's* α : 0.929, ordinal, 95% CI [0.880, 0.966]). We adopt the agreed human evaluations as the gold standard, and eight disagreements (out of 150) occurred between the LLM and human annotators. At the AC level (5 apps $\times$ 30 requirements $\times$ 3 AC), agreement between the two human annotators was similarly high (*Cohen's* κ : 0.921, pooled, nominal, with accuracy: 96.7%, 435 out of 450 AC).

Table 10.2: Effectiveness results for MLLM-based CUA GUI verification for detecting *met*, *unmet*, and *partially met* classes in GUI applications across *Requirements (Req)* and *Acceptance Criteria (AC)* in five apps with *precision (P)*, *recall (R)*, and *F1-measure (F1)*.

	Met (Req)			Unmet (Req)			Partial (Req)			Met (AC)			Unmet (AC)		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
App-1	78.9	93.8	85.7	100.0	71.4	83.3	50.0	42.9	46.2	90.8	96.7	93.7	92.0	79.3	85.2
App-2	100.0	94.4	97.1	100.0	100.0	100.0	83.3	100.0	90.9	96.8	98.4	97.6	95.8	92.0	93.9
App-3	93.3	82.4	87.5	83.3	83.3	83.3	62.5	83.3	71.4	93.1	90.0	91.5	79.3	85.2	82.1
App-4	100.0	82.4	90.3	75.0	85.7	80.0	50.0	66.7	57.1	94.6	89.8	92.2	82.4	90.3	86.2
App-5	85.0	100.0	91.9	100.0	85.7	92.3	66.7	40.0	50.0	90.9	100.0	95.2	100.0	77.8	87.5
Avg.	91.5	90.6	90.5	91.7	85.2	87.8	62.5	66.6	63.1	93.3	95.0	94.0	89.9	84.9	87.0
SD	9.3	7.9	4.4	11.8	10.2	8.2	13.8	25.8	18.3	2.6	4.8	2.5	8.8	6.4	4.3

Stratified by app, κ ranged from 0.841 (*App 1*) to 0.972 (*App 2*), indicating high overall agreement with some heterogeneity. Table 10.1 shows summary statistics for the novel gold standard with word counts per NLR and AC, while Figure 10.4(a) shows the respective distributions and Figure 10.4(b) depicts the NLR label distributions across all five apps.

Evaluation Metrics. Based on the ground truth of each NLR and related AC, we evaluated the ability of *GUISpector* to identify the correct requirements statuses. Each app was evaluated in a single run. Requirements were classified into three categories (*met*, *unmet*, *partially met*), while AC were assigned two categories (*met*, *unmet*). We calculated *precision*, *recall*, and *F1-measure* for the three-class and the binary AC classification task.

10.3.2 RQ₂: Cost for MLLM-based CUA GUI Verification

Evaluation Metrics. To evaluate the costs associated with MLLM-based CUA GUI verification, we track several cost metrics alongside running the experiments, including the *token consumption*, *steps*, *time*, and *monetary cost*, and report the NLR-wise averages.

10.4 Results & Discussion

10.4.1 RQ₁: Effectiveness of MLLM CUA for GUI Verification

Table 10.2 shows a summary of per-app and average results regarding the effectiveness of MLLM-based CUA GUI verification across NLR and AC. In addition, Figure 10.5 illustrates respective bar charts for verification results on the requirements level, and Figure 10.6 depicts bar charts for verification results on the AC level. For the AC binary classification task, both classes *met* ($F1=94.0$) and *unmet* ($F1=87.0$) achieve high scores on average across the five considered applications, indicating that our approach reliably identifies fulfillment and violations at the AC level. Considering the effectiveness on

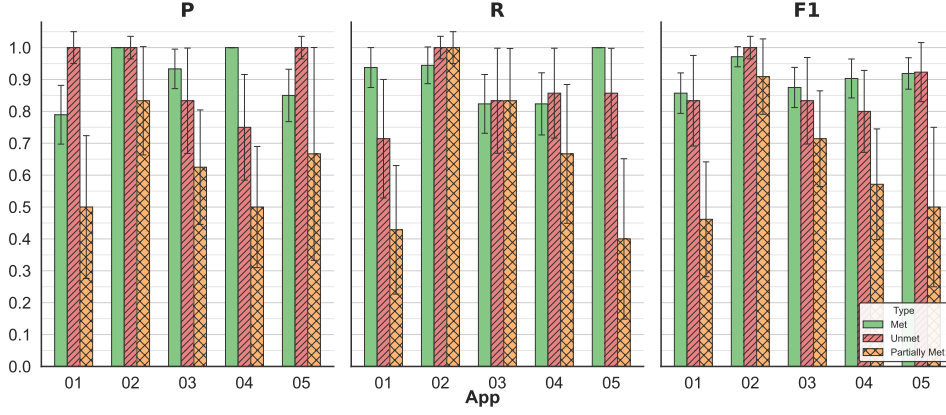


Figure 10.5: Bar charts (error bars represent one standard deviation) for automated MLLM-based CUA GUI verification on the requirements level across *met* (green), *unmet* (red), and *partially met* (orange) class for metrics *precision* (*P*), *recall* (*R*), *F1-measure* (*F1*).

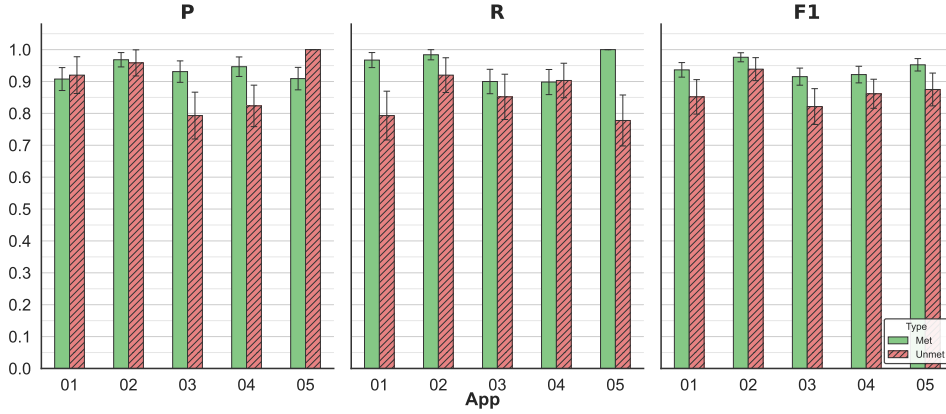


Figure 10.6: Bar charts (error bars represent one standard deviation) for automated MLLM-based CUA GUI verification on the Acceptance Criteria (AC) level across *met* (green) and *unmet* (red) class (binary) for metrics *precision* (*P*), *recall* (*R*), *F1-measure* (*F1*).

individual applications, it can be observed that *App-3* (*Recipe Generator*) performs worst (with the lowest *F1* scores for *met* ($F1=91.5$) and *unmet* ($F1=82.1$)), while *App-2* (*Budget Tracker*) performs best (with the highest *F1* scores for *met* ($F1=97.6$) and *unmet* ($F1=93.9$)).

For the three-class requirements task, *F1* scores remain high for *met* ($F1=90.5$) and *unmet* ($F1=87.8$) on average across the five considered applications, whereas *partially met* is slightly lower ($F1=63.1$), which may result from boundary confusions due to greater semantic ambiguity. For example, for the *met* and *unmet* classes on the requirements level, a single FP prediction by the model on the AC level leads to an erroneous prediction on the requirements level (also FP), rendering this the most fragile class of the three.

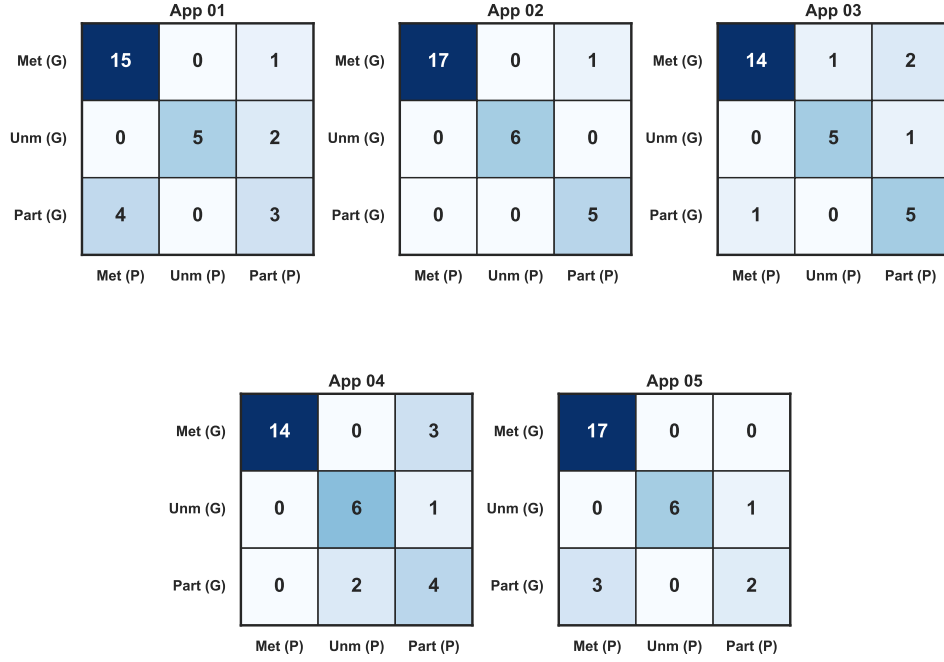


Figure 10.7: Confusion matrices for MLLM-based CUA GUI verification on the requirements level across five apps and three classes *met*, *unmet* (*Unm*), and *partially met* (*Part*), showing highest error rate on the *partially met* class. G = Gold standard, P = Prediction.

Figure 10.7 illustrates the confusion matrices on the requirements level and across all five apps. In particular, it highlights the highest error rate on the *partially met* class, which is often erroneously predicted by the model (reducing *precision*, rightmost column in the confusion matrices) or erroneously not recognized (reducing *recall*, lowest row in the confusion matrices). Overall, the obtained results in our experimental evaluation across both AC and requirements levels indicate that our MLLM-based CUA GUI verification approach possesses high effectiveness.

Next, we conducted an additional analysis of the trajectories obtained during the verification runs over the requirements in our gold standard to better understand the behavior of the MLLM-based CUA GUI verification approach. Figure 10.8 shows a series of stacked bar charts with distributions of the predicted and executed MLLM-based CUA GUI actions across all five applications. Notably, as expected, the most frequently predicted GUI action is *click*, which represents one of the most common interaction mechanisms on GUIs overall and is also supported by many different GUI components. The second most frequent action is represented by *scroll*. While the action *click* is also commonly utilized to conduct high-level navigation in applications (via the navigation drawer or menu), *scroll* is required to access the entire content on a fixed, local page. However, *App-2* represents an exception since *scroll* is among the least frequently utilized actions, representing that vertical or horizontal scrolling was not necessary and the content was entirely visible within the window. The third most frequently predicted

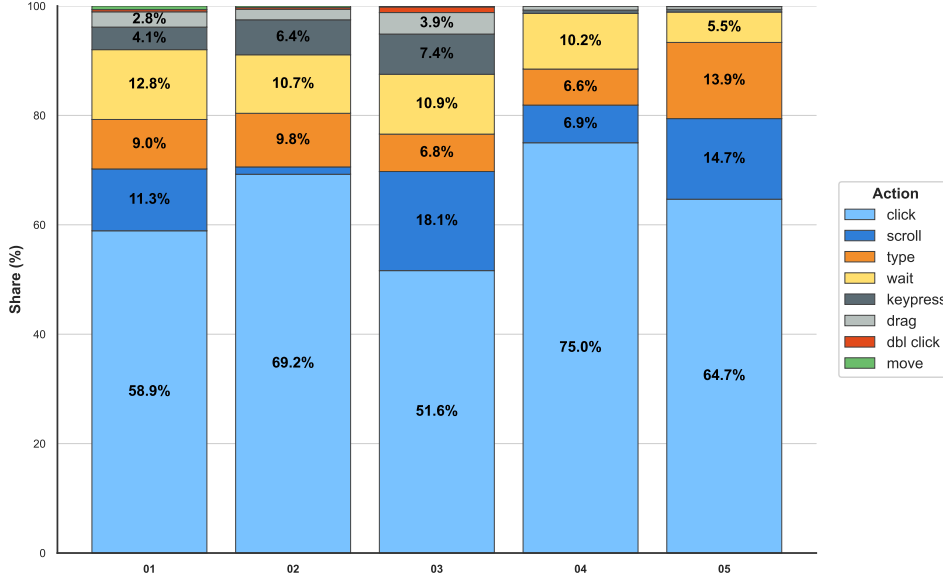


Figure 10.8: Stacked bar charts showing the distributions of the predicted and executed MLLM-based CUA GUI actions (including *click*, *scroll*, *type*, *keypress*, *drag*, *double click*, *move*, and *wait*) across the five GUI apps considered in our experimental evaluation runs.

action on average is *wait*, which is utilized, for example, when content was not yet rendered in the previous GUI state and therefore no meaningful reasoning could be generated. Subsequently, *type* and *keypress* actions follow, which are often required for providing custom input data, but less common compared to clickable GUI components. Actions *drag*, *double click*, and *move* were the least frequently predicted and often only required for very special cases (such as drag-and-drop of list items or calendar entries).

Moreover, in Figure 10.9, we provide an example trajectory extracted from an experiment run over the gold standard, particularly for the high-level NLR of being able to flag an ingredient with *Use Soon* with three AC. In addition, we provide annotations for the predicted parametrized actions at each state and optional reasoning. First, the agent clicks on the *text input field* to add an ingredient to the inventory list (as indicated by the reasoning). After the *text input field* is focused, the agent types “*Apple*” and clicks on the *Add to Inventory* button. Next, since the current view does not fully capture the entire page content, the agent predicts a downward *scroll* action, identifies and clicks the *Mark Use Soon* button on the created ingredient card (AC₁), identifies the distinct visual style (AC₂), and confirms the state after refreshing the page (AC₃), therefore fulfilling all three AC. At the end, the agent provides a detailed summary and gathered evidence per AC.

Answer to RQ₁: The proposed automated MLLM-based CUA GUI verification approach shows high effectiveness on the AC level (*met* ($F1=94.0$) and *unmet* ($F1=87.0$)) and requirements level (*met* ($F1=90.5$), *unmet* ($F1=87.8$), and *partially met* ($F1=63.1$)).



Figure 10.9: Example trajectory extracted from the gold standard run encompassing GUI states (i.e. screenshots), optional reasoning, and parametrized GUI actions from *App-3* (*Recipe Generator*) and the NLR: “The system shall enable users to mark ingredients with a ‘Use Soon’ flag for prioritization.[sic]” with AC₁: “Ingredient cards provide a ‘Use Soon’ control.[sic]”, AC₂: “Activating the control highlights the ingredient with a distinct visual style.[sic]”, and AC₃: “The ‘Use Soon’ state is saved and restored via browser storage.[sic]”.

10.4.2 RQ₂: Cost for MLLM-based CUA GUI Verification

To answer this research question, we gathered several cost-related metrics during the experiment runs shown in Table 10.3 and Figure 10.10. Execution effort varies by app (#steps from 19.5 for *App-3* to 34.2 for *App-5* on average). The observed spread is expected

Table 10.3: Performance metrics across applications (averaged over requirements): *#steps*, *time (s)*, *token consumption (k)*, and *cost (\$)* (OpenAI CUA: \$3/M input, \$12/M output).

	#Steps		Time (s)		#In-Tok. (k)		#Out-Tok. (k)		Cost (\$)	
	Avg.	SD	Avg.	SD	Avg.	SD	Avg.	SD	Avg.	SD
App-1	25.1	17.4	336.0	291.2	229.8	165.5	2.394	1.323	0.689	0.029
App-2	21.1	14.9	322.3	316.0	189.7	127.8	1.984	0.817	0.569	0.024
App-3	19.5	16.6	223.1	240.5	174.6	149.2	2.002	1.020	0.524	0.024
App-4	22.3	12.4	293.4	182.3	202.7	107.7	2.115	0.822	0.608	0.025
App-5	34.2	19.8	412.9	266.7	311.9	188.0	2.850	1.399	0.936	0.034
Avg.	24.4	16.2	317.6	259.3	221.7	147.6	2.269	1.076	0.665	0.027
SD	5.8	2.8	68.8	51.4	54.3	31.4	0.364	0.274	0.163	0.004

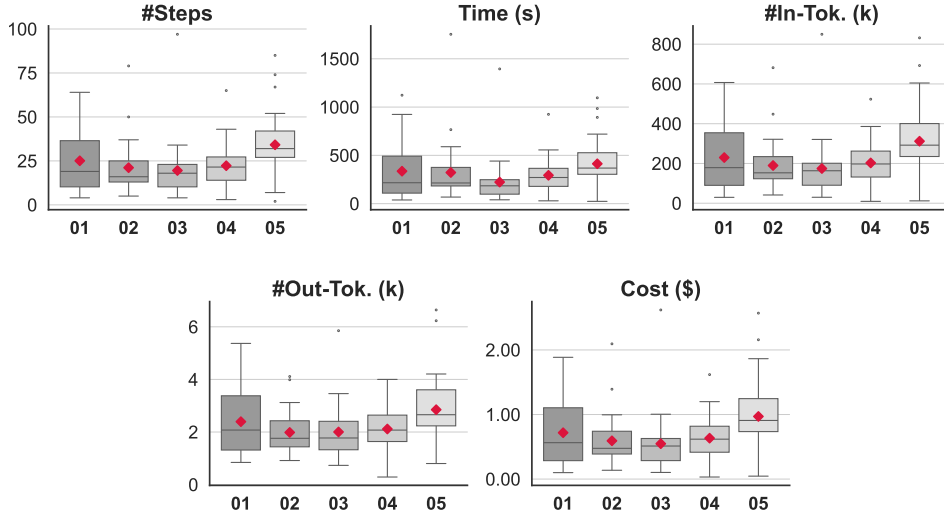


Figure 10.10: Boxplots for the performance metrics across all five considered GUI applications (averaged over all requirements per application): *#steps*, *time (s)*, *token consumption (k)*, *cost (\$)* (OpenAI CUA: \$3/M input, \$12/M output), red triangles represent means.

because each run starts from a clean state and certain apps require longer prerequisite flows before a requirement can be verified (e.g., always configuring a cleaning appointment prior to editing it). The elapsed *time* per requirement likewise differs across apps for the same reasons. Although per-requirement runtime may, in some instances, exceed that of human testers, the framework can make up for it by supporting straightforward parallelization. Costs are dominated by MLLM input tokens, with an average of \$0.67 per requirement verification. We expect reductions by merging related verification runs.

Answer to RQ₂: The costs range from \$0.524 (min. average) to \$0.936 (max. average) and from 22.3 (min. average) to 34.2 (max. average) *#steps* per requirement verification.

10.5 Threats to Validity

Internal Validity. Due to the absence of any dataset combining GUI implementations with requirements, the dataset utilized in our evaluation relies on requirements, AC, and GUI implementations generated by the *Codex* (OpenAI, 2025a) model, which might introduce bias and errors. Furthermore, the employed MLLM CUA is based on the *GPT-4o* (OpenAI, 2024) model. While the model used to create the dataset differs from the agent model used in our verification approach, they are from a similar model family (*OpenAI GPT*). However, the length of requirements and AC indicate a good level of complexity. Moreover, two paper authors independently investigated the requirements for correctness, verified the requirements status in the implemented system, and resolved annotation conflicts, showing a very high IAA, which indicates a high-quality annotation.

External Validity. While our evaluation dataset encompasses five different applications from diverse domains, a larger dataset with even more applications from more domains could improve the generalizability of the results. Moreover, the requirements are generated by the *Codex* (OpenAI, 2025a) model. Therefore, the considered requirements in our evaluation might differ from real-world requirements, which potentially restricts generalizability. However, two paper authors independently investigated the requirements manually, which revealed a high level of quality and annotation correctness.

10.6 Limitations

While our approach enables automated verification of NLR in highly dynamic GUI applications, it also carries substantial costs through the usage of an MLLM-based CUA. However, the long runtime of tests (up to several minutes on average across the evaluation dataset) can be mitigated by heavy parallelization through increasing the number of worker nodes in *Celery*. Furthermore, the monetary (API) costs entailed by employing a proprietary, state-of-the-art MLLM-based CUA might be considered high. However, running these tests manually with developers or writing automation scripts with developers also entails high costs. In addition, in this work, we focused on investigating the general potential and effectiveness of MLLM-based CUA for GUI verification, while we neglected optimization. However, we observed high potential for efficiency improvements: many requirements encompass similar sub-trajectories, which could be exploited for efficient merges, which could save a large number of verification steps. Moreover, an external MLLM memory could optimize the predicted action sequences for efficiency.

10.7 Related Work

Since we already discussed related work regarding testing and verification of GUI applications previously (see Chapter 9 Section 9.7), we provide a brief summarization. Specifically, many GUI testing approaches have been proposed before in research including automated test scripts (Xie and Memon, 2007), random exploration (Mao et al.,

2016), model-based techniques (Zeng et al., 2016; Gu et al., 2019), and scripted bug replay methods (Gomez et al., 2013; Feng and Chen, 2022). However, with our proposed *GUISpector* approach, we enable NLR-based verification of GUI applications, while previous methods focused on identifying runtime failures (system crashes). Therefore, our approach requires profound NLU capabilities to process provided NLR and effectively run verifications in GUI applications. While recent LLM-based methods have been proposed that run tests based on NL descriptions (Feng et al., 2024, 2025), they rely on textual GUI hierarchy abstractions, which oversimplify the GUI state and limit the capabilities for detecting requirement violations related to visual and dynamic aspects. In addition, with the proposed *GUISpector* approach, to the best of our knowledge, we are the first to enable fully autonomous GUI implementation-verification loops for LLM-based programming agents with an MCP server and provide a user-friendly tool.

10.8 Conclusion

The work presented in this chapter was driven by challenge **C2-2**: *How can NLR-based verification be automated for highly dynamic, fully-fledged GUI applications?* To tackle this question, in this work, we proposed *GUISpector*, a novel MLLM-based CUA GUI verification approach capable of processing complex NLR. Moreover, we provide a tool prototype implementation of the approach and an MCP server, which enables fully autonomous implementation-verification loops for LLM-based programming agents. Finally, the experimental evaluation indicates the high effectiveness of our approach, achieving high verification accuracy across AC and requirements level verification tasks.

Part IV

DISCUSSION AND CONCLUSION

CHAPTER II

DISCUSSION

While we presented our solution approaches for the different challenges for automated NLR-based GUI prototyping and verification in previous chapters, in this chapter, we provide a brief discussion about interesting aspects spanning the entire work of the thesis.

II.1 GUI Retrieval and GUI Generation Trade-offs

To address the challenge of automated GUI prototyping from NLR, we initially proposed GUI retrieval approaches (see Chapter 3, 4, and 5), followed by LLM-based GUI generation approaches (see Chapter 6, 7, and 8). This raises the question whether one approach is generally better and should be preferred with respect to NLR-based GUI prototyping. While retrieval approaches benefit from the low computational cost and speed (low latency), enabling the computation of millions of relevance scores within seconds (although with less representational power for both NLR and GUI prototypes), they are restricted to the content of associated GUI repositories, which are static GUI representations untailored to arbitrary, custom NLR. In contrast, this represents a crucial advantage of LLM-based GUI generation approaches, which — given the impressive NLU capabilities of modern LLMs — can adjust and directly tailor generated GUI prototypes to particular details formulated in the requirements. However, both retrieval and generation approaches are inherently limited regarding their potential domain coverage, given their data-driven nature (GUI repository for retrieval and pretraining data for LLMs), while LLM-based generation possesses higher adaptation capabilities.

However, we argue that retrieval-based approaches are particularly beneficial as GUI prototyping support in domains covered well by the underlying GUI repository, since these approaches can retrieve and show relevant GUI prototypes rapidly and enable immediate comparison of many similar GUIs, which potentially improves elicitation and helps to identify additional relevant requirements. While LLM-based approaches could be parallelized to generate many variations, this entails much higher cost and still carries latency. However, for requirements that are less commonly known or represent many deviations from more standardized GUIs, LLM-based approaches should be preferred. An LLM-based approach could be created to decide which approach to utilize per re-

quirement. In addition, GUI retrieval and LLM-based generation approaches can also be effectively integrated. In particular, we presented the RAGG approach (see Chapter 6), which enables the incorporation of external GUI prototyping knowledge into the LLM, which is especially helpful in domains not well covered in the pretraining data of the LLM. Moreover, retrieval-based methods could be utilized initially to discover broadly relevant GUI prototypes, which are subsequently adapted with an LLM. To summarize, GUI retrieval and GUI generation approaches both have advantages and disadvantages, which should be carefully considered and weighed when choosing an appropriate approach.

II.2 LLM-based Approaches Augment Human Capabilities

While we presented several approaches for GUI prototyping automation, we focused on integrating LLMs into existing GUI prototyping workflows to augment the capabilities of requirements analysts and prototype developers (such as approaches in Chapter 7 and 8). Particularly, we focused on reducing labor-intensive, manual tasks, enabling analysts and prototype developers to focus more on the actual creative aspects. Moreover, while LLM-based approaches encompass tremendous NLU capabilities, they still produce erroneous outputs, emphasizing the importance of close collaboration between humans and LLMs for GUI prototyping. With approaches such as presented in Chapter 7, we provide a framework that integrates LLMs closely into GUI prototyping environments, while ensuring human control in the generation process and for subsequent GUI editing.

II.3 Cost-Effectiveness of LLM-based Approaches

While LLM-based approaches for automated GUI prototyping (such as MLLM-based GUI reranking, feature prediction, and ZS-based GUI generation) and verification are highly effective, they also entail high costs, as shown across several evaluations, including long *runtime* (especially for verification) and high *monetary cost*. In particular, API costs play a crucial role, since state-of-the-art LLMs are typically proprietary models with dedicated, paid APIs. However, the gains in effectiveness across different tasks are also high, therefore requiring careful consideration when choosing an LLM-based approach. This highlights the importance of research for effective Small Language Models (SLMs).

II.4 Evaluation Challenges for GUI Prototyping Approaches

One crucial methodological challenge for evaluating automated GUI prototyping approaches is the subjectivity encompassed in deciding whether GUI prototypes are *relevant*, *accurate*, or *good* with respect to a provided NLR. While certain aspects are more objective (such as the number of GUI components contained in a GUI), others are more subjective (such as requirements that are expressed through that GUI). To embrace and mitigate this subjectivity when rating GUIs, we conducted different types of evaluations (such as relevance annotations with IR metrics, user studies, and Likert scale perception) and included large numbers of distinct annotators, typically via high-quality crowdsourcing.

II.5 Hallucinations in LLM-based Approaches

Hallucinations represent a popular issue and central reliability risk occurring in LLMs. In particular, hallucinations refer to generated LLM output that appears fluent and plausible while being factually incorrect or not grounded in the relevant sources (Ji et al., 2023; Huang et al., 2025; Kalai et al., 2025). Prior research differentiates between *faithfulness*, referring to model responses that are not supported by or contradict the provided input material, and *factuality*, referring to generated outputs that are factually incorrect with respect to external world knowledge (Ji et al., 2023).

In this thesis, we proposed several approaches that are built upon and rely heavily on LLMs, which renders hallucinations as a relevant issue to be considered. In general, the investigated tasks and proposed LLM-based approaches differ from many typical settings that are particularly prone to hallucinations. First, the models are not primarily employed for factual question answering with an objectively correct output, where the validity of the generated model response depends on fetching facts from the internal model parameters. In contrast, the answers are often grounded through concrete artifacts such as GUIs, requirements, or design specifications. In addition, many of the considered tasks, such as GUI relevance judgment given an NLR, are inherently subjective and therefore lack an objectively correct answer (also multiple human raters would judge differently). Moreover, our approaches often include validation steps (e.g., automatically validating if generated *Material Design* component configurations adhere to the available components and represent valid configurations) or emphasize human-in-the-loop (see Section II.2), ensuring that generated outputs are verified by humans. In the following, we provide more detailed descriptions for each of the LLM-based approaches.

For *GUI-ReRank* (see Chapter 4), hallucinations could mainly occur in the GUI description generation phase or while decomposing the requirements. For example, the generated descriptions might include functionality which is not present in the GUI. However, in this task, the LLM output is directly grounded to the GUI screenshots, rather than retrieving facts from the internal model parameters. In addition, we manually inspected a random sample of the descriptions and requirements decompositions, which did not reveal hallucinated fragments. In contrast, GUI relevance judging or reranking represents a subjective task, not possessing an objectively correct answer. Therefore, we do not consider deviations from the ground truth as hallucinations.

In *SERGUI* (see Chapter 5), hallucination risks are primarily related to the LLM-based GUI feature recommendations, given the NLR, an initially selected GUI, and already specified GUI features. Since these recommendations are intended to stimulate REI with the user rather than automatically creating final GUI specifications, unsupported or less relevant recommendations are mitigated through human-in-the-loop. In addition, the recommended GUI features are grounded through the provided context. Moreover, the recommendation task considered is also subjective in nature. During the user study, we did not observe any recommendations that were entirely out of scope.

For the proposed ZS approaches for GUI generation (see Chapter 6), the hallucination risks lie mainly in whether the created GUI prototypes are sufficiently grounded

in the NLR. For example, the employed LLM might introduce GUI elements or design choices that have not explicitly been stated in the NLR. However, since our approach focuses particularly on the elicitation phase, exploring potential requirements options and design choices derived by the LLM are valuable for stimulation. In addition, we considered abstracted, high-level NLR, which apparently forces the LLM to rely on assumptions for generating concrete low-level GUI prototypes. However, since the prototypes are intended to be employed in discussions with stakeholders, we would incorporate human feedback and mitigate LLM assumptions. Moreover, the proposed techniques provide grounding for the generation process (e.g., RAGG, which provides concrete GUI screenshots relevant to the NLR in the context), and we evaluated the prototypes with human judgments, trying to assess the subjective nature of GUI prototype quality.

For our proposed *Figma* GUI prototyping support approaches (see Chapter 7 and 8), the hallucination risks mainly concern the correct generation of *Material Design* components and automatic derivation of additional requirements from high-level descriptions. For example, an LLM might create component configurations with attributes, icons, and layouts that are not supported by the employed component library. In addition, the implemented components might not adequately represent the intended functionality given the NLR. For the requirements derivation, the LLM could propose low-level requirements irrelevant to the high-level description. However, to mitigate these risks, we constrain the generation process with multiple strategies. For example, we provide the *Material Design* component library specification via a two-stage RAG approach and existing GUI context to the LLM, enabling grounding of the responses. Moreover, we automatically validate the generated components against the specification, restricting the LLM output to solely valid component configurations. Finally, we propose a human-in-the-loop approach, which enables developers or designers to rapidly verify and adapt the created components or derived requirements.

Considering LLM-based static GUI verification (i.e. predicting the implementation status of a US in a GUI and extracting GUI components, see Chapter 9), the main hallucination risks lie in false predictions (e.g., classifying a US as implemented while it is not fulfilled in the GUI or extracting false GUI components). While both tasks involve a higher degree of factuality compared to tasks discussed earlier, the tasks still encompass a certain level of ambiguity or subjectivity. In addition, the LLM responses are grounded in the provided context of the GUI and the US. While for higher temperature settings some CoT matching approaches created random output sequences (resulting in $F1=0$), we did not observe hallucinations for the rest of the evaluation dataset. Errors stem mainly from underspecified textual GUI representations. In addition, this approach is constructed again as human-in-the-loop, ensuring that humans conduct output verification.

Finally, in *GUISpector* (see Chapter 10), the primary hallucination risks concern unsupported NLR verification predictions. Hallucinations could occur when the MLLM agent runs trajectories in the apps irrelevant to the NLR. However, the LLM verification result is grounded in specific GUI screenshots, executed actions, and overall evidence collected during the verification run. To mitigate hallucination risks, *GUISpector* follows a human-in-the-verification-loop approach, allowing users to inspect verification runs.

CHAPTER 12

CONCLUSION

GUI prototyping represents an important activity for stimulating REI and ensuring that captured requirements reflect the actual needs of stakeholders (RVal) via tangible, visual prototypes. However, while effective, traditional GUI prototyping approaches are time-consuming and costly. Furthermore, verifying that the implemented GUI adheres to its specified requirements is another related challenge. While critically important, traditional GUI testing methods lack the ability to process complex NLR. The leading challenge behind automating these activities is the representation of the complex, unrestricted NLR on the one hand — which requires strong NLU capabilities — and the similarly complex GUI prototypes, encompassing multi-dimensional aspects and being highly dynamic. With the advent of powerful LLMs, which are effective across various NLP tasks, many automation approaches became feasible. Therefore, in this work, we explored the challenges of automated NLR-based GUI prototyping and verification mainly with LLM-based approaches. Specifically, we addressed the following seven sub-challenges:

(C1-1) NLR-based GUI Retrieval Gap: when considering automation of GUI prototyping from NLR, an initial solution approach is represented by employing IR methods. While existing research has proposed several GUI retrieval approaches (such as *Swire* (Huang et al., 2019) and *GUIFetch* (Behrang et al., 2018)), they largely focused on different input representations (such as GUI images and hand-drawn sketches), while neglecting NLR. Therefore, employing traditional IR methods (such as TF-IDF and BM25) provides potential. However, GUI prototypes require an effective textual representation for enabling IR methods. Moreover, there is a gap between NLR and textual representations of GUI prototypes, which represents a challenge for traditional text-based IR scoring models. To tackle this challenge, we proposed a BERT-LTR model, which we trained and finetuned on GUI relevance data with respect to NLR obtained through crowdsourcing platforms. By leveraging the PLM BERT — with its state-of-the-art NLU capabilities before the advent of LLMs — we effectively reduced the gap between text representations of NLR and GUI prototypes, as indicated by the effectiveness improvements on the novel GUI retrieval gold standard. Furthermore, the created dataset enabled the training of a model that learned improved relevance scoring for the particular GUI problem at hand.

(C1-2) *Semantic Representation Gap in GUI Retrieval and Reranking:* while the previously discussed BERT-LTR models reduced the gap between the NLR and textual GUI prototype representations to improve GUI ranking effectiveness, both representations rely on simplifications and therefore cannot fully capture their semantics. In particular, the GUI text representations utilized in the previous approach are based on an extraction method exploiting the GUI hierarchy data. While this method already enhances the text representation compared to relying solely on displayed text, it simultaneously neglects the multi-dimensional characteristics of GUI prototypes, such as functional and non-functional aspects, including GUI components and their inter-relationship, displayed texts, icons, layouts, and the overall design. In addition, the expressiveness of NLR is restricted, particularly with respect to negation or exclusion criteria, which are neglected by earlier approaches. Hence, the main challenge is represented by the semantic gap between NLR and multi-dimensional GUI prototypes. To tackle this challenge, we proposed an MLLM-based GUI reranking approach, which can utilize multi-dimensional, detailed descriptions for GUI prototypes derived by an MLLM, or GUI screenshots directly, to substantially improve the GUI ranking effectiveness, clearly outperforming BERT-LTR models. To integrate the multi-dimensional GUI representation and complex NLR already for the retrieval phase, we additionally proposed embedding-based constrained GUI retrieval. The proposed retrieval and reranking methods enable the processing of a semantically richer NLR and GUI prototype representation, with effectiveness gains shown for reranking. Finally, we integrated both approaches into a novel unified tool.

(C1-3) *GUI Prototyping Automation for Self-Elicitation:* the previously discussed NLR-based GUI retrieval and reranking approaches enable the rapid mapping of NLR to corresponding GUI prototypes. While the novel data-driven GUI prototyping tool *Ra Wi* facilitates prototyping via GUI retrieval, it lacks guidance for the elicitation process integrated with GUI prototyping. Furthermore, while previous research introduced the notion of SER (Rietz and Maedche, 2019), their approach focuses on the simple *laddering* interview technique (Corbridge et al., 1994) to enable stakeholders to uncover their own requirements. We previously established the importance of visual representations of the requirements in the form of GUI prototypes to mitigate ambiguity and misunderstandings. Hence, since their approach relies completely on NLR, it cannot profit from the benefits of prototyping. To enable automated SER integrated with the benefits of GUI prototyping, we proposed *SERGUI*, a novel approach that integrates GUI retrieval methods with a guided dialogue-based chat interface and LLM-based feature recommendations to stimulate elicitation. Moreover, users are enabled to conduct feature-level searches, integrate aspects from different GUIs into a coherent GUI prototype specification, and benefit from a GUI retrieval approach extended by feature-based reranking. We demonstrated the effectiveness of the approach through a user study, indicating high effectiveness for feature recommendation, feature visualization, and reranking, representing integral components for supporting SER with automated GUI prototyping. Our evaluation also indicates the high perceived usability of *SERGUI* based on the SUS.

(C1-4) *Efficient and Effective Adaptation of LLMs for NLR-based GUI Generation:* while GUI retrieval approaches provide a cost-efficient solution for challenge **C1**, they carry the disadvantage of providing static GUI prototypes with limited support for special NLR, requiring GUI adaptation. With the advent of LLMs – possessing impressive NLU and text generation capabilities – formulating the rapid mapping of NLR to GUI prototypes as a translation and text generation problem became feasible. Previous research proposed approaches to train LLMs from scratch (Brie et al., 2023) and finetune pretrained LLMs (Feng et al., 2023) to produce low-fidelity GUIs. However, these approaches solely support the creation of low-fidelity GUIs and require resource-intensive training or finetuning. Therefore, we investigated how to more efficiently optimize LLMs for effectively generating high-fidelity GUIs through several ZS-prompting-based methods. In particular, we proposed RAGG, which is a novel method that integrates previously developed GUI retrieval methods with the generative capabilities of modern LLMs, enabling the LLM to leverage relevant external GUI prototyping knowledge. Furthermore, we adapted SC for GUI prototyping, which employs the LLM itself in an iterative prototyping and feedback approach. Our large-scale evaluation, which is based on over 20,400 GUI annotations from 101 UI/UX experts, demonstrated the effectiveness of the proposed ZS prompting methods over several ZS baselines to efficiently improve LLM-based GUI generation.

(C1-5) *Efficient Adaptation of NLR-based GUI Generation via LLMs for Editable Prototypes:* while pretrained LLMs offer high effectiveness for generating GUI prototypes in common representation languages (such as HTML), which LLMs have been heavily exposed to during pretraining, these representations are not well suited for direct customization or editing, typically conducted in visual graphical prototyping editors. Moreover, these editors rely on proprietary GUI prototype representations, on which LLMs are not trained. Therefore, this raises the challenge of how to efficiently adapt LLMs to generate proprietary GUI representations that can directly be integrated into common GUI prototyping editors and workflows. To tackle this challenge, we proposed a novel two-stage RAG approach, which enables the efficient integration of external GUI component libraries by initially solely providing an abstracted library view to the LLM for selection, followed by retrieving the full GUI component specifications for selected components, which are employed for generating proprietary GUI representations that can directly be integrated into the popular prototyping environment *Figma*. Furthermore, we integrated this approach into an LLM-based prototyping assistant plugin, which is also capable of identifying the implementation status of requirements and matching components to respective GUI components. The evaluation of our approach demonstrated substantial prototyping effectiveness improvements, clearly outperforming prototyping workflows without LLM integration. Our evaluation demonstrated that the proposed two-stage RAG-based approach substantially increases total LLM token usage efficiency.

(C2-1) *NLR Fulfillment Verification in Static GUIs:* the discussed GUI retrieval and generation approaches facilitate rapid GUI prototyping, which benefits REI and RVal. However, another time-consuming and often manual activity is SVer, ensuring that the

implemented system adheres to its specification. Among verification activities, GUI testing represents a particularly effort-demanding task (Memon, 2001, 2007). Previous research mainly focused on improving automated GUI testing efficiency and coverage (Xie and Memon, 2007; Mao et al., 2016; Zeng et al., 2016), enabling the detection of system crashes. However, verifying the correctness of an implemented GUI application against a collection of complex NLR was neglected. Such testing activities are often still manual tasks or require automation scripts, which themselves are effort-demanding to create and require constant maintenance. Moreover, the advent of LLMs and continuous improvements in NLU capabilities provide large potential to address this challenge and increase automation. Therefore, to tackle this challenge, we conducted an initial feasibility study by employing LLM-based methods to automatically verify requirements implementation in static *Rico* GUIs. Our evaluation demonstrated high performance, with the approach achieving high effectiveness in identifying whether a specific requirement is implemented in a GUI and which GUI components correspond to that particular requirement.

(C2-2) NLR Fulfillment Verification in Fully-Fledged, Dynamic GUI Applications: while the previously discussed feasibility study showed promising results, it represents a simplified scenario with static *Rico* GUIs, which differ from real-world, highly dynamic GUI applications. Therefore, we extended the approach to tackle the verification of NLR for web-based GUI applications. In particular, we proposed an MLLM-based CUA approach for NLR-based GUI verification, which leverages the image understanding and reasoning capabilities of state-of-the-art MLLMs to autonomously predict meaningful interactions with the GUI, given the current trajectory consisting of GUI states (screenshots), reasoning, and predicted actions. In addition, we implemented an MCP server for enabling fully autonomous implementation-verification loops for LLM-based programming agents. Our evaluation demonstrates the effectiveness of MLLM-based CUA for GUI verification, achieving high scores at both the requirements and AC levels.

To summarize, in this thesis, we identified key challenges in automated NLR-based GUI prototyping and GUI verification, necessitating substantial NLU capabilities. We proposed novel GUI retrieval approaches based on PLMs (BERT) and improved the effectiveness with MLLM-based GUI reranking. Moreover, we investigated LLM-based approaches for GUI generation. Finally, we proposed an automated GUI verification approach via an MLLM-based CUA. An integral component of the proposed methods is the use of pretrained LLMs, enabling effective processing of NLR and GUI prototypes.

Given the work proposed in this thesis, important ideas for future work and potential improvements are identified. As presented in our discussion, while the pretrained LLMs are highly effective across the investigated problems, they simultaneously entail high costs. Reducing the costs associated with running these models and increasing their efficiency represents an important direction for future research. Adapting SLMs to improve task-specific performance (such as NLR-based GUI prototyping and GUI verification) – while acknowledging the reduction of more general capabilities of the model – offers substantial potential to enhance the efficiency while potentially maintaining high task effectiveness.

BIBLIOGRAPHY

1990. Ieee standard glossary of software engineering terminology. *IEEE Std 610.12-1990*, pages 1–84.
2020. Ergonomics of human-system interaction — part 220: Processes for enabling, executing and assessing human-centred design within organisations (iso 9241-220:2019); deutsche fassung en iso 9241-220:2019. Standard, 116 pages.
- Zahra Shakeri Hossein Abad, Sania Moazzam, Christina Lo, Tianhan Lan, Elis Frroku, and Heejun Kim. 2018. Loud and interactive paper prototyping in requirements elicitation: What is it good for? In *2018 IEEE 7th International Workshop on Empirical Requirements Engineering (EmpiRE)*, pages 16–23. IEEE.
- Omolola A Adedokun and Wilella D Burgess. 2012. Analysis of paired dichotomous data: A gentle introduction to the mcnemar test in spss. *Journal of MultiDisciplinary Evaluation*, 8(17):125–131.
- Adobe XD. 2015. Adobe xd. <https://www.adobe.com/de/products/xd.html>. Accessed: 2021-03-21.
- W Richards Adrion, Martha A Branstad, and John C Cherniavsky. 1982. Validation, verification, and testing of computer software. *ACM Computing Surveys (CSUR)*, 14(2):159–192.
- Ritu Agarwal and Mohan R Tanniru. 1990. Knowledge acquisition using structured interviewing: an empirical investigation. *Journal of Management Information Systems*, 7(1):123–140.
- Cem Akkaya, Alexander Conrad, Janyce Wiebe, and Rada Mihalcea. 2010. Amazon mechanical turk for subjectivity word sense disambiguation. In *Proceedings of the NAACL HLT 2010 workshop on creating speech and language data with Amazon’s Mechanical Turk*, pages 195–203.
- Ian F Alexander and Richard Stevens. 2002. *Writing better requirements*. Pearson Education.

- Domenico Amalfitano, Anna Rita Fasolino, Porfirio Tramontana, Salvatore De Carmine, and Atif M Memon. 2012. Using gui ripping for automated testing of android applications. In *Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering*, pages 258–261.
- Shun-ichi Amari. 1993. Backpropagation and stochastic gradient descent method. *Neurocomputing*, 5(4-5):185–196.
- Anthropic. 2025a. Claude 4. <https://www.anthropic.com/news/claude-4>. Accessed: 2025-12-18.
- Anthropic. 2025b. Claude models — overview documentation. <https://platform.claude.com/docs/en/about-claude/models/overview>. Accessed: 2025-12-04.
- Inc. Anywhere. 2025. Cursor. <https://cursor.com>. Accessed: 2025-12-18.
- Diego Martin Arroyo, Janis Postels, and Federico Tombari. 2021. Variational transformer networks for layout generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13642–13652.
- Hiteshwar Kumar Azad and Akshay Deepak. 2019. Query expansion techniques for information retrieval: A survey. *Information Processing & Management*, 56(5):1698–1735.
- Muneera Bano, Didar Zowghi, Alessio Ferrari, Paola Spoletini, and Beatrice Donati. 2019. Teaching requirements elicitation interviews: an empirical study of learning from mistakes. *Requirements Engineering*, 24(3):259–289.
- Dirk Baumer, W Bischofberger, Horst Lichter, and Heinz Zullighoven. 1996. User interface prototyping-concepts, tools, and experience. In *Proceedings of IEEE 18th International Conference on Software Engineering*, pages 532–541. IEEE.
- Michel Beaudouin-Lafon and WE Mackay. 2002. Prototyping development and tools. *Handbook of Human-Computer Interaction*, pages 1006–1031.
- Kent Beck. 2000. *Extreme programming explained: embrace change*. addison-wesley professional.
- Farnaz Behrang, Steven P Reiss, and Alessandro Orso. 2018. Guifetch: supporting app design and development through gui search. In *Proceedings of the 5th International Conference on Mobile Software Engineering and Systems*, pages 236–246.
- Carlos Bernal-Cárdenas, Kevin Moran, Michele Tufano, Zichang Liu, Linyong Nan, Zhehan Shi, and Denys Poshyvanyk. 2019. Guigle: a gui search engine for android apps. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pages 71–74. IEEE.

BIBLIOGRAPHY

- Niels Ole Bernsen, Hans Dybkjær, and Laila Dybkjær. 1994. Wizard of oz prototyping: How and when. *Proc. CCI Working Papers Cognit. Sci./HCI, Roskilde, Denmark*, page 67.
- James Betker, Gabriel Goh, Li Jing, Tim Brooks, Jianfeng Wang, Linjie Li, Long Ouyang, Juntang Zhuang, Joyce Lee, Yufei Guo, et al. 2023. Improving image generation with better captions. *Computer Science*. <https://cdn.openai.com/papers/dall-e-3.pdf>, 2(3):8.
- Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, et al. 2024. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954*.
- Hafiz Anas Bilal, Muhammad Ilyas, Qandeel Tariq, and Muhammad Hummayun. 2016. Requirements validation techniques: An empirical study. *International Journal of Computer Applications*, 148(14).
- Christopher M Bishop. 1994. Mixture density networks.
- Christopher M Bishop and Nasser M Nasrabadi. 2006. *Pattern recognition and machine learning*, volume 4. Springer.
- Elizabeth Bjarnason, Krzysztof Wnuk, and Björn Regnell. 2011. Requirements are slipping through the gaps—a case study on causes & effects of communication gaps in large-scale software development. In *2011 IEEE 19th international requirements engineering conference*, pages 37–46. IEEE.
- Barry Boehm. 2007. A survey of agile development methodologies. *Laurie Williams*, 45:119.
- Barry W Boehm. 1984a. Software engineering economics. *IEEE transactions on Software Engineering*, (1):4–21.
- Barry W. Boehm. 1984b. Verifying and validating software requirements and design specifications. *IEEE software*, 1(1):75.
- Barry W Boehm and Philip N. Papaccio. 1988. Understanding and controlling software costs. *IEEE transactions on software engineering*, 14(10):1462–1477.
- Paul Brie, Nicolas Burny, Arthur Sluÿters, and Jean Vanderdonckt. 2023. Evaluating a large language model on searching for gui layouts. *Proceedings of the ACM on Human-Computer Interaction*, 7(EICS):1–37.
- John Brooke. 1996a. SUS: A 'Quick and Dirty' Usability Scale. In Patrick W. Jordan, B. Thomas, Ian Lyall McClelland, and Bernard Weerdmeester, editors, *Usability Evaluation In Industry*, 0 edition, pages 207–212. CRC Press, London, England.
- John Brooke. 1996b. Sus: a "quick and dirty" usability. *Usability evaluation in industry*, 189.

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Sebastian Bruch, Xuanhui Wang, Michael Bendersky, and Marc Najork. 2019. An analysis of the softmax cross entropy loss for learning-to-rank with binary relevance. In *Proceedings of the 2019 ACM SIGIR international conference on theory of information retrieval*, pages 75–78.
- Sara Bunian, Kai Li, Chaima Jemmali, Casper Hartevelt, Yun Fu, and Magy Seif El-Nasr. 2021. Vins: Visual search for mobile user interface design. *arXiv preprint arXiv:2102.05216*.
- Jose Cambronero, Hongyu Li, Seohyun Kim, Koushik Sen, and Satish Chandra. 2019. When deep learning met code search. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 964–974.
- Zhe Cao, Tao Qin, Tie-Yan Liu, Ming-Feng Tsai, and Hang Li. 2007. Learning to rank: from pairwise approach to listwise approach. In *Proceedings of the 24th international conference on Machine learning*, pages 129–136.
- Pär Carlshamre and Joachim Karlsson. 1996. A usability-oriented approach to requirements engineering. In *Proceedings of the Second International Conference on Requirements Engineering*, pages 145–152. IEEE.
- Claudio Carpineto, Renato De Mori, Giovanni Romano, and Brigitte Bigi. 2001. An information-theoretic approach to automatic query expansion. *ACM Transactions on Information Systems (TOIS)*, 19(1):1–27.
- Erin A. Carroll, Celine Latulipe, Richard Fung, and Michael Terry. 2009. Creativity factor evaluation: towards a standardized survey metric for creativity support. In *Proceedings of the seventh ACM conference on Creativity and cognition*, pages 127–136, Berkeley California USA. ACM.
- Vitor R Carvalho, Matthew Lease, and Emine Yilmaz. 2011. Crowdsourcing for search evaluation. In *ACM Sigir forum*, volume 44, pages 17–22. ACM New York, NY, USA.
- Celery. 2025. Celery documentation: Introduction. <https://docs.celeryq.dev/en/main/getting-started/introduction.html>. Accessed: 2025-12-18.
- Chunyang Chen, Sidong Feng, Zhenchang Xing, Linda Liu, Shengdong Zhao, and Jinshui Wang. 2019a. Gallery dc: Design search and knowledge discovery through auto-created gui component gallery. *Proceedings of the ACM on Human-Computer Interaction*, 3(CSCW):1–22.

BIBLIOGRAPHY

- Chunyang Chen, Ting Su, Guozhu Meng, Zhenchang Xing, and Yang Liu. 2018. From ui design image to gui skeleton: a neural machine translator to bootstrap mobile gui implementation. In *Proceedings of the 40th International Conference on Software Engineering*, pages 665–676. ACM.
- Jieshan Chen, Chunyang Chen, Zhenchang Xing, Xin Xia, Liming Zhu, John Grundy, and Jinshui Wang. 2020. Wireframe-based ui design search through image autoencoder. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 29(3):1–31.
- Xiangping Chen, Qiwen Zou, Bitian Fan, Zibin Zheng, and Xiaonan Luo. 2019b. Recommending software features for mobile applications based on user interface comparison. *Requirements Engineering*, 24:545–559.
- Zhanpeng Chen, Chengjin Xu, Yiyang Qi, and Jian Guo. 2024. Mllm is a strong reranker: Advancing multimodal retrieval-augmented generation via knowledge-enhanced reranking and noise-injected training. *arXiv preprint arXiv:2407.21439*.
- Davide Chicco. 2021. Siamese neural networks: An overview. *Artificial neural networks*, pages 73–94.
- Mike Cohn. 2004. *User stories applied: For agile software development*. Addison-Wesley Professional.
- C Corbridge, G Rugg, Nigel P Major, NR Shadbolt, and AM Burton. 1994. Laddering: technique and tool use in knowledge acquisition. *Knowledge Acquisition*, 6(3):315–341.
- Adrien Coyette, Suzanne Kieffer, and Jean Vanderdonckt. 2007. Multi-fidelity prototyping of user interfaces. In *IFIP Conference on Human-Computer Interaction*, pages 150–164. Springer.
- Adrien Coyette, Jean Vanderdonckt, and Quentin Limbourg. 2006. Sketchixml: a design tool for informal user interface rapid prototyping. In *International Workshop on Rapid Integration of Software Engineering Techniques*, pages 160–176. Springer.
- Fabiano Dalpiaz and Sjaak Brinkkemper. 2018. Agile requirements engineering with user stories. In *2018 IEEE 26th International Requirements Engineering Conference (RE)*, pages 506–507. IEEE.
- Hafsa Dar, M Ikramullah Lali, Humaira Ashraf, Muhammad Ramzan, Tehmina Amjad, and Basit Shahzad. 2018. A systematic study on software requirements elicitation techniques and its challenges in mobile application development. *IEEE Access*, 6:63859–63867.
- Alan Davis, Oscar Dieste, Ann Hickey, Natalia Juristo, and Ana M Moreno. 2006. Effectiveness of requirements elicitation techniques: Empirical results derived from a systematic review. In *14th IEEE International Requirements Engineering Conference (RE'06)*, pages 179–188. IEEE.

- Richard C Davis, T Scott Saponas, Michael Shilman, and James A Landay. 2007. Sketch-wizard: Wizard of oz prototyping of pen-based user interfaces. In *Proceedings of the 20th annual ACM symposium on User interface software and technology*, pages 119–128.
- Fabian de Bruijn and Hans L Dekkers. 2010. Ambiguity in natural language software requirements: A case study. In *International Working Conference on Requirements Engineering: Foundation for Software Quality*, pages 233–247. Springer.
- Sourav Debnath, Paola Spoletini, and Alessio Ferrari. 2021a. From Ideas to Expressed Needs: an Empirical Study on the Evolution of Requirements during Elicitation . In *2021 IEEE 29th International Requirements Engineering Conference (RE)*, pages 233–244, Los Alamitos, CA, USA. IEEE Computer Society.
- Sourav Debnath, Paola Spoletini, and Alessio Ferrari. 2021b. From ideas to expressed needs: an empirical study on the evolution of requirements during elicitation. In *2021 IEEE 29th International Requirements Engineering Conference (RE)*, pages 233–244. IEEE.
- Biplab Deka, Zifeng Huang, Chad Franzen, Joshua Hibsman, Daniel Afegan, Yang Li, Jeffrey Nichols, and Ranjitha Kumar. 2017. Rico: A mobile app dataset for building data-driven design applications. In *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology*, pages 845–854.
- Biplab Deka, Zifeng Huang, and Ranjitha Kumar. 2016. Erica: Interaction mining mobile apps. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology*, pages 767–776.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186.
- Oxford English Dictionary. 1989. Oxford english dictionary. *Simpson, Ja & Weiner, Esc*, 3.
- Djellel Difallah, Elena Filatova, and Panos Ipeirotis. 2018. Demographics and dynamics of mechanical turk workers. In *Proceedings of the eleventh ACM international conference on web search and data mining*, pages 135–143.
- Chenhe Dong, Yinghui Li, Haifan Gong, Miaoxin Chen, Junxin Li, Ying Shen, and Min Yang. 2022. A survey of natural language generation. *ACM Computing Surveys*, 55(8):1–38.
- Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Baobao Chang, et al. 2024. A survey on in-context learning. In

BIBLIOGRAPHY

- Proceedings of the 2024 conference on empirical methods in natural language processing*, pages 1107–1128.
- Benjamin D Douglas, Patrick J Ewell, and Markus Brauer. 2023. Data quality in on-line human-subjects research: Comparisons between mturk, prolific, cloudresearch, qualtrics, and sona. *Plos one*, 18(3):e0279720.
- M José Escalona and Nora Koch. 2003. Requirements engineering for web applications—a comparative study. *Journal of web Engineering*, pages 193–212.
- Scott Faranello. 2012. *Balsamiq wireframes quickstart guide*. Packt Publishing Ltd.
- Sidong Feng and Chunyang Chen. 2022. Gifdroid: Automated replay of visual bug reports for android apps. In *Proceedings of the 44th International Conference on Software Engineering*, pages 1045–1057.
- Sidong Feng, Mingyue Yuan, Jieshan Chen, Zhenchang Xing, and Chunyang Chen. 2023. Designing with language: Wireframing ui design intent with generative large language models. *arXiv preprint arXiv:2312.07755*.
- Sidong Feng et al. 2024. Enabling cost-effective ui automation testing with retrieval-based llms: A case study in wechat.
- Sidong Feng et al. 2025. Agent for user: Testing multi-user interactive features in tiktok. *arXiv preprint arXiv:2504.15474*.
- Alessio Ferrari, Paola Spoletini, and Sourav Debnath. 2022. How do requirements evolve during elicitation? an empirical study combining interviews and app store analysis. *Requirements Engineering*, 27(4):489–519.
- Alessio Ferrari, Paola Spoletini, and Stefania Gnesi. 2016. Ambiguity and tacit knowledge in requirements elicitation interviews. *Requirements Engineering*, 21(3):333–355.
- António MS Ferreira, Alberto Rodrigues da Silva, and Ana CR Paiva. 2022. Towards the art of writing agile requirements with user stories, acceptance criteria, and related constructs. In *ENASE*, pages 477–484.
- Figma. 2016. Figma. <https://www.figma.com/>. Accessed: 2021-03-21.
- Figma, Inc.. 2024. Figma. <https://www.figma.com>. Accessed: 2024-09-11.
- Donald Firesmith. 2007. Common requirements problems, their negative consequences, and the industry best practices to help solve them. *J. Object Technol.*, 6(1):17–33.
- William Foddy and William H Foddy. 1993. *Constructing questions for interviews and questionnaires: Theory and practice in social research*. Cambridge university press.
- Kevin Forsberg and Harold Mooz. 1991. The relationship of system engineering to the project cycle. *Center for Systems Management*, 5333:4–6.

- Martin Fowler, Jim Highsmith, et al. 2001. The agile manifesto. *Software development*, 9(8):28–35.
- Lukas Galke, Ahmed Saleh, and Ansgar Scherp. 2017. Word embeddings for practical information retrieval. *INFORMATIK 2017*.
- Donald C Gause and Gerald M Weinberg. 1989. Exploring requirements. *Dorset House*, 249.
- Tom Gilb and Dorothy Graham. 1993. *Software inspections*. Addison-Wesley Reading, Massachusetts.
- Inc. GitHub. 2025. Github copilot. <https://github.com/features/copilot>. Accessed: 2025-12-18.
- Martin Glinz. 2011. A glossary of requirements engineering terminology. *Standard Glossary of the Certified Professional for Requirements Engineering (CPRE) Studies and Exam, Version*, 1:56.
- Martin Glinz and Roel J Wieringa. 2007. Guest editors’ introduction: Stakeholders in requirements engineering. *IEEE software*, 24(2):18–20.
- Joseph A Goguen and Charlotte Linde. 1993. Techniques for requirements elicitation. In *[1993] Proceedings of the IEEE International Symposium on Requirements Engineering*, pages 152–164. IEEE.
- Pavel Golik, Patrick Doetsch, and Hermann Ney. 2013. Cross-entropy vs. squared error training: a theoretical and experimental comparison. In *Interspeech*, volume 13, pages 1756–1760.
- Lorenzo Gomez et al. 2013. Reran: Timing-and touch-sensitive record and replay for android. In *2013 35th international conference on software engineering (ICSE)*, pages 72–81. IEEE.
- Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2020. Generative adversarial networks. *Communications of the ACM*, 63(11):139–144.
- Google. 2025a. Gemini api — supported models overview (gemini 2.5 pro). <https://ai.google.dev/gemini-api/docs/models?hl=de#gemini-2.5-pro>. Accessed: 2025-12-04.
- Google. 2025b. Gemini api — supported models overview (gemini 3 pro). <https://ai.google.dev/gemini-api/docs/models?hl=de#gemini-3-pro>. Accessed: 2025-12-04.
- Google LLC. 2025. Material 3 design kit. <https://m3.material.io/blog/material-3-figma-design-kit>. Accessed: 2025-01-20.

BIBLIOGRAPHY

- Saul Greenberg. 1996. Teaching human computer interaction to programmers. *Interactions*, 3(4):62–76.
- Jiuxiang Gu, Zhenhua Wang, Jason Kuen, Lianyang Ma, Amir Shahroudy, Bing Shuai, Ting Liu, Xingxing Wang, Gang Wang, Jianfei Cai, et al. 2018a. Recent advances in convolutional neural networks. *Pattern recognition*, 77:354–377.
- Tianxiao Gu et al. 2019. Practical gui testing of android applications via model abstraction and refinement. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 269–280. IEEE.
- Xiaodong Gu, Hongyu Zhang, and Sunghun Kim. 2018b. Deep code search. In *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*, pages 933–944. IEEE.
- Kamal Gupta, Justin Lazarow, Alessandro Achille, Larry S Davis, Vijay Mahadevan, and Abhinav Shrivastava. 2021. Layouttransformer: Layout generation and completion with self-attention. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1004–1014.
- Shuguang Han, Xuanhui Wang, Mike Bendersky, and Marc Najork. 2020. Learning-to-rank with bert in tf-ranking. *arXiv preprint arXiv:2004.08476*.
- Zellig S Harris. 1954. Distributional structure. *Word*, 10(2-3):146–162.
- Sandra G. Hart. 2006. Nasa-Task Load Index (NASA-TLX); 20 Years Later. *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, 50(9):904–908.
- H Rex Hartson and Eric C Smith. 1991. Rapid prototyping in human-computer interface development. *Interacting with computers*, 3(1):51–91.
- Muhammad Haseeb. 2025. Context engineering for multi-agent llm code assistants using elicit, notebooklm, chatgpt, and claude code. *arXiv preprint arXiv:2508.08322*.
- James H Hicinbothom and Wayne W Zachary. 1993. A tool for automatically generating transcripts of human-computer interaction. In *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, volume 37, pages 1042–1042. SAGE Publications Sage CA: Los Angeles, CA.
- Ann Hickey and Douglas Dean. 1998. Prototyping for requirements elicitation and validation: A participative prototype evaluation methodology.
- Ann M Hickey and Alan M Davis. 2003. Elicitation technique selection: how do experts do it? In *Proceedings. 11th IEEE International Requirements Engineering Conference, 2003.*, pages 169–178. IEEE.
- Jim Highsmith. 2013. *Adaptive software development: a collaborative approach to managing complex systems*. Addison-Wesley.

- David N Hinkle. 1965. The change of personal constructs from the viewpoint of a theory of implications. *Unpublished doctoral dissertation, Ohio State University*.
- Yaoshiang Ho and Samuel Wookey. 2019. The real-world-weight cross-entropy loss function: Modeling the costs of mislabeling. *IEEE access*, 8:4806–4813.
- Xinyi Hou et al. 2025. Model context protocol (mcp): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*.
- Frank Houdek and Klaus Pohl. 2000. Analyzing requirements engineering processes: a case study. In *Proceedings 11th International Workshop on Database and Expert Systems Applications*, pages 983–987. IEEE.
- Forrest Huang, John F Canny, and Jeffrey Nichols. 2019. Swire: Sketch-based user interface retrieval. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pages 1–10.
- Forrest Huang, Gang Li, Xin Zhou, John F Canny, and Yang Li. 2021. Creating user interface mock-ups from high-level text descriptions with deep-learning models. *arXiv preprint arXiv:2110.07775*.
- Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55.
- Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*.
- Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436*.
- Paul Jaccard. 1901. Étude comparative de la distribution florale dans une portion des alpes et des jura. *Bull Soc Vaudoise Sci Nat*, 37:547–579.
- Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM computing surveys*, 55(12):1–38.
- Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515*.
- Conny Johansson and Christian Bucanac. 1999. The v-model. *IDE, University Of Karlskrona, Ronneby*.

BIBLIOGRAPHY

- K Sparck Jones and Cornelis Joost Van Rijsbergen. 1976. Information retrieval test collections. *Journal of documentation*.
- T Capers Jones. 2007. *Estimating software costs*. McGraw-Hill, Inc.
- Adam Tauman Kalai, Ofir Nachum, Santosh S Vempala, and Edwin Zhang. 2025. Why language models hallucinate. *arXiv preprint arXiv:2509.04664*.
- Massila Kamalrudin and John Grundy. 2011. Generating essential user interface prototypes to validate requirements. In *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*, pages 564–567. IEEE.
- Erik Kamsties. 2005. Understanding ambiguity in requirements engineering. In *Engineering and Managing Software Requirements*, pages 245–266. Springer.
- Erik Kamsties and Barbara Peach. 2000. Taming ambiguity in natural language requirements. In *Proceedings of the Thirteenth international conference on Software and Systems Engineering and Applications*, volume 1315.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.
- David J Kasik. 1982. A user interface management system. In *Proceedings of the 9th annual conference on Computer graphics and interactive techniques*, pages 99–106.
- Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2022. Decomposed prompting: A modular approach for solving complex tasks. *arXiv preprint arXiv:2210.02406*.
- Nadzeja Kiyavitskaya, Nicola Zeni, Luisa Mich, and Daniel M Berry. 2008. Requirements for tools for ambiguity identification and measurement in natural language requirements specifications. *Requirements engineering*, 13(3):207–239.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213.
- Kristian Kolthoff. 2019. Automatic generation of graphical user interface prototypes from unrestricted natural language requirements. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1234–1237. IEEE.
- Kristian Kolthoff, Christian Bartelt, and Simone Paolo Ponzetto. 2020. Gui2wire: Rapid wireframing with a mined and large-scale gui repository using natural language requirements. In *35th IEEE/ACM International Conference on Automated Software Engineering (ASE '20)*. ACM.

- Kristian Kolthoff, Christian Bartelt, and Simone Paolo Ponzetto. 2021. Automated retrieval of graphical user interface prototypes from natural language requirements. In *International Conference on Applications of Natural Language to Information Systems*, pages 376–384. Springer.
- Kristian Kolthoff, Christian Bartelt, and Simone Paolo Ponzetto. 2023. Data-driven prototyping via natural-language-based gui retrieval. *Automated Software Engineering*, 30(1):13.
- Kristian Kolthoff, Christian Bartelt, Simone Paolo Ponzetto, and Kurt Schneider. 2024a. Self-elicitation of requirements with automated gui prototyping. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 2354–2357.
- Kristian Kolthoff, Felix Kretzer, Christian Bartelt, Alexander Maedche, and Simone Paolo Ponzetto. 2025a. Gui-rerank: Enhancing gui retrieval with multi-modal llm-based reranking.
- Kristian Kolthoff, Felix Kretzer, Lennart Fiebig, Christian Bartelt, Alexander Maedche, and Simone Paolo Ponzetto. 2024b. Zero-shot prompting approaches for llm-based graphical user interface generation. *arXiv preprint arXiv:2412.11328*.
- Kristian Kolthoff, Felix Kretzer, Simone Paolo Ponzetto, Alexander Maedche, and Christian Bartelt. 2025b. Guispector: An mllm agent framework for automated verification of natural language requirements in gui prototypes. *arXiv preprint arXiv:2510.04791*.
- Kristian Kolthoff et al. 2024c. Interlinking user stories and gui prototyping: A semi-automatic llm-based approach. In *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, pages 380–388. IEEE.
- Kristian Kolthoff et al. 2025c. Guide: Llm-driven gui generation decomposition for automated prototyping. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pages 1–4. IEEE.
- Konva.js. 2015. Konva.js - html5 2d canvas js library for desktop and mobile applications. <https://konvajs.org/>. Accessed: 2020-03-23.
- Fabrice Kordon et al. 2002. An introduction to rapid system prototyping. *IEEE Transactions on Software Engineering*, 28(9):817–821.
- Gerald Kotonya and Ian Sommerville. 1998. *Requirements engineering: processes and techniques*. Wiley Publishing.
- Helena Chmura Kraemer, David J Kupfer, Diana E Clarke, William E Narrow, and Darrel A Regier. 2012. Dsm-5: how reliable is reliable enough? *American Journal of Psychiatry*, 169(1):13–15.

BIBLIOGRAPHY

- Felix Kretzer, Kristian Kolthoff, et al. 2025. Closing the loop between user stories and gui prototypes: an llm-based assistant for cross-functional integration in software development. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–19.
- Klaus Krippendorff. 2011. Computing krippendorff’s alpha-reliability.
- Solomon Kullback. 1951. Kullback-leibler divergence. *Tech. Rep.*
- Peter A Lachenbruch. 2014. McNemar test. *Wiley StatsRef: Statistics Reference Online*.
- Oliver Laitenberger and Jean-Marc DeBaud. 2000. An encompassing life cycle centric survey of software inspection. *Journal of systems and software*, 50(1):5–31.
- Frederick Wilfrid Lancaster and Emily Gallup. 1973. Information retrieval on-line. Technical report.
- James A Landay and Brad A Myers. 1994. Interactive sketching for the early stages of user interface design. Technical report, Carnegie-Mellon Univ Pittsburgh PA Dept of Computer Science.
- James A Landay and Brad A Myers. 1995. Interactive sketching for the early stages of user interface design. In *Proceedings of the SIGCHI conference on Human factors in computing systems*, pages 43–50.
- J Richard Landis and Gary G Koch. 1977. The measurement of observer agreement for categorical data. *biometrics*, pages 159–174.
- Anne Lauscher. 2021. Language representations for computational argumentation.
- Chunggi Lee, Sanghoon Kim, Dongyun Han, Hongjun Yang, Young-Woo Park, Bum Chul Kwon, and Sungahn Ko. 2020. Guicomp: A gui design assistant with real-time, multi-faceted feedback. *arXiv preprint arXiv:2001.05684*.
- Luis A Leiva, Asutosh Hota, and Antti Oulasvirta. 2020. Enrico: A dataset for topic modeling of mobile ui designs. In *22nd International Conference on Human-Computer Interaction with Mobile Devices and Services*, pages 1–4.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474.
- Huayang Li, Yixuan Su, Deng Cai, Yan Wang, and Lemao Liu. 2022. A survey on retrieval-augmented text generation. *arXiv preprint arXiv:2202.01110*.
- Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhui Chen. 2024. Long-context llms struggle with long in-context learning. *arXiv preprint arXiv:2404.02060*.

- Toby Jia-Jun Li, Lindsay Popowski, Tom Mitchell, and Brad A Myers. 2021. Screen2vec: Semantic embedding of gui screens and gui components. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pages 1–15.
- James Lin, Mark W Newman, Jason I Hong, and James A Landay. 2000. Denim: Finding a tighter fit between tools and practice for web site design. In *Proceedings of the SIGCHI conference on Human factors in computing systems*, pages 510–517.
- Aldo Lipani. 2016. Fairness in information retrieval. In *Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval*, pages 1171–1171.
- Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35.
- Xiao Liu, Fanjin Zhang, Zhenyu Hou, Li Mian, Zhaoyu Wang, Jing Zhang, and Jie Tang. 2021. Self-supervised learning: Generative or contrastive. *IEEE transactions on knowledge and data engineering*, 35(1):857–876.
- R Lo, Richard Webby, and R Jeffery. 1996. Sizing and estimating the coding and unit testing effort for gui systems. In *Proceedings of the 3rd International Software Metrics Symposium*, pages 166–173. IEEE.
- Jieyi Long. 2023. Large language model guided tree-of-thought. *arXiv preprint arXiv:2305.08291*.
- Pericles Loucopoulos and Vassilios Karakostas. 1995. *System requirements engineering*, volume 199. McGraw-Hill London.
- Haoyu Lu, Wen Liu, Bo Zhang, Bingxuan Wang, Kai Dong, Bo Liu, Jingxiang Sun, Tongzheng Ren, Zhuoshu Li, Hao Yang, et al. 2024. Deepseek-vl: towards real-world vision-language understanding. *arXiv preprint arXiv:2403.05525*.
- Garm Lucassen, Fabiano Dalpiaz, Jan Martijn EM van der Werf, and Sjaak Brinkkemper. 2016. The use and effectiveness of user stories in practice. In *International working conference on requirements engineering: Foundation for software quality*, pages 205–222. Springer.
- Lucene. 2011. Apache lucene. <https://lucene.apache.org/>. Accessed: 2022-08-31.
- Fei Lv, Hongyu Zhang, Jian-guang Lou, Shaowei Wang, Dongmei Zhang, and Jianjun Zhao. 2015. Codehow: Effective code search based on api understanding and extended boolean model (e). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 260–270. IEEE.
- Linda A Macaulay. 2012. *Requirements engineering*. Springer Science & Business Media.

BIBLIOGRAPHY

- Guillermo Macbeth, Eugenia Razumiejczyk, and Rubén Daniel Ledesma. 2011. Cliff's delta calculator: A non-parametric effect size program for two groups of observations. *Universitas Psychologica*, 10(2):545–555.
- Christopher D Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to information retrieval*. Cambridge university press.
- Markus Mannio and Uolevi Nikula. 2001. Requirements elicitation using a combination of prototypes and scenarios. In *WER*, pages 283–296.
- Ke Mao, Mark Harman, and Yue Jia. 2016. Sapienz: Multi-objective automated testing for android applications. In *Proceedings of the 25th international symposium on software testing and analysis*, pages 94–105.
- Brian Marick. 1998. When should a test be automated. *Proceedings of The 11th International Software/Internet Quality Week*, pages 1–20.
- Ggaliwango Marvin, Nakayiza Hellen, Daudi Jjingo, and Joyce Nakatumba-Nabende. 2023. Prompt engineering in large language models. In *International conference on data intelligence and cognitive informatics*, pages 387–402. Springer.
- Michael McCurdy, Christopher Connors, Guy Pyrzak, Bob Kanefsky, and Alonso Vera. 2006. Breaking the fidelity barrier: an examination of our current characterization of prototypes and an example of a mixed-fidelity success. In *Proceedings of the SIGCHI conference on Human Factors in computing systems*, pages 1233–1242.
- James E McDonald, Paul DJ Vandenberg, and Melissa J Smartt. 1988. The mirage rapid interface prototyping system. In *Proceedings of the 1st annual ACM SIGGRAPH symposium on User Interface Software*, pages 77–84.
- Collin McMillan, Mark Grechanik, Denys Poshyvanyk, Chen Fu, and Qing Xie. 2011. Exemplar: A source code search engine for finding highly relevant applications. *IEEE Transactions on Software Engineering*, 38(5):1069–1087.
- Atif M Memon. 2003. Advances in gui testing. *Adv. Comput.*, 58:149–201.
- Atif M Memon. 2007. An event-flow model of gui-based applications for testing. *Software testing, verification and reliability*, 17(3):137–157.
- Atif Muhammed Memon. 2001. *A comprehensive framework for testing graphical user interfaces*. University of Pittsburgh.
- Meta. 2025a. Llama 2 — meta llama models. <https://www.llama.com/llama2/>. Accessed: 2025-12-04.
- Meta. 2025b. Llama 3 — meta llama models. <https://www.llama.com/models/llama-3/>. Accessed: 2025-12-04.

- Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013a. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.
- Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013b. Distributed representations of words and phrases and their compositionality. In *Advances in neural information processing systems*, pages 3111–3119.
- minGPT - GitHub Repository. mingpt - github repository. <https://github.com/karpathy/minGPT>. Accessed: 2024-07-20.
- Tom M Mitchell. 1997. Does machine learning really work? *AI magazine*, 18(3):11–11.
- Mockplus. 2014. Mockplus. <https://www.mockplus.com/>. Accessed: 2021-03-21.
- J Michael Moore and Frank M Shipman. 2000. A comparison of questionnaire-based and gui-based requirements gathering. In *Proceedings ASE 2000. Fifteenth IEEE International Conference on Automated Software Engineering*, pages 35–43. IEEE.
- Kevin Moran, Carlos Bernal-Cárdenas, Michael Curcio, Richard Bonett, and Denys Poshyvanyk. 2018. Machine learning-based prototyping of graphical user interfaces for mobile apps. *arXiv preprint arXiv:1802.02312*.
- Parisa Moslehi, Bram Adams, and Juergen Rilling. 2020. A feature location approach for mapping application features extracted from crowd-based screencasts to source code. *Empirical Software Engineering*, 25(6):4873–4926.
- Abdelrahman Abdallah Bhawna Piryani Jamshid Mozafari and Mohammed Ali Adam Jatowt. 2025. How good are llm-based rerankers? an empirical analysis of state-of-the-art reranking models.
- Kizito Ssamula Mukasa and Hermann Kaindl. 2008. An integration of requirements and user interface specifications. In *2008 16th IEEE International Requirements Engineering Conference*, pages 327–328. IEEE.
- Glenford J Myers, Tom Badgett, Todd M Thomas, and Corey Sandler. 2004. *The art of software testing*, volume 2. Wiley Online Library.
- MySQL. 2025. Mysql server. <https://github.com/mysql/mysql-server>. Accessed: 2025-12-18.
- Bashar Nuseibeh and Steve Easterbrook. 2000. Requirements engineering: a roadmap. In *Proceedings of the Conference on the Future of Software Engineering*, pages 35–46.
- Peter O’Donovan, Aseem Agarwala, and Aaron Hertzmann. 2015. Designscape: Design with interactive layout suggestions. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*, CHI ’15, page 1221–1224, New York, NY, USA. Association for Computing Machinery.

BIBLIOGRAPHY

- Dan Olteanu, Holger Meuss, Tim Furche, and François Bry. 2002. Xpath: looking forward. In *International Conference on Extending Database Technology*, pages 109–127. Springer.
- OpenAI. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- OpenAI. 2024. Gpt-4o. <https://openai.com/index/hello-gpt-4o/>. Accessed: 2024-09-11.
- OpenAI. 2025a. Codex. <https://openai.com/codex/>. Accessed: 2025-12-18.
- OpenAI. 2025b. Cua: Introducing a universal interface for ai to interact with the digital world.
- OpenAI. 2025c. Gpt-4.1. <https://openai.com/index/gpt-4-1/>. Accessed: 2025-12-18.
- OpenAI. 2025d. Openai platform documentation: Gpt-4.1. <https://platform.openai.com/docs/models/gpt-4.1>. Accessed: 2025-12-04.
- OpenAI. 2025e. Openai platform documentation: Gpt-4o. <https://platform.openai.com/docs/models/gpt-4o>. Accessed: 2025-12-04.
- OpenAI. 2025f. Openai platform documentation: Gpt-5. <https://platform.openai.com/docs/models/gpt-5>. Accessed: 2025-12-04.
- Dillon Otto, Tanner Rowlett, and Stefan Nagy. 2025. Guifuzz++: Unleashing grey-box fuzzing on desktop graphical user interfacing applications.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744.
- Peter O’Donovan, Aseem Agarwala, and Aaron Hertzmann. 2014. Learning layouts for single-pagegraphic designs. *IEEE transactions on visualization and computer graphics*, 20(8):1200–1213.
- Carla Pacheco, Ivan García, and Miryam Reyes. 2018. Requirements elicitation techniques: a systematic literature review based on the maturity of the techniques. *IET Software*, 12(4):365–378.
- Stefan Palan and Christian Schitter. 2018. Prolific. ac—a subject pool for online experiments. *Journal of behavioral and experimental finance*, 17:22–27.
- Srishti Palani, David Ledo, George Fitzmaurice, and Fraser Anderson. 2022. ”i don’t want to feel like i’m working in a 1960s factory”: The practitioner perspective on creativity support tool adoption. In *Proceedings of the 2022 CHI Conference on Human Factors*

- in Computing Systems*, CHI '22, New York, NY, USA. Association for Computing Machinery.
- Gabriele Paolacci, Jesse Chandler, and Panagiotis G Ipeirotis. 2010. Running experiments on amazon mechanical turk. *Judgment and Decision making*, 5(5):411–419.
- Eyal Peer, Joachim Vosgerau, and Alessandro Acquisti. 2014. Reputation as a sufficient condition for data quality on amazon mechanical turk. *Behavior research methods*, 46(4):1023–1031.
- Nina Phan, Peter Bailey, and Ross Wilkinson. 2007. Understanding the relationship of information need specificity to search query length. In *Proceedings of the 30th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 709–710.
- Klaus Pohl. 1996. *Requirements engineering: An overview*. RWTH, Fachgruppe Informatik Aachen.
- Klaus Pohl. 1997. *Process-centered requirements engineering*. John Wiley & Sons, Inc.
- Klaus Pohl. 2010. *Requirements engineering: fundamentals, principles, and techniques*. Springer Publishing Company, Incorporated.
- Klaus Pohl. 2016. *Requirements engineering fundamentals: a study guide for the certified professional for requirements engineering exam-foundation level-IREB compliant*. Rocky Nook, Inc.
- Hendrik Post, Carsten Sinz, Florian Merz, Thomas Gorges, and Thomas Kropf. 2009. Linking functional requirements and software verification. In *2009 17th IEEE International Requirements Engineering Conference*, pages 295–302. IEEE.
- Prolific Academic Ltd. Prolific.
- Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, et al. 2024. Large language models are effective text rankers with pairwise ranking prompting. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 1504–1518.
- Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.
- Steven Rakitin. 2001. *Software verification and validation for practitioners and managers*. Artech.

BIBLIOGRAPHY

- Veenu Rani, Syed Tufael Nabi, Munish Kumar, Ajay Mittal, and Krishan Kumar. 2023. Self-supervised learning: A succinct review. *Archives of Computational Methods in Engineering*, 30(4):2761–2775.
- Cyrus Rashtchian, Peter Young, Micah Hodosh, and Julia Hockenmaier. 2010. Collecting image annotations using amazon’s mechanical turk. In *Proceedings of the NAACL HLT 2010 Workshop on Creating Speech and Language Data with Amazon’s Mechanical Turk*, pages 139–147.
- Alon Ravid and Daniel M Berry. 2000. A method for extracting and stating software requirements that a user interface prototype contains. *Requirements Engineering*, 5(4):225–241.
- Partha Pratim Ray. 2025. A review on vibe coding: Fundamentals, state-of-the-art, challenges and future directions. *Authorea Preprints*.
- Redis. 2025. Redis. <https://github.com/redis/redis>. Accessed: 2025-12-18.
- Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*.
- Tim Rietz and Alexander Maedche. 2019. Ladderbot: A requirements self-elicitation system. In *2019 IEEE 27th International Requirements Engineering Conference (RE)*, pages 357–362. IEEE.
- Daniel Ritchie, Ankita Arvind Kejriwal, and Scott R Klemmer. 2011. d. tour: Style-based exploration of design example galleries. In *Proceedings of the 24th annual ACM symposium on User interface software and technology*, pages 165–174.
- Stephen E Robertson, Steve Walker, Susan Jones, Micheline M Hancock-Beaulieu, Mike Gatford, et al. 1995. Okapi at trec-3. *Nist Special Publication Sp*, 109:109.
- Thiago Rocha Silva, Marco Winckler, and Hallvard Trætteberg. 2019. Ensuring the consistency between user requirements and graphical user interfaces: A behavior-based automated approach. In *Computational Science and Its Applications – ICCSA 2019*, pages 616–632, Cham. Springer International Publishing.
- Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695.
- Joel Ross, Lilly Irani, M Six Silberman, Andrew Zaldivar, and Bill Tomlinson. 2010. Who are the crowdworkers? shifting demographics in mechanical turk. In *CHI’10 extended abstracts on Human factors in computing systems*, pages 2863–2872.

- Winston W Royce. 1987. Managing the development of large software systems: concepts and techniques. In *Proceedings of the 9th international conference on Software Engineering*, pages 328–338.
- Jim Rudd, Ken Stern, and Scott Isensee. 1996. Low vs. high-fidelity prototyping debate. *interactions*, 3(1):76–85.
- Sachidanandam Sakthivel. 1991. A survey of requirements verification techniques. *Journal of Information Technology*, 6(2):68–79.
- Gerard Salton and Christopher Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Information processing & management*, 24(5):513–523.
- Ranjan Sapkota, Konstantinos I Roumeliotis, and Manoj Karkee. 2025. Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic ai. *arXiv preprint arXiv:2505.19443*.
- William Saunders, Catherine Yeh, Jeff Wu, Steven Bills, Long Ouyang, Jonathan Ward, and Jan Leike. 2022. Self-critiquing models for assisting human evaluators. *arXiv preprint arXiv:2206.05802*.
- Pete Sawyer, Ian Sommerville, and Stephen Viller. 1997. Requirements process improvement through the phased introduction of good practice. *Software Process: Improvement and Practice*, 3(1):19–34.
- Kurt Schneider. 2007. Generating fast feedback in requirements elicitation. In *International working conference on requirements engineering: Foundation for software quality*, pages 160–174. Springer.
- Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, et al. 2024. The prompt report: a systematic survey of prompt engineering techniques. *arXiv preprint arXiv:2406.06608*.
- Hinrich Schütze, Christopher D Manning, and Prabhakar Raghavan. 2008. *Introduction to information retrieval*, volume 39. Cambridge University Press Cambridge.
- Ken Schwaber and Mike Beedle. 2001. *Agile software development with Scrum*. Prentice Hall PTR.
- Vinícius CVB Segura, Simone DJ Barbosa, and Fabiana Pedreira Simões. 2012. Uiskei: a sketch-based prototyping tool for defining and evaluating user interface behavior. In *Proceedings of the International Working Conference on Advanced Visual Interfaces*, pages 18–25. ACM.
- Unnati S Shah and Devesh C Jinwala. 2015. Resolving ambiguities in natural language software requirements: a comprehensive survey. *ACM SIGSOFT Software Engineering Notes*, 40(5):1–7.

- Imran Sheikh, Irina Illina, Dominique Fohr, and Georges Linares. 2016. Learning word importance with the neural bag-of-words model. In *ACL, Representation Learning for NLP (Repl4NLP) workshop*.
- Michael R Sheldon, Michael J Fillyaw, and W Douglas Thompson. 1996. The use and interpretation of the friedman test in the analysis of ordinal-scale data in repeated measures designs. *Physiotherapy Research International*, 1(4):221–228.
- Nakatani Shuyo. 2010. Language detection library for java. Retrieved Jul, 7:2016.
- Thiago Rocha Silva, Jean-Luc Hak, Marco Winckler, and Olivier Nicolas. 2017. A comparative study of milestones for featuring gui prototyping tools. *Journal of Software Engineering and Applications*, 10(6):564–589.
- Amit Singhal et al. 2001. Modern information retrieval: A brief overview. *IEEE Data Eng. Bull.*, 24(4):35–43.
- Sketch. 2019. Sketch. <https://www.sketch.com/>. Accessed: 2021-03-21.
- Ray Smith. 2007. An overview of the tesseract ocr engine. In *Ninth international conference on document analysis and recognition (ICDAR 2007)*, volume 2, pages 629–633. IEEE.
- Carolyn Snyder. 2003. *Paper prototyping: The fast and easy way to design and refine user interfaces*. Morgan Kaufmann.
- Ian Sommerville. 2011. Software engineering 9th edition. ISBN-10, 137035152:18.
- OMG Available Specification. 2007. Omg unified modeling language (omg uml), super-structure, v2. 1.2. *Object Management Group*, 70:16.
- Jonette M Stecklein, Jim Dabney, Brandon Dick, Bill Haskins, Randy Lovell, and Gregory Moroney. 2004. Error cost escalation through the project life cycle. In *14th Annual International Symposium, JSC-CN-8435*.
- Ilya Sutskever, James Martens, and Geoffrey E Hinton. 2011. Generating text with recurrent neural networks. In *Proceedings of the 28th international conference on machine learning (ICML-11)*, pages 1017–1024.
- Pedro Szekely. 1994. User interface prototyping: Tools and techniques. In *Workshop on Software Engineering and Human-Computer Interaction*, pages 76–92. Springer.
- Gemini Team, Rohan Anil, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Leonor Teixeira, Vasco Saavedra, Carlos Ferreira, João Simões, and Beatriz Sousa Santos. 2014. Requirements engineering using mockups and prototyping tools: developing a healthcare web-application. In *International Conference on Human Interface and the Management of Information*, pages 652–663. Springer.

- Simone Teufel. 2007. An overview of evaluation methods in trec ad hoc information retrieval and trec question answering. *Evaluation of text and speech systems*, pages 163–186.
- Kashyap Todi, Daryl Weir, and Antti Oulasvirta. 2016. Sketchplore: Sketch and explore with a layout optimiser. In *Proceedings of the 2016 ACM Conference on Designing Interactive Systems*, DIS '16, page 543–555, New York, NY, USA. Association for Computing Machinery.
- Alberto Tonon, Gianluca Demartini, and Philippe Cudré-Mauroux. 2015. Pooling-based continuous evaluation of information retrieval systems. *Information Retrieval Journal*, 18(5):445–472.
- Gerrit Van der Veer and Hans Van Vliet. 2001. The human-computer interface is the system; a plea for a poor man’s hci component in software engineering curricula. In *Proceedings 14th Conference on Software Engineering Education and Training. In search of a software engineering profession* (Cat. No. PR01059), pages 276–286. IEEE.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.
- Veniamin Veselovsky, Manoel Horta Ribeiro, and Robert West. 2023. Artificial artificial intelligence: Crowd workers widely use large language models for text production tasks.
- Gursimran Singh Walia and Jeffrey C Carver. 2009. A systematic literature review to identify and classify software requirement errors. *Information and Software Technology*, 51(7):1087–1109.
- Dolores R Wallace and Roger U Fujii. 1989. Software verification and validation: an overview. *Ieee Software*, 6(3):10–17.
- Bryan Wang, Gang Li, Xin Zhou, Zhouong Chen, Tovi Grossman, and Yang Li. 2021. Screen2words: Automatic mobile ui summarization with multimodal learning. In *The 34th Annual ACM Symposium on User Interface Software and Technology*, pages 498–510.
- Haifeng Wang, Jiwei Li, Hua Wu, Eduard Hovy, and Yu Sun. 2023. Pre-trained language models and their applications. *Engineering*, 25:51–65.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022a. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.
- Yihui Wang, Shanquan Gao, Xingtong Li, Lei Liu, and Huaxiao Liu. 2022b. Missing standard features compared with similar apps? a feature recommendation method based on the knowledge from user interface. *Journal of Systems and Software*, 193:111435.

BIBLIOGRAPHY

- Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. 2021. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.
- Jialiang Wei, Anne-Lise Courbis, Thomas Lambolais, Binbin Xu, Pierre Louis Bernard, and Gérard Dray. 2023. Boosting gui prototyping with diffusion models. In *2023 IEEE 31st International Requirements Engineering Conference (RE)*, pages 275–280. IEEE.
- Paweł Weichbroth and Marcin Sikorski. 2015. User interface prototyping. techniques, methods and tools. *Studia Ekonomiczne. Zeszyty Naukowe Uniwersytetu Ekonomicznego w Katowicach*, pages 184–198.
- Jason Weston, Emily Dinan, and Alexander H Miller. 2018. Retrieve and refine: Improved sequence generation models for dialogue. *arXiv preprint arXiv:1808.04776*.
- R Duncan William. 1996. *A guide to the project management body of knowledge*.
- Peter Windsor and Graham Storrs. 1992. Prototyping user interfaces. In *IEE Colloquium on Software Prototyping and Evolutionary Development*, pages 4–1. IET.
- Jane Wood and Denise Silver. 1995. *Joint application development*. John Wiley & Sons, Inc.
- RF Woolson. 2007. Wilcoxon signed-rank test. *Wiley encyclopedia of clinical trials*, pages 1–3.
- Chenfei Wu, Shengming Yin, Weizhen Qi, Xiaodong Wang, Zecheng Tang, and Nan Duan. 2023. Visual chatgpt: Talking, drawing and editing with visual foundation models. *arXiv preprint arXiv:2303.04671*.
- Qing Xie and Atif M Memon. 2007. Designing and comparing automated test oracles for gui-based software applications. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 16(1):4–es.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- An Yang et al. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2(3).
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2024. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36.

- Mingyue Yuan, Jieshan Chen, and Aaron Quigley. 2024. Maxprototyper: A multi-agent generation system for interactive user interface prototyping. *arXiv preprint arXiv:2405.07131*.
- Brad Vander Zanden and Brad A Myers. 1991. The lapidary graphical interface design tool. In *Proceedings of the SIGCHI conference on human factors in computing systems*, pages 465–466.
- Xia Zeng et al. 2016. Automated test input generation for android: Are we really there yet in an industrial case? In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 987–992.
- Hongyu Zhang, Anuj Jain, Gaurav Khandelwal, Chandrashekhar Kaushik, Scott Ge, and Wenxiang Hu. 2016. Bing developer assistant: improving developer productivity by recommending sample code. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 956–961.
- Tianming Zhao, Chunyang Chen, Yuanning Liu, and Xiaodong Zhu. 2021. Guigan: Learning to generate gui designs using generative adversarial networks. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 748–760. IEEE.
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2).
- Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. 2023. Minigpt-4: Enhancing vision-language understanding with advanced large language models. *arXiv preprint arXiv:2304.10592*.
- Dirk Zimmermann and Lennart Grötzbach. 2007. A requirement engineering approach to user centered design. In *International Conference on Human-Computer Interaction*, pages 360–369. Springer.
- Didar Zowghi and Chad Coulin. 2005. Requirements elicitation: A survey of techniques, approaches, and tools. In *Engineering and managing software requirements*, pages 19–46. Springer.

APPENDIX A

PUBLISHED RESOURCES

We provide an overview of the resources and artifacts published in the context of this thesis, including code, datasets, experimental results, and easily reusable tool prototypes.

Chap.	Resource Name	Type	Location	Challenges
3	RaWi	Code+ Dataset	https://github.com/kristiankolthoff/RaWi	C1-1
3	GUI2WiRe	Code	https://github.com/kristiankolthoff/GUI2WiReTool	C1-1
4	GUI-ReRank	Code+ Dataset	https://github.com/kristiankolthoff/GUI-ReRank	C1-2
5	SERGUI	Code	https://github.com/kristiankolthoff/SERGUI-Prototyping	C1-3
6	ZS-Prompting	Code+ Dataset	https://github.com/kristiankolthoff/ZS-Prompting	C1-4
7	GUI-US-Loop	Code	https://github.com/kristiankolthoff/Closing-the-Loop-US-GUI	C1-5
8	GUIDE	Code	https://github.com/kristiankolthoff/GUIDE-Prototyping	C1-5
9	Interlinking	Code+ Dataset	https://github.com/kristiankolthoff/IUS-GUI-Prototyping	C2-1
10	GUISpector	Code+ Dataset	https://github.com/kristiankolthoff/GUISpector	C2-2

Table A.1: Overview of published resources and artifacts in the context of this thesis.

A. PUBLISHED RESOURCES

In addition to the above resources, we also created and published several videos accompanying our research paper publications, introducing not only the respective novel approaches and techniques, but also the functionality of the developed research tool prototypes.

Chap.	Resource Name	Location	Challenges
3	GUI ₂ WiRe	https://youtu.be/2nN-Xr2Hk7I	C1-1
4	GUI-ReRank	https://youtu.be/_7x9UCh82ug	C1-2
5	SERGUI	https://youtu.be/pzAAB9Uht80	C1-3
7	GUI-US-Loop	https://youtu.be/QQd007gJLHQ	C1-5
8	GUIDE	https://youtu.be/C9RbhMxqpTU	C1-5
10	GUISpector	https://youtu.be/JByYF6BNQeE	C2-2

Table A.2: Overview of published approach demo videos in the context of this thesis.

APPENDIX B

APPENDIX FOR CHAPTER 3

B.1 Experimental Details

Model 1	Model 2	AP	MRR	H@1	H@3	H@5	H@10	H@15
BM25	nBoW	<0.001*	1.000	1.000	1.000	0.914	1.000	1.000
BM25	TF-IDF	0.034*	0.494	1.000	0.118	0.300	0.149	1.000
PRF	BM25	1.000	1.000	1.000	1.000	1.000	1.000	1.000
PRF (s)	BM25	0.608	1.000	1.000	1.000	1.000	1.000	0.809
PRF (w)	BM25	1.000	1.000	1.000	1.000	1.000	1.000	1.000
PRF (sw)	BM25	1.000	1.000	1.000	1.000	1.000	1.000	1.000
SBERT	BM25	0.382	0.636	1.000	0.927	0.669	0.092*	0.169
BERT-LTR (1)	BM25	0.050	0.044*	1.000	0.315	0.012*	0.007*	0.053
BERT-LTR (2)	BM25	0.016*	0.131	1.000	0.106	< 0.001*	0.003*	0.053
BERT-LTR (3)	BM25	0.039*	0.168	1.000	0.206	0.012*	0.001*	0.053
BERT-LTR (1)	SBERT	0.388	0.572	1.000	1.000	0.272	1.000	1.000
BERT-LTR (2)	SBERT	0.305	0.530	1.000	0.927	0.011*	1.000	1.000
BERT-LTR (3)	SBERT	0.388	0.572	1.000	1.000	0.265	0.182	1.000

Table B.1: *Holm*-corrected p -values from *Wilcoxon signed-rank test* over NLR-based GUI retrieval gold standard with AP , MRR , and $HITS@k$. Particularly, we test whether *model 1* is greater than *model 2*. **Bold*** indicates significance at $\alpha = 0.05$ after *Holm* correction.

Model 1	Model 2	P@3	P@5	P@7	P@10
BM25	nBoW	1.000	0.309	0.486	0.306
BM25	TF-IDF	0.149	0.028*	0.012*	0.011*
PRF	BM25	1.000	1.000	0.946	0.448
PRF (s)	BM25	1.000	1.000	0.946	0.251
PRF (w)	BM25	1.000	1.000	0.946	0.459
PRF (sw)	BM25	1.000	1.000	0.946	0.306
SBERT	BM25	0.948	0.797	0.195	0.013*
BERT-LTR (1)	BM25	0.407	0.052	0.031*	0.008*
BERT-LTR (2)	BM25	0.065	0.089	0.037*	0.002*
BERT-LTR (3)	BM25	0.706	0.061	0.025*	<0.001*
BERT-LTR (1)	SBERT	1.000	0.608	0.946	0.577
BERT-LTR (2)	SBERT	0.368	1.000	0.946	0.459
BERT-LTR (3)	SBERT	1.000	0.282	0.764	0.441

Table B.2: *Holm*-corrected p -values from *Wilcoxon signed-rank test* over NLR-based GUI retrieval gold standard with *precision at rank k* ($P@k$). Particularly, we test whether *Model 1* is greater than *Model 2*. **Bold*** indicates significance at $\alpha = 0.05$ after *Holm* correction.

Model 1	Model 2	NDCG@3	NDCG@5	NDCG@10	NDCG@15
BM25	nBoW	0.877	0.213	<0.001*	<0.001*
BM25	TF-IDF	0.054	0.012*	0.002*	0.001*
PRF	BM25	0.969	0.816	1.000	0.942
PRF (s)	BM25	0.969	0.525	0.455	0.204
PRF (w)	BM25	0.877	1.000	1.000	1.000
PRF (sw)	BM25	0.969	1.000	1.000	1.000
SBERT	BM25	0.384	0.213	0.004*	0.004*
BERT-LTR (1)	BM25	0.042*	0.007*	<0.001*	<0.001*
BERT-LTR (2)	BM25	0.011*	0.007*	<0.001*	<0.001*
BERT-LTR (3)	BM25	0.090	0.012*	<0.001*	<0.001*
BERT-LTR (1)	SBERT	0.247	0.213	0.706	0.424
BERT-LTR (2)	SBERT	0.091	0.213	0.455	0.165
BERT-LTR (3)	SBERT	0.495	0.228	0.409	0.399

Table B.3: *Holm*-corrected p -values from *Wilcoxon signed-rank test* over NLR-based GUI retrieval gold standard with metric *NDCG@ k* . Particularly, we test whether *Model 1* is greater than *Model 2*. **Bold*** indicates significance at $\alpha = 0.05$ after *Holm* correction.

Minute	$\bar{C}_A$	$\tilde{C}_A$	$\bar{C}_B$	$\tilde{C}_B$	$\Delta\bar{C}$	$\Delta\tilde{C}$	δ	W	p	p_{Holm}
3	15.87	17.00	5.05	4.00	10.82	12.00	0.807	692.0	<0.001	<0.001*
4	17.68	19.00	7.18	6.50	10.50	12.00	0.785	685.5	<0.001	<0.001*
5	19.74	22.00	9.82	9.00	9.92	10.50	0.756	656.5	<0.001	<0.001*
6	21.16	23.00	12.55	11.50	8.61	8.00	0.672	659.0	<0.001	<0.001*
7	22.97	24.00	15.11	14.00	7.87	7.00	0.659	688.5	<0.001	<0.001*

Table B.4: **Holm**-corrected p -values from pairwise one-sided *Wilcoxon signed-rank tests* comparing tool A (*RaWi*) with tool B (*Mockplus*) on paired task data per minute on the $\#GUI\text{-}comps$ metric ($\alpha = 0.05$). $\bar{C}$ and $\tilde{C}$ represent the mean and median values, respectively. $\Delta\bar{C}$ and $\Delta\tilde{C}$ denote mean and median differences. δ denotes Cliff's Delta. **Bold** values with * indicate significance at $\alpha = 0.05$ after applying the *Holm* correction.

Minute	$\bar{C}_A$	$\tilde{C}_A$	$\bar{C}_B$	$\tilde{C}_B$	$\Delta\bar{C}$	$\Delta\tilde{C}$	δ	W	p	p_{Holm}
3	15.50	16.50	4.76	4.00	10.74	12.50	0.810	692.0	<0.001	<0.001*
4	16.89	18.00	6.45	6.00	10.45	11.50	0.810	693.0	<0.001	<0.001*
5	18.97	19.00	8.71	7.50	10.26	10.50	0.805	661.5	<0.001	<0.001*
6	20.45	21.50	10.45	9.50	10.00	9.00	0.834	732.0	<0.001	<0.001*
7	22.11	23.50	12.89	12.00	9.21	9.00	0.808	695.0	<0.001	<0.001*

Table B.5: **Holm**-corrected p -values from pairwise one-sided *Wilcoxon signed-rank tests* comparing tool A (*RaWi*) with tool B (*Mockplus*) on paired task data per minute on the $\#GUI\text{-}comps\text{-}div$ metric ($\alpha = 0.05$). $\bar{C}$ and $\tilde{C}$ represent the mean and median values, respectively. $\Delta\bar{C}$ and $\Delta\tilde{C}$ denote mean and median differences, respectively. δ denotes Cliff's Delta. **Bold** values with * indicate significance at $\alpha = 0.05$ after applying the *Holm* correction.

Minute	$\bar{C}_A$	$\tilde{C}_A$	$\bar{C}_B$	$\tilde{C}_B$	$\Delta\bar{C}$	$\Delta\tilde{C}$	δ	W	p	p_{Holm}
3	10.45	11.00	0.29	2.00	10.16	10.50	0.680	656.0	<0.001	<0.001*
4	12.97	13.00	3.32	3.00	9.66	10.00	0.660	591.5	<0.001	<0.001*
5	15.68	14.50	6.76	6.50	8.92	8.50	0.605	598.5	<0.001	<0.001*
6	17.74	18.00	10.50	10.50	7.24	6.50	0.512	578.5	<0.001	<0.001*
7	19.76	20.00	13.18	13.00	6.58	5.00	0.482	545.0	<0.001	<0.001*

Table B.6: **Holm**-corrected p -values (and raw p -values) from pairwise one-sided *Wilcoxon signed-rank tests* comparing tool A (*RaWi*) with tool B (*Mockplus*) on paired task data per minute on the metric $\#GUI\text{-}comps$ adjusted by the $\#GUI\text{-}comps\text{-}neg$ metric ($\alpha = 0.05$). $\bar{C}$ and $\tilde{C}$ represent the mean and median values, respectively. $\Delta\bar{C}$ and $\Delta\tilde{C}$ denote mean and median differences, respectively. δ denotes Cliff's Delta. **Bold** values with * indicate significance at $\alpha = 0.05$ after applying the *Holm* correction.

Term	β	SE	z	p	CI _L	CI _U
Intercept	12.435	1.050	11.838	<0.001	10.376	14.494
Approach B	-11.090	0.874	-12.691	<0.001	-12.803	-9.377
Task 2	-0.188	0.505	-0.372	0.710	-1.178	0.802
GUI 2	2.216	0.504	4.394	<0.001	1.227	3.204
Condition 2	4.640	1.153	4.024	<0.001	2.380	6.900
Condition 3	4.460	1.153	3.868	<0.001	2.200	6.720
Condition 4	0.468	1.223	0.382	0.702	-1.930	2.865
Time	1.768	0.252	7.013	<0.001	1.274	2.263
Approach B $\times$ Time	0.779	0.357	2.184	0.029	0.080	1.478

Table B.7: Results of the linear mixed-effects model for the *#GUI-comps* productivity metric. The model includes *approach*, *time*, *task*, *GUI*, and *condition* as fixed effects, as well as an interaction term between *approach* and *time* ($\#GUI-comps \sim Approach \times Time_c + Task + GUI + Condition + (1 | Participant)$). Participant was modeled as a random intercept to account for repeated measurements. Coefficients (β), standard errors (SE), Wald z statistics, p -values, and 95% confidence intervals are reported. Approach A (*RaWi*), task 1, GUI 1, and condition 1 serve as reference levels. Time was centered at minute 3 (representing the starting point of collection). Approach B represents *Mockplus*.

Term	β	SE	z	p	CI _L	CI _U
Intercept	12.090	0.912	13.263	<0.001	10.303	13.877
Approach B	-10.851	0.791	-13.718	<0.001	-12.401	-9.301
Task 2	0.371	0.457	0.812	0.417	-0.525	1.267
GUI 2	1.795	0.456	3.932	<0.001	0.900	2.689
Condition 2	4.600	0.971	4.737	<0.001	2.697	6.503
Condition 3	3.690	0.971	3.800	<0.001	1.787	5.593
Condition 4	0.412	1.030	0.401	0.689	-1.606	2.431
Time	1.676	0.228	7.345	<0.001	1.229	2.124
Approach B $\times$ Time	0.350	0.323	1.084	0.278	-0.283	0.983

Table B.8: Results of the linear mixed-effects model for the *#GUI-comps-div* productivity metric. The model includes *approach*, *time*, *task*, *GUI*, and *condition* as fixed effects, as well as an interaction term between *approach* and *time* ($\#GUI-comps-div \sim Approach \times Time_c + Task + GUI + Condition + (1 | Participant)$). Participant was modeled as a random intercept to account for repeated measurements. Coefficients (β), standard errors (SE), Wald z statistics, p -values, and 95% confidence intervals are reported. Approach A (*RaWi*), task 1, GUI 1, and condition 1 serve as reference levels. Time was centered at minute 3 (representing the starting point of collection). Approach B represents *Mockplus*.

Term	β	SE	z	p	CI _L	CI _U
Intercept	5.621	0.934	6.018	<0.001	3.790	7.452
Approach B	-0.737	0.725	-1.016	0.310	-2.158	0.685
Task 2	1.194	0.419	2.850	0.004	0.373	2.016
GUI 2	1.042	0.418	2.490	0.013	0.222	1.862
Condition 2	-1.600	1.068	-1.498	0.134	-3.694	0.494
Condition 3	-0.880	1.068	-0.824	0.410	-2.974	1.214
Condition 4	-3.562	1.133	-3.144	0.002	-5.783	-1.342
Time	-0.571	0.209	-2.729	0.006	-0.981	-0.161
Approach B $\times$ Time	-0.179	0.296	-0.605	0.545	-0.759	0.401

Table B.9: Results of the linear mixed-effects model for the *#GUI-comps-neg-div* productivity metric. The model includes *approach*, *time*, *task*, *GUI*, and *condition* as fixed effects, as well as an interaction term between *approach* and *time* (*#GUI-comps-neg-div* $\sim$ Approach $\times$ Time_c + Task + GUI + Condition + (1 | Participant)). Participant was modeled as a random intercept to account for repeated measurements. Coefficients (β), standard errors (SE), Wald z statistics, p -values, and 95% confidence intervals are reported. Approach A (*RaWi*), task 1, GUI 1, and condition 1 serve as reference levels. Time was centered at minute 3 (representing the starting point). Approach B represents *Mockplus*.

APPENDIX C

APPENDIX FOR CHAPTER 4

C.1 Experimental Details

Model 1	Model 2	AP	MRR	H@1	H@3	H@5	H@10	H@15
G4.I (I)	BERT-LTR-1	<0.001*	<0.001*	<0.001*	<0.001*	0.001*	1.000	1.000
G4.I (I)	BERT-LTR-2	<0.001*	<0.001*	<0.001*	<0.001*	0.012*	1.000	1.000
G4.I (I)	BERT-LTR-3	<0.001*	<0.001*	<0.001*	<0.001*	0.001*	1.000	1.000
G4.I (T)	BERT-LTR-1	<0.001*	<0.001*	<0.001*	<0.001*	0.001*	1.000	1.000
G4.I (T)	BERT-LTR-2	<0.001*	<0.001*	<0.001*	<0.001*	0.012*	1.000	1.000
G4.I (T)	BERT-LTR-3	<0.001*	<0.001*	<0.001*	<0.001*	0.001*	1.000	1.000
G4.I (I)	G4.I (T)	0.050	0.785	1.000	0.841	1.000	1.000	1.000
G4.I (I)	G4.IM (I)	0.043*	0.785	1.000	0.317	1.000	1.000	1.000
G4.I (I)	G4.IN (I)	<0.001*	<0.001*	<0.001*	0.034*	0.793	1.000	1.000
G4.IM (I)	G4.IN (I)	<0.001*	<0.001*	<0.001*	0.087	0.793	1.000	1.000
G4.I (T)	G4.IM (T)	0.055	0.709	1.000	0.236	1.000	1.000	1.000
G4.I (T)	G4.IN (T)	<0.001*	0.009*	0.026*	0.009*	0.159	1.000	1.000
G4.IM (T)	G4.IN (T)	<0.001*	0.020*	0.037*	0.087	0.159	1.000	1.000

Table C.1: *Holm*-corrected p -values from *Wilcoxon signed-rank test* with *AP*, *MRR*, and *HITS@k*. Particularly, we test whether *model 1* is greater than *model 2*. **Bold*** indicates significance at $\alpha = 0.05$ after *Holm* correction. Abbreviations: I=Image, T=Text, M=Mini, N=Nano, G4.I=GPT-4.I.

Model 1	Model 2	P@3	P@5	P@7	P@10
G4.I (I)	BERT-LTR-1	<0.001*	<0.001*	<0.001*	<0.001*
G4.I (I)	BERT-LTR-2	<0.001*	<0.001*	<0.001*	<0.001*
G4.I (I)	BERT-LTR-3	<0.001*	<0.001*	<0.001*	<0.001*
G4.I (T)	BERT-LTR-1	<0.001*	<0.001*	<0.001*	<0.001*
G4.I (T)	BERT-LTR-2	<0.001*	<0.001*	<0.001*	<0.001*
G4.I (T)	BERT-LTR-3	<0.001*	<0.001*	<0.001*	<0.001*
G4.I (I)	G4.I (T)	0.057	0.159	0.129	0.198
G4.I (I)	G4.IM (I)	0.041*	0.082	0.035*	0.223
G4.I (I)	G4.IN (I)	<0.001*	<0.001*	<0.001*	<0.001*
G4.IM (I)	G4.IN (I)	<0.001*	<0.001*	<0.001*	<0.001*
G4.I (T)	G4.IM (T)	0.008*	0.296	0.312	0.223
G4.I (T)	G4.IN (T)	<0.001*	<0.001*	<0.001*	<0.001*
G4.IM (T)	G4.IN (T)	<0.001*	<0.001*	<0.001*	<0.001*

Table C.2: *Holm*-corrected p -values from *Wilcoxon signed-rank test* with precision at rank k ($P@k$). Particularly, we test whether *model 1* is greater than *model 2*. **Bold*** indicates significance at $\alpha = 0.05$ after *Holm* correction. Abbreviations: I=Image, T=Text, M=Mini, N=Nano, G4.I=GPT-4.I.

Model 1	Model 2	NDCG@3	NDCG@5	NDCG@10	NDCG@15
G4.I (I)	BERT-LTR-1	<0.001*	<0.001*	<0.001*	<0.001*
G4.I (I)	BERT-LTR-2	<0.001*	<0.001*	<0.001*	<0.001*
G4.I (I)	BERT-LTR-3	<0.001*	<0.001*	<0.001*	<0.001*
G4.I (T)	BERT-LTR-1	<0.001*	<0.001*	<0.001*	<0.001*
G4.I (T)	BERT-LTR-2	<0.001*	<0.001*	<0.001*	<0.001*
G4.I (T)	BERT-LTR-3	<0.001*	<0.001*	<0.001*	<0.001*
G4.I (I)	G4.I (T)	0.036*	0.011*	0.028*	0.036*
G4.I (I)	G4.IM (I)	0.036*	0.011*	0.047*	0.059
G4.I (I)	G4.IN (I)	<0.001*	<0.001*	<0.001*	<0.001*
G4.IM (I)	G4.IN (I)	<0.001*	<0.001*	<0.001*	<0.001*
G4.I (T)	G4.IM (T)	0.036*	0.054	0.006*	0.041*
G4.I (T)	G4.IN (T)	<0.001*	<0.001*	<0.001*	<0.001*
G4.IM (T)	G4.IN (T)	<0.001*	<0.001*	<0.001*	<0.001*

Table C.3: *Holm*-corrected p -values from *Wilcoxon signed-rank test* with metric $NDCG@k$. Particularly, we test whether *model 1* is greater than *model 2*. **Bold*** indicates significance at $\alpha = 0.05$ after *Holm* correction. Abbreviations: I=Image, T=Text, M=Mini, N=Nano, G4.I=GPT-4.I.

APPENDIX D

APPENDIX FOR CHAPTER 6

D.1 Experimental Details

Metric	Description	n	χ^2	p
A	Feature Completion	50	23.81	<0.001
B	Feature Extensiveness	50	92.49	<0.001
C	Feature Implementation	50	21.00	<0.001
D	Information Organization	50	16.15	0.003
E	Visual Appeal	50	37.45	<0.001
F	Minimal Errors	50	15.98	0.003
G	Overall Satisfaction	50	25.96	<0.001
H	Complete App	50	25.51	<0.001

Table D.1: Results of the *Friedman test* on full dataset (50 GUIs), assessing overall differences between ZS prompting (ZS baselines, PD and SC) across metrics. p -values indicate whether at least one model differs significantly from the others for a given metric.

Metric	Description	n	χ^2	p
A	Feature Completion	15	16.30	0.006
B	Feature Extensiveness	15	28.28	<0.001
C	Feature Implementation	15	13.55	0.019
D	Information Organization	15	4.19	0.523
E	Visual Appeal	15	21.96	<0.001
F	Minimal Errors	15	8.48	0.132
G	Overall Satisfaction	15	13.19	0.022
H	Complete App	15	24.91	<0.001

Table D.2: Results of the *Friedman test* on *cleaned dataset* (15 GUIs), assessing overall differences between ZS prompting (including RAGG) across evaluation metrics. p -values indicate whether at least one model differs significantly from the others for a given metric.

Model 1	Model 2	A	B	C	D	E	F	G	H
ZS	PD1	1.000	<0.001*	1.000	0.142	1.000	0.096	1.000	0.867
ZS	PD2	1.000	0.317	1.000	0.142	0.268	0.490	1.000	0.316
ZS	SC	0.003*	<0.001*	0.028*	0.872	0.003*	0.639	0.003*	0.036*
ZS-CoT	PD1	1.000	<0.001*	1.000	0.142	1.000	0.459	0.703	0.200
ZS-CoT	PD2	1.000	0.100	1.000	0.142	1.000	0.639	1.000	0.927
ZS-CoT	SC	0.002*	<0.001*	0.033*	0.697	0.001*	0.490	<0.001*	0.001*
PD1	PD2	1.000	0.002*	1.000	0.872	0.274	0.639	1.000	0.269
PD1	SC	0.002*	<0.001*	0.009*	0.016*	0.004*	0.012*	0.002*	0.172
PD2	SC	0.002*	<0.001*	0.002*	0.010*	<0.001*	0.017*	<0.001*	<0.001*

Table D.3: *Holm*-corrected p -values from paired post-hoc *Wilcoxon signed-rank tests* (two-sided) on full dataset (50 GUIs). Abbreviations: *A*=Feature Completion, *B*=Feature Extensiveness, *C*=Feature Implementation, *D*=Information Organization, *E*=Visual Appeal, *F*=Minimal Errors, *G*=Overall Satisfaction, *H*=Complete App. PD1=Prompt Decomposition (ZS), PD2=Prompt Decomposition (ZS-CoT) and SC ($k = 4$). **Bold*** indicates significance at $\alpha = 0.05$ after *Holm* correction.

Model 1	Model 2	A	B	C	E	G	H
ZS	RAGG	0.020*	0.029*	0.019*	0.006*	0.022*	0.006*
ZS-CoT	RAGG	0.021*	0.023*	0.062	0.018*	0.050	0.012*
PD1	RAGG	0.181	0.485	0.050*	0.091	0.335	0.064
PD2	RAGG	0.181	0.113	0.062	0.025*	0.147	0.006*
SC	RAGG	0.793	0.485	0.414	0.301	0.572	0.015*

Table D.4: *Holm*-corrected p -values from paired post-hoc *Wilcoxon signed-rank tests* (two-sided) on *cleaned dataset* (15 GUIs). Abbreviations: *A*=Feature Completion, *B*=Feature Extensiveness, *C*=Feature Implementation, *D*=Information Organization, *E*=Visual Appeal, *F*=Minimal Errors, *G*=Overall Satisfaction, *H*=Complete App. PD1=Prompt Decomposition (ZS), PD2=Prompt Decomposition (ZS-CoT), SC ($k = 4$) and RAGG ($k = 7$). **Bold*** indicates significance at $\alpha = 0.05$ after *Holm* correction.

Model 1	Model 2	A	B	C	D	E	F	G	H
RAGG ($k = 1$)	RAGG ($k = 3$)	0.104	0.420	0.416	1.000	0.759	1.000	0.545	0.750
RAGG ($k = 1$)	RAGG ($k = 5$)	0.026*	0.174	0.097	1.000	0.704	0.275	0.545	0.601
RAGG ($k = 1$)	RAGG ($k = 7$)	0.011*	0.174	0.021*	0.481	0.032*	0.447	0.159	0.041*
RAGG ($k = 3$)	RAGG ($k = 5$)	0.670	1.000	0.422	1.000	0.759	0.529	0.714	0.694
RAGG ($k = 3$)	RAGG ($k = 7$)	0.206	1.000	0.021*	0.217	0.056	0.529	0.396	0.098
RAGG ($k = 5$)	RAGG ($k = 7$)	0.206	1.000	0.086	0.934	0.385	1.000	0.714	0.545

Table D.5: *Holm*-corrected p -values from paired post-hoc *Wilcoxon signed-rank tests* (two-sided) comparing RAGG with different numbers of retrieved examples ($k = 1, 3, 5, 7$). Abbreviations: *A*=Feature Completion, *B*=Feature Extensiveness, *C*=Feature Implementation, *D*=Information Organization, *E*=Visual Appeal, *F*=Minimal Errors, *G*=Overall Satisfaction, *H*=Complete App. **Bold*** indicates significance at $\alpha = 0.05$ *Holm* correction.

Model 1	Model 2	A	B	C	D	E	F	G	H
SC ($k = 0$)	SC ($k = 1$)	0.044*	< 0.001*	0.030*	1.000	0.031*	0.593	0.005*	0.046*
SC ($k = 0$)	SC ($k = 2$)	0.004*	< 0.001*	0.007*	0.154	< 0.001*	0.695	< 0.001*	< 0.001*
SC ($k = 0$)	SC ($k = 3$)	0.002*	< 0.001*	0.030*	0.981	< 0.001*	0.689	< 0.001*	< 0.001*
SC ($k = 0$)	SC ($k = 4$)	0.004*	< 0.001*	0.030*	1.000	0.004*	1.000	0.004*	0.031*
SC ($k = 1$)	SC ($k = 2$)	1.000	0.002*	1.000	0.211	0.045*	1.000	0.126	0.004*
SC ($k = 1$)	SC ($k = 3$)	0.693	< 0.001*	0.853	0.981	0.045*	1.000	0.126	0.028*
SC ($k = 1$)	SC ($k = 4$)	1.000	0.002*	1.000	1.000	0.530	1.000	1.000	0.492
SC ($k = 2$)	SC ($k = 3$)	1.000	0.088	1.000	1.000	1.000	1.000	1.000	0.492
SC ($k = 2$)	SC ($k = 4$)	1.000	0.134	1.000	0.981	1.000	1.000	0.289	0.036*
SC ($k = 3$)	SC ($k = 4$)	1.000	0.768	1.000	1.000	1.000	1.000	0.640	0.248

Table D.6: *Holm*-corrected p -values from paired post-hoc *Wilcoxon signed-rank tests* (two-sided) comparing SC with different loop counts ($k = 0, 1, 2, 3, 4$), where $k = 0$ corresponds to the underlying ZS baseline (as base prototype input to the SC approach). Abbreviations: *A*=Feature Completion, *B*=Feature Extensiveness, *C*=Feature Implementation, *D*=Information Organization, *E*=Visual Appeal, *F*=Minimal Errors, *G*=Overall Satisfaction, *H*=Complete App. **Bold*** indicates significance at $\alpha = 0.05$ after *Holm* correction (within each metric across pairwise k comparisons).

Method	A	B	C	D	E	F	G	H
ZS-CoT (+ Content)	0.050*	0.010*	0.302	0.059	0.002*	0.045*	0.004*	< 0.001*
PD-ZS (+ Content)	0.013*	0.065	0.132	0.468	0.003*	0.040*	0.008*	0.005*
RAGG (+ Content)	0.194	0.282	0.266	0.779	0.235	0.228	0.127	0.325
SC (+ Content)	0.467	0.357	0.131	0.240	0.051	0.068	0.101	0.032*

Table D.7: Paired *Wilcoxon signed-rank tests* (two-sided) comparing prompting techniques (ZS-CoT, PD-ZS, RAGG and SC) *with* and *without* content generation. Abbreviations: *A*=Feature Completion, *B*=Feature Extensiveness, *C*=Feature Implementation, *D*=Information Organization, *E*=Visual Appeal, *F*=Minimal Errors, *G*=Overall Satisfaction, *H*=Complete App. RAGG and SC both use $k = 3$. **Bold*** indicates significance at $\alpha = 0.05$.

APPENDIX E

APPENDIX FOR CHAPTER 9

E.1 Experimental Details

Model 1	Model 2	n_{10}	n_{01}	Disc.	χ^2	p	p_{Holm}
ZS	FS-5	6	5	11	0.000	1.000	1.000
ZS	FS-10	9	5	14	0.643	0.423	1.000
ZS	CoT-0.0	21	11	32	2.531	0.112	1.000
ZS	CoT-0.5	23	11	34	3.559	0.059	1.000
ZS	CoT-1.0	16	11	27	0.593	0.441	1.000
ZS	CoT-1.3	22	8	30	5.633	0.018	0.370
FS-5	FS-10	5	2	7	0.571	0.450	1.000
FS-5	CoT-0.0	21	12	33	1.939	0.164	1.000
FS-5	CoT-0.5	21	10	31	3.226	0.072	1.000
FS-5	CoT-1.0	16	12	28	0.321	0.571	1.000
FS-5	CoT-1.3	25	12	37	3.892	0.049	0.970
FS-10	CoT-0.0	21	15	36	0.694	0.405	1.000
FS-10	CoT-0.5	21	13	34	1.441	0.230	1.000
FS-10	CoT-1.0	16	15	31	0.000	1.000	1.000
FS-10	CoT-1.3	24	14	38	2.132	0.144	1.000
CoT-0.0	CoT-0.5	8	6	14	0.071	0.789	1.000
CoT-0.0	CoT-1.0	5	10	15	1.067	0.302	1.000
CoT-0.0	CoT-1.3	18	14	32	0.281	0.596	1.000
CoT-0.5	CoT-1.0	7	14	21	1.714	0.190	1.000
CoT-0.5	CoT-1.3	20	18	38	0.026	0.871	1.000
CoT-1.0	CoT-1.3	22	13	35	1.829	0.176	1.000

Table E.1: *Holm*-corrected p -values from the *McNemar* tests (with continuity correction) on paired correctness for US detection. Particularly, we test whether *Model 1* differs from *Model 2*. n_{10} : *Model 1* correct/*Model 2* incorrect; n_{01} : *Model 1* incorrect/*Model 2* correct, Disc. = $n_{10} + n_{01}$. Abbrev.: ZS=Zero-Shot, FS=Few-Shot, CoT=Chain-of-Thought.

Model 1	Model 2	$\overline{F1}_1$	$\overline{F1}_2$	$\Delta \overline{F1}$	W	p	p_{Holm}
ZS _A	ZS _B	0.755	0.743	0.012	2053.5	0.278	1.000
ZS _A	FS-5	0.755	0.765	-0.009	2483.5	0.558	1.000
ZS _A	CoT-0.0	0.755	0.727	0.028	1893.0	0.0632	0.505
ZS _A	CoT-0.5	0.755	0.688	0.067	2950.0	<0.001	0.000948*
ZS _A	CoT-1.0	0.755	0.690	0.066	3313.0	<0.001	0.00602*
ZS _A	CoT-1.3	0.755	0.654	0.102	4309.5	<0.001	<0.001*
ZS _B	FS-5	0.743	0.765	-0.022	3683.0	0.735	1.000
ZS _B	CoT-0.0	0.743	0.727	0.016	3344.5	0.147	0.736
ZS _B	CoT-0.5	0.743	0.688	0.055	4184.0	0.00152	0.0198*
ZS _B	CoT-1.0	0.743	0.690	0.054	4861.0	0.00408	0.0449*
ZS _B	CoT-1.3	0.743	0.654	0.089	5447.0	<0.001	0.000833*
FS-5	CoT-0.0	0.765	0.727	0.038	3795.5	0.0214	0.214
FS-5	CoT-0.5	0.765	0.688	0.077	4303.0	<0.001	0.00699*
FS-5	CoT-1.0	0.765	0.690	0.075	5166.0	0.00122	0.0171*
FS-5	CoT-1.3	0.765	0.654	0.111	5316.0	<0.001	<0.001*
CoT-0.0	CoT-0.5	0.727	0.688	0.039	2552.0	0.00348	0.0418*
CoT-0.0	CoT-1.0	0.727	0.690	0.037	2625.0	0.0462	0.416
CoT-0.0	CoT-1.3	0.727	0.654	0.073	3804.5	<0.001	0.00416*
CoT-0.5	CoT-1.0	0.688	0.690	-0.001	2510.5	0.651	1.000
CoT-0.5	CoT-1.3	0.688	0.654	0.034	3824.5	0.0859	0.515
CoT-1.0	CoT-1.3	0.690	0.654	0.036	4412.5	0.0651	0.505

Table E.2: *Holm*-corrected p -values from pairwise one-sided *Wilcoxon signed-rank tests* on paired per-US $F1$ scores, testing whether *Model 1* has greater $F1$ than *Model 2*. $\Delta \overline{F1} = \overline{F1}_1 - \overline{F1}_2$. **Bold** values with * indicate significance at $\alpha = 0.05$ after applying *Holm* correction. Abbreviations: ZS=Zero-Shot, FS=Few-Shot and CoT=Chain-of-Thought.